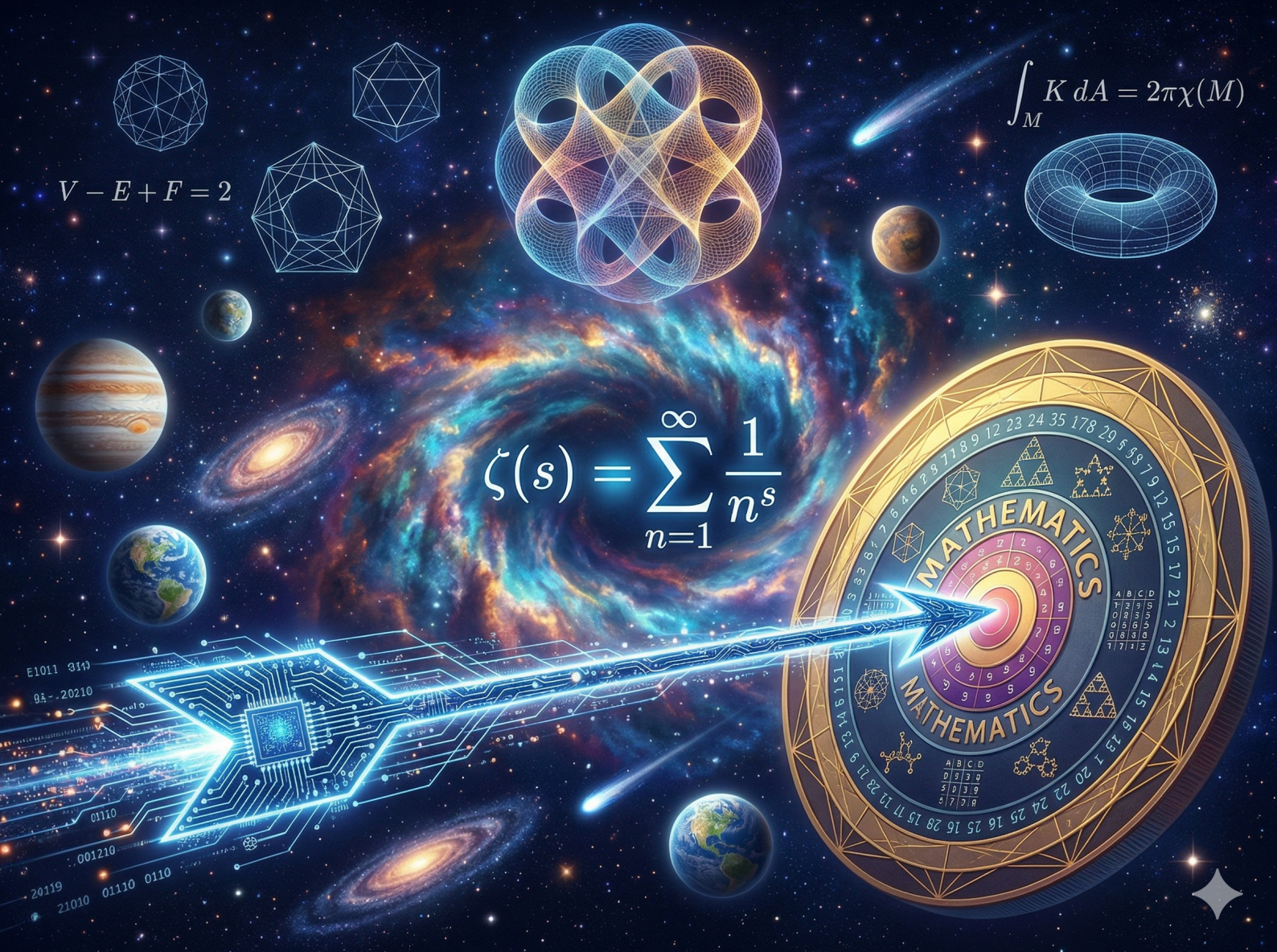




Lectures on AI for Mathematics

作者：Xiaoyang Chen & Xiang Jiang

组织：同济大学数学科学学院

时间：2026/4/11

版本：1.0



目录

第一章 概述	1
1.1 数学研究的核心要素	1
1.2 AI 在数学研究中扮演的角色	1
1.3 核心思想：数学问题作为主线	5
1.4 争议与思考：AI 时代数学研究的新议题	5
第二章 机器学习简介	7
2.1 机器学习概述	7
2.2 线性模型	8
2.2.1 监督学习中的线性模型	8
2.2.2 无监督学习中的线性模型：主成分分析 (PCA)	12
2.2.3 优化算法：梯度下降法	14
2.2.4 模型评估与选择	15
2.3 人工神经网络	16
2.4 强化学习简介	24
2.5 生成式人工智能简介	27
2.6 大语言模型简介	29
2.7 工程技巧	36
第三章 AI 发现数学规律	46
3.1 概述	46
3.2 AI 发现扭结不变量之间的隐藏关系：基于监督学习	46
3.3 AI 发现新算法：基于强化学习和 LLM	51
第四章 AI 证明数学定理	61
4.1 概述	61
4.2 大语言模型的数学能力：技术实现	63
4.3 当前成功案例 1：AlphaProof	65
4.4 当前成功案例 2：基于 LLM 的智能体	68
4.5 大语言模型的创造力	69
第五章 AI 构造数学猜想反例	73
5.1 概述	73
5.2 AI 构造图论猜想反例：基于强化学习	73
5.3 PatternBoost 方法	78
5.4 AI 构造反例：基于 LLM 智能体	82
第六章 AI for PDE (数值计算)	85
6.1 PDE 问题概览与传统数值方法	85
6.2 PINNs 的基本框架与数学原理	97
6.3 应用实例	102

第一章 概述

长久以来，数学被视为纯粹的人类智力巅峰。数学家们依靠洞察力、直觉与严密的逻辑，在抽象的世界里证明猜想、创造理论。当前人工智能，特别是以数据驱动的机器学习技术，取得了突破性进展。一个自然而然的问题浮现出来：人工智能与数学的结合将如何重塑我们探索数学世界的方式？

本书旨在系统性地梳理并回答这个问题。我们将以数学研究作为主线，阐述人工智能介入的重要价值。这并非工具性的“A 如何帮助 B”的简单叠加，而是探究一个更深层次的结构性问题：当数学问题的复杂性与规模达到一定程度时，传统的发现与证明范式面临哪些内在局限？人工智能方法为解决这些局限提供了哪些新的思想工具和可能路径？通过构建一张清晰的“AI for Math”领域的认知图谱，我们希望读者能理解，这不仅仅是一项技术应用，更是一场关于“数学如何被研究”的方法论变革。

1.1 数学研究的核心要素

为了理解人工智能如何发挥作用，我们首先需要理解数学研究工作的本质。虽然每位数学家的具体工作千差万别，但其核心活动大致可以归结为三类相互关联但又各具特点的环节：发现、证明与证否。这三者构成了一个循环往复、相互促进的探索闭环。

- 发现数学规律：**这是数学创造性的源泉。数学家从具体的例子、观察到的模式、或物理世界的现象出发，通过直觉、类比和洞察，提出新的概念、猜想或理论。例如，观察一系列数字的规律后猜想一个通项公式，或是在研究几何对象时提出一个关于其不变量的命题。这一过程的本质是在一个巨大、可能无限的可能性空间中进行搜索和筛选，目标是对其中“有数学意义”或“可能为真”的结构进行定位。人类的直觉在这里扮演了核心的启发式搜索角色，它允许数学家快速聚焦于有希望的“区域”，但也可能因认知偏见或问题的复杂性而遗漏某些非直观但重要的模式。
- 证明定理：**一旦一个猜想被提出，数学的核心价值——严谨性——便要求我们确立其真实性。证明是一个将直觉、猜想转化为无可辩驳的逻辑真理的过程。它确保结论在给定公理和定义下的必然性。证明活动本身也可以被视为一种复杂的搜索：在由公理、定理和推理规则构成的逻辑空间里，寻找一条从已知事实通往待证命题的路径。随着数学理论的高度抽象化和分支的细化，证明变得越来越长、越来越复杂，依赖越来越多的引理和跨领域的知识。
- 构造反例：**构造反例通常极具挑战性，因为它要求我们在一个庞大的候选对象集合中，精确地找到一个特例。人类的构造往往依赖于精巧的归纳、递归技巧，但当约束复杂或搜索空间巨大时，构造反例通常变得极为困难。

1.2 AI 在数学研究中扮演的角色

1. 从机器证明到深度学习

1.1 机器证明的发展历程

机器证明的历史可以追溯到 17 世纪莱布尼茨的“推理演算”构想，但直到 20 世纪计算机技术发展后才真正实现突破。1950 年代，人工智能先驱艾伦·纽厄尔等人首次尝试用计算机程序证明数学定理。而 1976 年，数学家借助计算机完成了四色定理的证明，这一里程碑事件引起了数学界的广泛关注。四色定理的证明需要的穷举分析，远超人类手工处理的能力范围，这一成功案例充分展示了计算机在解决超大规模数学问题中的独特价值。

机器证明的发展经历了几个重要阶段：从早期的穷举法证明，到吴文俊的代数化方法，再到后来的自动推理和交互式证明系统。随着计算机性能的指数级提升，机器证明的能力也在不断增强。以 Coq、Isabelle、lean 等为代表的形式化验证方法，可以确保证明的绝对严谨性，避免了传统数学证明中可能存在的疏漏。

机器证明的价值不仅体现在提高数学研究效率上，更深刻地改变了数学研究的思维方式。它突破了人类认知的局限性，能够处理极其复杂的计算和推理过程；同时它也促进了数学的严格化发展，使数学证明变得更加

可靠和可验证。未来随着人工智能技术的发展，机器证明必将在数学研究中发挥更加重要的作用。

1.2 深度学习革命

深度学习作为机器学习的重要分支，其发展历程可追溯至 20 世纪 40 年代神经网络概念的提出，但真正迎来爆发式发展是在 2010 年后。这一技术革命的核心在于通过构建多层非线性变换的神经网络架构，使计算机获得了前所未有的特征学习和模式识别能力。2012 年 AlexNet 在 ImageNet 竞赛中的突破性表现，标志着深度学习时代的正式来临。随后，随着算力的提升、算法的优化和大数据的积累，深度学习在计算机视觉、自然语言处理等领域取得了一系列令人瞩目的成就。

在数学研究领域，深度学习的价值主要体现在三个关键维度。首先，在发现新数学规律方面，深度学习展现了惊人的潜力。2021 年 DeepMind 团队在《Nature》发表的研究中，通过分析扭结理论中的不变量，发现了人类数学家长期未能察觉的数学联系。

在数学证明验证方面，深度学习与形式化证明系统的结合开创了新局面。通过将神经网络与形式化验证相结合，AI 系统能够辅助完成复杂的证明过程。这种“神经-符号”系统既保留了严格的逻辑性，又具备了强大的数学推理能力。

在构造反例方面，深度学习同样表现出色。通过强化学习算法，AI 系统能够在巨大的组合空间中高效搜索潜在的反例结构。在图论领域，这种方法已经成功构造了多个著名猜想的反例。

深度学习的独特价值在于它突破了人类思维的固有局限。它能够处理人类难以驾驭的高维数据空间，发现非直观的数学模式，并在证明和反例构造中提供新的思路。随着神经符号系统等新技术的发展，深度学习正在推动数学研究进入人机协同的新时代。这项技术不仅提高了研究效率，更重要的是拓展了人类对数学本质的理解，为这个最古老的学科注入了崭新的活力。

2. AI 如何辅助数学研究？

数学研究作为一个系统性的认知活动，主要包括三个核心环节：发现新的数学规律、验证这些规律的正确性、以及构造反例。在本小节我们简单总结当前 AI 在这三个关键领域展现出的能力：

2.1 AI 发现数学规律

2021 年，DeepMind 团队在《Nature》杂志发表突破性成果，他们开发的 AI 系统能够帮助数学家发现扭结不变量之间的新联系。这项研究证明，AI 可以引导数学家的直觉，发现人类可能忽略的数学联系。

扭结 (knot) 是三维空间中的闭合曲线，数学家通常用琼斯多项式 (Jones polynomial) 双曲体积 (hyperbolic volume) 等不变量来描述其性质。DeepMind 将扭结的代数不变量和几何不变量编码为高维向量，并利用深度神经网络 (DNN) 学习它们之间的潜在关系，即学习如何从一种不变量 (如琼斯多项式) 预测另一种不变量 (如双曲体积)。数学家基于 AI 的预测，进一步构建了严格的数学证明，最终在扭结理论中发现了不变量之间的新联系。

在另一项发表在《Nature》的论文里，DeepMind 利用深度强化学习 (Deep Reinforcement Learning, DRL) 发现了更高效的矩阵乘法算法，突破了人类数学家在该领域的最佳记录。这一成果展示了 AI 如何通过自主探索发现人类未曾想到的数学优化策略。矩阵乘法是计算机科学和数学中的基础运算，传统算法 (如 Strassen 算法) 已经优化了数十年。对于两个 $n \times n$ 矩阵相乘，标准方法需要 $O(n^3)$ 次标量乘法。DeepMind 的目标是找到更少的乘法组合，从而降低计算复杂度。DeepMind 采用 AlphaTensor (基于 AlphaZero 的改进模型)，其核心组件包括：

- 状态表示 (State Representation)
 - 将矩阵乘法问题编码为三维张量 (Tensor)，每个元素代表可能的乘法组合。
 - 例如， 2×2 矩阵乘法可表示为 $4 \times 4 \times 4$ 张量，其中每个维度对应输入/输出矩阵的元素。
- 动作空间 (Action Space):
 - 每一步动作对应一个基本乘法操作 (如标量乘法)。
 - AI 的目标是通过一系列动作分解张量，找到最少的乘法步骤。
- 奖励函数 (Reward Function):
 - 主要优化目标：减少乘法次数 (即张量分解的秩)。
 - 额外奖励：发现结构化模式 (如对称性)，以便推广到更大矩阵。

通过将蒙特卡洛树搜索（MCTS）与深度学习结合，DeepMind 最终发现了更高效的矩阵乘法算法。

2.2 AI 自动证明定理

现代人工智能系统在数学证明领域取得了显著进展，其技术实现主要体现在两个重要方向：奥林匹克级几何题求解和形式化数学证明。这些突破性进展展示了 AI 在数学推理方面的强大能力。

在几何证明方面，DeepMind 开发的 AlphaGeometry 系统代表了当前最先进的技术水平。该系统采用神经符号混合架构，结合了语言模型和符号引擎的逻辑推理能力。具体而言，AlphaGeometry 首先分析几何图形，生成潜在的辅助构造点（如中点、垂足等）。然后，符号推理引擎会将这些构造点纳入演绎数据库，应用几何公理和定理进行严格的逻辑推导。系统在训练过程中使用了超过 1 亿个合成几何问题，这些问题通过随机定理生成算法创建，确保了训练数据的多样性和复杂性。值得注意的是，在 2023 年的测试中，AlphaGeometry 在 30 道 IMO 几何题中解决了 25 道。

在形式化验证领域，以 Lean 为代表的交互式定理证明器扮演着验证内核的核心角色。这类系统基于依赖类型理论，能够对数学推理进行终极的机器检查——一旦用户或外部工具提供了完整的证明步骤，Lean 内核会以不容置疑的方式验证其逻辑正确性。

为了提升这种验证过程的效率，近年来 AI 技术被引入作为证明助手，辅助人类或自动完成证明的构建。其协作模式如下：

- 形式化编码与语义理解：首先将非正式的数学命题转化为严格的、机器可读的形式化陈述（如 Lean 的代码）。AI（如预训练的大语言模型）通过理解这些语义，为下一步的证明策略提供智能建议。
- 智能搜索与策略优化：面对复杂的证明空间，利用 AI 进行启发式搜索，预测哪些证明路径更有可能成功，从而避免穷举式的低效尝试。
- 自动化填补与最终验证：AI 工具负责填补具体的证明步骤。一旦这些步骤被完整提交给 Lean 内核，内核便执行严格的形式化验证，确保整个证明过程无懈可击。

在这种模式下，AI 负责“探索”和“猜想”，而 Lean 负责“验证”和“确证”。两者的结合，既保留了形式化验证的绝对可靠性，又大幅提升了构造证明的效率。

这些 AI 系统的核心技术突破在于：1）将神经网络的直觉能力与符号系统的严格推理相结合；2）构建了大规模的数学问题数据集来训练和验证系统性能。当前的研究重点正在向更复杂的数学领域拓展，预示着 AI 将成为数学研究不可或缺的智能助手。

2.3 AI 构造猜想反例

近年来，强化学习算法在图论领域取得了突破性进展，特别是在构造组合数学猜想反例方面展现出独特优势。这种方法通过精心设计的算法框架，能够在巨大的组合可能性空间中高效地搜索潜在的反例结构，其核心技术主要包括：

问题建模与状态表示：

首先将图论问题转化为适合机器学习的形式。对于图论猜想，状态空间通常表示为图的邻接矩阵或特征向量。例如，在构造反例时，系统会初始化一个 n 个顶点的图，其中每个可能的边都作为一个独立的决策变量。

强化学习框架设计：

采用强化学习算法，其中：

- 动作空间：包括添加/删除边、改变顶点属性等图修改操作
- 奖励函数：精心设计为多目标优化形式，包含：
 - 主要奖励：违反目标猜想的程度（如违反某个不等式的大小）
 - 辅助奖励：保持图的其他性质（如连通性、正则性等）
 - 惩罚项：控制图的复杂度（如边数、顶点数）

通过强化学习，AI 成功构造了多个著名猜想的反例，展示了 AI 在探索数学问题上的独特优势。

3. 大模型时代的数学革命

数学，这门探索宇宙真理的古老学科，正在经历一场由人工智能大模型带来的深刻变革。从 ChatGPT 到 AlphaGeometry，这些人工智能系统不仅改变了数学研究的方式，更在重新定义数学发现的本质。这场变革的核

心在于，人工智能正在突破人类认知的局限，以前所未有的方式整合数学知识、发现隐藏模式，并创造性地解决开放性问题。这种转变不仅提高了数学研究的效率，更重要的是拓展了数学探索的疆界，为这门最严谨的科学注入了新的活力。

在知识整合方面，人工智能大模型正在成为打破学科壁垒的“超级助手”。传统数学研究面临的一个主要挑战是学科的高度专业化。随着数学的发展，各个分支领域变得越来越精深，一个领域的专家可能对相邻领域的进展知之甚少，而这种知识割裂常常阻碍重大突破的产生。著名数学家希尔伯特曾指出：“数学是一个有机整体，它的生命力正来自于各个部分之间意想不到的联系。”然而，在实践层面，发现这些联系往往需要研究者具备罕见的广博学识和非凡的洞察力。

大语言模型通过其海量的知识储备和强大的关联能力，正在改变这一局面。以 GPT 为代表的大模型可以即时调用数万篇数学论文的知识，建立跨领域的知识图谱，发现不同分支间的深层联系。例如，当研究者研究代数几何中的某个难题时，AI 可能提示：“这个结构与拓扑学中的同调理论有惊人的相似性”；或者在研究数论问题时指出：“这个猜想与量子计算中的相位估计算法存在对应关系”。这种跨学科的联想能力，在过去需要数学家数十年的广泛阅读和深入思考才能获得，现在则可以通过 AI 的辅助在短时间内实现。

DeepMind 开发的 FunSearch 系统生动展示了这种知识整合的威力。该系统通过巧妙结合大型语言模型的创造性思维和评估代码的精确验证，在组合数学中发现了新的上限构造，解决了开放多年的帽集问题。这一突破性进展的关键在于，AI 能够自由地在离散数学、算法设计和信息理论等看似不相关的领域间建立连接，发现人类研究者可能忽略的关联模式。AI 在这里扮演的不是计算器的角色，而更像是一个具有跨学科视野的合作者，它能够从完全不同的角度审视问题。这种知识整合的价值不仅体现在具体问题的解决上，更重要的是它正在改变数学研究的范式。首先，AI 辅助可以大大缩短数学家的“学习曲线”，使研究者能够快速掌握相关领域的基础知识；其次，它能够揭示不同数学分支之间的“隐藏桥梁”，为新的研究方向提供线索；最后，这种跨领域的知识融合往往能催生全新的数学工具和方法，推动学科的创新发展。

4. AI 能够创造新的数学吗？

创造性思维是人类智慧皇冠上最耀眼的明珠。从欧几里得的公理化体系到高斯的微分几何，从黎曼的复变函数到格罗滕迪克的概形理论，数学史上的每一次重大突破都闪耀着创造性思维的光芒。这种思维究竟包含哪些关键要素？当前的人工智能又为何难以企及？

真正的数学创造性思维至少包含三个相互关联的层面：

1. 概念抽象能力是数学创造的基石。优秀的数学家能从具体问题中抽离出本质特征，形成新的数学概念。比如欧拉看到柯尼斯堡七桥问题时，没有停留在具体的桥与河，而是抽象出拓扑的基本概念。这种从具体到抽象的飞跃，需要深刻的直觉洞察力。
2. 类比迁移能力使数学思想在不同领域间流动。19 世纪，黎曼将高斯曲面理论的思想迁移到复变函数研究，开创了黎曼曲面理论。这种跨领域的联想能力，依赖于对数学本质的深刻理解，而非表面特征的简单对应。
3. 构建全新的数学理论是最高级的数学创造。格罗滕迪克在代数几何中建立的概形理论，不仅解决具体问题，更构建了全新的理论框架。这种体系化的创造需要宏观的数学视野和严谨的逻辑思维相结合。

数学创造性思维的本质，在于将直觉的飞跃与逻辑的严谨完美结合。当前 AI 还无法复制人类数学家那种“灵感乍现”的创造过程。研究数学创造性思维不仅关乎数学发展本身，也是推动人工智能向更高层次进化的重要途径。

总而言之，从吴文俊的机器证明到今天的 AlphaGeometry，AI 与数学的融合正在创造新的研究范式。虽然 AI 目前还无法完全替代数学家的创造力和洞察力，但它已经成为不可或缺的研究伙伴。未来，随着技术的进步，我们或许将见证 AI 提出全新的数学理论，开启数学研究的新纪元。在这个人机协作的新时代，数学的发展将不再受限于个人智慧，而是由人类与 AI 共同推动。这场变革不仅会改变数学研究的方式，更可能帮助我们揭开宇宙更深层的数学奥秘。

1.3 核心思想：数学问题作为主线

贯穿本书，我们将始终坚持一个核心叙事原则：以数学问题本身为出发点，考察 AI 技术如何作为一种新的方法论来解决这些问题。这意味着我们思考的逻辑链条是：

数学问题 → 适配的 AI 技术 → 具体实现过程

例如面对“理解扭结几何不变量与代数不变量之间的关系”这一深层数学问题，其挑战在于从非结构化的数据中学习不变量之间的联系，这引导我们使用神经网络进行学习并提炼数学洞察。

这种以问题为导向的视角，有助于我们避免陷入对 AI 技术的盲目崇拜或工具性堆砌，而是始终关注：这项技术究竟为我们理解某个数学结构带来了何种新的、本质性的帮助？

1.4 争议与思考：AI 时代数学研究的新议题

在前面三个部分的讨论中，我们系统梳理了数学研究的核心要素、AI 在数学研究中扮演的多重角色，以及将数学问题作为主线的核心思想。至此，我们已经看到 AI 与数学深度融合所带来的巨大潜力——从辅助猜想生成到加速定理证明，从概念隐喻发现到计算实验探索。

然而，正如每一次技术革命都会带来深刻的社会与学术变革，AI 进入数学领域也引发了一系列值得深思的争议。这些问题并非简单的技术难题，而是关乎数学这门学科最本质的定义——谁是创造者？如何建立信任？未来的数学家应具备何种能力？

沿着这一思路，首先浮现的是从“辅助工具”到“合作者”的边界模糊问题。当 AI 系统不仅仅是执行计算，而是能够独立提出核心猜想、发现关键证明步骤时，我们不得不重新审视数学发现的归属与作者身份。若一篇论文的核心洞见由 AI 生成，人类作者应如何署名？是将其视为工具而不予署名，还是应该承认 AI 的“合著者”地位？更进一步，如果 AI 基于海量人类已有知识生成新结果，其知识产权的归属又该如何界定？这些问题虽然目前尚未在学术界形成普遍共识，但随着 AI 在数学研究中扮演的角色日益深入，它们正从理论探讨逐渐走向现实关切。

与归属问题相伴而生的，是验证与信任体系的重新构建。数学作为最严谨的学科，其知识大厦建立在“证明必须正确”这一基石之上。传统上，一篇论文的正确性依赖于同行评议——几位专家阅读并认可其逻辑推导。但当证明过程涉及复杂的 AI 模型、海量的计算实验，甚至包含人类难以直接理解的内在表征时，传统的审阅机制便显露出局限性。数学界需要思考：什么程度的 AI 参与是可以接受的？为了确保结果的可复现性，作者需要公开哪些材料——仅仅是最终结果，还是包括训练代码、数据、模型权重乃至超参数配置？如何区分 AI 的“有效启发”和“潜在错误”？这些问题正在催生学术界的初步讨论，其未来走向将深刻影响数学研究的可信度基础。

最后，也是最深远的影响，体现在数学教育领域。当 AI 能够解决越来越多的标准数学问题，甚至在某些领域展现出超越人类的能力时，我们不禁要问：数学教育的重点应该转向哪里？是继续训练学生掌握那些机器已经可以完美执行的技巧，还是转向培养那些 AI 尚不具备的能力？批判性思维、问题提出来源于对数学本质的理解、跨领域的隐喻式联想、以及验证 AI 输出合理性的判断力，这些能力的重要性将愈发凸显。未来的数学教育或许不再是“传授已知解法”，而是“培养与 AI 协作解决问题的能力”——这要求我们重新思考课程设置、评价方式乃至数学素养的定义。

面对这些争议，我们需要避免两种极端态度：既不应盲目拥抱、忽视潜在风险，也不应因噎废食、抗拒变革。正如望远镜的发明没有取代天文学家，而是极大地扩展了人类的视野，AI 的目标也应是增强数学家的能力，而非替代数学家本身。关键在于如何建立一套既能充分利用 AI 潜力，又能维护学术严谨性和人文价值的框架。

我们正站在一个激动人心的交叉路口。数学，这门从古希腊时代延续至今的最古老、最严谨的学科，正在与人工智能进行一场前所未有的深度融合。这种融合不仅将改变数学研究的工具和方法，更将重塑数学发现的认识论基础——什么是证明？什么是理解？什么是创造？在接下来的章节中，我们将一同探索这些问题，见证并参与这场深刻影响数学未来面貌的变革。

参考文献

- [1] Kenneth Appel and Wolfgang Haken. “The solution of the four-color-map problem”. In: *Scientific American* 237.4 (1977), pp. 108–121.
- [2] François Chollet. “On the measure of intelligence”. In: *arXiv preprint arXiv:1911.01547* (2019).
- [3] Alex Davies et al. “Advancing mathematics by guiding human intuition with AI”. In: *Nature* 600.7887 (2021), pp. 70–74.
- [4] Alhussein Fawzi et al. “Discovering faster matrix multiplication algorithms with reinforcement learning”. In: *Nature* 610.7930 (2022), pp. 47–53.
- [5] Gal Raayoni et al. “Generating conjectures on fundamental constants with the Ramanujan Machine”. In: *Nature* 590.7844 (2021), pp. 67–73.
- [6] Bernardino Romera-Paredes et al. “Mathematical discoveries from program search with large language models”. In: *Nature* 625.7995 (2024), pp. 468–475.
- [7] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [8] Trieu H Trinh et al. “Solving olympiad geometry without human demonstrations”. In: *Nature* 625.7995 (2024), pp. 476–482.
- [9] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems*. Vol. 30. 2017, pp. 5998–6008.
- [10] Adam Zsolt Wagner. “Constructions in combinatorics via neural networks”. In: *arXiv preprint arXiv:2104.14516* (2021).
- [11] 陈小杨. “AI 重塑数学研究”. In: *科学 (中国)* (2025).

第二章 机器学习简介

2.1 机器学习概述

在上一章，我们勾勒了“AI for Math”这一新兴交叉领域的广阔图景。其背后的核心都源于一套被称为机器学习的方法论。机器学习通过数学工具，建立一套从数据中自动学习规律，并据此对未来进行预测或决策的计算框架。理解这一框架的数学本质，是掌握本书所有后续内容的前提。

机器学习核心思想是：从数据中自动学习一个从输入空间到输出空间的“最佳”映射。这种“基于数据自动优化一个映射”的人工智能方法，打开了利用计算能力处理复杂模式的大门。这也正是人工智能能够介入数学研究的起点：寻找一个好的映射，能够从已有数学对象和数据中学习数学的内在规律。

任何机器学习任务都可以分解为三个基本部分：数据，模型，算法，它们共同定义了一个具体并可计算的数学优化问题。

1. 数据

数据是学习的原料，通常以样本集合给出： $\mathcal{D} = \{(x^1, y^1), (x^2, y^2), \dots, (x^N, y^N)\}$ 。这里， N 是样本数量。这里 x^i 为输入数据， y^i 为输出数据。在实际问题中，数据样本通常用一个向量表示，负责将现实问题（如图像、文本、数学公式）转化为算法可处理的数值形式。

2. 模型

模型 $f(x; \theta)$ 是我们用来近似真实数据分布的映射，其中 θ 是需要优化的参数。常见的数学模型有：

- 线性模型：最简单的例子是 $f(x; w, b) = w^T x + b$ ，其中 w, b 为未知参数。这类模型形式简单、易于解释，适用于数据具有较强线性关系的情形。比如：
 - 房价预测：根据房屋面积（ x ）预测价格（ y ），假设每平方米价格固定，则总价与面积近似呈线性关系。
 - 学习成绩分析：研究学习时长（ x ）与考试成绩（ y ）的关系，在合理范围内，时间越长成绩可能线性提高。
 - 广告效果评估：分析广告投入（ x ）与产品销量（ y ）之间是否存在稳定的线性增长趋势。
- 非线性模型：当数据关系较为复杂，简单的直线无法描述其变化规律时，需要引入非线性模型。例如多项式模型：

$$f(x; w) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M,$$

其中 $w_0, w_1, \dots, w_M$ 为待定系数。它能够逼近非常复杂的函数，是处理高维、非结构化数学对象的有力工具。比如：关于物体自由落体，物体下落距离（ y ）与时间（ x ）的关系为 $y \propto x^2$ ，需要用二次多项式描述。

3. 算法

算法负责根据数据 $\mathcal{D}$ ，在预先设定的数学模型 $f(x; \theta)$ 中自动寻找最优的参数 θ 。这引出了下一个核心问题：如何定义“最优”？

我们希望模型在所有可能遇到的数据上都表现良好。在概率框架下，假设数据由未知的真实分布 $p(x, y)$ 生成，最优性定义为最小化期望风险（Expected Risk）或期望损失：

$$\mathcal{R}(\theta) = \mathbb{E}_{(x, y) \sim p(x, y)} \mathcal{L}(y, f(x; \theta))$$

其中 $\mathcal{L}(y, \hat{y})$ 称为损失函数，衡量预测输出 $\hat{y} = f(x; \theta)$ 与真实输出 y 的差异。因此，理想目标为

$$\theta^* = \arg \min_{\theta} \mathcal{R}(\theta).$$

然而，真实分布 $p(x, y)$ 通常未知。我们仅有的信息是来自该分布的采样：训练集 $\mathcal{D}$ 。一个自然而实用的

近似是用经验风险（Empirical Risk）替代期望风险：

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}; \theta))$$

这导出了机器学习的核心准则——经验风险最小化（Empirical Risk Minimization, ERM）：

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

ERM 将学习问题转化为一个纯粹的数学优化问题。其背后的统计学信念是：通过优化模型在训练集上的表现，我们期望模型在未知数据上也有良好表现，即具备泛化能力。过拟合正是 ERM 原则在有限样本上面临的核心挑战：模型可能过于“迎合”训练数据中的特定样本（包括噪声），损害了泛化性能。

总之，数据提供了学习的素材，模型定义了表达的形式，算法则指明了优化的路径。这三个要素共同构成了机器学习的基本框架，但框架本身并不限定学习的具体方式——正如有了纸、笔和书写规则，我们还需要知道要写什么类型的文章。这就引出了一个更上层的区分：根据数据形态和学习目标的不同，机器学习可以分为三种基本范式，它们决定了三要素以何种方式组合在一起解决问题。

1. 监督学习 (Supervised Learning)

监督学习最显著的特点就是数据由标注的输入-输出对 (x, y) 组成，目标是学习从输入空间到输出空间的最佳映射 $f: \mathcal{X} \rightarrow \mathcal{Y}$ 。在监督学习中，我们收集的数据样本既包含真实输入，也包含标注的真实输出。这是监督学习的显著特点。根据输出空间的类型，主要分为：

- 回归问题： $\mathcal{Y} \subseteq \mathbb{R}$ ，即输出样本为连续变化的数值。例如，根据历史天气数据预测明天温度，或根据函数在有限点的取值预测其积分值。
- 分类问题： $\mathcal{Y} = \{1, 2, \dots, C\}$ ，即输出样本为离散值。例如，判断一封邮件是否为垃圾邮件，或对一个数学命题进行真伪分类。

2. 无监督学习 (Unsupervised Learning)

在无监督学习中，我们只有输入数据 $\{x^{(1)}, \dots, x^{(N)}\}$ ，没有标注的真实输出数据 y 。由于真实输出 y 没有提前标注，我们事先并不知晓 y 的取值。这是无监督学习与监督学习的显著区别。无监督学习的目标是发现数据中隐藏的结构、模式或分布，例如：

- 聚类：将数据划分成若干个簇，使得同一簇内的样本彼此相似，不同簇间的样本差异较大。这相当于在数据空间中发现自然的“分组”或“类别”，而无需预先定义这些类别是什么。
- 降维：将高维数据投影到低维空间，同时尽可能保留数据中的重要信息（如方差、流形结构）。这有助于可视化、去噪和减少后续计算复杂度。
- 密度估计：生成观测数据的潜在概率分布。

3. 强化学习 (Reinforcement Learning, RL)

强化学习处理的是一个智能体（Agent）在环境中学习，通过执行动作来改变环境状态，并从环境中获得奖励信号。目标是学习一个策略模型（从状态到动作的映射），以最大化长期累积奖励。

强化学习的训练数据来源于智能体与环境的互动，并不需要事先采集，这是强化学习与其他学习范式的显著区别。在强化学习中，奖励函数是关键要素之一，用来优化策略模型中的参数（在监督学习与无监督学习中，通常用损失函数来优化预测模型中的参数）。

2.2 线性模型

本节我们简要描述机器学习中的线性模型，一类重要的非线性模型-人工神经网络将在下一节阐述。

2.2.1 监督学习中的线性模型

1. 线性回归模型

线性回归是最基础的回归模型，其核心假设是输出变量 y 可以表示为输入特征向量 x 的线性组合：

$$y = w^T x + b.$$

该模型采用平方损失函数： $\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$ ，其经验风险定义为：

$$\mathcal{R}_{\text{emp}}(w, b) = \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - (w^T x^{(n)} + b) \right)^2$$

通过 Lagrangian 乘子法，可以求解最优参数 w, b ，最小化此经验风险函数。亦可以采用梯度下降法等迭代优化算法进行求解（关于梯度下降法，参考本节后续材料）。

2. 分类模型：感知器

感知器是一类重要的分类模型，从几何角度出发，通过迭代学习，自动找到一个能将两类数据正确分开的线性超平面。数学定义如下：

感知器是一个函数 $f: \mathbb{R}^d \rightarrow \{-1, +1\}$ ，它将一个 d 维实值输入向量 $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ 映射到一个二元输出（通常取 $+1$ 和 -1 ，代表两个类别）。

它的计算分为两个步骤：

(1). 加权求和（净输入）：计算输入特征的线性组合。

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

其中， $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ 称为权重向量， b 称为偏置。权重 w_i 衡量了对应特征 x_i 对决策的重要性，偏置 b 决定了决策平面相对于原点的偏移。

(2). 激活函数：对净输入 z 应用一个阶跃函数（或称为 Heaviside 函数）。

$$f(\mathbf{x}) = \text{sign}(z) = \begin{cases} +1, & \text{if } z \geq 0 \\ -1, & \text{if } z < 0 \end{cases}$$

从几何的角度来解释感知器，感知器本质上是在寻找样本空间中的一个决策超平面，感知器的决策规则 $\mathbf{w}^T \mathbf{x} + b = 0$ 在特征空间中定义了一个决策超平面，其中各个参数的含义如下：

- 权重向量 $\mathbf{w}$ ：垂直于该决策超平面的法向量，指明了分类的正方向。
- 偏置 b ：决定了超平面到原点的距离。当 $b = 0$ 时，超平面过原点。
- 决策：对于任意数据点 $\mathbf{x}$ ，若它落在法向量 $\mathbf{w}$ 所指的一侧（即 $\mathbf{w}^T \mathbf{x} + b > 0$ ），则被分类为 $+1$ ；否则分类为 -1 。

因此，感知器的学习目标就是找到这样一个超平面，能够将所有正类样本（ $y = +1$ ）和负类样本（ $y = -1$ ）正确地分割在两侧。

学习算法：权重更新规则

感知器采用错误驱动的学习算法。给定一个带有标签的训练数据集 $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots\}$ ，其算法流程如下：

1. 初始化：将权重 $\mathbf{w}$ 和偏置 b 初始化为零或小的随机数。
2. 迭代：对训练集中的每个样本 $(\mathbf{x}, y)$ （或按随机顺序）：
 - (a). 计算预测值： $\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$ 。
 - (b). 更新权重（当且仅当预测错误时）：

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (y - \hat{y}) \cdot \mathbf{x}$$

$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

其中， $\eta > 0$ 是一个关键的超参数，称为学习率，用于控制每次更新的步长。

在理解了感知器的基本形式后，我们来深入剖析其更新规则的几何直观——为什么这样一个看似简单的规则能够逐步修正分类错误？

当某个样本被误分类时，预测值 $\hat{y}$ 与真实标签 y 必然不一致，此时 $(y - \hat{y})$ 的值为 $+2$ 或 -2 （考虑到 $\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$ 取值为 ± 1 ）。为了更清晰地理解参数调整的几何意义，我们不妨从两种误分类情形分别考察：

情形一：真实标签 $y = +1$ ，但模型预测为 $\hat{y} = -1$

这意味着当前模型给出的线性输出 $w^T x + b < 0$ ，即样本点位于决策平面的错误一侧。

更新规则为： $w \leftarrow w + \eta \cdot x$ ， $b \leftarrow b + \eta \cdot (1)$

几何解释：这一更新相当于将整个决策平面向着远离当前样本点的方向“推动”。为了理解这一点，我们需要观察更新后样本点相对于新平面的位置变化。

更新前，我们有 $w^T x + b < 0$ 。更新后，新的线性输出为：

$$(w + \eta x)^T x + (b + \eta) = w^T x + \eta \|x\|^2 + b + \eta = (w^T x + b) + \eta(\|x\|^2 + 1)$$

由于 $\|x\|^2 + 1 > 0$ ，且 $\eta > 0$ ，新的输出值在原有负值的基础上增加了一个正项。这意味着样本点相对于新平面的“得分”提高了——虽然它可能仍然为负，但已经向正方向移动了一步。如果这一步的调整幅度足够大，样本点就可能越过决策平面，进入正半空间。

从平面的角度看：权重向量 w 的方向决定了平面的法向，偏置 b 决定了平面沿法向的位置。当我们增加 w 在 x 方向上的分量时，相当于将平面绕某个轴旋转，使其法向更倾向于 x 的方向；同时增加 b 则相当于将平面沿着法向平移。二者的协同效果是：让平面朝着远离 x 的方向移动，从而试图将 x 纳入正确的一侧。

情形二：真实标签 $y = -1$ ，但模型预测为 $\hat{y} = +1$

此时 $w^T x + b > 0$ ，样本点同样位于决策平面的错误一侧，但方向相反。

更新规则为： $w \leftarrow w - \eta \cdot x$ ， $b \leftarrow b + \eta \cdot (-1) = b - \eta$

几何解释：这一更新同样是将决策平面推离当前样本点，但方向与情形一相反。

更新前， $w^T x + b > 0$ 。更新后，新的线性输出为：

$$(w - \eta x)^T x + (b - \eta) = w^T x - \eta \|x\|^2 + b - \eta = (w^T x + b) - \eta(\|x\|^2 + 1)$$

新的输出值在原有正值的基础上减去一个正项，使样本点的“得分”降低。这意味着样本点向负方向移动，逐渐靠近甚至越过决策平面进入负半空间。

从平面角度看：减去 x 方向的分量相当于让平面的法向 w 偏离 x 的方向，同时减小偏置 b 则相当于将平面沿法向负方向平移。这两种调整共同作用，使平面朝着远离当前样本点的方向移动，试图将其纳入负半空间。

理解上述两种更新后，我们可以描绘出感知器学习过程中决策平面的动态变化：

每一次遇到误分类样本，算法便根据该样本的位置和误判方向，对决策平面施加一次局部调整。这种调整总是让当前误分类样本向着正确的一侧移动——具体表现为该样本相对于新平面的线性输出值向零靠近，直至越过边界。然而，这种调整是以牺牲其他样本的分类状态为代价的：原本分类正确的某些样本可能因此被推回错误一侧。这正是感知器算法的特点——它不追求一步到位的全局最优，而是通过不断试错、反复修正，最终在数据线性可分的前提下达到一个平衡状态：所有样本都位于决策平面的正确一侧。

从几何直观上看，整个学习过程就像一块倾斜的平板（决策平面）在数据点之间不断翻转、平移，每次遇到被压错的点就稍微抬起一端，直到所有点都落在平板正确的一侧为止或达到迭代上限。

这一反复调整的过程最终能否停止？感知器收敛定理 [Novikoff, 1963] 给出了肯定回答：如果训练数据线性可分，即存在某个超平面能够完美分开两类样本，那么无论初始平面如何，上述更新规则都保证在有限步迭代后收敛到一个能够正确分类所有训练样本的解。

然而，收敛定理只保证了“存在一个解”，并未承诺这个解是“最优的”。事实上，感知器最终找到的决策平面严重依赖于样本出现的顺序和初始参数。它可能恰好落在两类样本的边缘地带——距离某些样本点极近，甚至紧贴着某一类别的边界。这样的解虽然在训练集上表现完美，但在面对未见过的测试数据时，往往因缺乏鲁棒性而表现不佳。这种局限性后来催生了支持向量机（SVM）的诞生：SVM 不仅寻找能够分类的超平面，更追求其中“间隔最大”的那个，从而在数学上保证了更强的泛化能力。感知器与 SVM 之间的这一差异，恰恰反映了机器学习从“寻找可行解”到“寻找最优解”的思想演进。

3. 分类模型：支持向量机 (SVM)

感知器为我们提供了一个寻找分类超平面的方法，但它只能保证找到“某个”能将两类样本分开的平面，而不关心这个平面是否“好”。当存在无数个可能的分类超平面时，我们自然会问：哪一个是最好的？

最大间隔的思想。支持向量机从几何视角出发，给出了一个明确的答案：最好的分类超平面应当是那个离

两类样本都尽可能远的平面。这个“离得远”的程度用间隔来度量——它定义为离超平面最近的样本点到该平面的距离。最大化间隔的直觉在于，这样的超平面具有最强的鲁棒性：当新样本在边界附近轻微波动时，它更不容易被误分类，从而可能拥有最优的泛化能力。值得注意的是，对于线性可分数据，能够最大化间隔的超平面是唯一的，如图2.1（引自邱锡鹏《神经网络与深度学习》图3.6）所示。

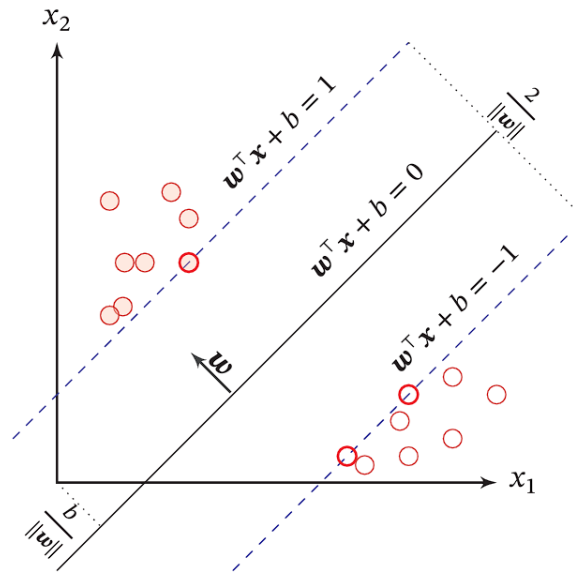


图 2.1: 支持向量机示例

(1). 线性可分情况：硬间隔 SVM

当数据线性可分时，SVM 的优化问题可以表述为：

$$\begin{aligned} \min_{w, b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y^{(n)}(w^T x^{(n)} + b) \geq 1, \quad \forall n = 1, \dots, N \end{aligned}$$

这里需要解释一下约束条件中的“1”从何而来。我们知道，对于任意一个能将两类样本正确分类的超平面 $w^T x + b = 0$ ，总可以通过等比例缩放 w 和 b 使其满足 $|w^T x + b| \geq 1$ 对所有样本成立，且等号至少在距离平面最近的点上取得。这个“1”本质上是对函数间隔的归一化，它消除了参数缩放带来的不确定性，使优化问题有了确定的形式。此时，间隔恰好等于 $1/\|w\|$ ，因此最小化 $\|w\|^2$ 等价于最大化间隔。

这是一个凸二次规划问题——目标函数是二次的，约束条件是线性的，因此存在唯一全局最优解。求解后得到的决策函数为 $f(x) = \text{sign}(w^T x + b)$ ，其中只有那些位于间隔边界上的样本（即满足 $y^{(n)}(w^T x^{(n)} + b) = 1$ 的点）对最终的超平面有贡献，它们被称为支持向量。

(2). 线性不可分情况：软间隔 SVM

现实世界的的数据往往并非完美线性可分，可能存在噪声、重叠甚至个别异常点。如果坚持使用上述“硬间隔”约束，问题将无解。为此，我们需要允许模型在某些样本上犯错——这正是软间隔 SVM 的出发点。

引入非负的松弛变量 $\xi_n \geq 0$ ，每个样本对应一个 ξ_n ，它衡量了该样本违反约束的程度。具体来说，我们将原来的严格约束放宽为：

$$y^{(n)}(w^T x^{(n)} + b) \geq 1 - \xi_n, \quad \forall n = 1, \dots, N$$

- 当 $\xi_n = 0$ 时，样本位于正确一侧且满足间隔要求；
- 当 $0 < \xi_n < 1$ 时，样本在正确一侧但位于间隔内部（即介于决策平面和间隔边界之间）；
- 当 $\xi_n \geq 1$ 时，样本被误分类（位于决策平面的错误一侧）。

松弛变量的引入让模型有了“容错空间”，但我们需要控制总的犯错程度。于是优化目标变为在最大化间隔和最小化总误差之间寻求平衡：

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n$$

这里 $\sum \xi_n$ 是对所有样本违反约束程度的总惩罚，而惩罚系数 $C > 0$ 则是一个需要人为设定的超参数，它调节着两个目标之间的权重：

- 较大的 C 意味着对误分类的容忍度低，模型会尽力将每个样本分类正确，即使这意味着间隔变小；
- 较小的 C 则允许更多的误分类，以换取更大的间隔，从而获得更强的泛化能力。

C 的选择至关重要——它本质上控制着模型对噪声的敏感程度，是平衡拟合能力与泛化能力的关键旋钮。

核技巧：从线性到非线性

最大间隔的思想固然优美，但它仍然局限于线性分类器。如何让 SVM 处理非线性可分的数据？直觉告诉我们，如果在原始特征空间中数据无法线性分开，或许可以通过某种变换将其映射到一个更高维的空间，使它们在那里变得线性可分。

这正是核技巧的核心思想。考虑一个非线性映射 $\phi: X \rightarrow \mathcal{F}$ ，将原始空间的数据点映射到高维特征空间 $\mathcal{F}$ 。在这个新空间中，我们可以执行标准的线性 SVM。然而，直接计算 $\phi(x)$ 并在高维空间中做内积往往计算量巨大，甚至可能面临维度灾难。

核技巧的关键洞察在于：无论是 SVM 的优化问题还是最终的决策函数，数据都是以内积的形式出现的——即 $x_i^T x_j$ 的形式。如果我们能够找到一个函数 $K(x_i, x_j)$ ，它直接等于高维空间中的内积 $\langle \phi(x_i), \phi(x_j) \rangle$ ，那么我们就无需显式计算 $\phi(x)$ ，也无需知道映射的具体形式，只需要计算这个核函数即可。这不仅避免了维度过高带来的计算负担，甚至允许我们隐式地工作在无穷维空间中。

常用的核函数包括：

- 线性核： $K(x, z) = x^T z$ ，退化为普通线性 SVM；
- 多项式核： $K(x, z) = (x^T z + c)^d$ ，引入多项式组合特征；
- 高斯径向基核（RBF）： $K(x, z) = \exp(-\gamma \|x - z\|^2)$ ，能够逼近任意非线性函数，是最常用的核函数之一，其中参数 γ 控制单个样本的影响范围。

引入核技巧后，SVM 的决策函数变为：

$$f(x) = \text{sign} \left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \right)$$

其中 α_i 是对偶问题中求解得到的拉格朗日乘子，只有那些 $\alpha_i > 0$ 的样本（即支持向量）才会对决策结果产生影响。这体现了 SVM 的一个优美特性：解具有稀疏性，最终模型仅由少数关键样本决定。

从感知器到硬间隔 SVM，再到软间隔 SVM 和核技巧，我们见证了机器学习思想的一次重要跃迁：从“寻找一个可行解”到“寻找最优解”，从线性到非线性，从参数化模型到基于样本的核方法。支持向量机以其优美的数学形式和坚实的理论基础，成为机器学习发展史上的里程碑。

2.2.2 无监督学习中的线性模型：主成分分析（PCA）

与监督学习不同，无监督学习面对的数据没有标签——我们只有输入 x ，却没有对应的输出 y 来告诉模型应该学什么。那么，在没有“标准答案”的情况下，模型还能学到什么呢？答案是：数据本身蕴含的结构、模式或分布。无监督学习的目标，就是让模型自主发现这些隐藏的信息。

无监督学习涵盖了许多不同的任务，其中最具代表性的两类是降维和聚类。降维试图在保留关键信息的前提下，将高维数据压缩到低维空间；聚类则试图将相似的样本自动归为一类。这里我们重点介绍降维中最经典的方法之一：主成分分析。

在现实问题中，我们经常面对高维数据——一张图片可能有成千上万个像素，一份文本可能有上万种词汇。高维数据不仅难以可视化，还会带来巨大的计算负担，更重要的是，其中往往包含大量冗余信息。降维的目标就

是找到一种方式，将数据从高维空间映射到低维空间，同时尽可能保留数据中最重要的信息。

打个比方：假设你需要向别人描述一个人的外貌。你可以逐像素地描述一张照片，但这显然效率极低。更好的方式是抓住关键特征——脸型、眼睛颜色、发型等几个维度——就能大致还原一个人的样貌。降维做的正是类似的事情：找到那些最能刻画数据差异的方向，舍弃那些变化不大或噪声主导的方向。

降维的价值体现在多个方面：

- 可视化：将数据降至二维或三维，可以直接观察样本的分布和聚类情况；
- 去噪：舍弃变化小的方向往往同时去除了噪声；
- 计算效率：低维数据大幅降低后续算法的计算成本；
- 特征提取：降维后的新特征有时比原始特征更有代表性。

主成分分析是最常用、最经典的线性降维方法。它的核心思想非常直观：如果我们只能选择一个方向来投影数据，应该选择哪个方向，才能最大程度地保留原始数据的差异？

想象一群散落在二维平面上的点。如果把它们投影到一条直线上，有些投影方向会让所有点挤在一起（方差小），有些方向则会让点尽量散开（方差大）。显然，后者保留了更多原始数据的分布信息——因为点之间的相对远近关系被更好地保留下来。主成分分析要找的，正是这样一些方向：数据在这些方向上投影的方差尽可能大，并且这些方向彼此正交。如图2.2（引自邱锡鹏《神经网络与深度学习》图9.1）所示：

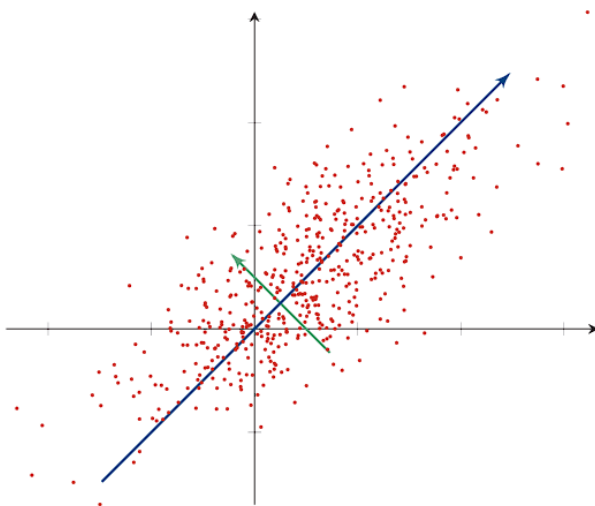


图 2.2: 主成分分析示意图

这些方向被称为主成分。第一个主成分是方差最大的方向；第二个主成分是与第一个正交的前提下方差次大的方向；以此类推。

那么，如何从数学上找到这些方向呢？这需要借助协方差矩阵。给定 N 个 d 维样本 $x^{(1)}, x^{(2)}, \dots, x^{(N)}$ ，我们首先计算样本的均值 $\bar{x} = \frac{1}{N} \sum_{i=1}^N x^{(i)}$ ，然后将每个样本中心化（减去均值）。中心化后的数据构成了一个矩阵，它的协方差矩阵定义为：

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \bar{x})(x^{(i)} - \bar{x})^T$$

这个矩阵的维度是 $d \times d$ ，它刻画了原始特征之间的协方差关系——对角线元素是各特征的方差，非对角线元素是特征间的协方差。

主成分分析的关键在于：数据在某个方向上的投影方差，恰好等于该方向与协方差矩阵的某种关系。数学上可以证明，协方差矩阵 Σ 的特征向量指明了方差变化的方向，而对应的特征值则度量了在该方向上的方差大小。特征值越大，意味着这个方向保留的信息越多。

因此，PCA 的求解步骤非常清晰：

1. 计算数据的协方差矩阵 Σ ；

2. 求解 Σ 的所有特征值和特征向量；
3. 将特征值按从大到小排序，选取前 k 个最大的特征值对应的特征向量；
4. 将这些特征向量组成投影矩阵 $W \in \mathbb{R}^{d \times k}$ ；
5. 对任意样本 x ，降维后的表示为 $z = W^T(x - \bar{x})$ 。

这样，我们就将原始 d 维数据压缩到了 k 维 ($k \ll d$)，同时尽可能保留了数据中的主要变化信息。

主成分分析还有一个等价的解释：它试图找到一种低维表示，使得从低维表示重构回原始空间时，产生的误差尽可能小。换句话说，如果我们把降维看作一种压缩，那么好的压缩应该能够尽量还原原始数据。PCA 正是在这个意义上最优的线性降维方法——在所有线性降维方法中，它在最小化重构误差（即原始数据与重构数据之间的欧氏距离平方和）的意义上达到最优。

这一视角与“最大化方差”是等价的：保留最大方差的方向，也就是重构误差最小的方向。两者从不同角度诠释了同一个本质——主成分分析试图捕捉数据中最本质的变化模式。

注记：无监督学习的另一个典型应用是聚类分析，即将数据划分成若干个簇，使得同一簇内的样本彼此相似，不同簇间的样本差异较大。这相当于在数据空间中发现自然的“分组”或“类别”，而无需预先定义这些类别是什么。我们在此简单介绍一类特殊的聚类方法（尽管该方法不是线性模型）：

K-means 聚类：目标是将 N 个样本划分至 K 个簇，最小化样本到其所属簇中心的距离平方和：

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x^{(n)} - \mu_k\|^2$$

其中 $r_{nk} \in \{0, 1\}$ 表示样本 n 是否属于簇 k ， μ_k 为簇 k 的中心。算法通过交替执行以下步骤求解：

1. 分配步骤：固定 $\{\mu_k\}$ ，将每个样本分配至最近的簇中心；
2. 更新步骤：固定 $\{r_{nk}\}$ ，重新计算每个簇的中心为该簇样本的均值。

K-means 简单高效，但对初始中心敏感，需预设簇数 K ，且对非球状簇或不同密度的簇效果有限。

2.2.3 优化算法：梯度下降法

在前面的讨论中，我们建立了模型，也明确了学习的目标——通常是最小化某个损失函数。然而，对于绝大多数有实用价值的模型，这个最小化问题都无法直接通过解析求解（像解一元二次方程那样得到一个明确的公式解）。我们面临的现实是：需要找到一种方法，让模型逐步迭代地逼近最优解。这正是优化算法的用武之地。梯度下降法及其众多变体，构成了机器学习优化的核心。

想象你站在一座山上的某个位置，浓雾弥漫，看不清整个山势，但你能感觉到脚下地面的坡度。想要最快地走到山谷，你会怎么做？很自然地，你会朝着最陡的下坡方向迈出一步，然后在新位置重新感知坡度，再迈一步——如此反复，直到感觉不到明显的下坡为止。

这就是梯度下降法的直观写照。在数学上，梯度 $\nabla_{\theta} \mathcal{R}(\theta)$ 指向函数值上升最快的方向，其反方向自然就是下降最快的方向。梯度的每个分量，就是损失函数对相应参数的偏导数——它告诉我们，微调这个参数会让损失增加还是减少，变化幅度有多大。

基于这一思想，梯度下降的更新规则可以写作：

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta_t)$$

这里， θ_t 表示第 t 步迭代时的参数， $\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta_t)$ 是当前参数下经验风险函数（即训练集上的平均损失）的梯度。而学习率 $\alpha > 0$ 则控制着我们每一步迈多大——步子太小，收敛太慢；步子太大，可能一步跨过山谷甚至跌出山外。

梯度下降的具体实现方式，取决于我们每次用多少数据来计算梯度。根据数据使用量的不同，可以区分出三种基本变体。

- 批量梯度下降使用全部训练数据计算梯度。这样做的好处是梯度方向准确，每一步都朝着全局最优的方向迈进；但代价也显而易见——当数据量巨大时，每次迭代都需要遍历整个数据集，计算开销大得难以承受。

- 随机梯度下降 [Nemirovskietal.,2009] 则走向另一个极端：每次只随机选取一个样本，用它的损失梯度作为全局梯度的估计。这一下，计算变得极其轻量，更新可以非常频繁。更重要的是，单样本梯度引入的随机性，反而有助于算法跳出局部极小值，有机会找到更好的解。但代价是更新方向震荡剧烈，收敛过程不够平稳。
- 小批量梯度下降 [Bottou,2010] 是前两者的折中：每次使用一小批数据（例如 32、64 或 128 个样本）计算梯度。它既通过批量平均降低了方差，使更新方向相对稳定，又保持了较高的计算效率。

进阶优化方法

基础的梯度下降虽然有效，但在复杂问题上会遇到两个棘手挑战：一是容易在陡峭山谷间来回振荡，收敛缓慢；二是对所有参数使用统一的学习率，无法适应不同参数的更新需求。为此，研究者提出了一系列更“聪明”的优化方法。

动量法（Momentum）：动量法的灵感来自物理学：想象一个小球从山坡滚下，它会积累动量，遇到反向坡度时不会立刻停止，而是凭借惯性冲过去。动量法引入速度变量 v 来模拟这一效应：

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

当前梯度负责“加速”，而历史梯度累积成的速度则保持着运动方向。超参数 β （通常取 0.9 左右）控制着对历史信息的依赖程度。动量法的效果是：在梯度方向一致的维度上加速收敛，在梯度方向震荡的维度上抑制振荡——就像给下山的小球增加了惯性，让它跑得更稳更快。

Adam（Adaptive Moment Estimation）则结合了动量与自适应学习率。它维护两个状态变量：

- 一阶矩估计 m_t ：梯度的指数加权移动平均，相当于带有动量的梯度；
- 二阶矩估计 v_t ：梯度平方的指数加权移动平均，反映了梯度的方差（即每个参数的历史更新幅度）。

具体更新过程如下。首先计算当前梯度 $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$ ，然后更新两个矩估计：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

这里 β_1 和 β_2 是衰减率。由于 m_t 和 v_t 在初始阶段会偏向于 0，需要进行偏差校正：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

最后，更新参数：

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中 ϵ 是一个很小的常数（如 10^{-8} ），防止除零错误。

Adam 的精妙之处在于：每个参数都有自己的自适应学习率。对于历史上梯度较大（更新频繁）的参数， $\sqrt{\hat{v}_t}$ 较大，有效学习率自动降低；对于梯度较小（更新缓慢）的参数，有效学习率自动升高。这种自适应机制使得 Adam 对初始学习率的选择不那么敏感，在各种问题上都能表现良好，因而成为训练神经网络的首选优化器之一（参看下一节人工神经网络）。

2.2.4 模型评估与选择

在机器学习实践中，我们面临的根本问题不是“模型能否记住训练数据”，而是“模型能否应对从未见过的新样本”——这就是泛化能力。如何准确评估模型的泛化能力，并在此基础上选择最优的模型配置，构成了模型评估与选择这一核心议题。

数据的三重划分

要评估泛化能力，首先需要明确：不能用训练过模型的数据来测试它——这就像用做过的试卷考试，无法反映真实水平。因此，我们需要将手头的数据进行合理划分。

- 训练集是模型学习的素材。模型通过它来更新参数、拟合数据分布，相当于学生的课本和练习题。
- 验证集则扮演着“模拟考”的角色。它不参与参数学习，但用来指导模型选择和超参数调优——当我们需要决定用几阶多项式、学习率设为多少时，验证集上的表现就是决策依据。之所以需要验证集，是因为如果直接用测试集调参，相当于提前知道了考题答案，最终的评估就失去了意义。
- 测试集是最终的“高考”。它在整个训练和调参过程中必须完全不可见，仅在模型完全确定后使用一次，用来给出对泛化能力的无偏估计。测试集的表现，才是我们对外宣称的模型性能。

这三重划分构成了评估的基本框架，确保了从模型开发到最终评估的各个环节都有清晰的数据边界。

上述单次划分虽然简单，但存在一个隐患：结果可能依赖于某次特定的划分方式。如果验证集恰好包含了一些容易或困难的样本，评估结果就会产生偏差。为了缓解这一问题， k 折交叉验证应运而生。

它的做法如下：将训练数据随机均分为 k 份（通常取 5 或 10）。然后进行 k 次训练-验证循环——第 1 次用第 1 份作验证集，其余作训练集；第 2 次用第 2 份作验证集，以此类推。每次循环都记录模型在验证集上的性能指标，最终取 k 次结果的平均值作为性能估计。

这种方法的优势在于：每个样本都有机会进入验证集，性能估计更加稳定可靠；同时，所有数据都被用于训练和验证，避免了数据浪费。当数据量有限时，交叉验证尤其有价值。

需要强调的是，交叉验证仍然是在训练数据内部进行的——它用于模型选择和调参，最终的测试集依然要保留不动，直到最后一次评估。

参数与超参数

在机器学习的语境中，“参数”和“超参数”是两个容易混淆但本质不同的概念。理解它们的区别，有助于把握模型学习的层次结构。

参数是模型的内在变量，它们构成模型本身的核心。线性模型中的权重系数 w 和偏置 b 、多项式模型中的各项系数，都属于参数。这些变量的特点在于：它们的值完全由训练数据驱动，通过优化算法（如梯度下降）自动学习得到。换句话说，参数是模型从数据中“总结”出来的知识。

超参数则完全不同。它们是训练开始前人为设定的配置选项，控制着训练过程本身和模型的结构。学习率 α 、梯度下降中的批量大小、多项式模型的阶数 M 、正则化系数——这些都属超参数。它们的值无法通过数据直接学习，需要依靠人的经验或某种搜索策略来确定。

可以这样理解二者之间的关系：参数是模型的内核，超参数是塑造内核的外部指令。超参数决定了模型以何种方式、在何种约束下去学习参数。正因为超参数的选择直接影响学习效果，我们需要借助验证集或交叉验证来“调优”它们——尝试不同的超参数组合，观察验证集上的表现，最终选出最合适的那一套。

这种分层结构贯穿于整个机器学习实践：在超参数确定的框架下，模型通过数据学习参数；而超参数本身，则需要更高层次的验证机制来优化。二者相辅相成，共同构成了从数据到模型的完整链条。

2.3 人工神经网络

目前我们已经介绍了机器学习的基础范式，其核心在于从数据中学习一个最佳映射。线性模型因其简洁性和可解释性成为基石，但它们处理复杂非线性关系的能力有限。本章节将聚焦于一种功能强大且富有革命性的非线性模型家族——人工神经网络（Neural Networks）。

人工神经网络的研究灵感来源于对生物大脑中神经元网络运作方式的简化模拟。人工神经网络扮演着一个独特且关键的角色：它是一个极其灵活的非线性函数逼近器，也是一个强大的特征转换引擎。这意味着，当面临复杂的问题——例如，需要从高维、非结构化的数据中寻找隐藏模式（如识别图像中的几何结构），或者逼近一个没有显式公式但可被样本描述的未知函数（如求解偏微分方程）——人工神经网络提供了一种从数据中直接“学习”解决方案的途径。

人工神经网络，尤其是深度的神经网络，构成了当前大多数人工智能突破性进展的基础。从图像识别、自然

语言处理，到 AlphaGo 这样的博弈系统，再到用于数学发现的 FunSearch 和 AlphaTensor，人工神经网络都发挥了核心价值。本节将从数学视角出发，剥离工程细节，深入探讨神经网络的三大核心要素：激活函数、网络结构和学习算法，并阐释其背后的数学思想与理论保障。

在介绍人工神经网络之前，我们先介绍它的前身：逻辑回归。

尽管名称中含有“回归”，但逻辑回归实为经典的二分类模型。其核心思想是用模型的输出拟合样本属于正类的对数几率（log-odds）。模型首先计算线性得分 $z = \mathbf{w}^T \mathbf{x} + b$ ，然后通过 Sigmoid 函数 $\sigma(z) = 1/(1 + e^{-z})$ 将其映射到 $(0, 1)$ 区间，解释为正类的概率：

$$p(y = 1|x; \mathbf{w}, b) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

分类决策规则为：若 $p(y = 1|x) > 0.5$ 则预测为正类，否则为负类。

逻辑回归采用经典的交叉熵损失函数。对于单个样本，真实标签 $y \in \{0, 1\}$ ，预测概率 $\hat{y} = p(y = 1|x)$ ，则损失为：

$$\mathcal{L}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

该函数衡量了预测概率分布与真实标签分布之间的差异，最小化交叉熵等价于极大似然估计。其梯度形式简洁： $\partial \mathcal{L} / \partial \mathbf{w} = (\hat{y} - y)\mathbf{x}$ 。

在二分类问题的基础上，逻辑回归可以进一步推广到多分类任务，这便引出了 Softmax 回归（也称为多类逻辑回归）。当样本类别数 $C > 2$ 时，我们需要为每个类别计算得分 $z_c = \mathbf{w}_c^T \mathbf{x} + b_c$ ，并通过 Softmax 函数将这些得分转换为一个概率分布：

$$p(y = c|x; \mathbf{W}, b) = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)} = \text{softmax}(z)_c$$

这样，Softmax 回归将多个类别的预测统一到一个规范化的概率框架下。相应地，其损失函数也采用交叉熵形式，对于真实类别 c ，损失为：

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^C \mathbb{I}(y = c) \log p(y = c|x) = - \log p(y = c|x)$$

其中 $\mathbb{I}(\cdot)$ 为指示函数。Softmax 回归简洁而有效，因此在深度学习中常作为多分类任务输出层的标准配置。

人工神经网络是逻辑回归函数的推广，包含如下三大核心要素：激活函数、网络结构和学习算法。

从生物神经元到人工神经元

神经网络的基本计算单元是人工神经元，其设计灵感来源于生物神经元。简化的生物神经元接收来自其他神经元的电信号，当信号强度总和超过某个阈值时，该神经元会被激活，并向下游传递信号。人工神经元则是对这一过程的数学抽象与建模。

一个典型的人工神经元（早期也称为感知机，Perceptron）的数学描述如下：

给定一个输入向量 $\mathbf{x} = [x_1, x_2, \dots, x_d]^T \in \mathbb{R}^d$ ，神经元首先计算一个加权和，并加上一个偏置项（bias）。这个中间结果被称为神经元的净输入（net input）或预激活值（pre-activation）：

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

其中， $\mathbf{w} = [w_1, w_2, \dots, w_d]^T \in \mathbb{R}^d$ 是权重向量，每个 w_i 衡量了对应输入 x_i 的重要性或连接强度； $b \in \mathbb{R}$ 是偏置，用于控制神经元被激活的难易程度。图2.3是一个典型的神经元结构示例（引自邱锡鹏《神经网络与深度学习》图 4.1）：

然而，如果神经元仅仅输出线性组合 z ，那么由多个这样的线性单元组成的网络，本质上仍是线性模型的组合，无法突破线性变换的局限。为了使网络具备逼近任意复杂函数的能力，必须引入非线性。因此，神经元会将净输入 z 传递给一个激活函数 $f(\cdot)$ ，得到最终的输出或活性值（activation）：

$$a = f(z) = f(\mathbf{w}^T \mathbf{x} + b)$$

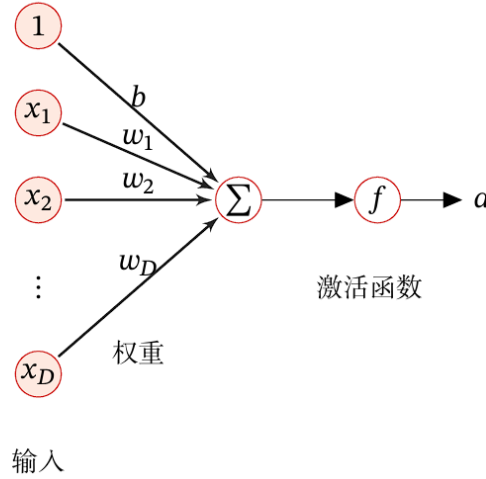


图 2.3: 神经元结构

正是激活函数 f 的非线性特性，赋予了神经网络强大的表达能力，使其能够学习数据中复杂的模式和层次化特征。

核心非线性组件：激活函数

激活函数的选择至关重要，它直接决定了神经元如何响应输入信号，并深刻影响整个网络的训练效果与性能。理想的激活函数通常需要具备非线性、连续可导（或几乎处处可导）、计算简单且导数易于计算等性质。以下是几种经典且广泛使用的激活函数。

1. Sigmoid 函数（Logistic 函数）

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid 函数将任意实数 z “挤压”到 $(0, 1)$ 区间内，其输出可以解释为一种概率。在早期神经网络中非常流行。其导数为 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 。然而，当输入 z 的绝对值很大时，其导数趋近于 0，这会导致在深层网络训练中引发“梯度消失”问题。

2. 双曲正切函数（Tanh 函数）

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Tanh 函数将输入映射到 $(-1, 1)$ 区间，且以 0 为中心（均值为 0），这在实践中有时能使网络训练得更快。其导数为 $\tanh'(z) = 1 - \tanh^2(z)$ 。

3. 修正线性单元（ReLU）[Nair et al., 2010]

$$\text{ReLU}(z) = \max(0, z)$$

ReLU 是当前最流行的激活函数之一。其计算极其简单：当 $z > 0$ 时，输出为 z ；否则输出为 0。其导数在 $z > 0$ 时为 1，在 $z < 0$ 时为 0。ReLU 的非饱和性（正区间导数为常数 1）有效缓解了梯度消失问题，加快了训练速度。其缺点是，当输入为负时，梯度为 0，可能导致某些神经元永远无法被激活（“神经元死亡”问题）。

4. Leaky ReLU 与 ELU

为了改善 ReLU 的“死亡”问题，提出了变体。Leaky ReLU 在负区间给予一个小的斜率（如 0.01）：

$$f(z) = \max(0.01z, z)$$

指数线性单元（ELU）在负区间使用一个指数衰减函数：

$$f(z) = \begin{cases} z, & z \geq 0 \\ \gamma(\exp(z) - 1), & z < 0 \end{cases},$$

其中 γ 是一个超参数。

这些非线性激活函数是神经网络构建复杂、分层特征表示的基础。一个单一的神经元就像一个“微型分类器”或“特征检测器”，而通过组合无数这样的神经元，网络能够学习和表达极其复杂的特征。

网络结构：从全连接到局部连接

单个神经元能力有限，将大量神经元按照特定的拓扑结构连接起来，就形成了人工神经网络。网络结构定义了神经元之间信息流动的路径和方式。

全连接前馈神经网络

全连接前馈神经网络是最基本也是最核心的网络结构之一，通常所说的“多层感知机”（MLP）或“深度神经网络”（DNN）即指此类网络。

结构特点：网络由多层神经元组成，通常包括一个输入层、一个或多个隐藏层（Hidden Layers）和一个输出层。同一层内的神经元之间没有连接。每一层的每个神经元都与前一层所有神经元相连接。信息从输入层开始，逐层单向传播，无反馈，因此称为“前馈”。这种结构可以用一个有向无环图表示。

数学描述：考虑一个有 L 层的网络（输入层不计入层数）。

M_l ：第 l 层神经元的个数（ $l=0$ 为输入层）。 $f_l(\cdot)$ ：第 l 层神经元的激活函数。 $\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$ ：连接第 $l-1$ 层到第 l 层的权重矩阵。其元素 $W_{ij}^{(l)}$ 表示第 $l-1$ 层第 j 个神经元到第 l 层第 i 个神经元的连接权重。 $\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$ ：第 l 层的偏置向量。 $\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$ ：第 l 层神经元的净输入向量。 $\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$ ：第 l 层神经元的输出（活性值）向量，且有 $\mathbf{a}^{(0)} = \mathbf{x}$ （输入）。

网络的前向传播过程可以递归地描述为：

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

最终，网络的输出为 $\mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$ ，其中 ϕ 表示整个网络的复合函数。

全连接前馈神经网络结构如图2.4（引自邱锡鹏《神经网络与深度学习》图4.7）所示。

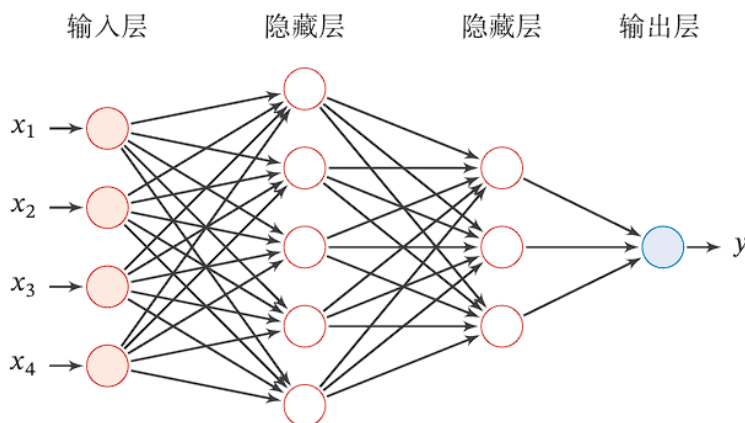


图 2.4: 前馈神经网络

全连接神经网络的强大能力有坚实的数学理论支撑。万有逼近定理指出：对于一个具有至少一个包含足够多神经元的隐藏层，并使用非线性激活函数（如 Sigmoid、ReLU 等）的全连接网络，它可以以任意精度逼近定义在紧致集上的任何连续函数。

定理条件与直观理解：该定理的经典形式通常要求激活函数 $\varphi(\cdot)$ 是非常数、有界、单调递增的连续函数（Sigmoid 函数满足此条件，ReLU 虽无界但后续扩展理论也支持）。同时，逼近是在一个“紧致集”（例如，有界闭区间）上成立的。这意味着，无论你想要逼近的函数多么复杂、多么不规则，只要网络足够宽（隐藏层神经元足够多），总可以找到一个网络，其输入输出关系与你的目标函数的误差在允许的任意范围之内。一个简单的直

觉是：每个隐藏层神经元可以看作一个“基函数”（如 Sigmoid 函数构成的一个“S 形”波峰或波谷），通过调整足够多这样的基函数的权重和偏置，我们可以用它们的线性组合来“雕刻”出任意复杂的连续函数形状。这一定理奠定了神经网络作为“万能函数逼近器”的理论基础，解释了为什么它可以处理从图像分类到数学方程求解等千变万化的问题。当然，定理保证了“存在性”，但并没有告诉我们如何找到（即训练出）这个网络，这需要依赖下一节介绍的学习算法。

卷积神经网络（CNN）

全连接网络在处理如图像、音频等具有局部相关性和平移不变性的数据时存在显著缺陷：参数过多导致计算和存储开销巨大，且难以有效捕捉局部特征。

卷积神经网络通过引入卷积层，巧妙地解决了这些问题。其核心思想来源于生物视觉系统的“感受野”机制。卷积神经网络的核心是卷积操作：

一维卷积：对于输入序列 x 和滤波器（或卷积核） w ，卷积输出 $y_t = \sum_{k=1}^K w_k x_{t-k+1}$ 。滤波器在输入上滑动，在每个位置计算点积，从而提取局部特征。

二维卷积：对于图像等二维输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$ 和一个小的滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$ （例如 $U=3, V=3$ ），卷积运算 $\mathbf{Y} = \mathbf{W} * \mathbf{X}$ 定义如下：

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} \cdot x_{i-u+1, j-v+1}$$

这里有一个直观的例子：假设输入图像 $\mathbf{X}$ 是一个 6×6 的矩阵，滤波器 $\mathbf{W}$ 是一个 3×3 的矩阵。滤波器从输入图像的左上角开始，覆盖一个 3×3 的区域，对应位置元素相乘后求和，得到输出特征图 $\mathbf{Y}$ 的第一个元素 y_{11} 。然后滤波器向右滑动一步（步长为 1），计算 y_{12} ，以此类推，直至扫描完整张图像，生成一个新的特征图。

全连接层和卷积层对比如图 2.5（引自邱锡鹏《神经网络与深度学习》图 5.5）所示。

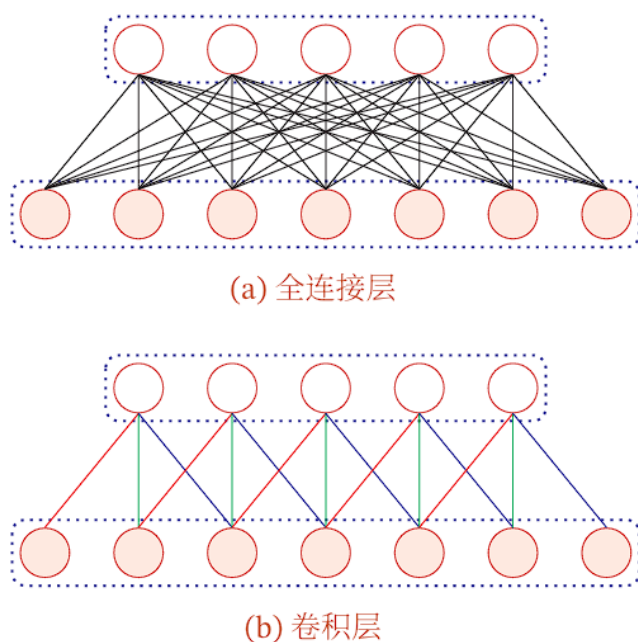


图 2.5: 卷积层

卷积结构特性带来的优势有：

1. 局部连接：每个神经元只与输入的一个局部区域（感受野）连接，而不是全部。这大幅减少了参数量。
2. 权重共享：同一个滤波器在输入的不同位置共享同一组权重 w_{uv} 。这意味着无论目标出现在图像的左上角还是右下角，都由同一个“特征检测器”来识别，这赋予了模型平移不变性。同时，这进一步压缩了参数量。

3. 层次化特征提取：典型的 CNN 由多个卷积层、汇聚层（Pooling Layer，如最大池化，用于下采样，增强平移鲁棒性）和全连接层（用于最终分类或回归）交叉堆叠而成。浅层卷积学习到的是边缘、角点等低级特征；深层卷积则将低级特征组合成更复杂的高级特征，如物体的部件或整体。这种结构使得 CNN 在图像、视频分析等领域取得了巨大成功。
4. 趋向全卷积：现代先进 CNN（如 ResNet、DenseNet）的架构趋向于“小卷积核、大深度”，并减少甚至消除全连接层，使用全局平均池化等操作，使网络对输入尺寸更加灵活。

CNN 本质上是一种具有强结构先验（局部性和平移不变性）的函数逼近器。它在高维输入空间（如图像像素空间）上施加了强大的约束，使得网络学习的函数具有特定的对称性。这种“对称性”可以理解为：如果我们平移输入图像，网络输出的特征表示也会发生相应的平移，而不会改变其内在的语义信息。

循环神经网络（RNN）：处理序列数据的记忆模型

全连接网络和卷积网络主要处理独立同分布的样本。但对于时间序列、自然语言、语音等序列数据，样本之间具有顺序依赖关系。循环神经网络通过引入“记忆”机制来处理这种动态时序信息。

其核心思想为：RNN 具有“循环”连接，使得网络能够将过去时刻的信息传递到当前时刻。网络在每一个时间步 t 接收当前输入 $\mathbf{x}_t$ 和上一个时间步的隐藏状态 $\mathbf{h}_{t-1}$ ，计算得到当前隐藏状态 $\mathbf{h}_t$ 和输出 $\mathbf{y}_t$ 。

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c})$$

其中 f, g 是激活函数。隐藏状态 $\mathbf{h}_t$ 被看作是网络在时间 t 的“记忆”，它浓缩了截止到当前时刻的所有历史输入信息。循环神经网络的结构如图 2.6（引自邱锡鹏《神经网络与深度学习》图 6.2）所示。

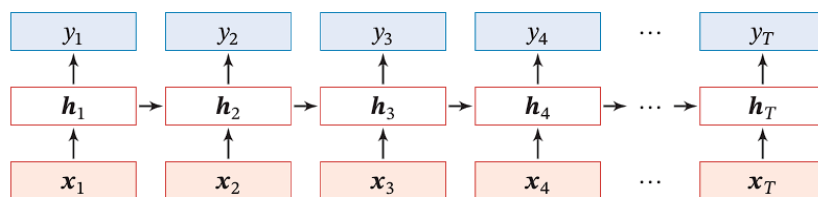


图 2.6: 循环神经网络

类似于前馈网络，RNN 也有其通用近似定理：一个具有足够多 sigmoid 型神经元的全连接循环网络，可以以任意精度逼近任何非线性动力系统。这从理论上保证了 RNN 有能力模拟复杂的时序过程。

基础 RNN 在训练长序列时，由于误差通过时间反向传播涉及多次矩阵连乘，极易出现梯度消失或爆炸问题，导致其难以学习长期依赖关系。LSTM 通过引入精妙的“门控机制”来解决此问题。LSTM 单元的核心是细胞状态 $\mathbf{C}_t$ ，它像一条传送带，在整个链上保持信息流动。LSTM 通过三个“门”来调节信息：

1. 遗忘门 $\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$ ：决定从细胞状态中丢弃哪些信息。
2. 输入门 $\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$ ：决定哪些新信息将被存入细胞状态。
3. 输出门 $\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$ ：基于当前细胞状态，决定输出什么信息到隐藏状态。

细胞状态的更新公式为： $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$ ，其中 $\tilde{\mathbf{C}}_t$ 是候选新信息。

LSTM 的关键在于加法更新：细胞状态通过遗忘门的逐元素乘法和输入门的逐元素加法来更新。这种加法连接使得梯度在反向传播时，可以在细胞状态路径上保持流动而不易消失（梯度可以通过加性路径直接传递）。

GRU 是 LSTM 的简化变体，将遗忘门和输入门合并为“更新门”，结构更简单，效果相当。这些门控机制赋予了 RNN 记住长期信息并选择性地遗忘或保留的能力。

RNN 及其变体（LSTM、GRU）在机器翻译、语音识别、文本生成等领域至关重要。在数学领域，它们可用于符号序列的生成（如生成数学表达式）、时间序列预测，或作为强化学习智能体的组成部分（如 AlphaGo 的策略网络包含了 CNN 和 RNN 来评估棋局）。

学习算法：如何找到那个“正确”的网络

万有逼近定理告诉我们，理论上存在一个神经网络可以解决我们的问题。但如何从无数可能的网络中，找到那个性能优异的网络？这正是学习算法的任务。神经网络的训练过程可以概括为三个核心步骤：前向传播计算预测结果，反向传播计算参数梯度，以及参数更新优化网络性能。

第一步：前向传播与损失函数——定义“好”与“坏”

训练开始前，我们首先需要有一个定量的标准来衡量网络输出与真实目标之间的差距，这就是损失函数 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 的作用。对于不同任务，我们选择不同的损失函数：回归问题常用均方误差，分类问题则常用交叉熵损失。损失函数：定义“好”与“坏”

以一个 C 类分类问题为例。在前向传播过程中，输入样本 $\mathbf{x}$ 依次通过网络各层，最终在最后一层通常使用 Softmax 函数将净输入转换为类别概率分布 $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$ 。随后，交叉熵损失计算预测分布与真实标签之间的差异：

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{c=1}^C y_c \log \hat{y}_c$$

其中 $\mathbf{y}$ 是真实的 one-hot 标签向量。

给定包含 N 个样本的训练集 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，我们通常最小化经验风险，即所有样本损失的平均值，并可能加上正则化项（如权重衰减 $\frac{1}{2}\lambda\|\mathbf{W}\|_F^2$ ）来防止过拟合：

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2}\lambda\|\mathbf{W}\|_F^2$$

至此，前向传播完成了它的使命：它不仅生成了预测结果 $\hat{\mathbf{y}}$ ，还计算出了衡量网络性能的损失值 $\mathcal{R}$ ，并为后续步骤缓存了必要的中间变量。

第二步：反向传播算法——计算梯度的高效引擎

有了损失函数 $\mathcal{R}$ ，我们的目标就是找到一组参数 $(\mathbf{W}, \mathbf{b})$ 使其最小化。然而， $\mathcal{R}$ 是关于所有网络参数的复杂非凸函数，直接计算每层参数的梯度极其困难。

反向传播算法正是解决这一挑战的关键。其本质是微积分中的链式法则在深度复合函数上的高效实现。算法从输出层开始，反向逐层传播误差信号：

1. 计算误差项：首先定义第 l 层的误差项 $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$ ，即损失函数关于该层净输入的梯度。
2. 误差反向递推：利用链式法则，可以得到误差项从后向前传递的递推公式：

$$\delta^{(l)} = f'_l(\mathbf{z}^{(l)}) \odot \left((\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \right)$$

其中 $\odot$ 表示逐元素相乘。这个公式的直观含义是：当前层的误差，取决于当前层激活函数的导数（局部梯度）与来自上一层的误差的加权组合。

3. 计算参数梯度：基于误差项 $\delta^{(l)}$ ，可以简洁地计算出该层参数的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

反向传播算法的精妙之处在于，它通过一次前向传播和一次反向传播，即可高效计算出所有参数的梯度，其计算复杂度与前向传播相当。这为后续的参数更新提供了关键的梯度信息。

第三步：参数更新——梯度下降驱动网络进化

获得梯度之后，最后一步就是利用这些梯度信息来更新网络参数，从而降低损失函数值。最常用的优化方法是梯度下降法及其各种变体。其核心思想朴素而深刻：参数沿着损失函数梯度的反方向（即下降最快的方向）进行更新。

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial \mathcal{R}}{\partial \mathbf{W}^{(l)}}, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}}{\partial \mathbf{b}^{(l)}}$$

其中 $\alpha > 0$ 是学习率，它控制着参数更新的步长大小——步子太小收敛过慢，步子太大则可能越过最优值甚至导致发散。

在实际训练中，我们通常不会一次性在所有样本上计算梯度（批量梯度下降），而是采用小批量随机梯度下

降 (Mini-batch SGD)：每次随机选取一小批样本，计算其平均梯度并更新参数。这一策略在计算效率和收敛稳定性之间取得了良好的平衡。

以上三个步骤——前向传播计算损失、反向传播计算梯度、参数更新优化网络——构成了神经网络训练的基本循环。这个循环反复进行，损失值逐渐下降，网络性能逐步提升，最终找到那个能够解决实际问题的“正确”网络。

优化器与挑战

训练深度神经网络不仅需要理解基本的梯度下降框架，更面临着诸多数学与工程上的核心挑战。优化算法的持续改进，正是应对这些挑战、推动深度学习发展的关键所在。

标准梯度下降（也称批量梯度下降）每一步都需要使用整个训练集计算梯度。当数据集规模庞大时，这种做法计算成本极高，且无法适应在线学习场景。因此，实践中更常用的是随机梯度下降 (Stochastic Gradient Descent, SGD) 及其变体——小批量随机梯度下降 (Mini-batch SGD)。后者每次随机抽取一小批数据计算梯度并更新参数。虽然引入的随机性会带来一定噪声，但大幅提升了迭代效率，而且适当的噪声反而有助于逃离局部极值点。

然而，标准 SGD 仍有改进空间。为了让训练过程更加智能、高效，研究者提出了多种自适应优化算法。

1. 带动量的 SGD (Momentum)

动量方法模拟了物理世界中的惯性概念。它在更新参数时，不仅考虑当前梯度，还累积历史梯度方向，形成一种“速度”效应。更新规则为：

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla_{\theta} \mathcal{R}, \quad \theta \leftarrow \theta + \mathbf{v}$$

其中 β 是动量衰减系数（通常取 0.9 左右）。这种机制有助于加速收敛、减少震荡，使网络更容易穿越平缓区域和鞍点，尤其在损失 landscape 存在狭长峡谷时表现优异。

2. Adam (Adaptive Moment Estimation)

Adam 是目前最流行的优化器之一 (Kingma & Ba, 2015)，它巧妙结合了动量和自适应学习率的优势。Adam 维护两个状态变量：梯度的一阶矩（均值）估计 m_t 和二阶矩（未中心化方差）估计 v_t ：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

其中 g_t 为当前梯度。随后对这两个矩估计进行偏差校正，最终更新参数：

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Adam 能自动为每个参数调整学习率，对超参数选择相对鲁棒，在自然语言处理、计算机视觉等众多任务上表现优异，成为深度学习实践的首选优化器之一。

3. 学习率调度

动态调整学习率 α 是另一项重要策略。常见做法包括：阶梯式衰减（每隔若干轮次乘以衰减因子）、余弦退火、或根据验证集性能自适应调整（如当验证损失停滞时降低学习率，即 ReduceLROnPlateau）。合理的学习率调度往往能带来显著的性能提升。

通过这些技术的有机结合，现代深度学习训练能够在收敛速度、稳定性和最终性能之间取得更好的平衡。

深度学习在实际训练中还面临其他几项重要挑战：

- 非凸优化：神经网络的损失函数是高维空间中的非凸函数，存在大量局部极小值和鞍点。然而，实践和部分理论研究表明，许多局部极小值在损失值上相差不大，且泛化性能可能都不错，而自适应优化器有助于逃离某些鞍点。
- 梯度消失与梯度爆炸：在深层网络中，误差信号 $\delta^{(l)}$ 需要通过多层连乘进行反向传播。如果每层梯度的范数小于 1，连乘会导致梯度指数级衰减，即梯度消失；反之若大于 1，则可能导致梯度指数级增长，即梯度爆炸。这一问题严重阻碍了深层网络的训练。ReLU 激活函数、恰当的权重初始化（如 Xavier、He 初始化）、批量归一化 (Batch Normalization) 以及残差连接 (ResNet 的核心思想) 等技术被证明能有效缓解这些问题。
- 超参数调优：学习率 α 、网络层数、宽度、正则化系数 λ 等超参数的选择对模型性能影响巨大。其优化本

身是一个元优化问题，通常通过网格搜索、随机搜索或更高级的贝叶斯优化等方法进行。

神经网络的应用掠影

神经网络有着许多重要应用，以下是几个标志性结果：

图像分类与模式识别：这是卷积神经网络（CNN）最为经典的应用场景。从数学角度看，图像分类本质上是在学习一个从高维像素空间到离散类别空间的复杂函数映射。该能力被广泛应用于识别数学符号、解析图表，甚至从科学图像（如天文观测图像、生物显微图像）中自动挖掘潜在模式，极大地提升了科研数据的处理效率。

AlphaGo 与强化学习：DeepMind 提出的 AlphaGo 结合了 CNN（用于评估棋盘局面的策略网络与价值网络）与蒙特卡洛树搜索。它的成功表明，神经网络能够逼近甚至超越人类专家级别的复杂决策函数。从数学角度看，这一突破启发了将定理证明、算法搜索等数学问题建模为强化学习问题。在此框架下，神经网络作为智能体，通过与环境（如证明器或计算环境）的交互，自主学习最优求解策略。代表性工作包括 AlphaProof、AlphaTensor 和 FunSearch 等。

符号回归与方程发现：以 AI Feynman 和 Ramanujan Machine 为代表的工作，利用神经网络对数据进行拟合，随后尝试从训练好的网络结构中提取或诱导出简洁且人类可读的数学表达式，如物理定律或数学恒等式。这类方法在“黑箱”模型与符号数学之间架起了一座桥梁，有助于自动发现新的科学规律。

组合对象构造：在组合数学中，许多难题需要构造满足特定性质的极值结构，如大型图或特殊的集合族。神经网络，尤其是生成模型，可以学习生成符合约束条件的候选构造，从而为数学家提供新的猜想和研究思路。相关研究如“Constructions in combinatorics via neural networks”即展示了这一方向的潜力。

科学计算：物理信息神经网络（PINNs）将描述物理过程的偏微分方程（PDE）作为正则化项引入损失函数。网络在拟合观测数据的同时，必须满足 PDE 所定义的物理规律。这使得 PINNs 提供了一种无需网格、数据驱动的灵活方案，适用于 PDE 正问题求解与参数反演等科学计算任务。

尽管取得了诸多成功，神经网络仍然面临一系列深刻的挑战：

- **可解释性：**神经网络的决策过程通常被视为一个“黑箱”，难以提供人类可理解的推理链条。如何让神经网络不仅输出预测结果，还能给出符合数学严谨性的“证明”或“解释”，是当前神经符号方法研究的核心难题。缺乏可解释性限制了其在数学证明、医疗诊断等高风险领域的应用。
- **泛化理论的缺乏：**尽管深度神经网络在实践中表现出极强的泛化能力，但其理论基础仍不完善。传统的统计学习理论给出的泛化边界对于过参数化的深度模型往往过于宽松，无法解释实际观察到的成功。虽然已有初步的探索，例如对线性网络或两层 ReLU 网络等简化模型的分析，以及关于“隐式正则化”（梯度下降倾向于收敛到平坦最小值，这通常对应更好的泛化）和“双下降”现象的研究，但目前尚未建立起统一、普适的深度网络泛化理论。
- **计算代价：**训练大型神经网络需要海量的计算资源和数据，这不仅带来了高昂的经济成本，也引发了对其环境影响的关注。高能耗与硬件资源的集中化，限制了部分研究群体和地区的参与，使得深度学习的可及性成为一个不容忽视的问题。

2.4 强化学习简介

强化学习的核心思想是让一个智能体在与环境的持续交互中，通过试错来学习如何采取行动，以达成长期目标。它不同于有监督学习（需要带标签的数据）和无监督学习（寻找数据内在结构），其学习信号来源于环境对智能体动作的反馈——奖励。

强化学习的基本框架与核心要素

强化学习的交互过程可以抽象为一个循环：智能体在时刻 t 观察环境状态 s_t ，并据此选择动作 a_t 执行。环境接收到动作后，转移到新状态 s_{t+1} ，并给出一个即时奖励 r_t 。智能体的终极目标是最大化整个过程中累积的奖励总和。图 2.7（引自邱锡鹏《神经网络与深度学习》图 14.2）给出了智能体与环境的交互：

这一过程在数学上由马尔可夫决策过程（MDP）完美刻画，图 2.8（引自邱锡鹏《神经网络与深度学习》图 14.3）给出了马尔可夫决策过程的图模型表示，其五元组 $(S, \mathcal{A}, P, R, \gamma)$ 即为强化学习的基本要素。

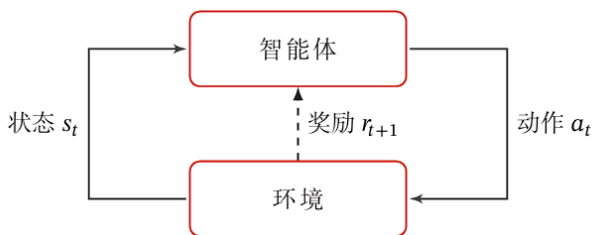


图 2.7: 智能体与环境的交互示意图

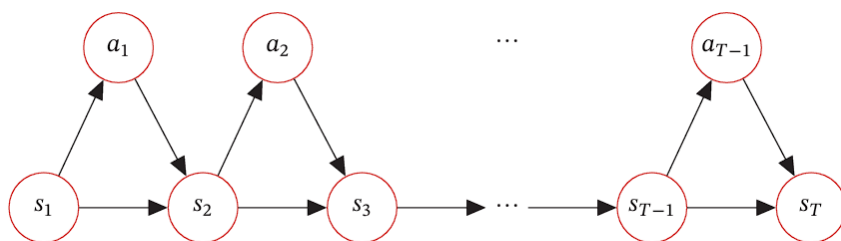


图 2.8: 马尔可夫图

1. 状态空间 (State Space, $\mathcal{S}$): 所有可能环境情况的集合。状态 s 应包含做出决策所需的全部信息（满足马尔可夫性）。

实例形式:

棋盘游戏（如围棋、国际象棋）：状态可以是整个棋盘所有棋子的布局。

视频游戏（如 Atari 游戏）：状态可以是连续几帧的原始像素图像，用以捕捉动态信息。

机器人控制：状态可以是机器人各个关节的角度与角速度，以及传感器（如摄像头、激光雷达、惯性测量单元）采集到的外部环境信息。

推荐系统：状态可以包括用户的画像特征、历史点击序列以及当前的上下文信息（如时间、设备、位置等）。

2. 动作空间 (Action Space, $\mathcal{A}$): 智能体在给定状态下可以执行的所有可能动作的集合。

实例形式:

离散动作空间：动作是可枚举的。例如：迷宫游戏中的上，下，左，右；围棋中的在 361 个交叉点落子，停一手。

连续动作空间：动作是实值向量，通常用于需要精细控制的场景。。例如：自动驾驶中的方向盘转角（连续值）、油门和刹车深度（连续值）；机械臂控制中各关节电机的目标力矩或位置增量。

3. 状态转移概率 (Transition Probability, $P(s' | s, a)$): 定义了环境的动态模型。表示在状态 s 下执行动作 a 后，环境转移到状态 s' 的概率。

实例形式:

在已知模型的简单问题中（如格子世界），它可以是一个精确的概率表。在绝大多数复杂问题中（如真实物理世界、复杂游戏），这个转移模型是未知的、复杂的、随机的。强化学习算法通常不需要事先知道 P ，而是通过与环境交互采样到的经验 (s, a, r, s') 来间接学习最优行为。

4. 奖励函数 (Reward Function, $R(s, a, s')$): 环境提供给智能体的即时反馈信号，是强化学习中的根本驱动力和设计核心。它量化了在状态 s 下采取动作 a 并导致状态转变为 s' 的即时好坏。

奖励函数的具体作用为:

- 定义目标：奖励函数本质上定义了智能体需要完成的任务。智能体的一切学习都围绕“最大化累积奖励”展开，因此奖励函数的设计直接决定了智能体最终学到什么策略。
- 引导学习：通过正向奖励（如得分、接近目标）和负向奖励（如碰撞、丢分），它像灯塔一样引导智能体探索有益的行为，避免有害的行为。
- 提供评估标准：价值函数 $V(s)$ 和 $Q(s, a)$ 都是对“未来累积奖励”的期望，奖励是这些评估函数的基石。

下面我们以几个具体的例子来说明强化学习的奖励规则。

对于迷宫导航游戏：到达目标则 +10 分；每走一步 -0.1 分（鼓励快速找到目标）；撞墙则 -1 分，奖励规则如下：

$$R(s, a, s') = \begin{cases} +10, & \text{如果 } s \text{ 是目标状态} \\ -1, & \text{如果 } s' \text{ 是撞墙状态} \\ -0.1, & \text{其他情况} \end{cases}$$

对于围棋/象棋：定义获胜 +1 分；失败 -1 分；平局或中间状态得 0 分。这是一个稀疏奖励的典型例子，大部分步没有即时反馈，智能体必须学会为最终结果进行长远规划。

对于自动驾驶：安全行驶在车道内则 +0.01 分；偏离车道 -0.1 分；舒适（平缓加速/刹车）+0.001 分；紧急刹车或碰撞则 -10 分。

对于能耗管理：规定完成特定任务 + 固定奖励；消耗的能量 -（能量消耗量 × 系数）。

5. 折扣因子 (Discount Factor, $\gamma \in [0, 1)$): 用于计算未来奖励在当前时刻的价值。 γ 越接近 0，智能体越“短视”，只重视眼前奖励；越接近 1，智能体越“有远见”，更考虑长期回报。其作用是在数学上保证无限时间步的累积奖励是有界的；在实际中便于平衡即时与远期收益。

在上述五个要素的定义下，智能体的核心任务是学习一个策略 $\pi(a|s)$ ，即在每个状态下选择动作的规则（可以是确定性函数或概率分布）。其目标是最大化期望累积折扣奖励（也称为回报）：

$$\mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right]$$

为了评估和寻找最优策略，引入了两个关键的价值函数：

- 状态价值函数 $V^{\pi}(s)$: 在策略 π 下，从状态 s 出发的期望回报。
- 动作价值函数 $Q^{\pi}(s, a)$: 在策略 π 下，从状态 s 执行动作 a 后的期望回报。

最优策略 π^* 对应的 Q^* 函数满足贝尔曼最优方程，这为许多算法提供了理论基石。

主要算法类别

1. 基于值函数的方法（如 Q-learning, DQN）：思想：不直接学习策略，而是先学习最优价值函数 $Q^*(s, a)$ ，最优策略自然就是选择让 Q 值最大的动作： $\pi^*(s) = \arg \max_a Q^*(s, a)$ 。其思想是不直接学习策略，而是先学会评估“哪个动作更值钱”——即学习最优价值函数 $Q^*(s, a)$ 。最优策略自然就是：在每个状态下，选择价值最高的那个动作。这类方法适合离散动作空间，且能重复利用历史经验，学习效率较高。
2. 策略梯度方法（如 REINFORCE）：其思想是直接参数化策略 $\pi_{\theta}(a|s)$ （如用一个神经网络），通过计算期望回报对策略参数 θ 的梯度，并使用梯度上升来优化策略。这类方法天然适合连续动作空间，能学习随机策略，但通常方差较大、训练不够稳定。
3. 演员-评论家方法（Actor-Critic）：该方法结合了以上两者。“演员”（Actor）即策略函数，负责根据状态选择动作；“评论家”（Critic）即价值函数（如 $V(s)$ 或 $Q(s, a)$ ），负责评估演员的表现并指导其更新。该方法用评论家提供的低方差估计来降低策略梯度的方差，显著提高了学习效率和稳定性。这是当前主流强化学习算法的基本架构（如 A3C, PPO, SAC 等）。

强化学习是一个以目标为导向的交互式学习范式。其基本原理围绕 MDP 五元组展开，智能体通过与环境交互获得的奖励信号来学习一个最大化长期回报的策略。奖励函数的设计是任务成败的关键，它如同设计者的“指挥棒”，需要精确、审慎地反映最终目标。从经典的 Q-learning 到现代的深度演员-评论家算法，各类方法都在尝试更高效、更稳定地解决这一核心问题，并在游戏、机器人、自动驾驶、资源管理等领域取得了瞩目成就。

2.5 生成式人工智能简介

根据任务的类型，机器学习可以划分为两大类：判别式人工智能，生成式人工智能。判别式人工智能的目标是学会在不同数据点之间“划清界限”，并对未知数据做出判别。它回答的是“这是什么？”的问题。例如，训练一个模型区分猫和狗的图片，模型学习的是“哪些像素特征组合更可能对应猫，哪些对应狗”，并建立一条决策边界。当看到新图片时，它判断这张图落在边界的哪一侧。判别式模型关心的是类别之间的分界线，而非数据本身的完整面貌。生成式人工智能的目标则更为宏大：理解并模仿数据的本质结构与分布。它学习训练数据是如何生成的——即其背后的概率分布。然后，它可以从学到的分布中采样，创造出全新的、符合数据规律的实例。如果说判别式模型回答“这是什么”，生成式模型则回答“如何创造这类事物”。通过学习海量数据中潜藏的模式，它能够生成原本不存在但符合该数据规律的文本、图像、代码、音乐乃至科学假设。生成式 AI 的成果已如潮水般涌入我们的工作与生活，其影响横跨多个领域：

- 1. 自然语言处理：
 - 智能对话与创作：以 ChatGPT、Claude 为代表的对话模型，能进行深度问答、撰写文章、翻译、编程等。
 - 代码生成：GitHub Copilot 等工具可根据注释或上下文，自动生成、补全或解释代码，极大提升了开发效率。
- 2. 视觉与多媒体创作：
 - 文生图/视频：Midjourney、Stable Diffusion、Sora 等模型，仅凭一段文字描述即可生成高度逼真或富有艺术感的图像与动态视频。
 - 设计辅助：自动生成产品原型、营销素材、室内设计图等。
- 3. 科学与发现：
 - 分子与材料设计：例如，生成具有特定属性的新药分子或电池材料结构，加速研发进程。
 - 数学与算法发现：如 DeepMind 的 AlphaEvolve，通过生成新的算法，直接改进了 50 年来未被优化的核心计算任务。

生成式 AI 的飞跃，主要建立在三大技术路线的突破之上。

生成对抗网络：博弈中进化

生成对抗网络（Generative Adversarial Networks, GANs）由 Ian Goodfellow 等人于 2014 年提出，其核心思想充满博弈论的智慧：让两个神经网络相互竞争、共同进化。

基本架构：GAN 由两个角色构成：

- 生成器（Generator）：一个试图“伪造”数据的艺术家。它接收一个随机噪声向量 z （通常从高斯分布或均匀分布中采样），将其转化为一个假样本 $G(z)$ ，例如一张假图像。生成器的目标是让生成的样本足够逼真，以至于无法被识破。
- 判别器（Discriminator）：一个试图“鉴伪”的专家。它接收真实样本 x 和生成器产生的假样本 $G(z)$ ，并判断每个样本是真实的还是伪造的。判别器的输出是一个概率值 $D(x)$ ，表示样本为真的置信度。

生成对抗网络的流程如图 2.9（引自邱锡鹏《神经网络与深度学习》图 13.8）所示：

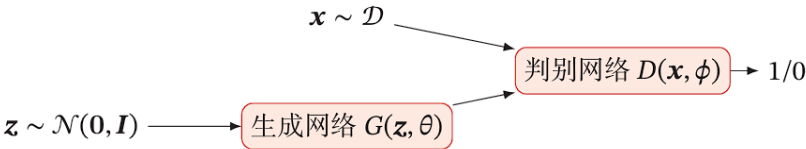


图 2.9: 生成对抗网络

在对抗训练过程中这两个网络进行着一场零和博弈：

- 生成器不断改进其伪造技巧，试图欺骗判别器，让它把假样本误判为真

- 判别器不断磨练其鉴别能力，试图识破生成器的把戏

整个训练过程可以形式化为一个极小极大博弈问题：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

其中 p_{data} 是真实数据分布， p_z 是噪声分布。直观理解：判别器 D 试图最大化正确分类真实样本和假样本的对数似然；生成器 G 则试图最小化判别器正确识别假样本的概率（即最大化判别器犯错的机会）。

训练的动态平衡：理想情况下，这场博弈最终会达到纳什均衡——生成器产生的样本与真实数据难以区分，判别器对任何样本的判断概率都接近 0.5（完全无法确定真假）。此时，生成器就学会了真实数据的分布。

然而，这场博弈的平衡极其微妙，GAN 的训练也因此以不稳定著称。生成器和判别器的能力需要同步成长——如果判别器太强，它会轻易识破所有伪造样本，导致生成器无法获得有效梯度而停止学习；如果生成器太强，它可能利用判别器的某些弱点，只学会生成少数几种能够骗过判别器的样本，这种现象被称为模式崩溃。此外，当判别器过于自信时，其输出的梯度可能趋近于零，导致生成器陷入梯度消失的困境，训练难以为继。研究者们提出了众多改进方案，如 WGAN、LSGAN、StyleGAN 等，通过改进损失函数、网络结构和训练策略来缓解这些问题。

经典应用实例：

- 人脸生成：StyleGAN 系列模型能够生成分辨率高达 1024×1024 的逼真人脸，这些面孔并不存在于现实中，但细节之丰富令人惊叹。用户可以控制生成结果的年龄、姿态、发型、表情等属性。
- 图像翻译：Pix2Pix 模型可以将素描转化为逼真的照片（如将边缘图转化为猫的图片），或将白天地图转化为夜间地图。CycleGAN 更进一步，实现了无需成对数据的图像风格迁移——例如将普通马的照片转化为斑马，或将莫奈的画风应用到任何风景照片上。
- 超分辨率：SRGAN 等模型能将低分辨率图像恢复出高分辨率细节，在医学影像、卫星图像等领域有重要应用。

尽管面临训练不稳定的挑战，GAN 仍以其独特的对抗式学习范式和惊艳的生成效果，成为图像生成领域的一座里程碑。

扩散模型：从混沌到有序

扩散模型（Diffusion Models）是近年来异军突起的技术路线，已成为当前图像生成领域的“王牌”——Stable Diffusion、DALI 2、Midjourney 等主流文生图工具均基于此技术。其灵感源于非平衡热力学中的扩散过程。其核心思想为：扩散模型包含两个相反的过程——前向扩散过程和逆向生成过程。

前向扩散过程：这个过程是固定的、不可训练的。我们从一张真实图像 x_0 开始，逐步向其添加微小的高斯噪声。经过 T 步，原始图像被彻底破坏，变成一张纯噪声图 $x_T \sim \mathcal{N}(0, I)$ 。这一过程可以看作信息的渐进式销毁，每一步的噪声添加是有固定规律的：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

其中 β_t 是一个预先定义的、随时间递增的噪声调度参数。一个精妙之处在于，由于高斯分布的可加性，我们可以直接从 x_0 一步计算出任意时刻 t 的加噪结果：

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

其中 $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ 。这个闭式解极大地简化了训练过程。

逆向生成过程：这才是模型真正学习的部分——如何从纯噪声 x_T 出发，逐步去噪，最终还原出清晰的图像 x_0 。每一步逆向过程 $p_\theta(x_{t-1}|x_t)$ 被建模为一个高斯分布，其均值和方差由神经网络预测：

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

在实际实现中（如 DDPM, Denoising Diffusion Probabilistic Models），模型通常被训练去预测添加到 x_0 上的噪声 ϵ ，而非直接预测图像本身。训练目标简化为：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

这意味着模型学习的是：给定带噪图像 x_t 和时间步 t ，预测当时添加的噪声是什么。掌握了这个能力后，在生成时就可以逐步减去预测的噪声，从混沌中重建秩序。

如果想将扩散模型拓展到文生图任务上，上述基础扩散模型只能无条件生成图像。要实现“根据文字描述生成图像”，需要引入条件控制。这通常通过交叉注意力机制实现：

- 用文本编码器（如 CLIP 或 T5）将用户输入的文字描述转换为文本嵌入向量
- 在扩散模型的去噪过程中，通过交叉注意力层，让图像特征与文本特征进行交互，使去噪过程朝着符合文本描述的方向进行

Stable Diffusion 进一步引入了潜在扩散模型（Latent Diffusion Model）的思想：不在像素空间直接进行扩散，而是先训练一个自编码器将图像压缩到低维潜在空间，然后在潜在空间中进行扩散。这大幅降低了计算成本，同时保持了生成质量。

生成过程实例：当用户在 Stable Diffusion 中输入“一只穿着宇航服的柯基犬”时，模型按顺序进行如下操作：

1. 文本编码器将这句话转换为向量表示
2. 模型从一个随机噪声潜在表示开始
3. 经过数十步迭代去噪，每一步都参考文本向量的引导
4. 最终潜在表示被解码器还原为像素图像——一只身穿迷你宇航服、神情认真的柯基犬跃然纸上

相比 GAN，扩散模型训练更稳定，不易发生模式崩溃，且能生成更多样化的结果。其逐步去噪的过程也赋予了用户更多控制权——可以在生成中途干预、进行图像修复、编辑等操作。当然，代价是生成速度较慢（需要数十至数百步迭代），但近年来提出的加速采样方法（如 DDIM、DPM-Solver）已显著提升了效率。

Transformer 架构与大语言模型

这是当前生成式 AI（尤其在语言领域）的基石。其核心的自注意力机制使模型能并行处理并深刻理解文本中所有单词之间的复杂关系。通过在 TB 级别的海量文本上进行预训练，大语言模型掌握了语言的通用知识和逻辑，再通过指令微调等对齐技术，使其能够遵循人类指令进行创造性工作。关于 Transformer 架构的详细原理，我们将在下一节大语言模型简介中探讨，此处不再赘述。

生成式 AI 正从一项炫酷的技术，演变为一种基础的创造性基础设施。它正在重塑内容创作、软件开发、科学研究的方式，其核心价值在于极大释放人类的创造力，将我们从重复性劳动中解放出来，专注于更高层次的战略、审美与创新决策。随着多模态融合的深入（如能同时理解和生成文本、图像、声音的模型），一个由自然语言驱动万物的智能交互时代已拉开序幕。

2.6 大语言模型简介

要理解大语言模型（LLM）如何工作，我们需要从两个层面入手：一是模型的整体架构，看信息如何流入流出；二是其核心机制，看模型如何理解语言。

一、大语言模型的整体框架：输入、处理、输出

大语言模型本质上是一个根据输入文本预测下一个词的神经网络系统。以 GPT 系列模型为例，其工作流程可以概括为三个阶段：

1. 输入处理：将文字转化为向量

计算机无法直接理解文字，因此第一步是将输入文本转换为数字表示。这个过程包括：

- 分词：将文本切分为更小的单元（如单词或子词），例如“我喜欢 AI”可能被切分为“[“我”，“喜欢”，“AI”]”，得到长度为 L 的 token 序列。
- 词嵌入：将每个 token 映射为一个高维向量。设词表大小为 V ，嵌入维度为 d_{model} ，则每个 token 根据其索引从一个嵌入矩阵 $E \in \mathbb{R}^{V \times d_{\text{model}}}$ 中查找对应的向量：

$$\mathbf{x}_i = E[\text{token}_i] \in \mathbb{R}^{d_{\text{model}}}$$

这些向量携带了词的语义信息——语义相近的词在向量空间中也彼此靠近。

- 位置编码：由于自注意力机制本身对顺序不敏感，需要额外注入位置信息。对于位置 i ，其位置编码 $\mathbf{p}_i$ 与词嵌入相加：

$$\mathbf{h}_i^{(0)} = \mathbf{x}_i + \mathbf{p}_i$$

位置编码可以是固定的正弦/余弦函数（如原始 Transformer）：

$$\text{PE}(i, 2j) = \sin\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right), \quad \text{PE}(i, 2j+1) = \cos\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right)$$

也可以是可学习的参数。最终得到输入表示矩阵 $H^{(0)} \in \mathbb{R}^{L \times d_{\text{model}}}$ 。

2. 核心处理：通过多层 Transformer 块提取特征

这是大语言模型的“大脑”部分。输入向量序列依次通过数十甚至上百个相同的处理模块——Transformer 块。每个块都会对序列进行一次“全局信息融合与局部特征变换”，让每个位置的表示不断吸收上下文信息。

3. 输出生成：预测下一个词

经过多层处理后，模型为每个位置输出一个高维表示。对于最后一个位置的输出 $\mathbf{h}_L^{(N)}$ ，模型通过一个线性变换加 Softmax 函数，将其转换为词表上所有词的概率分布：

$$P(\text{next token} = v | \text{context}) = \frac{\exp(\mathbf{w}_v^T \mathbf{h}_L^{(N)})}{\sum_{v'=1}^V \exp(\mathbf{w}_{v'}^T \mathbf{h}_L^{(N)})}$$

其中 $\mathbf{w}_v \in \mathbb{R}^{d_{\text{model}}}$ 是词 v 的输出权重向量（通常与输入嵌入矩阵共享）。模型会选择概率最高的词（或按概率采样）作为预测结果。例如，输入“我喜欢”，模型可能预测下一个词是“AI”（概率 0.3）、“吃”（概率 0.2）、“学”（概率 0.1）等。

这个“输入-处理-输出”的循环构成了大模型生成文本的基本方式：预测一个词，将新词加入输入，再预测下一个，如此往复，直至生成完整句子。

二、核心机制：注意力——让模型学会“关注重点”

在上述流程中，最关键的问题是：模型在处理每个词时，如何知道应该关注上下文中的哪些部分？这正是注意力机制要解决的问题。

思想起源：像人一样分配注意力

当我们阅读“那只看守所里的猫追自己的尾巴”这句话时，要理解“自己”指代什么，我们需要回顾前文找到“猫”——这就是注意力的朴素形式。注意力机制正是让模型学会这种“选择性关注”的能力：在处理当前位置时，动态地决定上下文中的哪些词更重要，并将它们的信息融合进来。

从数学上看，注意力机制是一种动态加权平均。它涉及三个基本角色：

- 查询（Query, Q）：代表当前关注点，相当于在问“我现在应该看哪里？”
- 键（Key, K）：代表每个上下文词的标识，相当于“我是谁”
- 值（Value, V）：代表每个上下文词的实际内容，相当于“我有什么信息”

给定一个查询向量 $\mathbf{q} \in \mathbb{R}^{d_k}$ 和一组键值对 $\{(\mathbf{k}_i, \mathbf{v}_i)\}_{i=1}^L$ ，其中 $\mathbf{k}_i \in \mathbb{R}^{d_k}$, $\mathbf{v}_i \in \mathbb{R}^{d_v}$ ，注意力输出为：

$$\text{Attention}(\mathbf{q}, K, V) = \sum_{i=1}^L \alpha_i \mathbf{v}_i$$

其中权重 α_i 通过查询与键的相似度计算并经 softmax 归一化得到：

$$\alpha_i = \frac{\exp(s(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^L \exp(s(\mathbf{q}, \mathbf{k}_j))}$$

相似度函数 $s(\cdot, \cdot)$ 最常用的是缩放点积：

$$s(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{d_k}}$$

这里的 $\sqrt{d_k}$ 是缩放因子，用于防止当向量维度较高时点积过大，导致 softmax 陷入梯度极小的饱和区。

在实际实现中，我们同时对整个序列的所有位置计算注意力，可以写成矩阵形式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 $Q \in \mathbb{R}^{L \times d_k}, K \in \mathbb{R}^{L \times d_k}, V \in \mathbb{R}^{L \times d_v}$ ，softmax 按行进行。

权重 α_i 直观地反映了当前查询对第 i 个信息的关注程度。通过可学习的线性变换，模型可以从输入序列中动态生成 Q, K, V 向量。这意味着，在生成序列的每一个位置时，模型都能灵活地“回顾”并聚焦于最相关的内容——这正是注意力机制的核心所在。

注意力机制的计算过程如图2.10（引自邱锡鹏《神经网络与深度学习》图 8.4）所示。

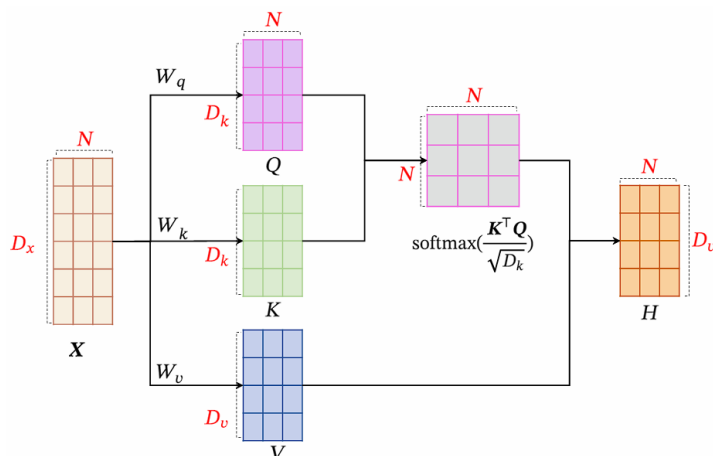


图 2.10: 注意力计算过程

自注意力

在大语言模型中，我们使用的是一种特殊形式——自注意力（Self-Attention）。这里的“自”意味着查询、键、值都来自同一输入序列的不同线性变换。设输入矩阵为 $X \in \mathbb{R}^{L \times d_{\text{model}}}$ ，我们通过三个可学习的权重矩阵 $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 将其投影到查询、键、值空间：

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

然后计算自注意力：

$$\text{SelfAttention}(X) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

这种设计的精妙之处在于：序列中的每个词既是“提问者”（发出查询），也是“被提问者”（提供键和值）。无论两个词在句子中相隔多远，自注意力都能让它们直接建立联系。在传统模型（如 RNN）中，“猫”对“自己”的影响需要经过多个时间步才能传递，信息容易衰减；而在自注意力中，这种影响一步直达，从根本上解决了长距离依赖问题。

多头注意力：从多个角度理解语言

与其只计算一组注意力，不如让模型从多个角度同时观察。多头注意力（Multi-Head Attention）将查询、键、值投影到 h 个不同的低维子空间（每个头的维度 $d_k = d_{\text{model}}/h$ ），在每个头上独立计算注意力，然后将所有头的

输出拼接并再次投影：

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

其中 $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 都是可学习的投影矩阵。

这类似于让多个“专家”同时阅读同一段文字：有的专家关注语法关系（主谓宾），有的专家关注指代关系（谁指谁），有的专家关注语义关联（同义词）。多头机制允许模型并行地捕获不同类型的关系，极大地增强了表达能力。

Transformer 块的结构

将上述组件组合起来，就得到了一个标准的 Transformer 编码器层（或称 Transformer 块）。其结构如下：

- 多头自注意力层：计算序列内所有位置之间的相互关系，让每个位置都能聚合其他位置的信息。
- 第一个残差连接与层归一化：将自注意力层的输入加到输出上（残差连接），然后进行层归一化。残差连接为梯度提供了“高速公路”，极大缓解了深层网络中的梯度消失问题；层归一化则稳定了训练过程。
- 前馈网络：一个简单的两层全连接网络，对每个位置的表示进行独立变换。这一步为模型引入了非线性，并增强了每个位置的表达能力。
- 第二个残差连接与层归一化：再次应用残差连接和层归一化，完成整个块的处理。

通过堆叠多个这样的 Transformer 块，模型可以逐层构建越来越抽象的表示——低层可能关注词法和短语，中层可能关注句法结构，高层则可能捕捉语义和篇章关系。

Transformer 网路架构如图2.11（引自 [Vaswani et al., 2017]）所示：

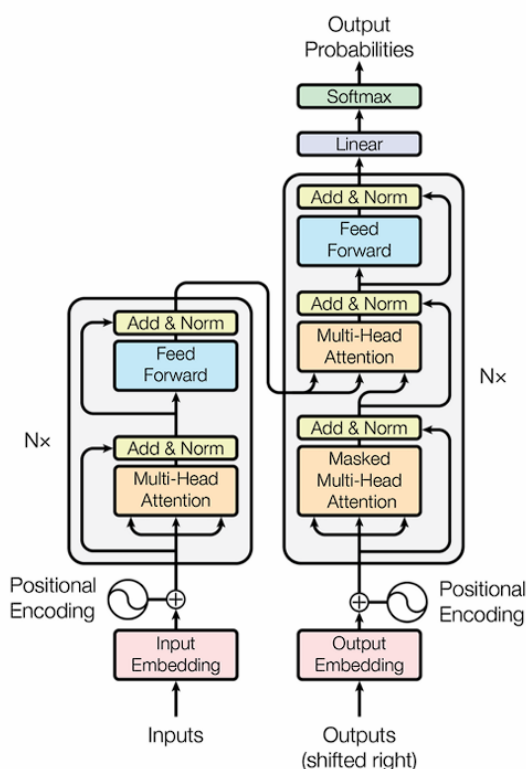


图 2.11: transformer 网络架构

理解上述机制后，我们就能明白 Transformer 架构为何能成为处理大规模序列数据（如自然语言、代码、数学符号序列）的理想架构：

- 并行计算：自注意力可以一次性计算所有位置之间的关系，而不像 RNN 那样必须逐个处理，这使大规模训练成为可能。

- 长距离依赖：任意两个位置可以直接交互，无论相隔多远，信息都不会衰减。
- 层次化理解：通过多层堆叠，模型可以从局部短语到全局语义，逐层构建对语言的深入理解。
- 可扩展性：Transformer 的架构设计天然适合堆叠更多层、使用更多头、处理更长的序列——这正是“大”语言模型中“大”字的体现。

正是这些优势，使得基于 Transformer 的大语言模型能够从海量文本中学习语言的规律，并在理解、生成、推理等任务上展现出惊人的能力。

三、大语言模型的训练机制与流程

拥有强大的 Transformer 架构之后，下一个核心问题浮出水面：如何通过工程化的训练流程，赋予模型语言理解、逻辑推理乃至数学能力？现代大语言模型的训练并非单一阶段，而是一个精心设计的多阶段流水线。每个阶段承担不同的使命，共同塑造出最终的能力强大的模型。

1. 预训练（Pretraining）：构建语言与知识的基础

预训练是整个训练流程中最耗资源、数据量最大的阶段，其目标是为模型打下坚实的语言基础和广泛的世界知识。这一阶段属于自监督学习——监督信号并非来自人工标注，而是从数据本身的结构中自动生成。

数据格式：使用大规模、多样化的文本语料库，包括网页、书籍、学术论文、代码仓库、社交媒体等。数据经过分词处理后，形成连续的 token 序列，作为模型的输入。这些数据无需人工标注。

训练目标：根据模型架构的不同，预训练目标主要有两种范式：

- 掩码语言建模（Masked Language Modeling, MLM）：以 BERT[Devlin J, Chang M W, Lee K, et al. BERT] 为代表的编码器模型采用此策略。具体做法是随机遮盖输入序列中的一部分 token（通常为 15%），然后训练模型预测被遮盖的原始 token。这类似于“完形填空”任务，使模型能够理解双向上下文信息。这里的标签——被遮盖的 token——正是从输入序列本身提取的。
- 下一个 token 预测（Next Token Prediction）：以 GPT 为代表的自回归解码器模型采用此策略。给定前文 token 序列，训练模型预测下一个最可能出现的 token。这类似于“文字接龙”，使模型天生具备文本生成能力。这里的标签——序列中实际的下一个 token——同样来自数据自身。由于生成任务的普适性，这一范式已成为当前大语言模型预训练的主流。

从数学上看，对于长度为 T 的序列 $\mathbf{x} = (x_1, x_2, \dots, x_T)$ ，自回归语言模型的训练目标是最大化以下似然：

$$\mathcal{L}_{\text{LM}} = \sum_{t=1}^T \log P(x_t | x_{1:t-1}; \theta)$$

其中 θ 表示模型参数， $P(x_t | x_{1:t-1}; \theta)$ 是模型预测的第 t 个位置的条件概率。注意到这里的监督信号 x_t 正是输入序列自身的一部分——这正是自监督学习的核心特征。

预训练如此大规模的模型，需要一系列精妙的工程手段支撑：

- 大规模分布式训练：当模型参数量达到千亿级别、训练数据达到数万亿 token 时，单块 GPU 的内存和算力已远远不够。必须采用复杂的并行策略，将计算分布到成千上万个 GPU 上：
 - 数据并行：将训练数据切分到多个设备，每个设备持有完整的模型副本，定期同步梯度
 - 张量并行：将单个层的权重矩阵切分到多个设备，协同完成矩阵运算
 - 流水线并行：将不同层分配到不同设备，形成计算流水线，同时处理多个微批次
- 混合精度训练：使用 FP16（半精度）或 BF16 浮点数格式进行前向和反向传播，可以大幅节省显存并加速计算。同时，在主副本中保留 FP32 格式的参数和优化器状态，保证数值稳定性。配合动态损失缩放技术，避免半精度下的梯度下溢。
- 激活检查点：在反向传播时，通常需要保存所有中间激活值以供梯度计算，这占用了大量显存。激活检查点技术选择性地保存部分激活值，在需要时重新计算其余部分，以时间换空间，在显存受限尤为有效。

2. 监督微调（Supervised Fine-Tuning, SFT）：对齐指令与输出格式

预训练模型虽然掌握了丰富的知识，但未必能很好地遵循人类指令或输出特定格式。监督微调（Supervised Fine-Tuning, SFT）阶段使用高质量的有标签数据对模型进行微调，使其行为与人类期望对齐。与预训练不同，这

一阶段属于标准的监督学习——每个训练样本都有明确的人工标注目标。

数据格式：使用人工精心编写的“指令-输出”对数据集。例如在数学领域，数据格式可能是：

- 指令内容示例：解方程 $x^2 - 5x + 6 = 0$
- 输出内容示例：因式分解得 $(x - 2)(x - 3) = 0$ ，所以解为 $x = 2$ 或 $x = 3$ 。

SFT 对数据质量要求极高，需要覆盖多样化的任务类型，且输出应包含正确的推理过程。数据集通常包含数万到数十万条这样的示例。

监督微调的训练是这样的一个过程，这是一个标准的有监督学习过程。将指令和期望输出拼接作为输入，训练模型在输出部分生成目标序列。损失函数仍为下一个 token 预测损失，但通常只计算输出部分的损失，避免对指令部分的无意义建模。

从数学上看，对于指令 x 和目标输出 y ，损失函数为：

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^{|y|} \log P(y_t | x, y_{1:t-1}; \theta)$$

其中工程技巧包括：

- 参数高效微调（Parameter-Efficient Fine-Tuning, PEFT）：大模型参数量巨大，完全微调所有参数成本高昂且容易过拟合。LoRA（Low-Rank Adaptation）是当前最广泛使用的技术。其核心思想是假设模型权重的更新 ΔW 具有低秩特性。因此，冻结原始权重 W_0 ，引入两个小得多的可训练矩阵 A 和 B ，使得 $\Delta W = BA$ ，更新后的权重为 $W = W_0 + BA$ 。在微调时，只更新 A 和 B ，可训练参数量减少到原来的万分之一甚至更少，大大降低了显存占用和过拟合风险。
- 高质量数据构建：SFT 数据的质量直接决定了微调效果的上限。除了人工编写，还可采用以下策略：
 - 模型辅助构建：利用更强的基础模型生成候选答案，再由专家筛选和修正
 - 社区资源挖掘：从数学竞赛题解、技术问答社区等高质量资源中收集整理
 - 数据增强：通过改写、翻译等方式扩充数据集，增加多样性

3. 强化学习（Reinforcement Learning, RL）：基于反馈的优化

SFT 之后，模型能够执行指令，但输出质量仍有提升空间——如正确性、简洁性、安全性、与人类偏好的一致性。强化学习（Reinforcement Learning, RL）阶段引入一个奖励信号来进一步优化模型，使其输出更符合高阶目标。这是提升模型数学推理能力、对齐人类价值观的关键阶段。在深入具体技术前，我们先回顾强化学习的基本框架。在语言模型场景中：

- 智能体（Agent）：语言模型本身，其策略 $\pi_\theta(a|s)$ 是在给定上下文 s （已生成的 token 序列）下选择下一个 token a 的概率分布
- 环境（Environment）：生成文本的交互过程，包括用户输入的提示和模型自回归生成的环境
- 状态（State）：当前的文本上下文，即已生成的所有 token
- 动作（Action）：选择下一个 token
- 奖励（Reward）：根据生成的完整序列计算的质量分数，由外部评估机制提供

目标是优化策略参数 θ ，最大化期望累积奖励：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

其中 $\tau = (s_0, a_0, s_1, a_1, \dots)$ 是一个完整的文本生成轨迹， $R(\tau)$ 是该轨迹获得的总奖励。

下面介绍几种典型的大模型强化学习的方法：

基于人类反馈的强化学习（RLHF）

RLHF 是 ChatGPT 等模型采用的核心对齐技术，它将人类偏好引入优化目标，使模型输出更符合人类价值观。RLHF 通常分为三个步骤：

1. 收集人类偏好数据：给定一个提示（Prompt），让 SFT 后的初始模型生成多个不同答案，由人类标注员对

这些答案进行排序（哪个更好、哪个次之）。这一过程产生大量偏好对 (x, y_w, y_l) ，其中 x 是提示， y_w 是更受偏好的回答， y_l 是较不受偏好的回答。

2. 训练奖励模型（Reward Model, RM）：使用收集到的偏好对训练一个独立的奖励模型 $r_\varphi(y|x)$ ，使其学会为更符合人类偏好的文本输出打更高的分。偏好建模通常采用 Bradley-Terry 模型：

$$P(y_w > y_l|x) = \sigma(r_\varphi(y_w|x) - r_\varphi(y_l|x))$$

其中 σ 是 sigmoid 函数。奖励模型的训练目标就是最大化这个似然概率：

$$\mathcal{L}_{RM}(\varphi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\varphi(y_w|x) - r_\varphi(y_l|x))]$$

训练完成后，奖励模型能够为任意文本输出提供一个标量分数，反映其与人类偏好的吻合程度。

3. 使用强化学习优化策略模型：冻结训练好的奖励模型 r_φ ，将 SFT 后的策略模型 π_{SFT} 作为微调起点。在给定提示 x 下，策略模型 π_θ 生成回答 y ，从奖励模型获得奖励 $r_\varphi(y|x)$ 。同时，为防止策略模型过度偏离初始的 π_{SFT} （以免失去语言能力或产生无意义输出），需要在奖励中添加一个 KL 散度惩罚项。因此，总奖励为：

$$R(y|x) = r_\varphi(y|x) - \beta \cdot \text{KL}(\pi_\theta(\cdot|x) \parallel \pi_{SFT}(\cdot|x))$$

其中 β 是惩罚系数，控制偏离程度。KL 散度项的计算公式为：

$$\text{KL}(\pi_\theta \parallel \pi_{SFT}) = \mathbb{E}_{y \sim \pi_\theta} \left[\log \frac{\pi_\theta(y|x)}{\pi_{SFT}(y|x)} \right]$$

然后使用策略梯度算法优化策略模型 π_θ 以最大化期望奖励 $J(\theta)$ 。近端策略优化（Proximal Policy Optimization, PPO）是目前最常用的算法，它通过引入重要性采样和裁剪机制，实现了稳定高效的策略更新。

直接偏好优化（Direct Preference Optimization, DPO）

RLHF 流程复杂，需要训练单独的奖励模型并使用强化学习算法，训练不稳定且超参数众多。直接偏好优化（DPO）提供了一种更简洁的替代方案，它绕过了奖励模型的显式建模，直接将人类偏好数据用于优化策略。

DPO 的核心洞见是：在 Bradley-Terry 偏好模型下，最优策略 π^* 与奖励函数 r^* 及参考策略 π_{ref} 之间存在一个封闭形式的解析关系：

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

其中 $Z(x)$ 是配分函数，与 y 无关。将这个表达式代入偏好概率公式，可以推导出一个直接基于策略 π_θ 的损失函数：

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

这里的 π_{ref} 通常是 SFT 后的初始模型。优化这个损失函数，直接驱使策略模型对优选回答 y_w 给出相对于 π_{ref} 更高的似然，对劣选回答 y_l 给出更低的似然。DPO 省去了奖励模型训练和复杂的强化学习循环，实现更简单、训练更稳定，已成为 RLHF 的热门替代方案，在许多任务上取得了与 RLHF 相当甚至更好的效果。

组相对策略优化（GRPO）与数学推理强化学习

在数学推理等任务中，答案质量往往是一个谱系，而非简单的对错二分，如何有效利用多样化的候选答案来指导模型学习。组相对策略优化（Group Relative Policy Optimization, GRPO）强调在同一输入条件下，对不同候选答案之间的相对质量进行建模。

其核心思想为：GRPO 不是基于二元偏好，而是基于答案组的相对质量进行优化。对于同一个数学问题，模

型生成或收集多个候选答案，根据其正确性、推理步骤的完整性、简洁性等标准，划分为不同的质量组。

对于问题 x ，模型会生成一组候选输出 $\{y_1, y_2, \dots, y_n\}$ ，每个候选答案都会通过某种方式（例如奖励模型或规则评估）被赋予一个质量评分 $r(x, y_i)$ 。GRPO 并不直接使用这些绝对奖励值进行优化，而是关注它们在同一组内的相对关系。

为此，可以定义每个候选答案的“相对优势”为其奖励与组内平均奖励之间的差值，即：

$$A(x, y_i) = r(x, y_i) - \frac{1}{n} \sum_{j=1}^n r(x, y_j)$$

这一量刻画了某个答案相对于同组其他答案的优劣程度。如果一个答案的得分高于平均水平，则其优势为正；反之则为负。

在此基础上，GRPO 的优化目标可以理解为：根据这一相对优势来调整模型对不同答案的概率分布。形式上，可以写为：

$$\mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [A(x, y) \cdot \log \pi_{\theta}(y|x)]$$

这一目标的直观含义是，当某个答案优于组内平均水平时，应提高其被生成的概率；而当其表现较差时，则降低其概率。通过这种方式，模型逐步学会在同一问题下偏好更高质量的输出。

从更高层的角度来看，GRPO 的核心思想在于用“相对比较”替代“绝对评估”。这种设计具有多方面优势。首先，它降低了对奖励函数精确尺度的依赖，使训练过程对奖励的数值范围更加鲁棒。其次，通过在同一输入下进行比较，可以有效减少训练中的方差问题，从而提升稳定性。此外，这种方法能够自然利用排序信息，而不仅仅依赖简单的正负标签，这在数学推理或代码生成等任务中尤为重要。

因此，可以将 GRPO 理解为一种基于组内相对排序的策略优化方法，它通过强化“更优答案应更可能被生成”这一原则，引导模型逐步学习更合理的生成行为。

GRPO 适合数学推理场景，因为：

- 从错误答案中学习“为什么错”与从正确答案中学习“如何对”同样重要
- 答案质量往往存在多个层次，二元偏好无法充分利用信息
- 组间对比引导模型学习更鲁棒、更高效的推理模式

GRPO 的思想可以灵活地与 DPO 或基于奖励模型的方法结合。例如，可以用一个经过校准的奖励模型为每个答案打分，然后根据分数动态分组，再进行组间对比优化。

综上所述，现代大语言模型的训练是一个多阶段、渐进式的过程，三种学习范式各司其职。这三个阶段层层递进、各有侧重，共同塑造出最终的大语言模型。预训练提供广度和深度，监督微调提供形式对齐，强化学习提供价值对齐和精细化优化。正是这一精心设计的训练流水线，使得模型能够在数学推理、代码生成、对话交互等复杂任务上展现出接近人类水平的能力。

表 2.1: 三种强化学习范式

阶段	学习范式	监督信号来源	核心目标
预训练	自监督学习	数据自身结构	学习语言规律和世界知识，奠定基础能力
监督微调	监督学习	标注	使模型学会遵循指令和输出格式
强化学习	强化学习	奖励信号	优化输出质量，对齐人类偏好和价值观

2.7 工程技巧

机器学习工程实践的核心思想，可以概括为“控制与引导”——通过一系列精心设计的技术手段，控制优化过程的内在不确定性，引导模型参数向具有良好泛化性能的区域收敛。这远非技术细节的简单堆砌，而是对优化动力学、泛化理论与计算资源管理的深刻理解与综合应用。

模型的最终性能，不仅取决于算法本身，更取决于工程实践中每一个环节的精心设计与协同配合。本节将围绕机器学习流程中的关键工程环节——数据准备、模型初始化、训练优化与正则化——展开深入探讨，揭示其

背后的数学直觉与设计原则。

一、数据准备：模型学习的基石

机器学习模型“学习”的是数据中的规律，因此数据的质量、规模与多样性直接决定了模型性能的上限。正如计算机科学中的格言所言：“垃圾进，垃圾出”（Garbage In, Garbage Out），无论算法多么精妙，如果输入数据存在缺陷，模型终究无法学习到正确的知识。

1. 数据收集：构建高质量的数据源泉

在面向数学任务的机器学习中，数据来源呈现出丰富的多样性，每种来源都有其独特的价值与挑战。

符号数据是最直接的数学知识载体，通常从结构化的数学数据库中提取。例如，整数序列在线百科全书（OEIS）收录了数十万条整数序列及其数学性质；arXiv 上的数学论文、教科书和定理证明文档中蕴含着海量的公式、定理和证明步骤。这些数据的特点是结构清晰、语义精确，但往往需要复杂的解析技术才能将其从 PDF 或 LaTeX 源码中还还原为可用的结构化形式。

生成数据则是根据特定数学规则或约束自动合成的数据，在数学推理任务中扮演着越来越重要的角色。例如，我们可以设计一个随机多项式生成器，产生形如 $ax^2 + bx + c = 0$ 的二次方程，并自动计算其判别式和根；或者生成特定图结构的邻接矩阵及其对应的图论性质（如直径、着色数）。生成数据的优势在于规模可扩展、标签天然存在，且可以精确控制数据分布的覆盖范围，但需要谨慎设计生成规则，避免产生与真实数学规律相悖的“伪数据”。

仿真数据常见于科学与工程问题的机器学习求解中，特别是涉及微分方程、物理模拟的任务。例如，在求解偏微分方程时，我们可以在计算域的网格点上采样，通过数值求解器获得方程的近似解，构成“输入-输出”对。仿真数据的质量取决于数值方法的精度和采样策略的合理性。

2. 数据清洗：剥离噪声，保留本质

收集到的原始数据往往是“脏”的，尤其当数据来源于互联网时。数据清洗的目标是减少数据中的噪声，让模型能够聚焦于真正的数学模式。对于数学文本，清洗工作涉及多个层面：

首先是格式规范化。数学表达式存在多种表示形式，例如乘号可能是“ $\times$ ”、“ $\cdot$ ”或“ $*$ ”，需要统一为标准形式；LaTeX 源码中的命令如“ $\frac{a}{b}$ ”需要被正确解析为结构化的分数表示。其次是噪声过滤。从互联网爬取的数学内容往往混杂着广告、导航栏、无关评论等干扰信息，需要设计启发式规则或训练分类器将其剔除。对于扫描版文献，光学字符识别（OCR）产生的错误也需要纠正——例如将“ x^2 ”误识别为“ $x2$ ”是常见问题，需要结合上下文或数学语法进行修复。

在大语言模型的训练中，数据清洗更是规模巨大且至关重要的工作。以训练一个千亿参数级的模型为例，训练数据量通常达到数万亿 token，清洗过程涉及去重（移除高度相似的文档）、毒性内容过滤、隐私信息脱敏、格式标准化等多个环节。这一过程需要设计高效的分布式处理流水线，在保证数据质量的同时控制计算成本。

3. 数据表征：架起数学世界与神经网络的桥梁

数据表征是将原始数学对象转化为模型可处理形式（通常是数值向量或张量）的关键步骤，也是 AI for Math 的核心挑战之一。一个好的表征应当保留数学对象的本质结构，同时便于神经网络处理。

文本与符号的表征是最常见的处理方式。对于数学表达式，如一阶逻辑公式 $\forall x \exists y (x + y = 0)$ ，我们首先通过分词器将其切分为离散的 token 序列：[“ $\forall$ ”，“ x ”，“ $\exists$ ”，“ y ”，“ $($ ”，“ x ”，“ $+$ ”，“ y ”，“ $=$ ”，“ 0 ”，“ $)$ ”]。这要求分词器能够理解数学符号的语法角色——例如区分作为变量的“ x ”和作为乘号的“ $\times$ ”。当前大语言模型广泛采用子词分词算法，如字节对编码（Byte-Pair Encoding, BPE），它能够从统计角度发现频繁出现的子词单元，有效处理罕见词汇和数学符号的组合。分词后的 token 序列通过嵌入层映射为密集向量，每个 token 被表示为一个高维向量，语义相近的 token 在向量空间中也彼此靠近。

图与组合结构的表征是处理离散数学对象的常用方式。一个图可以用邻接矩阵表示，其中元素 $A_{ij} = 1$ 表示节点 i 与 j 相连，否则为 0。对于带权图，邻接矩阵的元素可以是实数权重。对于多面体或更复杂的拓扑空间，其组合结构也可以用类似的 0-1 矩阵来描述——例如多面体的面-边关联矩阵。近年来兴起的图神经网络（Graph Neural Networks, GNN）专门处理这类结构，通过在图上进行消息传递和邻居聚合，学习节点、边乃至整个图的向量表示。

抽象代数结构的表征则更为困难，需要深厚的数学知识来设计。对于一个抽象群，我们无法直接将其输入神经网络，但可以计算其数值不变量——例如群元素的阶的分布、共轭类个数、特征标表等，将这些数值构成特征向量。对于一个流形，可以计算其拓扑不变量——如贝蒂数（各维同调群的秩）、同伦群、曲率积分等。另一种思路是将流形上的函数空间的系数作为表征，例如流形上拉普拉斯算子的特征函数（调和函数）的展开系数。这些表征的核心要求是在变换下保持结构的不变性——例如，无论我们如何重新标记群的元素，表征应该保持不变。

4. 数据增强：从有限数据中创造无限可能

当真实数据稀缺时，数据增强通过对现有数据进行合理的变换来人工扩展数据集，以提高模型的鲁棒性和泛化能力。其数学本质是在不改变数据标签“语义”的前提下，对输入空间施加一个保持不变的变换。

在真实数据稀缺的场景下，数据增强通过对现有数据进行合理的变换来人工扩展数据集，以提高模型的鲁棒性和泛化能力。其数学本质是：在不改变数据标签“语义”的前提下，对输入空间施加一个保持不变的变换。

在计算机视觉领域，数据增强已十分成熟——对图像进行旋转、翻转、裁剪、加噪声，这些变换不会改变图像类别（猫旋转后还是猫）。在数学任务中，数据增强同样可以发挥重要作用，其核心是利用数学对象的对称性、不变性和等价关系。

变量替换与缩放是最直观的增强方式。在代数表达式中，将变量名系统性地替换——例如将方程 $x^2 + y = 1$ 中的所有 x, y 替换为 a, b ，方程的解集结构保持不变。对等式两边同时乘以一个非零常数，或对方程组进行线性组合，也能生成新的等价形式。这利用了数学对象的同构关系——变量名只是标签，真正的数学结构在于变量间的相互作用。

等价形式重写则是利用数学恒等式进行变换。例如，将表达式 $\log(ab)$ 改写为 $\log a + \log b$ （在定义域内），或将 $\sin^2 x + \cos^2 x$ 改写为 1。这类增强教导模型关注表达式的等价类，而非具体的语法形式，有助于模型学习数学运算的代数规则。

逻辑变换在定理证明和逻辑推理任务中尤为有用。给定一个定理“若 A 则 B ”，我们可以生成其逆否命题“若非 B 则非 A ”作为新数据；对于“ A 当且仅当 B ”的等价关系，可以生成四个方向的蕴含关系。这类增强帮助模型理解逻辑结构的对称性。

参数扰动在数值计算和几何问题中效果显著。对于几何题，可以在合理范围内随机改变图形尺寸（如三角形边长）或角度，保持问题的几何关系不变；对于代数计算题，可以在数值稳定性允许的范围内扰动方程系数，生成新的实例。这迫使模型学习问题的结构特征而非具体的数值巧合。

数据增强是一种低成本注入数学知识（如对称性、不变性、等价关系）的有效手段，能够显著提升模型的泛化能力和数据效率。

二、模型初始化：训练旅程的起点

参数初始值决定了优化过程的起点。在非凸优化的广阔地形中，这个起点的选择至关重要——一个糟糕的起点可能导致模型收敛至性能较差的局部极小值，或在训练初期就遭遇数值不稳定的困境。正如登山者需要选择合适的营地开始攀登，模型初始化是为整个训练旅程奠定基础的关键一步。

1. 避免零初始化

直觉上，将所有权重和偏置初始化为零似乎是一个“无偏”的起点。然而，对于多层神经网络，这种初始化会导致严重的对称性问题，从而使模型无法有效学习。

以一个全连接层为例，若所有权重 $w_{ij} = 0$ ，则该层每个神经元的线性输入为

$$z_j = \sum_i w_{ij}x_i + b_j = b_j$$

可以看到，此时神经元的输出完全不依赖输入 (x)，而仅由偏置决定。

如果进一步假设所有偏置 b_j 也相同（例如均初始化为 0），那么该层所有神经元的输出将完全一致。在这种情况下，每个神经元在前向传播中产生相同的激活值；在反向传播中，由于它们接收到的误差信号和输入也完全相同，其梯度更新也将完全一致。因此，在整个训练过程中，这些神经元始终保持相同的参数值，学习到完全相同的特征。

即使偏置 b_j 不同，由于权重为零，神经元的输出仍然不包含任何输入特征的信息，网络在初始阶段等价于一个仅依赖常数偏移的模型，难以有效学习输入与输出之间的关系。

因此，问题的根本在于：当权重初始化为相同值（特别是零）时，模型无法打破神经元之间的对称性，导致多个神经元退化为功能等价的单元，从而显著降低模型的表达能力。这一现象被称为对称性未被打破（failure to break symmetry），也说明了为什么在实践中必须对权重进行随机初始化，以引入参数之间的差异性。

2. 随机初始化：尺度是关键

打破对称最直接的方法是随机初始化——从某个概率分布中为每个参数独立采样。然而，随机并不等于任意，尺度（scale）的选择成为决定成败的关键。假设我们从一个零均值、方差为 σ^2 的分布中采样权重 W 。考虑一个简单的前向传播： $z = Wx + b$ ，其中 x 是输入向量。若 σ 过大，权重值可能使 z 的幅值迅速膨胀，经过多层传播后激活值趋向无穷大，导致梯度爆炸；若 σ 过小，激活值则迅速收缩至零，导致梯度消失。这两种情况都会使训练陷入瘫痪——前者导致数值溢出，后者使参数更新几乎停滞。

那么，什么样的尺度才是“恰到好处”的？答案蕴含在方差分析之中。

3. 基于方差缩放的初始化

这类初始化方法的核心思想是：在初始化时，尽可能保持数据在前向传播中的激活值方差与反向传播中的梯度方差稳定。这本质上是设计一个精妙的统计平衡，让信号在网络中顺畅流动。

Xavier 初始化（也称为 Glorot 初始化）是这一思想的早期典范。它假设激活函数在原点附近是线性的且对称（如 $\tanh$ 函数），并推导出权重的方差应设置为：

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

其中 n_{in} 是输入神经元个数， n_{out} 是输出神经元个数。这一设置能够使前向传播中每层激活值的方差保持相等，同时反向传播中梯度的方差也保持相等。从均匀分布采样时，对应的范围是 $W \sim \mathcal{U}[-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}]$ 。

然而，Xavier 初始化针对的是线性激活函数。当 ReLU 成为主流后，研究者发现 Xavier 在 ReLU 网络上表现不佳。原因在于：ReLU 会将一半的负值输入置为零，使得输出的方差减半。为了补偿这一“信息损失”，He 初始化（也称为 Kaiming 初始化）应运而生：

$$\text{Var}(W) = \frac{2}{n_{\text{in}}}$$

对于均匀分布，对应范围为 $W \sim \mathcal{U}[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}]$ ；对于正态分布，则是 $W \sim \mathcal{N}(0, \frac{2}{n_{\text{in}}})$ 。He 初始化的直觉是：既然 ReLU 使方差减半，那就在初始化时将方差加倍，以维持整体方差的稳定。这一简单调整使得训练深层 ReLU 网络成为可能。

4. 正交初始化

方差缩放方法从统计角度保证了信号的稳定传播，而正交初始化则从几何视角提供了另一种优雅的思路。

考虑一个线性层 $y = Wx$ ，我们希望在整个网络中保持向量范数稳定——即 $\|y\|$ 与 $\|x\|$ 尽可能接近。如果 W 是正交矩阵（即 $W^T W = I$ ），那么对于任意向量 x ，有 $\|Wx\| = \|x\|$ 。正交矩阵完美保持了向量的范数，不会引入缩放效应。

将这一思想推广到神经网络初始化：我们将权重矩阵初始化为（近似）正交矩阵。具体实现时，可以从标准正态分布中采样一个随机矩阵，然后通过 QR 分解或奇异值分解将其正交化。虽然实际网络中的激活函数和非线性会破坏这种理想的范数保持性质，但正交初始化仍然极大缓解了梯度消失与爆炸问题，尤其适用于循环神经网络（RNN）和需要长期依赖建模的场景。

正交初始化的深层魅力在于，它不仅仅是数值技巧，更体现了对信息流动的几何理解——我们希望输入信号在网络中传播时，其“能量”得以保持，避免因逐层缩放而衰减或爆炸。这种几何直觉与方差缩放方法的统计直觉相辅相成，共同构成了现代深度学习初始化的理论基础。

三、训练过程优化：驾驭非凸地形

训练神经网络本质上是在高维非凸函数的地形上寻找最低点——这是一片布满山峰、山谷、鞍点和平原地带的复杂景观。训练过程优化的核心目标，就是让梯度下降及其变种在这片复杂地形上高效、稳定地行进，既能快速下降，又能避免陷入不良局部极值，最终抵达泛化良好的参数区域。

1. 优化算法家族：从朴素梯度下降到自适应方法

随机梯度下降与小批量梯度下降构成了所有现代优化算法的基础。与每次迭代使用全部数据的批量梯度下降不同，小批量梯度下降随机抽取一小批样本，用其梯度的平均值作为真实梯度的估计。设第 t 步的参数为 θ_{t-1} ，从训练集中随机采样的一个小批量样本集合为 B ，则梯度估计为：

$$g_t = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \mathcal{L}_i(\theta_{t-1})$$

其中 $|B|$ 表示小批量的大小， $\mathcal{L}_i$ 是第 i 个样本的损失函数值， $\nabla_{\theta} \mathcal{L}_i$ 表示损失函数对参数 θ 的梯度。这一做法带来了双重效果：一方面，计算成本大幅降低，使得在大规模数据集上的训练成为可能；另一方面，小批量采样引入的随机噪声实际上成为了优化过程的“有益扰动”——它能够帮助模型跳出较差的局部极小点，探索更广阔参数空间。小批量的大小体现了经典的偏差-方差权衡：大批量提供更准确的梯度估计（低方差），但计算成本高且可能陷入尖锐极小值；小批量引入更多噪声（高方差），但可能带来更好的泛化性能。

动量法的提出受到物理学的启发——想象一个小球从山坡滚下，它会累积速度，穿越平坦区域，抵抗局部颠簸的干扰。在数学上，动量法维护一个速度向量 v_t ，它是历史梯度的指数加权移动平均：

$$v_t = \beta v_{t-1} + \eta g_t, \quad \theta_t = \theta_{t-1} - v_t$$

其中 β 是动量衰减系数，控制历史信息的衰减速度； η 是学习率； g_t 是当前步的梯度。动量带来的效果是：在梯度方向一致的区域加速前进，在梯度方向震荡的区域平滑更新，从而有效缓解病态曲率问题，帮助模型穿越鞍点和平坦区域。

自适应学习率算法则将优化推向新的高度——它们为每个参数赋予独立的自适应学习率，根据参数的历史梯度信息动态调整更新步长。其中最杰出的代表是 Adam 算法。Adam 维护两个状态变量：梯度的一阶矩估计 m_t （即带动量的梯度均值）和二阶矩估计 v_t （即梯度平方的均值，反映梯度变化的幅度）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

其中 β_1 和 β_2 是衰减系数； g_t 是当前步的梯度； g_t^2 表示逐元素平方。由于 m_t 和 v_t 在初始阶段偏向于零，Adam 还引入了偏差校正步骤：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

最后更新参数为：

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

其中 η 是全局学习率， ϵ 是一个极小的常数用于防止除零。Adam 的精妙之处在于：对于梯度大且频繁变化的参数， $\sqrt{\hat{v}_t}$ 较大，有效学习率降低；对于梯度小且稳定的参数，有效学习率升高。这种自适应机制使 Adam 对超参数选择相对鲁棒，收敛速度快，成为当前最流行的默认优化器。当然，Adam 并非万能——在某些任务上（如计算机视觉的某些领域），经过精心调参的带动量随机梯度下降仍能取得更好的泛化性能。

2. 学习率调度

学习率是训练中最重要超参数之一，而学习率调度的核心理念朴素而深刻：训练初期使用较大学习率快速下降，探索广阔的参数空间；训练后期缩小学习率，在局部区域精细调整。

常见的学习率调度策略包括：

- 阶梯式衰减：每隔若干轮次将学习率乘以一个衰减因子（如 0.1）。这种策略简单有效，尤其适合长期训练。
- 余弦退火：按照余弦函数周期性地调整学习率，使学习率先缓慢下降，然后加速下降，最终趋于零。其平滑的衰减曲线有时能带来更好的收敛效果。
- 早停式调度：监控验证集性能，当性能停滞时主动降低学习率。这种自适应策略能够避免人为预设调度策略与训练动态不匹配的问题。

3. 梯度裁剪

在训练深度网络，尤其是循环神经网络或 Transformer 时，梯度爆炸是一个常见威胁——梯度范数可能指数级增长，导致参数更新步长过大，使损失函数瞬间冲至无穷大，训练崩溃。

梯度裁剪提供了一个简单而有效的解决方案。设梯度向量为 g ，其范数为 $\|g\|$ ，预设阈值为 c ，则裁剪后的梯度 $\tilde{g}$ 为：

$$\tilde{g} = \begin{cases} g, & \text{if } \|g\| \leq c \\ g \cdot \frac{c}{\|g\|}, & \text{if } \|g\| > c \end{cases}$$

这一操作不改变梯度的方向，仅将梯度的范数限制在阈值以内。可以将其理解为电路中的保险丝——当电流（梯度）过大时，保险丝熔断但保护了整个系统；梯度裁剪则是在梯度即将失控时，主动将其缩放回安全范围，保证训练的稳定性。

梯度裁剪的妙处在于，它承认了优化过程中的不确定性，并设置了一个“安全网”。在训练大语言模型时，梯度裁剪几乎是标配，既防止了梯度爆炸，又不会过度干扰正常的优化过程。

4. 课程学习

课程学习的灵感直接来源于人类的教学过程——我们不会让初学者直接面对最困难的题目，而是从简单概念开始，逐步增加难度。这一思想同样适用于机器学习：不让模型一开始就面对最困难的样本，而是从简单样本开始，逐步引入更复杂的案例。

课程学习的实现方式多种多样：

- 数据排序：根据难度指标（如句子长度、问题步骤数）对训练数据进行排序，初期采样简单样本，后期逐渐增加困难样本的比例。
- 动态采样：根据模型当前性能调整采样权重——表现好的样本类型可以降低采样概率，表现差的则提高概率。
- 渐进式任务：在强化学习中，可以先在简化环境中训练，再迁移到完整环境。

课程学习的数学直觉在于：简单样本提供了稳定的梯度信号，帮助模型在初期快速建立正确的表示；随着模型能力提升，困难样本的挑战性恰好与其当前水平匹配，避免了过早陷入不良局部极值。在数学推理任务中，课程学习尤为有效——例如，先让模型学习一元一次方程，再逐步引入二元一次方程组、二次方程、超越方程。

四、正则化：对抗过拟合，提升泛化

正则化的核心目标是减少过拟合，提高模型在未见数据上的泛化能力。其哲学本质是：对优化过程施加约束或干扰，引导模型走向更简单、平滑、一般化的解，而非仅仅记住训练数据。

1. 显式约束：L1 与 L2 正则化

最直观的正则化方式是在损失函数中添加参数的范数惩罚项，直接约束模型复杂度。L2 正则化（也称为权重衰减）在损失函数中加入权重的平方和。设原始损失函数为 $\mathcal{L}_{\text{original}}(\theta)$ ，则正则化后的损失为：

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}_{\text{original}}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

其中 λ 是正则化系数，控制惩罚的强度； $\|\theta\|_2^2 = \sum_i \theta_i^2$ 是参数向量的 L2 范数平方。其梯度更新变为：

$$\theta \leftarrow \theta - \eta \nabla \mathcal{L}_{\text{original}} - \eta \lambda \theta$$

可以看到，每一步更新都会额外减去一小部分权重值（ $\eta \lambda \theta$ ），驱使参数向零收缩。L2 正则化倾向于让所有

参数绝对值变小，使模型输出对输入变化的响应更加平滑，从而降低过拟合风险。从贝叶斯视角看，L2 正则化等价于对参数引入高斯先验。

L1 正则化则加入权重的绝对值之和：

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}_{\text{original}}(\theta) + \lambda \|\theta\|_1$$

其中 $\|\theta\|_1 = \sum_i |\theta_i|$ 是参数向量的 L1 范数。L1 正则化的独特之处在于，它倾向于驱使部分参数精确为零，实现特征选择，获得稀疏模型。这是因为 L1 范数在零点不可导，优化过程更容易将参数推至零点。L1 正则化等价于对参数引入拉普拉斯先验。

2. 隐式正则化：Dropout

Dropout 是深度学习中最富有想象力的正则化技术之一，它不直接修改损失函数，而是在训练过程中随机干扰网络结构。操作方式：在每次前向传播时，以概率 p （通常取 0.5）随机“丢弃”一部分神经元——即将这些神经元的输出暂时置为零。这意味着每次迭代都在训练一个不同的子网络，这些子网络共享权重。在测试时，所有神经元都被保留，但输出需要乘以 p ，以保持期望输出的一致性。直觉与效果：Dropout 的有效性来自多个层面：

- 防止协同适应：它强迫每个神经元不能过度依赖于少数其他特定的神经元，因为那些神经元可能在下次迭代中被丢弃。每个神经元必须学会与许多随机子集协同工作，提取的特征因此更加鲁棒和独立。
- 模型平均的近似：每次 dropout 相当于训练了一个不同的子网络。在测试时，使用完整的网络并配合权重缩放（乘以保留概率），可以近似地视为对这些指数级数量的子网络的输出进行了平均（在特定情况下对应几何平均）。类似于集成学习，而集成学习被证明能有效降低方差、提高泛化性能。
- 稀疏激活：Dropout 创造了一种稀疏的激活模式，与 L1 正则化有相似的精神——都是鼓励网络使用更少的特征来完成预测。

从信息论视角看，Dropout 可以理解为在训练过程中对网络结构施加随机扰动，是一种高效的数据无关的“数据增强”方式。

3. 早停法（Early Stopping）

早停法可能是最简单、最有效的正则化方法，其思想直白而深刻：在模型开始过度拟合训练数据之前停止训练。

具体做法：将数据分为训练集和验证集。在训练过程中，周期性地在验证集上评估模型性能（如损失或准确率）。当发现验证集上的误差不再下降，甚至开始上升时——尽管训练误差可能还在持续下降——就停止训练，并恢复到验证集性能最好的参数状态。

数学解释：在优化的早期阶段，参数朝着同时降低训练误差和测试误差的方向移动。模型在学习数据中的真实规律，这些规律在训练集和测试集上是一致的。然而，随着训练的持续，优化开始过度拟合训练数据中的噪声和特例，这些噪声只在训练集中存在。此时，参数向只降低训练误差但可能提高测试误差的方向移动，训练误差和验证误差开始背离。早停法本质上是在验证误差达到最小值时，截断优化迭代的次数，阻止这种背离的进一步恶化。

从优化视角看，早停法限制了参数在参数空间中移动的距离，类似于 L2 正则化对参数范数的约束。从贝叶斯视角看，它将参数先验集中在初始值附近。早停法的优雅之处在于，它不需要修改损失函数或网络结构，只需监控验证集性能即可获得正则化效果。

五、面向数学问题的大语言模型工程实践

将上述工程实践应用于数学大语言模型的训练，可以让我们更具体地理解这些技术如何协同工作，以及数学领域带来了哪些特殊挑战。

1. 数据准备：训练数学大语言模型需要海量、高质量的数学文本——从 arXiv 论文、数学教科书、Stack Exchange 讨论到代码库中的数学实现。这些数据需要经过精细的清洗，包括 LaTeX 表达式的规范化、数学符号的标准化、跨文档去重等。对于监督微调，需要精心构造数学问答对，不仅包含问题与答案，更要强调推理过程的呈现——思维链数据教导模型一步步推导，而非直接跳至结论。
2. 训练流程：如前所述，遵循“预训练 → 监督微调 → 强化学习”的三阶段流水线。在强化学习阶段，数学任

务的特殊性尤为突出——我们可以利用程序辅助执行，让模型生成可执行的 Python 代码来解决数学问题，然后通过代码解释器执行并验证结果。这提供了比人类偏好更客观、更可扩展的奖励信号：代码运行正确与否是确定的。还可以训练专门的验证器模型，评判生成答案的正确性和推理步骤的严谨性，作为强化学习中的奖励模型。

3. 正则化与稳定性：在数学推理中，过拟合可能表现为模型记住了特定问题的解法，而非学会通用的解题逻辑。Dropout 和早停法在此同样适用。此外，Transformer 架构本身的残差连接和层归一化已经内置了稳定训练的效果，减少了对某些显式正则化的依赖。
4. 课程学习：数学知识具有天然的层次结构——从算术到代数，从几何到微积分。这为课程学习提供了理想的应用场景。我们可以先让模型学习简单的四则运算，逐步引入方程、函数、导数、积分；或者在强化学习阶段，先让模型解决步骤较少的简单问题，再逐步增加问题难度。这种渐进式训练符合数学学习的认知规律，能够引导模型建立稳固的知识基础。

值得深思的是，这些工程实践并非孤立的技巧，而是一个相互关联的系统。好的初始化允许使用更大的初始学习率；残差连接和层归一化减少了梯度消失问题，也降低了对复杂正则化的依赖；早停法与学习率调度协同作用——当验证集性能停滞时，既可以停止训练，也可以降低学习率继续探索；而多阶段训练流程本身，就是一个宏观的“课程学习”——先让模型掌握基础语言能力（预训练），再学习遵循指令（监督微调），最后优化输出质量（强化学习）。

在 AI for Math 的探索中，结合具体数学问题的先验知识来设计和调整这些实践，是取得成功的关键。例如，设计数学特定的数据增强策略（如变量替换、等价形式重写），为物理信息神经网络设计特殊的损失函数，为数学推理模型设计基于程序验证的强化学习奖励。最终，所有这些工程实践的共同目标，是实现优化过程和模型行为的“控制与引导”——在有限的资源下，驯服复杂的非凸优化，引导模型走向具有强大泛化能力、可靠推理能力和数学问题解决能力的参数区域。

参考文献

- [1] Léon Bottou. “Large-scale machine learning with stochastic gradient descent”. In: *Proceedings of COMPSTAT'2010*. Springer. 2010, pp. 177–186.
- [2] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.
- [3] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [4] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. “Fast and accurate deep network learning by exponential linear units (ELUs)”. In: *arXiv preprint arXiv:1511.07289* (2015).
- [5] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2.4 (1989), pp. 303–314.
- [6] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2019, pp. 4171–4186.
- [7] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings*. 2010, pp. 249–256.
- [8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [9] Ian Goodfellow et al. “Generative adversarial nets”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.

- [10] Alex Graves. “Generating sequences with recurrent neural networks”. In: *arXiv preprint arXiv:1308.0850* (2013).
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising diffusion probabilistic models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [13] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [14] Phillip Isola et al. “Image-to-image translation with conditional adversarial networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 1125–1134.
- [15] Tero Karras, Samuli Laine, and Timo Aila. “A style-based generator architecture for generative adversarial networks”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 4401–4410.
- [16] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *International Conference on Learning Representations*. 2015.
- [17] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *Nature* 521.7553 (2015), pp. 436–444.
- [18] Christian Ledig et al. “Photo-realistic single image super-resolution using a generative adversarial network”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 4681–4690.
- [19] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proceedings of the International Conference on Machine Learning*. Vol. 30. 1. 2013, p. 3.
- [20] Vinod Nair and Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*. 2010, pp. 807–814.
- [21] Arkadi Nemirovski et al. “Robust stochastic approximation approach to stochastic programming”. In: *SIAM Journal on Optimization* 19.4 (2009), pp. 1574–1609.
- [22] Albert B Novikoff. *On convergence proofs on perceptrons*. Tech. rep. Stanford Research Institute Menlo Park CA, 1962.
- [23] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 27730–27744.
- [24] Alec Radford et al. *Improving language understanding by generative pre-training*. Tech. rep. OpenAI, 2018.
- [25] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 53728–53741.
- [26] Sebastian Ruder. “An overview of gradient descent optimization algorithms”. In: *arXiv preprint arXiv:1609.04747* (2016).
- [27] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536.
- [28] Andrew M Saxe, James L McClelland, and Surya Ganguli. “Exact solutions to the nonlinear dynamics of learning in deep linear neural networks”. In: *arXiv preprint arXiv:1312.6120* (2013).
- [29] Franco Scarselli et al. “The graph neural network model”. In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80.
- [30] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. 2016, pp. 1715–1725.

- [31] Zhihong Shao et al. “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [32] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. In: *The Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.
- [33] Ilya Sutskever et al. “On the importance of initialization and momentum in deep learning”. In: *International Conference on Machine Learning*. PMLR. 2013, pp. 1139–1147.
- [34] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT Press, 2018.
- [35] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.
- [36] Li-Ping Wang, Song Chen, Li-Nan Jiang, et al. “Parameter-Efficient Fine-Tuning in Large Language Models: A Survey of Methodologies”. In: *Artificial Intelligence Review* 58.8 (2025), p. 227.
- [37] Daniel M Ziegler et al. “Fine-tuning language models from human preferences”. In: *arXiv preprint arXiv:1909.08593* (2019).
- [38] 邱锡鹏. 神经网络与深度学习. 机械工业出版社, 2020.

第三章 AI 发现数学规律

3.1 概述

上一章对机器学习基本技术的介绍，为我们铺陈了一套强大的工具集。神经网络、深度学习、监督与无监督学习——这些方法在图像识别、自然语言处理等领域的成功，已充分证明它们从复杂数据中提取模式的能力。然而，当我们把目光转向数学研究，一个更深层的问题浮现出来：这些技术能否超越“高效计算器”的角色，真正参与到数学发现的核心过程中来？

要回答这个问题，我们需要重新审视数学研究本身的困境。数学规律的发现，历来依赖于两种核心能力的交织：直觉洞察与逻辑演绎。数学家通过对已知结构的深刻理解，产生关于未知关系的猜想，再通过严格的证明加以验证，形成“猜想-证明”的循环。然而，当数学研究的对象日益复杂——例如高维流形上的拓扑不变量、高次多项式环中的代数结构、非线性动力系统的长期行为——人类直觉的边界便开始显现。我们擅长想象三维空间的对称，却难以直观把握五十维空间的几何；我们能够推导低次方程的显式解，却难以洞察高维代数簇之间的隐含关联。

正是在这一困境中，人工智能展现出独特的价值。但这里的“人工智能”并非一个泛化的概念，而是指向一个正在形成的新兴学科领域：将机器学习作为认知手段，以数学结构本身为研究对象，以数学理解为核心目标。这一领域有着自己独特的方法论、理论关切和研究问题，它既不是 AI 技术的简单应用，也不是数学理论的被动延伸，而是一种真正意义上的交叉——一种新的“认知范式”的诞生。

这一新范式的核心在于：当我们面对一个超越人类直觉边界的数学问题时，我们可以借助 AI 的模式识别能力，从数学数据中“探测”出可能存在的规律信号；再通过可解释性技术，将这些信号“映射”回数学家可理解的概念，最终提炼为可供检验的数学猜想。这是一种 AI 辅助的数学发现——机器负责在高维空间中探索人类难以直接接触的结构，数学家则负责解释、提炼与证明。二者分工协作，共同拓展数学认知的边界。

这一协作逻辑，可以清晰地表述为：数学问题 $\rightarrow$ 适配的 AI 技术 $\rightarrow$ 具体实现过程。我们不从技术出发去寻找应用场景——那会导致工具性的堆砌；也不从理论出发去套用案例——那会遮蔽问题的本质。相反，我们从数学问题本身的困境出发，考察什么样的 AI 技术能够提供实质性的帮助。例如，面对“理解扭结几何不变量与代数不变量之间的关系”这一深层问题，其挑战在于从非结构化的数据中学习不同数学结构之间的映射关系——这恰恰是神经网络所擅长的。于是，我们设计实验，让 AI 在这些数据中寻找模式，再通过可解释性分析，揭示模型所依赖的关键特征，最终将这些特征转化为可供数学家检验的猜想。

这种以问题为导向的视角，揭示出 AI for Math 作为一种交叉学科的独特气质。在这里，AI 与数学的关系不再是单向的“应用”或“辅助”，而是一种深层的“协作”：AI 负责探测，数学家负责理解。

在接下来的几节中，我们将通过具体的案例，完整呈现这一协作范式。每一个案例都将遵循“数学问题 $\rightarrow$ AI 技术 $\rightarrow$ 实现过程”的叙事线索，展示 AI 如何作为一种新的方法论，帮助我们探索扭结不变量之间的深层关系、发现代数结构中的隐含规律、甚至预测某些数学猜想的存在性。这些案例不仅是技术的展示，更是一种新研究范式的见证——一个属于 AI for Math 的时代，正在悄然展开。

3.2 AI 发现扭结不变量之间的隐藏关系：基于监督学习

想象一根绳子在三维空间中打了一个复杂的结，两端再连接起来——这就是数学中的“扭结”，如何精确地描述和研究这个“结”？关键在于找到一些量，无论我们如何连续地拉伸、扭曲这根绳子（只要不剪断或不让它自身穿过），这些量都保持不变。这样的量被称为扭结不变量，它们就像是扭结的“指纹”，帮助我们区分不同的扭结，理解它们的深层结构。

主要的扭结不变量可以分为两大类：第一类是几何（双曲）不变量。对于绝大多数扭结，其补空间（即三维空间去掉扭结本身的那部分）可以被赋予一种非常优美的几何结构——双曲几何。就像一张马鞍面是二维双曲

空间，扭结的补空间可以是三维双曲空间。由此可以导出一系列精细的几何量，例如：

- 体积：双曲体积，度量该几何空间的大小，是扭结的一个基本几何特征。
- Chern-Simons 不变量：一个源自理论物理的拓扑不变量，与空间的“扭曲程度”有关。
- Cusp 几何：扭结本身可以想象成无限细的管子，其管状邻域的边界是一个环面，这个环面上的几何信息（如经向平移和纵向平移）编码了扭结在“无穷远处”的结构。
- 内射半径：衡量扭结补空间中“最窄处”的尺度，反映了几何上的“瓶颈”。

第二类是代数不变量，通过代数或组合方法定义，例如：

- 琼斯多项式：一个在 1984 年发现的多项式不变量，它的出现曾彻底改变了扭结理论。
- 符号差：一个整数不变量，它编码了扭结的许多重要代数性质，例如与扭结在四维空间中能否成为某个曲面的边界（即曲面的最小亏格）密切相关。

长期以来，数学家们一直隐约感到，这两个不同世界的不变量之间应该存在着深刻的联系。一个著名的例子是“体积猜想”，它大胆地提出：扭结的双曲体积可能隐藏在其琼斯多项式的某种渐近行为之中。然而，要发现新的、可严格证明的具体关系，却面临着巨大的挑战。原因在于，这些不变量之间的关系可能极其微妙，涉及高维空间中的非线性模式。从海量的、复杂的数据中，仅凭人类的观察和传统的统计方法，很难可靠地提取出具有数学价值的线索。我们擅长在三维空间中想象，却难以直观把握由多个不变量张成的高维空间中的结构。

这正是 AI 可以介入的节点。当数学问题的核心困境是“高维复杂性超出人类直觉”时，机器学习强大的模式识别能力便有了用武之地。

2021 年，DeepMind 团队在《Nature》杂志发表突破性成果，他们开发的 AI 系统能够帮助数学家发现扭结不变量之间的新联系。这项研究证明，AI 可以引导数学家的直觉，发现人类可能忽略的数学联系。整个研究遵循一条清晰的逻辑线索，我们可以将其拆解为四个关键步骤。

步骤 1：将数学猜想转化为机器学习问题与数据生成

任何探索都始于一个初始的直觉或假设。在这项研究中，数学家的初始假设是：“扭结的几何（双曲）不变量与其代数不变量（特别是符号差）之间存在某种尚未被发现的可预测关系。”为了检验这个假设，需要将其转化为一个机器学习问题。具体而言：

- 定义输入（ X ）：我们需要选择一组几何不变量，作为描述每个扭结的特征向量。研究人员选择了体积、Chern-Simons 不变量、描述 Cusp 几何的四个量（经向平移的实部和虚部、纵向平移的实部和虚部）、内射半径等。这样一来，每个扭结 K 就被表示为一个数值向量 $X(K)$ 。
- 定义输出（ Y ）：我们需要选择一个代数不变量作为预测目标。这里选择了符号差 $\sigma(K)$ 。选择符号差是明智的：它是一个整数，便于建模为分类问题；它易于计算；而且它蕴含丰富的数学信息，是一个“值得被理解”的量。
- 构建数据集：机器学习需要数据。研究人员构建了三个互补的数据集，以确保后续发现的稳健性和泛化能力：
 - 系统普查数据：来自“Regina”数据库，包含所有交叉数（衡量扭结复杂度的指标）不超过 16 的扭结，共计约 170 万个样本。这个数据集提供了对小而简单的扭结的“系统性覆盖”。
 - 随机复杂数据：使用专业扭结软件 SnapPy，随机生成具有约 80 个交叉点的复杂扭结，经过去重后得到约 100 万个样本。这个数据集代表了更复杂、更“一般”的扭结分布，用于验证发现的泛化能力。
 - 特定构造数据：通过控制编织参数，构造了数万个具有特定代数结构的扭结（如 4-编织、5-编织、6-编织）。这个数据集不用于训练，而是专门用于后续对猜想的“压力测试”——看看发现的规律在这些特殊扭结上是否仍然成立。

至此，一个抽象的数学直觉被转化为了一个具体的机器学习问题：给定一个扭结的几何特征向量 $X(K)$ ，能否训练一个模型，让它准确预测出这个扭结的符号差 $\sigma(K)$ ？

步骤 2：监督学习与模式探测

接下来，使用神经网络对准备好的数据集进行训练。在扭结案例中，研究人员采用了标准的全连接前馈神经网络。这是一个多分类问题，因为符号差是整数。模型的性能评估基于其在独立测试集上的分类准确率。

实验结果令人振奋：训练好的模型在预测符号差时，测试准确率达到了 78%，且预测错误从未超过 ± 2 。相

比之下，一个随机猜测的基线准确率远低于此（例如，在符号差取值范围内均匀猜测）。这个结果强烈暗示，在扭结的几何特征与其符号差之间，存在着一个可被机器学习模型捕捉到的、稳健的数学模式。这给了数学家继续深入探索的信心——他们的直觉方向很可能是有价值的。

步骤 3：可解释性分析与线索提取

模型的成功预测只是一个开始。更重要的是：模型是“如何”做出预测的？它依赖了哪些几何特征？这些特征之间是如何相互作用的？回答这些问题是将机器学习输出转化为数学猜想的关键。

为此，研究人员使用了一种名为“基于梯度的显著性分析”的可解释性技术。其核心思想非常直观：我们想知道每个输入特征对模型最终预测的“贡献”有多大。具体来说，对于数据集中的每个样本，我们计算模型的损失函数对每个输入特征的偏导数的绝对值 $|\partial L / \partial x_i|$ 。这个值的直观含义是：如果我们对某个特征 x_i 做一个微小的改变，模型的预测损失会发生多大的变化？变化越大，说明模型对这个特征越“敏感”，这个特征在决策中就越重要。

对于数据集 X 中的每个样本 x ，计算模型损失函数 L 相对于第 i 个输入特征 x_i 的梯度的绝对值 $|\partial L / \partial x_i|$ 。这个值反映了，如果稍微改变特征 x_i ，模型的预测损失会有多大变化。变化越大，说明模型对该特征越敏感，该特征可能越重要。

为了得到每个特征在整个数据集上的总体重要性，我们对所有样本的梯度绝对值取平均：

$$r_i = \frac{1}{|X|} \sum_{x \in X} \left| \frac{\partial L(x)}{\partial x_i} \right|$$

其中， r_i 就是第 i 个特征的平均梯度显著性分数， $|X|$ 表示数据集的样本总数。

对扭结模型进行上述显著性分析后，得到了清晰的模式。分析指出，在所有几何不变量中，有三个特征脱颖而出，其 r_i 值显著高于其他特征，对预测符号差至关重要：

- 经向平移的实部 $Re(\mu)$
- 经向平移的虚部 $Im(\mu)$
- 纵向平移的实部 $Re(\lambda)$

这三个量共同描述了扭结补空间中“Cusp”（无限细管状邻域的边界环面）的几何形状，即所谓的 Cusp 几何。换句话说，模型告诉我们：要预测符号差，最重要的信息几乎全部集中在 Cusp 的几何上。

为了验证这一发现，研究人员训练了第二个模型，这次只使用这三个 Cusp 几何特征作为输入。结果令人震惊：这个简化模型的预测准确率，与使用全部十几个几何特征的原模型几乎完全相同！这无疑是一个决定性的证据：关于符号差的信息，几乎完全浓缩在这三个 Cusp 几何量之中。这一步骤将数学家的注意力，从十几个可能的几何不变量，急剧聚焦到了三个核心特征上。问题被极大地简化了。

步骤 4：数学提炼与猜想/定理形成

现在，机器学习的任务已经完成。它探测到了模式，定位了关键特征。接下来，是数学家发挥其不可替代的创造性智慧的时刻。他们拿到 AI 提供的线索——“Cusp 几何是关键”——开始运用深厚的数学直觉和专业学识，进行深度分析和再创造。

首先，他们可视化这些关键特征与符号差的关系。观察发现，符号差与这些量之间并非简单的线性关系，而是呈现出某种复杂的模式。经过对数据分布的反复审视和对几何意义的深入挖掘，数学家们创造性地定义了一个全新的、复合的几何量，他们称之为自然斜率：

$$\text{slope}(K) = Re(\lambda/\mu)$$

这个新定义的量有着清晰的几何解释：在 Cusp 的欧几里得环面上，有一条特殊的曲线（从经线出发的测地线），当它绕行环面后再次与经线相交时，它走过的路径相当于若干倍经线加上一条纬线。这个“倍数”，就是自然斜率。它是一个由 Cusp 几何直接导出的、具有明确几何意义的量。

更令人兴奋的发现随之而来：当把符号差 $\sigma(K)$ 与这个新定义的自然斜率 $\text{slope}(K)$ 放在一起观察时，两者之间呈现出强烈的线性趋势！这促使数学家们提出了第一个具体的数学猜想：

初始猜想: 存在常数 c_1 和 c_2 , 使得对所有双曲扭结 K , 有

$$|2\sigma(K) - \text{slope}(K)| < c_1 \cdot \text{vol}(K) + c_2$$

这个猜想简洁而优美, 将代数不变量 (符号差) 与一个全新的几何不变量 (自然斜率) 及另一个基本几何量 (体积) 联系了起来。然而, 科学发现的旅程从来不是一帆风顺的。研究人员利用之前构建的“特定构造数据” (那些由编织构造的特殊扭结) 对这个猜想进行压力测试。结果发现, 在某些特殊扭结上, 这个猜想并不成立。反例的出现, 并没有宣告探索的失败, 反而引导数学家们进行更精细的分析。他们注意到, 在显著性分析中, 还有一个特征——内射半径 $\text{inj}(K)$ ——虽然不如 Cusp 几何那么突出, 但也显示出一定的重要性。结合这个线索, 他们最终提出了一个修正后的猜想:

最终猜想: 存在一个常数 c , 使得对所有双曲扭结 K , 有

$$|2\sigma(K) - \text{slope}(K)| \leq c \cdot \text{vol}(K) \cdot \text{inj}(K)^{-3}$$

随后, 研究人员在另外一篇数学论文中严格证明了上述猜想, 它成为了一个可靠的数学定理。这个定理具有直接的应用, 例如, 它隐含了符号差控制着扭结上的非双曲 Dehn 手术, 而自然斜率控制着 $\mathbb{R}_4^+$ 中以该扭结为边界的曲面的亏格。一个由 AI 辅助发现的新数学真理, 就此诞生。

回顾整个扭结研究案例, 我们可以清晰地看到一条贯穿始终的逻辑主线, 它确保了 AI 的辅助始终服务于数学发现的核心目标:

- **对象定义:** 研究的核心对象是双曲扭结 K 。这是整个探索的出发点。
- **表示学习:** 将抽象的扭结结构, 通过其几何不变量, 转化为机器学习模型可以处理的数值向量 $X(K)$ 。同时, 关心的目标属性是其代数不变量符号差 $Y(K)$ 。这一步是连接数学世界与数据世界的桥梁。
- **函数逼近:** 将“发现关系”这一模糊目标, 转化为一个明确的监督学习问题: 寻找一个函数 f , 使得 $f(X(K))$ 能够很好地预测 $Y(K)$ 。神经网络充当了这个未知函数 f 的高维近似器。
- **结构反演:** 通过可解释性技术, 对训练好的神经网络进行“探查”, 逆向工程出它做出决策所依赖的关键数学结构 (即 Cusp 几何)。这相当于将模型学习到的复杂高维模式, “翻译”回数学家可以理解的、具有明确几何意义的特征子集。
- **提出猜想:** 基于识别出的关键结构, 数学家运用领域知识, 创造新的复合概念 (自然斜率), 并构建出精确的、可用传统数学语言表述的猜想, 最终通过证明将其确立为定理。这完成了从“数据中的模式”到“数学中的陈述”的决定性跃迁。
- **验证与修正:** 利用特殊构造的数据和数学证明, 对猜想进行严格的验证和修正, 最终形成经得起推敲的数学成果。

这条主线强调, AI 的角色不是取代数学家的推理, 而是作为一个强大的模式探测器和聚焦透镜。它处理人类不擅长的海量、高维数据分析, 并指出值得深入挖掘的“矿脉”所在, 而最终的“采矿”和“冶炼” (即概念的创造、猜想的提出与定理的证明) 工作, 则由数学家凭借其深刻的专业直觉和严格的逻辑推理来完成。确切说来, AI 提供的线索引导数学家进行了以下创造性工作:

1. **定义新概念:** 基于关键特征 μ 和 λ , 数学家定义了一个新的、有几何意义的复合量——自然斜率: $\text{slope}(K) = \text{Re}(\lambda/\mu)$ 。
2. **提出并修正猜想:** 观察数据趋势后, 首先提出符号差 $\sigma(K)$ 与自然斜率存在近似线性关系并受体积约束的猜想。随后, 利用 AI 在另一数据集上的表现发现了该猜想的反例。
3. **猜想最终结论:** 结合另一个重要性较高的特征 (内射半径 $\text{inj}(K)$), 数学家成功猜出了如下结论:

$$|2\sigma(K) - \text{slope}(K)| \leq c \cdot \text{vol}(K) \cdot \text{inj}(K)^{-3}$$

在上述研究工作中, AI 模型的工作流程可以概括如下:

- **数据来源:** 构建多层次的数据基础
任何数据驱动的探索都始于数据。为了确保后续发现的全面性和稳健性, 研究人员构建了三个互补的数据集, 它们各自扮演着不同的角色:

- 系统普查数据：来自“Regina”数据库，包含所有交叉数（衡量扭结复杂度的常用指标）不超过 16 的扭结，共计约 170 万个样本。这个数据集的价值在于其“系统性”——它覆盖了所有结构相对简单的扭结，为研究提供了完整的基准。
- 随机复杂数据：使用专业扭结软件 SnapPy，随机生成具有约 80 个交叉点的复杂扭结，经过去重后得到约 100 万个样本。这个数据集代表了更复杂、更“一般”的扭结分布，用于验证发现的泛化能力——即发现的规律是否在更广泛的扭结族中依然成立。
- 特定构造数据：通过控制编织参数，构造了数万个具有特定代数结构的扭结（如 4-编织、5-编织、6-编织）。这个数据集不用于模型的初始训练，而是专门用于后续对猜想的“压力测试”——看看发现的规律在这些特殊构造的扭结上是否仍然成立，甚至能否找到反例。

这三个数据集在规模和分布上形成了有机的互补：普查数据提供了系统性覆盖，随机数据提供了泛化性验证，而构造数据则用于针对性的边界测试。这种多层次的数据策略，是确保后续机器学习发现具有统计稳健性和数学意义的重要基础。

- 输入与输出：将数学问题翻译为机器学习任务

有了数据，下一步是将数学问题“翻译”成机器学习能够理解的语言：

- 输入特征 (X)：每个扭结被表示为一个数值向量，由多个几何不变量构成。这些特征包括体积、Chern-Simons 不变量、描述 Cusp 几何的四个量（经向平移的实部 $Re(\mu)$ 和虚部 $Im(\mu)$ ，纵向平移的实部 $Re(\lambda)$ 和虚部 $Im(\lambda)$ ）、内射半径等。这个向量 $X(K)$ ，就是模型用来“观察”扭结的视角。
- 输出标签 (Y)：模型的预测目标是扭结的代数不变量——符号差 $\sigma(K)$ 。符号差是一个整数，因此这个问题被建模为一个多分类任务。

- 网络结构与模型：选择适配的工具

面对这个预测任务，研究人员选择了最经典也最成熟的工具：

- 模型类型：采用标准的全连接前馈神经网络。这种网络结构虽然简单，但理论上可以逼近任意复杂的函数关系，足以应对未知数学模式的探测需求。
- 训练任务：以几何特征向量为输入，以符号差的类别为输出，进行监督学习。模型的目标是学习一个从几何特征到符号差的映射函数 f ，使得 $f(X(K))$ 尽可能接近真实的 $\sigma(K)$ 。

- 可解释性分析：打开黑箱，定位关键

然而，模型的高预测准确率只是一个开始。更重要的是回答“为什么”——模型究竟是依赖哪些几何特征做出判断的？这一步，是将机器学习输出转化为数学洞察的关键。

- 分析方法：研究人员采用了基于梯度的显著性分析。计算损失函数对每个输入特征的梯度绝对值，并在数据集上平均，得到每个特征的“重要性分数”。
- 关键发现：分析结果显示，在所有几何特征中，有三个特征的显著性分数远远高于其他特征——它们像三座凸起的山峰，分别是经向平移的实部 $Re(\mu)$ 、经向平移的虚部 $Im(\mu)$ 、纵向平移的实部 $Re(\lambda)$ 。仅使用这三个特征重新训练模型，准确率与使用全部特征相当，证实了信息的浓缩性。

- 方法的边界：审慎看待 AI 的“发现”

在同一篇论文中，研究人员还将这套方法成功应用到了表示论领域，进一步验证了其普适性。然而，尽管这项研究取得了令人瞩目的成功，我们也必须清醒地认识到这种 AI 辅助发现方法的固有局限：

- 数据依赖性与表示偏差：模型学到的全部知识都来源于训练数据。如果数据未能覆盖某些重要类型的扭结，或者所选择的特征表示未能有效刻画扭结的本质属性，模型就可能得出片面甚至误导性的结论。
- 相关性与因果性的鸿沟：机器学习识别的是统计相关性，而非数学必然性。一个高精度的预测模型，仅仅表明在观测数据中，某些几何特征与符号差之间存在稳定的关联，但这并不能保证这种关联是一条普适的数学定理。最终的确认，必须依赖于严格的数学证明。
- 可解释性技术的间接性：梯度显著性等方法提供的“重要性”信息，本质上是一种基于模型行为的间接推断，而非对数学结构的直接解读。它提供的线索需要结合领域知识进行审慎判断和交叉验证，不能盲目接受。
- 从模式到猜想的创造性跳跃：AI 输出的是数据和特征重要性，而将其转化为一个形式优美、条件恰

当的数学猜想，是一个高度依赖数学家创造力和专业素养的过程。本研究从初始猜想到最后定理的修正，充分体现了这种创造性跳跃的复杂性和不可替代性。

- 计算成本与可复现性：大规模的数据生成和模型训练需要可观的计算资源。一项完整的研究必须详细记录数据生成的协议、模型参数和随机种子，以确保其发现可以被其他人复现和验证。

这些局限并非否定 AI 的价值，而是为我们划定了一条审慎的边界：AI 是强大的助手，但还不是万能的先知。它的发现需要被理解、被检验、被证明。

3.3 AI 发现新算法：基于强化学习和 LLM

矩阵乘法是计算机科学与数学中最基础、最核心的计算任务之一。从科学计算、线性代数、信号处理到现代人工智能中的神经网络，无数复杂计算最终都可以归结为大规模的矩阵乘法操作。因此，改进矩阵乘法的效率，即使只是微小的理论或实践提升，也能对广泛的科技领域产生巨大的“乘数效应”。

然而，自 1969 年德国数学家沃尔克·施特拉森（Volker Strassen）提出著名的 Strassen 算法以来，对于固定尺寸矩阵乘法算法的实质性改进已停滞了数十年。Strassen 算法的核心洞察是：计算两个 2×2 矩阵的乘积，不一定需要 8 次标量乘法（标准的“点积”方法），而是可以通过一种巧妙的组合方式，仅用 7 次乘法完成。这个算法的思想可以递归地应用于更大的分块矩阵，将一个 $N \times N$ 矩阵乘法的时间复杂度从经典的 $O(N^3)$ 降低到约 $O(N^{\log_2 7}) \approx O(N^{2.81})$ 。

这一突破激发了计算复杂性理论中一个名为“矩阵乘法指数”的深奥研究方向，即寻找能够将矩阵乘法复杂度降至 $O(N^\omega)$ 的最小指数 ω 。经过半个多世纪的研究，目前理论上的最好结果是 $\omega < 2.37286$ ，但这些渐近最优的构造通常是高度复杂的“存在性证明”或非显式方法，并不直接对应我们能在实际硬件中高效运行的显式算法。

从实践角度看，一个更根本、也更令人困惑的问题是：对于给定的固定尺寸的矩阵（例如 $3 \times 3, 4 \times 4, 5 \times 5$ ），完成乘法所需的最少乘法次数是多少？这个问题看似基础，答案却极难确定。例如，对于 3×3 矩阵，目前已知的最佳显式算法由拉德曼（J. Laderman）于 1976 年发现，需要 23 次乘法。尽管经过了近 50 年的努力，我们仍然不知道这是否就是最优解。对于 4×4 矩阵，最知名的方法是递归应用两次 Strassen 算法（即所谓的 Strassen-square 算法），需要 $7^2 = 49$ 次乘法，但这是否最优，同样未知。

为什么这样一个看似简单、定义明确的问题如此困难？其根源在于，寻找一个高效的矩阵乘法算法，本质上等同于寻找一个特定三维“张量”（一个多维数组）的低秩分解。这个张量称为矩阵乘法张量，它完全编码了“乘法”这一双线性运算。寻找矩阵乘法算法，就是寻找一组简单的“基元”（秩一张量），其和恰好等于这个复杂的矩阵乘法张量。每个基元对应算法中的一次乘法操作。因此，算法所需的乘法次数，就是该张量分解的秩。

将这个数学问题转化为搜索问题，其困难立刻显现。以 4×4 矩阵为例，其矩阵乘法张量 $\mathcal{T}_4$ 的大小是 $16 \times 16 \times 16$ 。若在某个有限集合（例如 $F = \{-2, -1, 0, 1, 2\}$ ）中搜索其秩为 R 的分解，可能的组合数量是天文数字。具体来说，每个分解由 $3R$ 个长度为 16 的向量组成。对于 $\mathcal{T}_4$ ，其动作空间（可能的秩一因子组合）比 $\mathcal{T}_3$ 大 10^{10} 倍，远超国际象棋或围棋等传统游戏。其搜索空间之大，远超任何基于穷举或传统组合搜索的人类方法和计算机方法。因此，矩阵乘法算法的发现长期依赖于数学家的精巧构造、特定领域知识的启发式搜索或连续的数值优化后人工调整取整，这些方法往往带有强烈的个人色彩，且难以推广和规模化。

这就引出了本节的核心问题：我们能否利用现代人工智能技术，自动化地探索这个巨大的算法空间，发现超越人类直觉的高效、正确的新算法？这个问题不仅关乎矩阵乘法本身，更代表着一种新的范式——将“算法发现”本身视为一个可以由机器来探索和优化的过程。

AlphaTensor 和 AlphaEvolve 正是这一范式下的两个标志性工作。它们从不同的技术路径出发——前者基于深度强化学习将问题游戏化，后者基于大语言模型引导的程序进化——共同证明了人工智能在自动发现数学算法和构造方面的巨大潜力，并在一个困扰学界数十年的经典问题上取得了实质性突破。

要深入理解 AlphaTensor 和 AlphaEvolve 的工作，我们必须先掌握“算法即张量分解”这一核心数学思想。这是连接具体计算问题与抽象搜索空间的桥梁。

矩阵乘法 $\mathbf{C} = \mathbf{AB}$ 是一个双线性运算：输出 $\mathbf{C}$ 中的每个元素 c_{ij} 对输入 $\mathbf{A}$ 和 $\mathbf{B}$ 都是线性的。任何双线性运算都可以由一个三维张量唯一确定。

具体来说，考虑 2×2 矩阵的乘法：

$$\mathbf{C} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

这里， $c_{11} = a_{11}b_{11} + a_{12}b_{21}$ ， $c_{12} = a_{11}b_{12} + a_{12}b_{22}$ ，依此类推。

我们可以将输入矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 的元素展平为向量。例如，按行优先顺序： $\mathbf{a} = (a_{11}, a_{12}, a_{21}, a_{22})^T = (a_1, a_2, a_3, a_4)^T$ ，类似地有 $\mathbf{b} = (b_1, b_2, b_3, b_4)^T$ 和 $\mathbf{c} = (c_1, c_2, c_3, c_4)^T$ 。

那么， $c_1 = c_{11} = a_1b_1 + a_2b_3$ 。这个关系可以用一个 $4 \times 4 \times 4$ 的张量 $\mathcal{T}_2$ 来完全编码：定义 $\mathcal{T}_2$ 的元素 $\mathcal{T}_{i,j,k}$ ，使得

$$c_i = \sum_{j=1}^4 \sum_{k=1}^4 \mathcal{T}_{i,j,k} \cdot a_j b_k$$

对于 c_1 的计算，需要 $\mathcal{T}_{1,1,1} = 1$ 和 $\mathcal{T}_{1,2,3} = 1$ ，其余 $\mathcal{T}_{i,j,k} = 0$ 。这个张量 $\mathcal{T}_2$ 就是 2×2 矩阵乘法的矩阵乘法张量。它是一个元素仅为 0 或 1 的稀疏张量，其非零项的位置精确描述了哪些输入项的乘积会贡献到哪个输出项。

更一般地， $n \times n$ 矩阵乘法对应一个大小为 $n^2 \times n^2 \times n^2$ 的张量 $\mathcal{T}_n$ 。类似地， $n \times m$ 矩阵与 $m \times p$ 矩阵的乘法对应张量 $\mathcal{T}_{n,m,p}$ 。

理解了张量如何编码矩阵乘法，下一个问题是：如何分解这个张量？

首先，我们需要定义最基本的构建块——秩一张量（rank-one tensor）。一个三维张量如果能够写成三个向量的外积，就称为秩一张量。具体来说，给定三个向量 $\mathbf{u} \in \mathbb{R}^I$ 、 $\mathbf{v} \in \mathbb{R}^J$ 、 $\mathbf{w} \in \mathbb{R}^K$ ，它们的外积 $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ 产生一个 $I \times J \times K$ 的三维张量，其 (i, j, k) 位置的元素为 $u_i v_j w_k$ 。

直观上，这是一个最简单的、不可再分解的张量基元。

如果一个张量 $\mathcal{T}$ 可以表示为 R 个秩一张量之和：

$$\mathcal{T} = \sum_{r=1}^R \mathbf{u}^{(r)} \otimes \mathbf{v}^{(r)} \otimes \mathbf{w}^{(r)}$$

那么我们说 $\mathcal{T}$ 的张量秩（tensor rank）至多是 R ，记作 $\text{Rank}(\mathcal{T}) \leq R$ 。这是矩阵秩概念在高维的自然推广。矩阵的秩是其列向量（或行向量）张成空间的最小维度，而张量的秩则是其分解为秩一项（外积）所需的最少项数。

这里出现了一个深刻而优美的联系：

定理 3.1

矩阵乘法张量 $\mathcal{T}_n$ 的一个秩 R 分解，直接对应一个计算两个 $n \times n$ 矩阵乘积的算法，该算法恰好使用 R 次标量乘法。

这个对应关系如何建立？让我们一步步推导。

给定分解 $\mathcal{T}_n = \sum_{r=1}^R \mathbf{u}^{(r)} \otimes \mathbf{v}^{(r)} \otimes \mathbf{w}^{(r)}$ 。根据张量定义，对于任意输入 $\mathbf{a}, \mathbf{b}$ ，输出 $\mathbf{c}$ 满足：

$$c_i = \sum_{j,k} \mathcal{T}_{i,j,k} a_j b_k = \sum_{j,k} \left(\sum_{r=1}^R u_i^{(r)} v_j^{(r)} w_k^{(r)} \right) a_j b_k = \sum_{r=1}^R u_i^{(r)} \left(\sum_j v_j^{(r)} a_j \right) \left(\sum_k w_k^{(r)} b_k \right)$$

这个表达式直接给出了一个算法：

1. 对于 $r = 1$ 到 R ，计算：

$$m_r = \left(\sum_j v_j^{(r)} a_j \right) \cdot \left(\sum_k w_k^{(r)} b_k \right)$$

这里，括号内是输入元素的线性组合（只有加法），然后两者相乘——这是一次标量乘法。总共 R 次这样的乘法。

2. 对于 $i = 1$ 到 n^2 ，计算：

$$c_i = \sum_{r=1}^R u_i^{(r)} m_r$$

这一步只涉及加法和标量乘法（乘以系数 $u_i^{(r)}$ ），不引入新的乘法。

这个算法恰好使用了 R 次乘法。向量 $\mathbf{u}^{(r)}$ 、 $\mathbf{v}^{(r)}$ 、 $\mathbf{w}^{(r)}$ 的系数定义了算法中具体的线性组合。例如，Strassen 的经典算法正是 $\mathcal{T}_2$ 的一个 $R = 7$ 的分解，其对应的 7 组 $(\mathbf{u}^{(r)}, \mathbf{v}^{(r)}, \mathbf{w}^{(r)})$ 定义了那七个著名的中间乘积 $M_1, \dots, M_7$ 。

更重要的是，一个小尺寸矩阵的快速算法可以通过“分块矩阵”的思想递归地应用于大矩阵。例如，将一个大 $N \times N$ 矩阵视为由 2×2 的子块组成，那么对每个子块应用 Strassen 的 2×2 算法（现在视作对“块”进行运算，每次块运算需要 7 次标量乘法），就可以递归地将总复杂度降至 $O(N^{\log_2 7}) \approx O(N^{2.81})$ 。

更一般地，如果一个 $n \times n$ 矩阵乘法算法使用 R 次乘法（即 $\text{Rank}(\mathcal{T}_n) \leq R$ ），那么通过递归分块，两个 $N \times N$ 矩阵相乘的渐近复杂度为 $O(N^{\log_n R})$ 。因此，寻找更低的 R ，直接影响着矩阵乘法的理论极限。

寻找张量的低秩分解是一个众所周知的难题。对于矩阵（二维张量），计算其秩可以在多项式时间内解决（例如通过奇异值分解）。但对于三维及以上的张量，计算其精确秩是 NP-hard 问题，在实践中也极为困难。

因此，算法的发现长期依赖于数学家的灵光一现、针对特定结构的启发式搜索或连续的数值优化后人工调整取整。这些方法效率低、可扩展性差，且严重依赖人类设计的、可能并非最优的启发式规则。这正是人工智能方法可以大显身手之处：自动化搜索这个人类直觉难以触及的广阔空间。

DeepMind 的 AlphaTensor 工作，其核心创新在于将张量分解这一 NP-hard 问题巧妙地形式化为一个单人游戏（称为 TensorGame），并利用基于 AlphaZero 的深度强化学习技术来玩这个游戏，从而自动化地搜索高效的分解。

TensorGame 的设计精妙地将“寻找低秩分解”这一数学目标转化为序列决策问题：

- 状态：游戏状态 S_t 是一个三维张量。初始状态 S_0 正是我们想要分解的目标张量 $\mathcal{T}$ （例如 4×4 矩阵乘法对应的张量 $\mathcal{T}_4$ ）。
- 动作：在每一步 t ，玩家（即 AI 智能体）选择一个动作——这个动作对应一个秩一张量，由三个向量 $(\mathbf{u}^{(t)}, \mathbf{v}^{(t)}, \mathbf{w}^{(t)})$ 的外积构成。这些向量的元素必须来自一个预设的离散集合 F ，例如 $\{-2, -1, 0, 1, 2\}$ 。离散化的目的很实际：保证最终发现的算法系数简洁、可解释，并避免浮点数精度带来的麻烦。
- 状态转移：执行动作后，状态更新为当前张量减去这个秩一张量：

$$S_t = S_{t-1} - \mathbf{u}^{(t)} \otimes \mathbf{v}^{(t)} \otimes \mathbf{w}^{(t)}$$

这个操作的直观含义是：“从剩余待分解的部分中移除一个基元”。每一步都在逐步“消解”目标张量。

- 终止与奖励：游戏的目标是在尽可能少的步数内将张量归零，即达到 $S_R = \mathbf{0}$ 。此时，动作序列就构成了一个合法的分解：

$$\mathcal{T} = \sum_{t=1}^R \mathbf{u}^{(t)} \otimes \mathbf{v}^{(t)} \otimes \mathbf{w}^{(t)}$$

这里的步数 R 正是分解的秩，对应算法的乘法次数。

奖励设计直接服务于这个目标：每一步给予 -1 的奖励。因此，整局游戏的总回报是 $-R$ ，最大化回报等价于最小化步数 R ——也就是找到尽可能低秩的分解。

如果游戏在预设的最大步数 R_{limit} 内未能归零，则根据剩余张量 $S_{R_{\text{limit}}}$ 的秩的一个上界给予额外惩罚。这引导智能体即使无法完全分解，也要尽可能降低剩余部分的复杂度。

这种形式化的一个关键优势是它的灵活性。奖励函数可以轻松修改以优化其他目标，而不仅仅是理论上的乘法次数。例如，可以在游戏结束时，将动作序列对应的算法在特定硬件上运行，并用其负运行时间作为附加奖励——从而直接发现为 GPU、TPU 等特定硬件定制化的、实践高效的算法。

AlphaTensor 建立在著名的 AlphaZero 框架之上，其核心是一个神经网络的蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）规划器。MCTS 是一种通过随机模拟与价值估计结合的树搜索策略，用于在大动作空间中高效地选择有希望的动作。

- 神经网络 f_θ ：它输入当前状态（张量 S_t ），输出两个关键信息：
 - 策略 π ：一个在所有可能动作 $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ 上的概率分布。它预测哪些动作更有希望导向好的分解。由于动作空间巨大，AlphaTensor 采用了 Sampled AlphaZero 的策略，让网络输出一个相对紧凑的候选动作集，而不是枚举全部。
 - 价值 z ：一个对未来回报（累积奖励）分布的估计。这本质上是网络对“从当前状态出发，最终能以多小的秩完成游戏”的信念。采用分布而非单一值，可以更好地捕捉不确定性。
- 蒙特卡洛树搜索：在每一步，AlphaTensor 并不直接使用神经网络输出的策略，而是以该策略和价值为引导，进行多轮模拟来构建一棵搜索树。在树中，它会更深入地探索那些看起来有希望的动作路径，并综合模拟结果得到一个比原始网络策略更优的搜索策略。这个过程平衡了“利用”（选择当前看来最好的）和“探索”（尝试新的可能性）。
- 训练循环：智能体通过自我对弈进行训练。在每局游戏中，它使用 MCTS 来选择动作。游戏结束后，得到的轨迹（状态、动作、最终奖励）被用作训练数据来更新神经网络参数 θ ，使其策略和价值预测越来越准确。这是一个不断自我改进的闭环。

直接将标准的 AlphaZero 应用于 TensorGame 会面临巨大挑战，因为其动作空间和状态表示的复杂度都远超棋盘游戏。AlphaTensor 为此引入了多项关键创新：

- 专门的神经网络架构：采用基于 Transformer 的架构，但针对三维张量输入进行了深度定制。
 - 输入投影：将 $S \times S \times S$ 的输入张量投影到三个 $S \times S$ 的二维网格上。每个网格对应于张量的一对“模式”（例如，行列分别对应输入矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 的索引）。
 - 广义轴向注意力：模型的核心是一系列注意力块，每个块在成对的网格之间进行操作。这比标准的全自注意力更高效，并且显式地建模了张量不同切片之间的关系。架构设计尊重了张量秩在切片重排下的不变性这一数学性质——这种定制的归纳偏置使网络能更有效地学习和推理张量结构。
- 合成演示与混合训练：张量分解是难的，但其逆过程——给定分解构造张量——却很容易。AlphaTensor 利用这一点，生成了海量的“合成演示”数据：随机生成秩一张量集合 $\{(\mathbf{u}^{(r)}, \mathbf{v}^{(r)}, \mathbf{w}^{(r)})\}$ ，然后构造对应的张量 $\mathcal{D} = \sum_r \mathbf{u}^{(r)} \otimes \mathbf{v}^{(r)} \otimes \mathbf{w}^{(r)}$ 。这样就得到了（张量，分解）对。

神经网络在两种数据上混合训练：一是在目标张量 $\mathcal{T}_n$ 上进行强化学习（通过自我对弈）；二是在合成演示数据上进行监督学习（模仿已知分解）。这种混合策略极大地提升了性能和样本效率，即使随机生成的张量与目标矩阵乘法张量性质不同。

- 基变换注入多样性：张量在不同的基（坐标系）下表示形式不同，但其秩不变。AlphaTensor 在每局游戏开始时，对目标张量 $\mathcal{T}_n$ 施加一个随机的可逆线性变换（基变换），然后在变换后的空间中玩游戏。找到分解后，再变换回标准基得到最终算法。

这一策略带来了双重优势：一是提供了一种强大的探索机制，因为同一个目标在不同基下呈现不同“面貌”；二是分解在变换后的基中可能系数更简单（限制在 F 内），但转换回原基后可能产生 F 之外的系数，从而意外地扩大了算法空间的覆盖范围。

- 动作规范化：为了减少网络学习冗余表达，对等价的动作（仅符号不同）进行了规范化处理。同时，从已完成的游戏通过交换动作顺序等方式提取额外训练数据，提高数据利用效率。

AlphaTensor 训练了一个单一的模型来分解多种尺寸 $(n, m, p \leq 5)$ 的矩阵乘法张量，并在不同数系（标准算术 $\mathbb{R}$ 和模 2 算术 $\mathbb{Z}_2$ ）中进行探索。其成果丰硕且具有里程碑意义：

- **重新发现经典算法**：AlphaTensor 从零开始自主重新发现了 Strassen 算法（ 2×2 ，秩 7）和 Laderman 算法（ 3×3 ，秩 23），验证了其方法在已知问题上的有效性。
- **首次超越 Strassen 的 4×4 算法**：这是最具理论突破性的成果。
 - 在模 2 算术（ $\mathbb{Z}_2$ ）中，AlphaTensor 发现了秩为 47 的 $\mathcal{T}_4$ 分解，首次改进了 Strassen 两层递归方法对应的秩 49。
 - 在标准算术（ $\mathbb{R}$ ）中，它改进了多个矩阵尺寸的最佳已知上界。例如，对于 $\mathcal{T}_{4,5,5}$ （ 4×5 乘 5×5 矩阵），将已知最佳乘法次数从 80 降至 76；对于 $\mathcal{T}_{5,5,5}$ ，从 98 降至 96。
- **丰富算法数据库与递归提升**：AlphaTensor 为每个尺寸发现了成千上万种不等价的分解（不能通过简单的

对称操作相互转换)。例如, 仅对 4×4 的秩 49 分解, 就发现了超过 14,000 种非等价形式。这揭示了算法空间的丰富性。通过智能地递归组合这些发现的小尺寸算法, AlphaTensor 进一步为 70 多个更大尺寸 ($n, m, p \leq 12$) 的矩阵乘法改进了已知上界。

- **超越标准矩阵乘法:** 框架的通用性得以验证。

- **斜对称矩阵-向量乘法:** AlphaTensor 发现了小尺寸的快速分解, 并从中归纳、证明了一个通用算法: 对于 $n \times n$ 斜对称矩阵与向量的乘法, 仅需 $\sim \frac{1}{2}n^2$ 次乘法, 达到渐近最优, 且改进了之前 n^2 次的算法。
- **发现离散傅里叶变换基:** 在有限域上, AlphaTensor 被用于寻找循环卷积张量的分解, 结果它自主发现了对应于离散傅里叶变换及其逆变换的因子, 展现了其识别深层数学结构的能力。

- **硬件定制化算法发现:** 通过修改奖励函数, 加入在特定硬件 (Nvidia V100 GPU 和 Google TPU v2) 上的实际运行时间作为反馈, AlphaTensor 成功发现了针对硬件优化的矩阵乘法算法。这些算法虽然理论乘法次数可能与 Strassen-square 相同 (例如 4×4 块矩阵乘法), 但由于其分解产生的运算序列更匹配硬件的内存层次、计算单元和编译器优化策略, 在实际测试中取得了显著的加速。这证明了 AI 不仅可以优化理论复杂度, 还能直接优化工程实践中的关键性能指标。

AlphaTensor 的成功表明, 深度强化学习可以攻克像张量分解这样的核心 NP-难计算数学问题, 并系统性地产出可证明正确、具有理论价值和实践意义的新知识。它将算法发现从一门依赖灵感的“艺术”, 转变为一个可扩展的、自动化的“科学”过程。

AlphaEvolve: 基于大语言模型的程序进化与算法发现

如果说 AlphaTensor 是“专才”, 针对张量分解问题定制了一套精密的强化学习解决方案, 那么 Google DeepMind 的 AlphaEvolve 则更像一个“通才”。它是一个基于大语言模型的代码进化智能体, 其设计目标更为广泛: 通过结合进化计算与 LLM 的代码生成与理解能力, 在程序空间中搜索能够解决各类科学和工程问题 (特别是那些可以自动评估的问题) 的更优算法或构造。

在深入技术细节之前, 我们需要先理解一个根本问题: 为什么不直接让大语言模型根据问题描述生成一个完美算法? 这背后有三个层面的考量:

- **可靠性的困境:** 复杂算法的“一次性生成”成功率极低。大语言模型的“幻觉”现象会导致大量无效输出——它们可能看起来合理, 但运行起来却错误百出。纯粹的生成式方法缺乏验证机制, 难以保证算法的正确性。
- **累积创新的必要性:** 算法发现很少是一蹴而就的。人类数学家的工作方式往往是渐进式的——在一个已有算法的基础上, 进行微小的改进, 积累到一定程度才形成突破。进化框架完美地模拟了这种“累积性创新”的过程: 一个微小的有效修改被保留下来, 成为下一次更大改进的基础。LLM 在已被验证有效的代码上进行迭代, 远比从头创造更可靠。
- **搜索引导的价值:** 纯粹的代码生成是漫无目的的。进化框架中的数据库和评估反馈, 为 LLM 的创造提供了明确的方向——它不是在海量可能性中随机游走, 而是朝着性能提升的目标持续优化。这种“目标导向的生成”, 正是智能搜索的核心。

基于这些考量, AlphaEvolve 确立了自己的核心哲学: 将解决方案表示为程序, 并利用 LLM 作为强大的、具有领域知识的“变异算子”, 在一个由自动评估函数引导的进化循环中, 持续改进这些程序。

AlphaEvolve 的工作流程可以清晰地分解为四个关键环节:

第一步: 任务定义与自动评估闭环

用户需要提供问题的核心: 一个自动评估函数 `evaluate(program)`。这个函数接收一个候选程序 (算法), 运行它, 并返回一个或多个标量分数来衡量其性能。正确性可以通过在执行中验证来保证——例如, 用发现的矩阵乘法算法计算随机实例, 并与标准结果比对, 任何错误都会被自动捕获并扣分。

这里用程序执行的真实结果替代了人类或 LLM 的主观评价, 从而能够进行长期、稳定的迭代优化, 而无惧 LLM 的“幻觉”或偏见。正是这个闭环, 将 AlphaEvolve 与纯粹的对话式 AI 区分开来。

第二步: 初始化

用户提供一个初始程序 (可以非常简单、低效, 甚至是朴素的实现), 并用特殊注释 (如 `# EVOLVE-BLOCK-START/END`)

标记出代码中允许进化的部分。其余部分作为固定框架，保持不变。这种设计既保证了进化的自由度，又确保了程序基本结构的稳定性。

第三步：进化循环

- 提示采样与构建：从“程序数据库”（一个存储了历史探索过的高分、多样程序及其分数的存档）中，采样一批表现优异的程序。构建一个富含信息的提示给 LLM，内容包括：任务描述和指令、一个或多个高分程序及其详细评估结果（分数、输出等）、可能的额外上下文（相关数学公式、论文片段、优化目标说明等）。最后，要求 LLM 分析这些程序并提出改进方案，以代码差异的形式呈现。
- LLM 生成与创造性演绎：LLM（如 Gemini 系列模型）消化提示后，生成代码修改建议。这里的关键在于，LLM 不仅能进行语法层面的小修改，还能深入理解程序逻辑，提出深刻的算法性变更——例如引入新的优化步骤、更改数据结构、添加启发式规则、甚至改变整个算法范式。它调用的不仅是当前上下文中的信息，更是其在海量代码上预训练所积累的算法知识库。
- 程序创建与评估：将 LLM 生成的结果应用到父程序上，产生新的候选程序。随后，级联评估机制启动：首先在少量、快速的测试用例上运行，以快速过滤掉有错误或性能极差的程序；通过初步筛选后，再在更全面、更耗时的测试集上进行正式评估。这种分层策略兼顾了效率与准确性。
- 数据库更新与进化策略：新程序及其分数被送入程序数据库。数据库的管理策略借鉴了 MAP-Elites 等先进进化算法的思想，旨在同时维持“精英存档”（每个性能区域的最优者）和总体多样性，以平衡“利用”（深耕已知优势区域）与“探索”（尝试全新的可能性）。

第四步：迭代与收敛

随着循环的持续，程序数据库中的算法性能不断提升。进化过程可以在达到预设的性能阈值后停止，也可以由人类研究者介入，审视进化出的算法并提炼新的数学洞察。

AlphaEvolve 被应用于与 AlphaTensor 相同的张量分解问题，但它采用的是一种更高阶的“元搜索”策略。这种策略的独特之处，在于它改变了“搜索什么”这个根本问题，包括：

- 搜索对象的跃迁：在 AlphaTensor 中，用户直接搜索分解因子 $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ 本身。但在 AlphaEvolve 中，用户不要求 LLM 直接输出分解因子，而是要求它进化出一个“能够找到张量分解的算法程序”。
初始程序可以非常简单——例如，一个现成的梯度下降优化器，使用 Adam 优化器最小化重构损失 $\|\mathcal{T} - \sum \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}\|^2$ ，并预设一个秩 R 。这个程序本身可能效率不高，但它是进化的起点。
- 评估对象的跃迁：相应地，评估函数不再直接衡量分解的质量，而是衡量这个“分解寻找算法”的性能。具体来说，评估函数会运行进化出的算法，在多个目标张量（如 $\mathcal{T}_{4,4,4}$ ）和多个随机种子上进行尝试，返回它能够找到的最佳秩，以及达到该秩的成功率。这种评估方式激励进化出更强大、更鲁棒的优化算法——那些能够稳定地、可重复地发现优质分解的算法。
- LLM 扮演的角色：在这个框架中，LLM 阅读当前表现最好的几个“分解寻找算法”，理解它们的结构（如使用何种优化器、损失函数有何正则项、如何初始化、是否包含随机重启等），然后提出修改建议。这些修改可以非常深入，远超人类专家日常调整的范畴，例如：
 - 将梯度下降改为拟牛顿法，以获得更快的收敛；
 - 在损失函数中加入促进系数离散化的正则项，引导算法找到系数简洁的分解；
 - 设计一种交替投影算法，在因子空间和张量空间之间来回迭代；
 - 甚至编写一个模仿 AlphaTensor 风格的小型搜索逻辑，将局部搜索与全局规划结合起来。

AlphaEvolve 的工作流程如图 3.1（引自 [alpha evolve 论文]）所示：

LLM 之所以能够进行这些创造性的修改，是因为它在预训练过程中“阅读”了海量的算法代码、优化文献和数学知识。它不是在凭空想象，而是在一个巨大的先验知识库中进行有依据的组合与创新。

在矩阵乘法问题上，AlphaEvolve 取得了与 AlphaTensor 互补且在某些方面更具历史意义的成果：

1. **广泛匹配与超越**：在测试的 54 种矩阵乘法尺寸中，AlphaEvolve 发现的程序在 38 种情况下匹配了已知最佳结果，并在 14 种尺寸上取得了实质性改进。例如，它将 $\langle 2, 4, 5 \rangle$ 的乘法次数从 33 降至 32，将 $\langle 3, 4, 7 \rangle$ 从 66 降至 63。这些改进虽然数字微小，但在算法优化领域，每一个“1”的减少都可能意味着理论边界的突

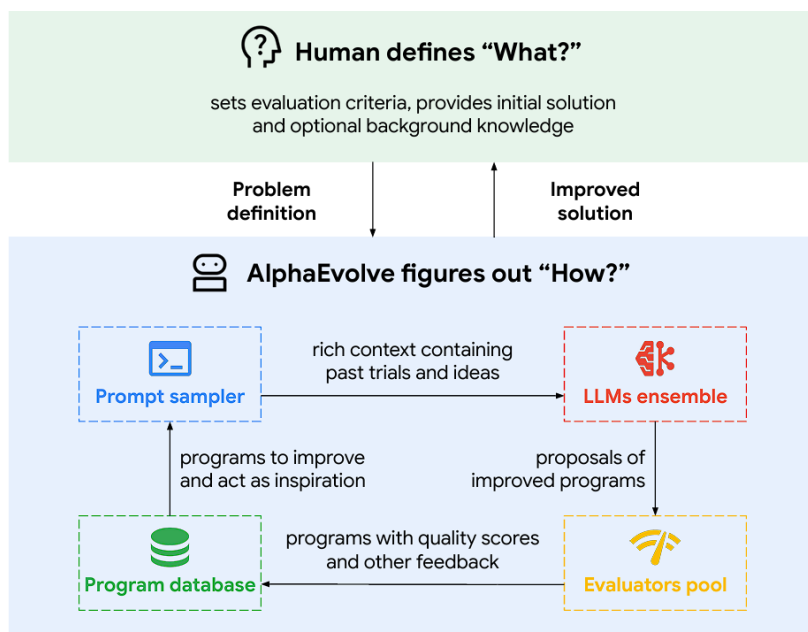


图 3.1: AlphaEvolve 工作流程

破。

- 历史性突破：4 × 4 复矩阵的秩 48 算法：**这是 AlphaEvolve 最引人注目的成果。它发现了第一个在复数域（以及实数域）上计算 4 × 4 矩阵乘法的秩 48 算法。自 1969 年 Strassen 算法提出以来，4 × 4 矩阵在通用数系上的最佳记录一直是秩 49（即 Strassen 两层递归的结果）。这个记录保持了半个多世纪，被无数数学家尝试突破而未果。AlphaEvolve 将这一记录改写为 48，由此递归可得复杂度约为 $O(N^{\log_4 48}) \approx O(N^{2.792})$ 的算法——一个比 Strassen 算法更快的矩阵乘法新方法。

AlphaEvolve 的真正威力，在于它的通用性。矩阵乘法只是它能力的第一个展示窗口。在更广阔的数学和工程领域，它同样展现了惊人的潜力。

1. 数学构造问题：研究团队在超过 50 个来自分析学、组合数学、数论、几何学的开放问题上对 AlphaEvolve 进行了测试。结果令人震撼：

- 在约 75% 的问题上，它重新发现了已知的最佳构造——这验证了方法的可靠性。
- 在约 20% 的问题上，它发现了新的、可证明更优的构造——这展示了方法的创造力。

具体案例包括：

- 改进了埃尔德什（Erdős）最小重叠问题的上界，略微改进了之前的记录。
- 在 11 维空间的吻球数（Kissing Number）问题上，将已知下界从 592 提高到 593。吻球数问题研究的是一个球周围可以同时接触多少个不相交的等大球，在高维空间中极其困难。
- 改进了多种几何填充问题（如六边形打包、圆内填圆、Heilbronn 三角形问题）的最佳已知密度或尺寸。

其方法论的关键在于：AlphaEvolve 被设计为进化“启发式搜索算法”，而不是直接进化“数学构造”本身。每个进化的程序被要求在固定时间内，基于前一个最佳构造进行局部搜索和优化。这种多阶段、自适应的策略，使得它能够自动攻击那些传统方法难以处理的复杂优化问题。

2. 关键工程优化，这是 AlphaEvolve 最令人惊叹的应用场景之一：它能够在完全没有先验硬件知识的情况下，仅通过自动评估反馈，优化复杂的工程系统。

- 数据中心调度：**为谷歌的 Borg 集群管理系统进化出一个简单的启发式函数，用于在调度时评估机器的适合度。这个由 AI 进化出的函数，平均回收了 0.7% 的全局计算资源——在一个超大规模数据中心中，这意味巨大的成本节约和价值创造。
- AI 训练内核优化：**为训练 Gemini 大模型的关键矩阵乘法核，进化出“分块策略”启发式。分块是优化矩

阵乘法在硬件上运行速度的核心技术，但最优分块策略高度依赖于硬件架构。AlphaEvolve 在没有被告知任何硬件细节的情况下，通过运行时间的自动反馈，找到了比人类专家设计更优的分块策略，使内核速度平均提升 23%，并减少了整体训练时间。

- 硬件电路设计：在高度优化的 TPU 算术电路 Verilog 代码中，AlphaEvolve 发现可以移除冗余逻辑的简化方案。
- 编译器中间代码优化：直接优化 XLA 编译器为 Transformer 注意力机制生成的中间表示代码，提升了执行效率。这意味着它不仅在算法层面进行优化，还能深入编译器层面，优化代码的实际执行方式。

这些案例共同证明了一点：AlphaEvolve 是一个能够将前沿大语言模型的创造力与自动评估的可靠性相结合的、强大的通用“科学发现与工程优化引擎”。它不局限于特定的数学问题，而是可以应用于任何能够用程序表达、能够自动评估的领域。

AlphaTensor 和 AlphaEvolve 代表了 AI 用于算法发现的两种不同但互补的范式。它们从迥异的起点出发，最终在解决同一类核心问题上汇合，并展现出各自的特色与潜力。

表 3.1: AlphaTensor 与 AlphaEvolve 的方法论对比

维度	AlphaTensor	AlphaEvolve
核心方法	深度强化学习（基于 AlphaZero/MCTS），专为特定问题形式（游戏）定制	大语言模型引导的程序进化，采用通用的问题解决框架
问题形式化	将“寻找分解”形式化为单人游戏（TensorGame），状态是张量，动作是秩一因子	将“解决问题”形式化为代码优化任务，状态/知识隐含在程序数据库中，动作是代码修改
搜索空间	直接搜索解空间（分解因子 u, v, w ）	搜索元空间（能生成或寻找解的程序）
核心技术	专门的神经网络（针对张量设计的 Transformer 变体），通过强化学习训练	通用的大语言模型（如 Gemini），其知识来自预训练，通过提示工程和上下文学习来调用
优势	针对性强，搜索效率高；理论保障性好；在目标领域内性能极致	灵活性极高，几乎无需为不同问题修改核心框架；能处理复杂的、多组件的代码库；可无缝结合丰富的自然语言领域知识；输出通常更具可解释性和可部署性
局限性	需要为每个新问题设计专门的游戏规则、状态表示和奖励函数；神经网络架构与问题紧密耦合，迁移成本高	严重依赖自动评估函数的存在和质量；进化过程可能较慢，且依赖于 LLM 生成的质量；对计算资源要求可能很高
输出性质	直接的数学对象（分解因子），可立即转换为算法	生成该数学对象的程序，或直接是优化后的可运行代码

这两种范式并非竞争关系，而是构成了一个方法论光谱的两极。它们各自捕捉了智能搜索的不同侧面——AlphaTensor 代表了“专注的深度”，AlphaEvolve 代表了“广度的灵活”。未来，这两种范式很可能走向深度融合：

- 搜索层次的互补：AlphaTensor 在“对象层”搜索，AlphaEvolve 在“元层”搜索。后者可以看作对前者的一种泛化和抽象。一个有趣的设想是：用 AlphaEvolve 来进化出更好的 AlphaTensor——例如，进化出更有效的神经网络架构、更巧妙的奖励函数设计、甚至新的 MCTS 变体。这样一来，元层的进化为对象层的搜索提供了更好的工具，而对象层的发现又反过来丰富了元层的知识。
- 构建混合工作流：一个强大的人机协作科研范式可能是这样的：
 - 探索阶段：使用类似 AlphaEvolve 的系统进行宽泛的、启发式的探索，快速产生大量候选思路或粗糙算法。这个阶段的重点是广度——尽可能覆盖不同的可能性，识别有潜力的方向。
 - 深化与验证阶段：对于有希望的候选方向，用更专门、更强大的方法（如定制化的 AlphaTensor 风格强化学习）进行集中、深入的搜索和优化。这个阶段的重点是深度——在选定的方向上尽可能逼近最优解。同时，使用形式化验证工具确保最终结果的正确性。

- 解释与猜想阶段：从 AI 发现的高性能算法或对象中，人类与 AI 协作提取模式、对称性和潜在的新数学猜想。AI 可能发现了一种新的结构，而人类数学家则负责理解这种结构背后的数学原理，并将其形式化为可证明的定理。
- 反馈与迭代阶段：人类数学家的新洞察被形式化为新的约束、启发式或评估标准，反馈给 AI 系统，开启新一轮更高效的搜索。这样，人类的直觉与 AI 的搜索能力形成了正向循环，相互增强。

AlphaTensor 和 AlphaEvolve 共同描绘了这样一个未来：算法发现不再是偶然的灵光一现，而是一个可以系统化推进、人机智能深度协同的探索过程。在这个未来中，AI 不是取代数学家的思考，而是扩展数学家的能力边界——它们负责在人类难以触及的广阔空间中探测可能性，而人类则负责理解、提炼和证明。正是这种协同，构成了 AI for Math 这一新兴领域的核心魅力。

AI 发现数学规律的工作方兴未艾，成果层出不穷。限于篇幅，我们无法一一详述。感兴趣的读者可以参阅本章末尾的参考文献，其中收录了近年来这一领域的重要进展。

参考文献

- [1] Charles Blundell et al. “Towards combinatorial invariance for Kazhdan-Lusztig polynomials”. In: *Representation Theory of the American Mathematical Society* 26 (2022), pp. 114–141.
- [2] Peter Bürgisser, Michael Clausen, and Mohammad Amin Shokrollahi. *Algebraic complexity theory*. Vol. 315. Springer Science & Business Media, 1997.
- [3] Tianlong Chen et al. “Learning to optimize: A primer and a benchmark”. In: *Journal of Machine Learning Research* 23.189 (2022), pp. 1–59.
- [4] Alexander Chervov et al. “CayleyPy RL: Pathfinding and Reinforcement Learning on Cayley Graphs”. In: *arXiv preprint arXiv:2502.18663* (2025).
- [5] Alexander Chervov et al. “A Machine Learning Approach That Beats Large Rubik’s Cubes”. In: *arXiv preprint arXiv:2502.13266* (2025).
- [6] Stephen G Coppola. *An annotated bibliography of fast matrix multiplication*. Available online. 2024.
- [7] Alex Davies et al. “Advancing mathematics by guiding human intuition with AI”. In: *Nature* 600.7887 (2021), pp. 70–74.
- [8] Alex Davies et al. “The signature and cusp geometry of hyperbolic knots”. In: *Geometry & Topology* (2024).
- [9] Bin Dong et al. “Machine Learning Assisted Exploration for Affine Deligne–Lusztig Varieties”. In: *Peking Mathematical Journal* (2025).
- [10] Alhussein Fawzi et al. “Discovering faster matrix multiplication algorithms with reinforcement learning”. In: *Nature* 610.7930 (2022), pp. 47–53.
- [11] Julian D Laderman. “A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications”. In: *Bulletin of the American Mathematical Society* 82.1 (1976), pp. 126–128.
- [12] Robert Lange, Yuki Imajuku, and Edoardo Cetin. “ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution”. In: *arXiv preprint arXiv:2509.19349* (2025).
- [13] Gang Liu et al. “Scientific Algorithm Discovery by Augmenting AlphaEvolve with Deep Research”. In: *arXiv preprint arXiv:2510.06056* (2025).
- [14] Alexander Novikov, Ngan Vũ, Martin Eisenberger, et al. “AlphaEvolve: A coding agent for scientific and algorithmic discovery”. In: *arXiv preprint* (2024).
- [15] Bernardino Romera-Paredes et al. “Mathematical discoveries from program search with large language models”. In: *Nature* 625.7995 (2024), pp. 468–475.

- [16] Julian Schrittwieser et al. “Mastering Atari, Go, chess and shogi by planning with a learned model”. In: *Nature* 588.7839 (2020), pp. 604–609.
- [17] David Silver et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. In: *Science* 362.6419 (2018), pp. 1140–1144.
- [18] Volker Strassen. “Gaussian elimination is not optimal”. In: *Numerische Mathematik* 13.4 (1969), pp. 354–356.

第四章 AI 证明数学定理

4.1 概述

1. 数学定理自动证明的发展历程

数学定理的证明，是数学这门学科最核心的活动。从欧几里得《几何原本》的公理化演绎，到怀特海与罗素《数学原理》的逻辑建构，再到当代数学家对千禧年难题的探索，“证明”始终是数学知识的基石——它不仅确认了一个命题的真伪，更重要的是揭示了命题之间深刻的逻辑联系，让数学成为一座坚实而优美的大厦。

自动定理证明，正是人工智能领域一个持续了近七十年的追求：让机器能够像人类数学家一样，从已知的公理和定理出发，通过逻辑推理得出新的结论。这一追求的历史，不仅见证了人工智能技术的演进，更折射出人们对“机器如何思考”这一根本问题的理解变迁。

自动定理证明的发展大致可划分为三个阶段，每一阶段都对应着对“机器如何理解数学”的不同技术路径和哲学思考。

第一阶段：早期探索与基于规则的自动化

这一时期的根本思想源于 20 世纪 30 年代图灵、丘奇等人奠定的可计算性理论基础。既然数学推理可以形式化为符号操作，那么理论上，机器应该能够执行这种操作。

1956 年，艾伦·纽厄尔、赫伯特·西蒙等人开发的“逻辑理论家”程序，成为这一理想的首次实现。它成功证明了罗素与怀特海《数学原理》第二章中的 38 个定理——这是机器第一次展现出逻辑推理的能力，也标志着人工智能作为一个独立学科的诞生。

这一阶段的核心方法论是符号主义与启发式搜索。计算机被视为一个在由公理、定理和推理规则构成的“状态空间”中寻找路径的搜索器：初始状态是已知事实，目标状态是要证明的定理，每一步动作是应用某条推理规则。然而，这种方法很快遭遇了根本性挑战：组合爆炸。即使对于中等复杂度的定理，可能的推理路径也是一个天文数字，其中绝大多数是死胡同。早期的系统严重依赖数学家精心设计的、针对特定领域的启发式规则来引导搜索，其“智能”本质上是人类专家知识的内化。这种依赖意味着，系统的能力边界被限制在人类能够事先预见和编码的范围内，难以真正突破人类的认知框架。

第二阶段：交互式定理证明器与形式化数学的兴起

面对组合爆炸的困境，研究范式在 20 世纪 70 至 80 年代发生了一次深刻的转变。研究者们逐渐意识到，在可预见的未来，追求完全自动化的证明可能是不现实的。于是，他们转而思考一个更务实的问题：如何让计算机成为数学家严谨性的守护者，而不是创造力的替代者？

这一思考催生了交互式定理证明器（Interactive Theorem Prover, ITP），如 Isabelle、HOL Light、Coq 和后来的 Lean。这些工具的核心设计思想极其优雅：一个经过严格验证的小型逻辑“内核”，只负责检查证明的每一步是否符合最基础的逻辑规则，而不关心这些步骤是如何被想到的。数学家的工作变成了与证明器进行交互：使用高级的“策略”（Tactics）来分解和证明目标，而证明器则像一个永不疲倦的校对员，确保每一步推理都绝对严谨。

这种协作模式带来了深远的影响。一方面，它让数学家能够专注于高阶的洞察和策略，将繁琐的细节检查交给机器。另一方面，借助 ITP，数学家完成了四色定理、开普勒猜想等里程碑式证明的完全形式化验证——这些证明的复杂性已经超越了任何人类单独检查的能力范围。

更重要的是，形式化数学库开始被系统地构建，如 Isabelle 的 Archive of Formal Proofs 和 Lean 的 Mathlib。这些库是已被机器严格验证的、可复用的数学知识基础设施，它们为后续的 AI 模型提供了高质量、结构化且绝对可靠的数据源——这是一个意义深远的奠基性工作，它让数学知识第一次以机器可读、可理解、可操作的形式被组织起来。

第三阶段：大语言模型与符号主义的融合

深度学习的兴起，特别是大语言模型和强化学习的发展，为自动定理证明带来了全新的可能性。研究者们意识到，人类数学家之所以能够在浩瀚的推理空间中快速找到正确的路径，依靠的不仅仅是逻辑演绎，更是一

种难以言说的“数学直觉”——对什么是有希望的证明方向的模糊感知。这种直觉，或许可以从数据中学习。

大语言模型通过在数以亿计的代码和数学文本上预训练，确实学会了数学语言的结构和常见的推理模式。它们可以被微调，将当前的证明状态作为输入，预测下一步可能有效的证明策略——换句话说，它们扮演了启发式策略网络的角色，为搜索提供了方向感。

而以 DeepMind 的 AlphaProof 为代表的系统，则将这一理念推向了极致。它将定理证明彻底形式化为一个强化学习问题：

- **环境：**Lean 等交互式定理证明器，提供确定性的状态转移和完美的正确性反馈——这是一个理想的强化学习环境，因为它有明确的目标（证明完成）和客观的奖励（证明成功与否）。
- **智能体：**一个大型神经网络，同时学习策略函数（选择下一步的战术）和价值函数（评估当前状态距离证明完成还有多远）。
- **训练：**通过在海量自动生成的形式化问题上进行自我对弈，智能体不断优化其证明搜索策略，学习在浩瀚的推理空间中高效导航。

这种神经符号主义的融合——神经网络的推理能力与形式化系统的绝对严谨性相结合——代表了当前自动定理证明的前沿。

2. 当前 AI 辅助定理证明的核心成果

AlphaProof 系统在这一路径上取得了令人瞩目的成就。在 2024 年国际数学奥林匹克竞赛中，它解决了三道非几何难题，包括那届比赛中最难的第六题。结合其专用的几何推理系统 AlphaGeometry 2，AlphaProof 的整体表现达到了银牌级别的水平。这是人工智能首次在顶级数学竞赛中达到奖牌水平——而且，它的每一个证明都经过了 Lean 内核的完全验证，具备绝对的逻辑可靠性。

这一突破的背后，是一系列方法论的创新：

- **自动形式化：**利用大语言模型将海量自然语言数学问题转化为形式化命题，构建了包含数千万个问题的训练集，突破了人工形式化的瓶颈。这意味着系统可以“阅读”和“理解”自然语言描述的数学问题，并将其转化为自己可以操作的符号形式。
- **神经引导的证明搜索：**将神经网络的模式识别能力与蒙特卡洛树搜索的前瞻规划能力相结合，极大地提高了在庞大证明空间中的搜索效率。神经网络告诉搜索“哪里可能值得探索”，而树搜索则负责系统地评估和比较不同的路径。
- **测试时强化学习：**针对单个极难问题，系统可以通过快速生成其变体并进行针对性的强化学习，实现“专项特训”。这种能力让系统能够动态地适应问题的特点，展示了强大的自适应能力。

3. 存在的核心局限与未来挑战

尽管成果显著，但当前 AI 定理证明系统仍面临根本性挑战：计算成本与可及性是一个现实问题。AlphaProof 级别的系统，其训练和运行（尤其是测试时强化学习）需要消耗巨额的计算资源。这使其更像一个“国家实验室级别”的科研装置，而非普通研究者或学生可以日常使用的工具。如何降低计算成本，让这项技术普惠化，是一个亟待解决的问题。

对形式化基础设施的依赖同样不可忽视。系统的能力边界受限于形式化数学库的覆盖范围。目前，成功主要集中在已有完善形式化的竞赛数学和经典数学领域。而要将前沿数学研究中的新概念、新理论进行形式化，本身就是一个极其耗时且需要深厚专业知识的巨大工程——这形成了一个悖论：AI 需要形式化数据才能学习，而形式化数据又需要大量人力才能创建。

最深远的挑战，或许在于从“问题求解”到“理论构建”的鸿沟。当前的 AI，无论多么强大，本质上仍然是超级的策略搜索和组合优化器。它们在一个相对封闭的形式化知识体系内解决问题，而非进行开疆拓土的探索。人类数学家的工作远不止于解决给定问题——他们还提出新问题，识别有意义的模式，评估不同数学方向的价值（所谓“数学品味”），甚至创造全新的概念和理论框架。这些活动要求一种元认知能力，是目前的技术远未达到的。

定理自动证明的历史是一部从符号搜索到神经学习、从独立验证到人机协同的演进史。以 AlphaProof 为代表的系统，通过将大语言模型的“直觉”与形式化系统的“严谨”深度融合，首次让机器在高质量数学推理上达

到了接近人类专家的水平。虽然前路依然漫长，特别是在高层创造性和理论构建方面，但它已经为数学研究开启了一扇新的大门。

4.2 大语言模型的数学能力: 技术实现

大语言模型在数学推理上展现出的能力，并非来自某种神秘的“数学直觉”，而是一套严谨、可工程化的训练流程系统塑造的结果。这一流程从灌输海量知识开始，到引导模型理解任务格式，最终通过强化学习实现与特定目标的精准对齐。理解这一流程，也就理解了当前 AI 数学能力的来源与边界。现代大语言模型的数学能力，通常通过一个精心设计的三阶段训练流程获得。每一阶段都有其独特的数据构成、算法目标和能力塑造重点。

第一阶段：大规模预训练——构建“知识基座”

任何能力的起点都是知识。预训练阶段的目标，是让模型成为一个“知识渊博”的通用学习者。

- 数据：海量无标注语料。模型被暴露在高达万亿 Token 级别的文本数据中，这些数据涵盖高质量网页、书籍、学术论文、开源代码等。规模是关键——只有足够多的数据，才能覆盖人类知识的广度和深度。
- 算法：下一个词预测与 Transformer 架构。预训练的核心任务看似简单：给定上文，预测下一个最可能出现的词。但这个“简单”的任务，在足够大的模型和足够多的数据上，却产生了惊人的涌现能力。支撑这一过程的引擎是 Transformer 架构，其自注意力机制允许模型动态评估序列中所有词之间的关系，高效捕获长距离依赖和复杂模式。通过在大规模集群上分布式训练，模型的数亿乃至数千亿个参数逐渐学习到将文本映射到高维语义空间的能力。

这一阶段产出的，是一个“知识渊博但未受规训”的基座模型。它拥有丰富的知识储备，知道大量关于数学的事实、公式和推理模式，但它尚不理解具体任务的格式——你可以问它任何问题，它的回答可能是事实、可能是虚构、可能是完全无关的内容。它需要被进一步“规训”。

第二阶段：监督微调——教授“任务格式”与“思维链”

如果说预训练是在灌输知识，那么监督微调就是在教会模型如何“使用”这些知识来回应人类的指令。

- 数据：高质量的指令-应答对。研究者构建数万至数十万条由人工精心构造的(指令, 输入, 期望输出)数据对。例如，指令是“解方程： $2x + 5 = 13$ ”，期望输出是“ $x = 4$ ”。这些数据示范了模型应该如何回应不同类型的任务。
- 算法：有监督的指令微调。在基座模型上，使用这些指令数据对其进行有监督微调。这个过程实质上是让模型学习一种新的条件概率分布： $P(\text{输出} | \text{指令}, \text{输入})$ 。模型逐渐学会，当看到“解方程”这个指令时，它应该输出一个数值答案；当看到“证明”这个指令时，它应该输出一串逻辑推导。

思维链微调是激发模型显式推理能力的关键创新。普通的指令微调只要求模型输出最终答案，而思维链微调则要求模型输出包含逐步推理过程的完整解答。

例如，对于同一个方程问题：

- 标准微调输出： $x = 4$
- 思维链微调输出：首先，将等式两边同时减去 5，得到 $2x = 8$ 。然后，两边同时除以 2，得到 $x = 4$ 。因此，答案是 4。

模型通过学习大量这样的思维链数据，内化了“对于复杂问题，应在最终答案前展示逻辑推导步骤”的范式。这并非赋予模型全新的推理能力，而是将预训练阶段已经隐式学习的逻辑能力，通过示范数据引导至显式输出。这种显式的推理过程，不仅让模型的决策更可解释，也为后续的强化学习提供了可以评估的中间步骤。

这一阶段结束后，模型转变为一个“遵循指令且能展示思考过程的学生”。但它输出的优化目标——例如，是否简洁、是否严谨、是否有创新性——尚未被精确定义。它知道如何回答，但还不知道什么是最好的回答。

第三阶段：强化学习——实现“任务精准对齐”

第三阶段的目标，是将模型的输出与特定的、可客观评估的任务目标进行强对齐。其核心是设计一个基于明确规则的奖励函数，然后通过强化学习优化模型，使其行为与这些规则高度一致。

- **奖励函数的设计**: 奖励函数 R 需要根据一套清晰的规则对模型的输出进行自动评分。奖励函数的设计直接决定了模型最终习得的行为。
 - **对于计算类问题**: 奖励函数通常非常简单和确定。规则是: $R(\text{输出}) = 1$ 如果最终答案数值正确, 否则 $R(\text{输出}) = 0$ 。系统可以自动提取模型输出中的最终数字答案, 与标准答案比对。
 - **对于逻辑证明类问题**: 奖励函数的设计则复杂得多, 这也是当前的技术前沿。
 - **形式化验证**: 在形式化数学场景下, 最严格的规则是将模型输出的整个证明过程 (如 Lean 或 Coq 代码) 提交给定理证明器内核进行验证。规则是: $R(\text{输出}) = 1$ 如果证明器完全接受该证明, 否则 $R(\text{输出}) = 0$ 。这是 AlphaProof 等系统采用的核心方法, 它提供了绝对可靠的正确性信号。
 - **自然语言证明的挑战**: 对于非形式化的自然语言证明, 自动判断其每一步推理是否有效极为困难。一种折中方法是检查证明中是否出现了关键推理步骤 (如 “使用了归纳法”、“应用了柯西-施瓦茨不等式”), 或通过另一个验证模型来检查前后步骤的逻辑一致性。然而, 这种规则的完备性和可靠性都有限。
 - **创新性判断的困境**: 目前几乎无法通过自动化规则有效评估创新性。创新性涉及与已知证明的 “不同” 程度、构思的 “巧妙性” 等, 这些都属于高阶、模糊的语义概念, 难以被量化为确定性的奖励信号。
- **强化学习算法**: 通常采用近端策略优化 (PPO) 等策略梯度算法。流程是: 将监督微调后的模型作为需要优化的策略; 针对一个任务提示, 策略模型生成一个输出; 基于规则的奖励函数对该输出进行自动打分; PPO 算法利用这个奖励信号更新模型参数, 使其未来更倾向于生成能获得高分的输出。

这种方法直接而高效, 特别适用于目标可被清晰量化的任务。它让模型的行为与 “正确性” 这一客观指标强绑定, 实现了从 “知道如何回答” 到 “知道如何正确回答” 的跃迁。

尽管基于规则的强化学习为模型提供了明确的对齐方向, 但其局限性在复杂推理任务中暴露无遗。理解这些局限, 有助于我们准确把握当前 AI 数学能力的边界。

1. 证明过程判断的模糊性与复杂性自然语言证明对于机器而言充满歧义。当模型输出 “显然可得” 或 “由标准引理” 时, 这些表述对人类读者可能足够清晰, 但对机器而言却难以验证。人类证明中常见的 “跳跃” —— 省略那些被认为 “显然” 的中间步骤 —— 在机器验证时需要被完全展开。自动判断一个跳跃是否可接受, 需要近乎完备的背景知识, 这本身就是一个 AI 完备问题。
2. 创新性评估的不可计算性什么是一个证明的 “创新”? 是使用了新颖的引理组合? 是化简了已知证明? 这些定义本身无法被完全形式化。要判断一个证明是否新颖, 需要与已有全部证明数据库进行语义层面的比对, 这在实践中不可行。奖励函数最多只能奖励 “与训练数据中常见证法不同” 的输出, 但这可能与真正的数学创新相去甚远, 甚至可能奖励了错误或更冗长的证法。
3. 奖励函数的稀疏性与探索挑战在证明任务中, 只有当最终证明完全正确时, 奖励函数才返回一个正信号 (+1), 在此之前的所有中间步骤奖励均为 0。这种稀疏奖励就像在浩瀚的宇宙中寻找一颗特定的行星 —— 指导信号极其微弱, 导致训练效率低下。

更微妙的问题是局部最优的陷阱。模型可能学会生成安全、冗长、模仿已知的证明来确保获得正确性奖励, 但这种策略会强烈抑制其探索更简洁、更巧妙新证法的动机。这与我们期望的 “创造性” 背道而驰 —— 模型被奖励机制引导向保守, 而非创新。

综观整个训练流程, 我们可以清晰地看到当前大语言模型数学能力的来源与边界, 它是一位极其勤奋的博学者。通过预训练, 它吸收了人类积累的海量数学知识; 通过监督微调, 它学会了如何以标准格式回应数学问题; 通过强化学习, 它优化了自己的行为以最大化正确性。

但距离成为拥有直觉与灵感的开创者, 它仍有本质性的差距。这个差距的核心在于: 我们能够定义和优化的, 是那些可以被明确规则量化的目标 —— 答案正确、证明通过。但高层次数学创造所需要的品质 —— 深刻洞察、巧妙构思、理论品味 —— 恰恰是无法被简化为明确规则的。

当我们将奖励函数定义为 “答案正确” 时, 我们优化的是可靠性与严谨性; 但 “创造力” 与 “深刻洞察” 如何定义、如何量化、如何作为优化目标, 仍然是未解之谜。这揭示了当前 AI 在数学推理乃至一般性科学发现中的能力边界: 它在封闭的、规则明确的问题空间内表现卓越, 但要在开放的、需要价值判断的创造性工作中发挥作用, 还需要方法论上的根本性突破。

未来的进展，可能依赖于如何将更抽象、更高阶的元认知目标——如“解释的优雅性”、“理论的统一性”、“概念的深刻性”——建模为可学习的信号。这不仅是技术挑战，更是对“什么是数学创造力”这一根本问题的重新思考。

4.3 当前成功案例 1: AlphaProof

在了解了自动定理证明的发展历程和大语言模型的技术实现之后，我们终于可以走进一个具体的、里程碑式的系统：DeepMind 开发的 AlphaProof。它不仅是当前 AI 定理证明领域的顶尖成果，更是我们之前讨论的诸多理念——形式化数学、神经引导搜索、强化学习——在一个统一框架下的深度融合。

AlphaProof 解决了 2024 年国际数学奥林匹克竞赛中三道高难度问题，包括那届比赛中最难的第六题，结合其专用的几何系统 AlphaGeometry2，整体表现达到了银牌级别。这是人工智能首次在顶级数学竞赛中达到奖牌水平，而且它的每一个证明都经过了 Lean 定理证明器的严格验证，具有绝对的逻辑可靠性。

这一成就的背后，是一个精心设计的技术架构。我们将以 AlphaProof 为主要案例，详细拆解其如何将抽象的数学推理转化为一个可计算、可优化的问题。

数学模型：将定理证明视为马尔可夫决策过程

任何强化学习系统的起点，都是将现实问题形式化为一个智能体可以交互的环境。AlphaProof 将定理证明定义为如下的马尔可夫决策过程：

- **状态 (S)**：状态 s_t 是 Lean 证明助理在时刻 t 的“战术状态”。它是一个文本字符串，清晰列出了当前所有待证明的目标以及每个目标下可用的假设。例如，在证明“对于所有自然数 n ， n 要么是偶数要么是奇数”时，初始状态可能是目标“ $\vdash \forall n \in \mathbb{N}, \text{Even}(n) \vee \text{Odd}(n)$ ”。应用一个“intro n ”战术后，状态变为“ $n : \mathbb{N} \vdash \text{Even}(n) \vee \text{Odd}(n)$ ”。这个文本表示包含了问题结构的全部信息——它告诉智能体：你现在有什么（假设），你需要证明什么（目标）。
- **动作 (A)**：动作 a_t 是一个可以在当前状态 s_t 下执行的 Lean 战术，同样表示为一个文本字符串。战术可以是基础的（如 intro、apply、have、exact），也可以是复杂组合的自动化策略（如 linarith、ring），甚至可以是用户自定义的。动作空间是开放且巨大的——这正是数学证明的丰富性所在。
- **状态转移 (P)**：转移函数 $P(s_{t+1}|s_t, a_t)$ 由 Lean 证明助理的执行引擎定义，是确定性的。给定当前状态和一个战术字符串，Lean 会尝试执行它。如果战术合法且成功，环境就转换到新的证明状态 s_{t+1} ；如果战术不适用或执行错误，当前尝试失败——这意味着智能体需要学会选择“可行”的动作。
- **奖励 (R)**：奖励函数的设计需要引导智能体找到尽可能短的证明。系统为每个非终止步骤设置一个小的负奖励（例如 $r_t = -1$ ）。当智能体成功完成整个定理的证明时，累积负奖励的减少本身就体现了优化的目标——找到证明所需的步数越少，总回报越高。一个更精巧的设计是处理“分目标”的情况：如果一个战术将目标分解为多个独立的子目标（例如证明 $P \wedge Q$ 分解为证明 P 和证明 Q ），其回报定义为所有子目标证明路径中最长分支的负步数之和。这鼓励智能体平衡各子目标的难度，而不是只优化总步数。
- **折扣因子 (γ)**：通常设为 1，因为证明是一个有限步长的片段任务，无需对未来进行指数衰减。

智能体的目标，是学习一个策略 $\pi(a|s)$ ，使得在整个证明片段上获得的期望累积奖励（回报）最大化，这等价于最小化找到证明所需的平均步数。

核心组件一：证明网络——理解状态与产生直觉

要学习策略 π ，智能体需要一个能够理解和表示证明状态的模型。AlphaProof 使用一个庞大的、拥有 30 亿参数的编码器-解码器 Transformer 模型作为其“证明网络”。这个网络扮演着两个关键角色，类似于人类数学家的两种能力：

1. **策略网络 (Policy Network)**：对应于“下一步该怎么做”的直觉。网络的解码器部分，接收由编码器产生的状态表示，并输出一个在可能战术上的概率分布 $\pi(a|s)$ 。这个分布不是均匀的，而是网络基于从数百万次训练中学习到的模式，对当前状态最可能有效的战术进行“排名”。它告诉智能体：在这个证明状态下，哪些战术更值得尝试。

2. 价值网络 (Value Network): 对应于“这个局面有多好/多难”的评估。网络的另一个输出头, 估计从当前状态 s 出发, 到最终完成证明的期望回报 $V(s) \approx E[G_t | s_t = s]$ 。直观上, $V(s)$ 反映了完成当前剩余证明的“难度估值”。一个接近 0 的 V 值意味着接近完成; 一个很大的负值意味着还有很长的路要走, 或者可能走进了死胡同。

这个网络的训练分为三个阶段。首先是大规模预训练, 在数百亿代码和数学文本语料上进行标准的语言建模, 使其掌握数学语言和逻辑结构的基本语法。然后是监督微调, 在 Mathlib 等人工撰写的形式化证明库中的“状态-下一个战术”配对数据上进行训练, 让网络初步学会模仿人类证明者的常见步骤。但最重要的阶段是接下来的强化学习, 这才是智能体形成自己“直觉”的关键——通过数百万次自我对弈的试错, 网络逐渐学会区分有希望的证明路径和死胡同。

核心组件二: 与环境的交互与搜索——规划与试错

仅仅有证明网络还不够, 它提供的策略和价值估计是即时、局部的。为了找到一个完整证明, 需要将这种局部直觉与前瞻性的搜索结合起来。AlphaProof 采用了受 AlphaZero 启发的蒙特卡洛树搜索算法, 但针对证明任务进行了关键改造。

搜索过程构建一棵树, 节点是证明状态, 边是战术动作。从初始定理状态 (根节点) 开始, 反复进行以下循环:

- 选择: 从根节点开始, 递归地选择子节点, 直到到达一个未完全展开的叶节点。选择的标准是权衡“利用” (选择价值估计高的分支) 和“探索” (选择访问次数少但有潜力的分支), 使用类似 PUCT 的公式, 其中策略网络的先验概率 $\pi(a|s)$ 作为探索的引导。这个机制确保智能体既不会固守已知的好路径, 也不会盲目尝试所有可能性。
- 扩展与评估: 当到达一个叶节点 s_L , 调用证明网络为其生成 K 个候选战术 (根据策略分布采样), 并在 Lean 环境中尝试执行。每个成功的战术产生一个新的子节点 (新的证明状态)。然后用价值网络为这个新状态 s_L 估值 $V(s_L)$ 。
- 回传: 将叶节点的估值 $V(s_L)$ 沿着选择的路径反向传播, 更新路径上各节点-动作对的访问次数和平均价值估计。

一个关键的改造是针对证明中常见的“分目标”。例如, 一个战术可能将目标“证明 $A \wedge B$ ”分解为两个独立的子目标“证明 A ”和“证明 B ”。在搜索树中, 这创建了一个特殊的“AND 节点”。从这个节点出发, 智能体需要选择先解决哪个子目标。搜索算法会优先处理估值最差 (即看起来最难) 的子目标, 因为整个证明的完成时间取决于最长分支——这体现了“先啃硬骨头”的证明策略, 与人类数学家的直觉一致。

搜索结束时, 根据根节点下各动作的访问次数, 可以得到一个经过前瞻搜索精炼后的改进策略。智能体可以据此选择下一步行动, 或者直接使用整个搜索树找到的证明路径。

AlphaProof 的卓越能力并非仅来自精妙的网络和搜索算法, 更源于其革命性的训练范式, 特别是两个核心思想: 通过自动形式化构建大规模训练课程, 以及测试时强化学习。

核心创新一: 自动形式化与大规模课程学习

要让强化学习智能体变得强大, 需要让它“见多识广”——在极其多样化、海量的问题上练习。然而, 人工将数学问题形式化成 Lean 语言是极其耗时的工作。AlphaProof 破解这一瓶颈的方法是自动形式化。它使用一个经过微调的大型语言模型 (基于 Gemini), 将约 100 万个自然语言数学问题 (来源广泛, 从中学数学到奥数题) 自动翻译成 Lean 语句。这个过程并不要求 100% 准确, 因为即使翻译有偏差, 生成的 Lean 语句本身也是一个语法正确、可验证的新数学命题。这套系统最终生成了约 8000 万个形式化问题, 构成了一个前所未有的、规模庞大的训练数据集。

一个中央“匹配器”系统动态地从这些问题库中选取题目分配给分布式“执行器”智能体去尝试证明或证伪。题目选择的优先级基于“趣味性”——那些尚未解决、或时而能解时而不能解的问题会被优先选择。对于多次尝试失败的问题, 系统会分配更多的计算资源 (搜索模拟次数) 去攻坚。智能体在尝试中产生的成功证明/证伪数据 (状态战术序列) 被收集起来, 用于持续更新证明网络的参数。这个过程形成了一个正反馈循环: 智能体解决问题的能力越强, 就能解决更多样、更难的问题, 从而产生更高质量的训练数据, 进一步改进网络。这种从

大规模、自主生成的经验中学习，正是 AlphaProof 超越以往基于固定数据集的监督学习方法的关键。

核心创新二：测试时强化学习

即便经过大规模训练，在面对一个全新的、极其困难的奥数题时，通用的智能体可能仍然无法在有限的搜索预算内找到证明。AlphaProof 的另一个突破是引入了测试时强化学习（TTTL）。其思想朴素而强大：既然通用训练能让智能体获得广泛的数学能力，那么能否针对单个“硬骨头”问题，进行一场短暂的、高强度的“特训”呢？

具体流程如下：当面对一个目标难题 T 时，系统首先会围绕 T 自动生成一个庞大的、相关的“变体问题”集合 $\{V_T\}$ 。这些变体通过大语言模型生成，可能是 T 的简化版本（如减少一个条件）、类比版本（如将代数结构换成拓扑结构）、推广版本，或是通过程序化方式对 T 的假设进行微小扰动得到的。这些变体与 T 本身共同构成一个针对该特定问题的、量身定制的训练课程。

然后，AlphaProof 启动一个聚焦的强化学习循环，但这次训练数据不再是 8000 万个通用问题，而仅仅是 T 及其变体集合。智能体（从一个训练好的通用模型初始化）在这个小范围、高相关性的课程上快速进行自我对弈和学习。这个过程可能持续数天，消耗大量计算资源。

其结果是，智能体能够深度适应 T 所涉及的特定数学结构和难点，常常能发现通用模型搜索几天也找不到的证明策略。这模仿了人类数学家钻研一个难题时的行为：尝试各种特例、寻找引理、从简单情况推广。测试时强化学习成功地将这种“问题特定”的深度探索机制自动化了。

AlphaProof 在多个维度上展示了其强大的能力：

- 解决高难度问题：在 2024 年国际数学奥林匹克竞赛中，它解决了三道非几何难题，包括全场最难的第 6 题。结合其专用的几何系统 AlphaGeometry 2，总分达到了银牌水平。这是人工智能首次在顶级数学竞赛中获得奖牌级别的表现。
- 验证可靠：所有输出都是经过 Lean 内核验证的正式证明，其可靠性等同于数学公理体系本身，彻底杜绝了“幻觉”或似是而非的推理。
- 发现新颖证明：通过强化学习和自我对弈，智能体有时能找到与人类已知证明不同、但同样正确且可能更简洁的证明路径。这种能力超越了单纯的模仿，展现了某种意义上的“创造性”。
- 效率随经验提升：随着训练的进行，智能体不仅解决能力变强，解决相同问题所需的搜索步数也在减少，表明其价值网络和策略网络确实学到了更有效的启发式信息。它不仅在“做对”，还在“做得更好”。

然而，在欣赏这些成就的同时，我们也必须清醒地认识到 AlphaProof 当前的局限：

- 巨大的计算成本：无论是训练阶段（数万 TPU 日）还是针对单题的测试时强化学习（数 TPU 日），所需的计算资源都极其庞大，远超学术机构的通常预算。这使得 AlphaProof 更像一个“国家实验室级别”的科研装置，而非普通研究者可用的工具，也引发了关于可及性和可复现性的担忧。
- 领域仍受限：目前的成功主要集中在奥林匹克数学和本科低年级竞赛数学的范围内。这些问题虽然很难，但属于“封闭世界”——所有需要的概念和定理都已在 Mathlib 等库中良好形式化。将系统拓展到数学研究前沿面临巨大挑战，因为那涉及定义新概念、提出新猜想，而不仅仅是解决已有形式化的问题。
- 几何处理的短板：由于当前形式化几何库（Mathlib）对奥林匹克风格的平面几何支持不足，AlphaProof 不得不将几何问题交给专门的 AlphaGeometry 2 系统处理。这种“分工”虽然解决了问题，但也暴露了形式化数学库的发展本身就是一个重大挑战：如果某个领域的知识尚未被形式化，AI 就无法触及。
- “创造性”的本质争议：AlphaProof 的“创造性”主要体现在搜索和组合已有知识以找到新路径上。这与人类数学家提出全新概念、构建全新理论框架的“高层创造性”仍有本质区别。正如 DeepMath-Creative 基准测试所揭示的，当前大语言模型在面对需要真正构造性思维（如构造特定反例）或开放性问题时，仍会频繁出现方向错误、逻辑缺陷或无效的冗长推理。其性能很大程度上依赖于对训练数据中模式的记忆和重组，而非深度的概念性理解或真正的灵感。这种“创造性”究竟是一种新的智能形式，还是对已有知识的精巧重组？这个问题，或许将伴随 AI for Math 的整个发展历程。

AlphaProof 的成功，标志着自动定理证明从“玩具问题”走向“竞赛难题”的关键一跃。它证明了神经符号融合的路径是可行的——神经网络的直觉引导，加上形式化系统的严谨验证，可以产生优越的表现。然而，从奥林匹克难题到前沿研究，从问题求解到理论构建，前方的道路依然漫长。但无论如何，一扇新的大门已经打开。

4.4 当前成功案例 2：基于 LLM 的智能体

数学研究长期以来依赖数学家个人的直觉、推理与创造力。然而，随着大语言模型（LLM）能力的飞速提升，一种全新的研究范式正在萌芽，由 AI 智能体深度参与甚至主导数学发现。本节将简要介绍 **Aletheia**：一个基于 **Gemini Deep Think** 构建的数学研究智能体，它能够针对研究级数学问题进行迭代式生成、验证和修正，并已成功产出完全由 AI 生成的数学论文。

基于大语言模型的智能体，是指以大型语言模型为核心，通过集成规划、记忆、工具使用等机制，使其能够感知环境、自主决策并执行复杂任务。**Aletheia** 正是遵循这一理念设计的智能体，其架构围绕三大核心支柱，既能应对高难度推理，又能借助外部工具确保结果的准确性与可验证性。

首先，**Aletheia** 配备了 **Gemini Deep Think** 作为其深度推理引擎，这是一个针对复杂数学和逻辑推理优化的高级语言模型，具备强大的多步推导能力和长上下文理解，能够处理奥数级难题和博士级别的专业问题，同时记忆和关联多个中间结果。其次，它实现了推理时扩展计算，通过多轮“思考-验证-修正”循环不断深化，直到得出可靠结论。此外，**Aletheia** 深度集成了 **Google** 搜索和网页浏览功能，使其能够实时验证定理、查找文献、避免编造错误，并通过外部计算工具校验复杂表达式。这种工具调用机制将模型从封闭的知识回忆中解放出来，使其能基于最新信息进行探索，从而有效终结“幻觉”与文献错误。

Aletheia 的工作流程是一个典型的迭代式生成-验证-修正闭环，适用于从具体计算到理论构建的多种任务：

1. 问题输入：研究者以自然语言提出问题。例如，“计算算术 **Hirzebruch** 比例性原理中的特征权重”或“分析 **Erdős** 问题数据库中第 47 号猜想的现状”。
2. 生成多轮推理：基于输入问题，模型首先生成一个初步的解题思路或证明草图。然后，利用推理时扩展计算，模型对每一步进行深入推演，如果遇到障碍，它会回溯并尝试新的路径。在生成过程中，模型可随时调用搜索工具检查是否存在已知结果，或验证某个中间引理是否已经被证明。
3. 验证与修正：完成初步结果后，模型进行自我验证：检查逻辑一致性、计算正确性。若发现错误或不一致，它回到推理阶段修正，重复迭代，直到满意为止。
4. 输出解答：最终输出完整的问题解答，并附上推理过程和文献引用。

这个流程看似简单，但其中蕴含的“自主迭代”能力，是将大语言模型从“问答工具”转变为“研究伙伴”的关键一步。**Aletheia** 已在多个前沿数学问题上展现了强大能力，以下两个案例最具代表性。

案例一：完全自主生成的数学论文

Aletheia 最引人注目的成就，是它完全自主地完成了一篇数学论文中的数学内容的生成，从问题理解、理论构建到最终计算，全程没有任何人工干预。

- 问题背景：**Feng**、**Yun** 和 **Zhang** 在先前的工作中建立了（高阶）算术 **Hirzebruch** 比例性原理，这是一个将 **shtukas** 模叠的算术体积与 **L**-函数的微分算子联系起来的深刻结果。该公式涉及某些被称为“特征权重”（**eigenweights**）的结构常数，但原作者仅在简单情形下计算了这些权重，一般情形尚未解决。
- **Aletheia** 的贡献：研究团队将任务交给 **Aletheia**，计算所有典型群下的特征权重。智能体在探索过程中，自主发现这些权重与对称群表示论之间存在深刻的联系。它进而运用代数组合学工具（如 **Young** 图、**Schur** 函子）推导出了一般情形下的封闭公式。整个研究过程，从问题理解、理论构建到最终计算，完全由 **Aletheia** 独立完成。

这一案例的意义不仅在于解决了一个具体问题，更在于它展示了 AI 能够进行概念性发现——它识别出了特征权重与表示论之间的关联，并主动调用相关理论工具构建解决方案。这超越了简单的模式匹配或计算执行，进入了数学研究的核心地带。

案例二：半自主攻克 **Erdős** 开放问题

如果说第一个案例展示了 AI 的“深度”，那么第二个案例则展示了 AI 的“广度”和“系统性”。

- 问题背景：**Erdős** 问题数据库由数学家 **Bloom** 整理，其中包含约 700 个标记为“开放”的猜想。这些问题难度不一，其中许多因文献分散或表述模糊而长期未解。对于人类数学家来说，逐一审视这 700 个问题是一项耗时且枯燥的工作。

- 研究策略：研究团队采用了一种混合方法：AI 先进行自然语言验证和文献检索，缩小搜索空间，再由人类专家评估正确性与新颖性。这种“AI 初筛 + 人类确认”的协作模式，充分发挥了双方的优势：AI 不知疲倦、阅读广泛，人类则具备深度判断和创造性理解的能力。

- Aletheia 的贡献：Aletheia 系统性地评估了全部 700 个开放问题，结果令人瞩目：

- 自主解决 4 个问题：AI 生成的解法看似新颖，经人类专家确认正确且此前未被记录。这些问题本身可能并非极其困难，但它们的“开放”状态意味着此前无人成功解决，AI 填补了这一空白。
- 辅助识别 9 个问题的已有解法：通过文献检索，AI 辅助发现这些问题实际上已在某些不为人知的论文中被解决，这些论文可能发表在小众期刊上，或用晦涩的语言写成，因此被主流数据库遗漏。其状态从“开放”被更新为“已解决”。

研究团队指出，这些问题的“开放”状态多源于文献的隐蔽性而非难度本身。换句话说，有些问题之所以长期未解，不是因为它们太难，而是因为它们的解法被埋没在了浩瀚的文献海洋中。AI 通过强大的搜索和推理能力，有效克服了这一点，它不仅能阅读和理解，还能跨越语言和期刊的障碍，建立起知识的连接。

Aletheia 智能体的成功标志着 AI 从“解题工具”向“研究伙伴”的跨越。它结合深度推理引擎、推理时扩展计算和工具调用能力，共同构建了一个能够处理前沿数学问题的自主系统。

从完全自主生成论文到系统性攻克 Erdős 问题，Aletheia 证明了大语言模型可以在数学发现中扮演实质性角色。未来，随着模型推理能力的进一步提升和工具生态的完善，我们有理由期待更多智能体深度参与数学研究。

4.5 大语言模型的创造力

当前，大语言模型在数学领域的应用研究呈现爆炸式增长。主流的评估基准（如 GSM8K、MATH 等）和模型优化方向，几乎都集中在模型的推理能力上——即模型能否一步步推导出正确答案，解决标准化的数学问题。这一方向的成果令人振奋，AlphaProof 在 IMO 上的表现就是最好的证明。

然而，数学不仅仅是逻辑推导的机械组合。数学的灵魂在于创造力——提出前所未有的概念，发明巧夺天工的方法，构造颠覆认知的反例。从欧几里得几何到非欧几何的飞跃，从整数到复数的概念扩展，从费马大定理到庞加莱猜想的最终证明，数学史上每一次重大突破，本质上都是创造力的胜利。如果说推理能力让 AI 成为了一名优秀的“解题者”，那么创造力才是让它成为真正“数学家”的关键。

但创造力是什么？它能否被评估？当前的 AI 模型是否具备这种能力？这些问题，正成为 AI for Math 领域下一个前沿的探索方向。

目前，学术界对 LLM 数学创造力的研究关注甚少，缺乏系统性的评估标准和专门的数据集。为了科学地评估创造力，我们需要一个可操作的分析框架。借鉴数学史和数学哲学的研究，DeepMath-Creative 这篇论文提出了一个基于三个关键维度的评估模型：

维度一：新概念的生成

这是最高层次的创造力，体现在引入前所未有的数学概念或思想，从而开辟全新的研究领域。概念是数学的语言，新概念的诞生往往意味着整个学科的重构。

黎曼度量的出现，为现代微分几何奠定了基础。它不仅在抽象层面定义了曲率，更重要的是，它为爱因斯坦的广义相对论提供了精确的数学语言——物质告诉时空如何弯曲，时空告诉物质如何运动。没有黎曼几何，广义相对论或许仍停留在哲学思辨的层面。

可微流形这一概念，将局部的微积分可行性与全局的复杂拓扑结构完美结合。它使得我们可以在球面、环面这样的弯曲空间上进行微积分运算，成为理论物理和几何建模的基石。今天，从机器学习的流形假设到宇宙的时空模型，无处不见流形的身影。

群论诞生于伽罗瓦对代数方程可解性的探索。这个当时年仅二十岁的数学家，在决斗前夜写下的手稿中，开创了一种全新的结构性思维。今天，群论统一了几何、代数和数论中的对称性研究，从晶体结构到量子力学，它的应用无处不在。

拓扑空间则超越了传统的距离概念（度量空间），只通过“开集”来定义连续性和邻近性。这种极致的抽象，将数学分析提升到了更本质的层面——原来，连续性的本质不在于距离，而在于“邻域”这个概念本身。

维度二：新方法的发明

创造力的另一种形式是发明全新的技术或工具，用以解决此前难以攻克的复杂问题。新方法往往不是直接给出答案，而是开辟了一条通向答案的道路。

例如广义函数论的发明，源于量子力学中狄拉克 δ 函数带来的困惑。这个函数在一点无穷大、其余点为零，积分却为 1——在经典函数论的框架下，它根本无法定义。数学家不得不突破“函数是点对点映射”的固有观念，将函数视为对测试函数的作用泛函。这一突破不仅为量子力学提供了严格的数学基础，更成为现代偏微分方程理论的核心工具。

Bochner 技术则是几何分析中的一个里程碑。它将几何（曲率）与分析（拉普拉斯算子）巧妙地联系起来，通过曲率条件来约束流形的拓扑性质。这种“以分析攻几何”的方法，为证明诸多深刻的几何定理提供了优雅的途径。

维度三：新例子的创建

通过构造具体的例子（尤其是反例）来检验命题的边界，是推动理论发展的重要驱动力。一个精妙的反例，往往比十个正例更能深化我们对概念的理解。

1956 年，约翰·米尔诺构造了一个七维流形，它在拓扑上与标准七维球面相同（是同胚的），但在微分结构上却不同（不是微分同胚的）。这一“奇异球”颠覆了人们对高维空间的直观理解——原来，同一个拓扑空间上可以存在不同的光滑结构。这一发现直接开创了奇异流形理论和微分拓扑这个全新领域。

更早的 1872 年，魏尔斯特拉斯构造了一个处处连续但处处不可导的函数。在此之前，数学家们普遍认为连续函数总是可导的，至少在大多数点上。这个反例直接冲击了当时的分析学基础，迫使数学家重新审视函数、连续和可导这些基本概念，推动了实分析理论的严谨化。它提醒我们：直觉可能会欺骗我们，只有严格的逻辑才能指引真理。

这三个维度，新概念、新方法、新例子，共同构成了数学创造力的完整光谱。它们层次不同，但同等重要。一个新概念可能开辟一个领域，一个新方法可能解决一批问题，一个新例子可能修正一个时代的方向。

人类数学家创造新概念和新方法的过程（如草稿、失败的尝试、灵光一现的瞬间）往往是混乱、非结构化和充满偶然性的。我们很难将这些复杂的认知轨迹还原成可供大语言模型训练的大规模、标准化数据集。更重要的是，当模型生成一个“新颖”的概念或方法时，我们很难判断：这是真正的原创性，还是仅仅是训练数据的巧妙重组？现有自动评估机制几乎无法区分这两种情况。

与抽象的概念和方法不同，具体的、可构造的数学例子是“有形的”。一个例子是否正确、是否构成反例，可以通过严格的数学验证来判断，这是数学这门学科独特的优势。这种可验证性，为机器创造力的评估提供了相对客观和可行的标准。

数学创造力的价值远不止于解决已知问题。回顾数学发展的每一个飞跃时刻——从非欧几何的诞生到拓扑学的兴起，从伽罗瓦群论的提出到怀尔斯对费马大定理的证明——无一不是创造力的结晶。正是新概念的引入、新方法的发明以及新例子的构造，不断重塑着数学的版图，拓展着人类理性的边界。在人工智能时代，赋予大语言模型数学创造力，其意义将更为深远：它不仅是检验机器是否具备真正智能的试金石，更可能催生一种“AI 数学家”或“AI 合作者”，能够以前所未有的速度探索数学猜想、生成证明思路，甚至在人类难以触及的复杂结构中发现隐藏的模式与联系，从而加速科学发现的进程，推动数学及相关学科（如理论物理、密码学、计算生物学）的变革性突破。

然而，通往这一愿景的道路上横亘着重大的核心技术挑战。其中最为核心的难题，莫过于在强化学习框架下难以定义和设计能够有效评估数学创造力的奖励函数。强化学习的有效性高度依赖于一个清晰、可量化且能够引导模型逐步逼近目标的奖励信号。但在数学创造力这一语境下，这一要求遭遇了根本性的困境：

1. 新颖性的主观性与语境依赖性：什么是“新颖”？一个构造对于模型自身可能是未知的，但对于整个数学界而言或许已是常识。真正的创造性需要跳出训练数据的分布，产生具有“历史性”意义的全新知识，而这种新颖性的判断往往需要深厚的数学洞察力，无法用简单的统计指标（如与训练数据的不相似性）来度量。一个构造是否“巧妙”、“深刻”，更是一种高阶的审美判断，难以形式化为标量奖励。
2. 长期影响的不可预测性：正如黎曼度量或奇异球的价值是在数十年后才被充分认识的，数学创造的真正影响往往在遥远的未来才能显现。在强化学习的每一步，模型无法预知其当前构造的微小片段在未来会产生

何种连锁反应。这种延迟且稀疏的奖励，使得传统的基于短期回报的优化方法（如策略梯度）几乎无从着力。

3. 探索空间的指数级稀疏性：数学对象（如流形、群、函数）的构造空间是无限且高度非结构化的。真正具有创造性的构造（如魏尔斯特拉斯函数）犹如沧海一粟。模型在如此巨大的空间中随机游走，几乎不可能仅靠“试错”偶然碰见一个有价值的例子。没有有效的内在动机或启发式引导，强化学习将陷入无尽的无效探索。
4. 正确性与创造性的张力：数学创造力必须建立在逻辑严密的基础之上。一个“新颖”但自相矛盾的反例毫无价值。因此，奖励函数必须在鼓励新颖性与强制正确性之间取得微妙平衡。过于强调正确性，模型将倾向于保守地复制已知知识；过于强调新颖性，模型则可能生成大量无意义的、逻辑断裂的构造。如何设计一个既能容忍探索中的错误，又能最终引导模型走向逻辑闭合的奖励机制，是当前技术的盲区。
5. 过程 vs. 结果的评估困境：数学创造的价值往往蕴含于思考过程本身——从模糊的直觉到严谨的构造，从失败的尝试到顿悟的瞬间。然而，现有的强化学习主要关注最终输出的结果（如反例是否正确）。一个能够捕捉模型在构造过程中表现出的“洞察力”、“类比迁移能力”或“对矛盾的处理能力”的奖励信号，几乎不存在。

这些挑战共同指向一个深层次的问题：我们尚缺乏一种能够理解和评价数学创造的计算理论。创造力不是一个可以简单量化的指标，而是一种复杂的、多维度的、与语境深度绑定的现象。要让它成为可计算、可优化的目标，我们需要对创造力本身有更深刻的理解。

未来的研究可能需要超越单一的、外部的奖励函数，转而探索内在动机机制（如基于好奇心或信息增益的奖励）、元认知机制（让模型学会评价自己的思考路径）以及人机协作评估（让人类数学家介入奖励塑造）等复合范式。

但无论如何，正视并攻克这一核心技术挑战，将是迈向具有真正数学创造力的智能系统的必经之路。因为最终，我们期待的不只是一个能解题的机器，而是一个能思考、能创造、能与我们共同探索数学未知领域的伙伴。这个伙伴可能不会像人类数学家那样拥有灵光一现的瞬间，但它可能以一种完全不同的方式——通过海量的探索、精确的模式识别、不知疲倦的组合——发现人类难以独自看到的数学真理。

参考文献

- [1] Mohammed Abouzaid, Andrew J Blumberg, Martin Hairer, et al. “First Proof”. In: *arXiv preprint arXiv:2602.05192* (2026).
- [2] Josh Achiam et al. *GPT-4 technical report*. Tech. rep. arXiv preprint arXiv:2303.08774, 2023.
- [3] Zhangir Azerbayev et al. “Llemma: An open language model for mathematics”. In: *The Twelfth International Conference on Learning Representations*. 2024.
- [4] Maissam Barkeshli, Michael R Douglas, and Michael H Freedman. “Artificial intelligence and the structure of mathematics”. In: *arXiv preprint arXiv:2604.06107* (2026).
- [5] Xiaoyang Chen and Xiang Jiang. “Can LLM Generate Interesting Mathematical Research Problems?” In: *arXiv preprint arXiv:2603.18813* (2026).
- [6] Xiaoyang Chen et al. “DeepMath-Creative: A Benchmark for Evaluating Mathematical Creativity of Large Language Models”. In: *arXiv preprint arXiv:2505.08744* (2025).
- [7] Ziru Chen et al. “Seed-Prover 1.5: Mastering Undergraduate-Level Theorem Proving via Learning from Experience”. In: *arXiv preprint arXiv:2512.17260* (2025).
- [8] Yuri Chervonyi, Trieu H Trinh, Miroslav Olšák, et al. “Gold-medalist performance in solving olympiad geometry with AlphaGeometry2”. In: *Journal of Machine Learning Research* 26.241 (2025), pp. 1–39.

- [9] Katherine M Collins, Albert Q Jiang, Simon Frieder, et al. “Evaluating language models for mathematics through interactions”. In: *Proceedings of the National Academy of Sciences* 121.36 (2024).
- [10] Yihong Dong and X Ma. “STP: Self-play LLM Theorem Provers with Iterative Conjecturing and Proving”. In: *International Conference on Machine Learning*. 2025.
- [11] Tony Feng. “Eigenweights for arithmetic Hirzebruch Proportionality”. In: *arXiv preprint arXiv:2601.23245* (2026).
- [12] Tony Feng et al. “Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems”. In: *arXiv preprint arXiv:2601.22401* (2026).
- [13] Tony Feng, Trieu H Trinh, G Bingham, et al. “Towards Autonomous Mathematics Research”. In: *arXiv preprint arXiv:2602.10177* (2026).
- [14] Simon Frieder et al. “Mathematical capabilities of ChatGPT”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 73421–73445.
- [15] K Georgiev et al. “Mathematical Exploration and Discovery at Scale”. In: *arXiv preprint* (2025).
- [16] Thomas Hubert, R Mehta, L Sartran, et al. “Olympiad-level formal mathematical reasoning with reinforcement learning”. In: *Nature* 651 (2026), pp. 607–613.
- [17] Yong Lin, Shange Tang, Chi Jin, et al. “Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving”. In: *arXiv preprint arXiv:2502.07922* (2025).
- [18] Jiawei Liu et al. “Numina-Lean-Agent: An Open and General Agentic Reasoning System for Formal Mathematics”. In: *arXiv preprint arXiv:2601.14027* (2026).
- [19] Filip Marić. “A survey of interactive theorem proving”. In: *Logic in Computer Science II*. Belgrade: Matematički institut SANU, 2015, pp. 173–223.
- [20] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.
- [21] Y Xin et al. “Scaling up Multi-Turn Off-Policy RL and Multi-Agent Tree Search for LLM Step-Provers”. In: *arXiv preprint arXiv:2509.06493* (2025).

第五章 AI 构造数学猜想反例

5.1 概述

数学的核心驱动力不仅是证明定理，更是划定定理的边界。构造反例，正是揭示“猜想在何处失效”的艺术。一份漂亮的证明告诉我们“真理止于此”，而一个精彩的反例则告诉我们“谬误始于彼”。

构造反例是数学逻辑中最具颠覆性的艺术——它不仅宣告一条路径的终结，更为真理划定不可逾越的边界。在数学史上，一个精巧的反例往往比十个常规例证更能揭示概念的内在裂隙，迫使公理体系修正、定义重构，甚至催生全新的研究范式。人工智能通过深度搜索、强化学习或生成式模型的暴力遍历与智能剪枝，能够悄无声息地潜入人类想象所不及的稀疏角落，这种方法并不试图取代数学家以灵感点燃反例的神来一笔，而是将反例发现从依赖偶然洞见的个人艺术，转化为可规模化、可系统复现的工程化流程。

5.2 AI 构造图论猜想反例：基于强化学习

组合数学与图论是数学中充满魅力与挑战的分支。它们研究的对象往往是离散的、有限的、结构化的。在这些领域中，数学家们提出了大量关于极值组合与图参数关系的猜想。这些猜想通常源于数值实验的观察、特定实例的归纳，或是对已知结论的推广。它们像一盏盏灯塔，指引着探索的方向。

然而，数学猜想的历史告诉我们：并非所有看似合理的命题都是正确的。要确定一个猜想是否成立，特别是要找到一个反例来推翻它，往往是一项极其艰巨的任务。问题的本质在于搜索空间极其庞大，而目标（一个违反猜想条件的结构）可能只是其中的“沧海一粟”。

以图论为例，一个包含 n 个顶点的简单无向图，其所有可能的图的数量是 $2^{\binom{n}{2}}$ ，这个数字随着 n 的增长而指数级爆炸。当 $n = 20$ 时，可能的图的数量已经超过了 10^{57} 。显然，通过人力枚举或传统的穷举算法来检验所有可能性是完全不可行的。

长期以来，数学家们依靠直觉、结构性的见解和理论推导来猜测可能的反例形态。他们也会运用启发式搜索、线性规划等算法工具进行辅助。例如，对于某些可以表达为线性规划的问题，商用求解器可以非常高效地找到最优解或反例。然而，许多组合问题无法简洁地表达为线性规划，或者其搜索空间的结构非常复杂，传统优化方法难以有效探索。

于是，我们面临一个核心问题：当面对一个广袤、离散且结构复杂的搜索空间，而我们需要在其中寻找一个满足特定（通常是“病态”或非直觉）条件的“珍稀”结构时，是否存在一种系统性的方法，能够不依赖于深刻的问题洞见，而是以“黑盒”的方式有效地进行探索和发现？

这样的方法应该具备以下特性：通用性（能处理广泛的问题表述）、无需过多领域知识（减少对问题特定启发式的依赖）、能有效探索高维离散空间、具备一定的“学习”能力以从失败尝试中积累经验。这恰恰是强化学习，特别是策略搜索方法可以发挥作用的舞台。

强化学习的核心思想是让一个智能体通过与环境的交互来学习如何执行一系列动作，以最大化累积的奖励。这一范式可以非常直观地映射到我们的数学构造问题上：

- 智能体：一个试图构造数学对象（如图、矩阵、集合族）的算法。
- 环境：定义了构造对象的规则（如构造一个 n 阶简单图）以及最终评估对象好坏的度量标准。
- 状态：在构造过程中某个时间点，已做出的部分决策（如已经确定的部分边）。
- 动作：在当前状态下，智能体可以做出的一个具体决策（如下一步是添加某条边，还是不加）。
- 奖励：当整个对象构造完成后，根据其是否违反猜想（或违反的程度）计算出的一个分数。我们的目标是最小化这个分数（对于下界猜想）或最大化它（对于上界猜想），以找到反例。

具体来说，构造一个组合对象的过程可以被看作按某种顺序生成一个字符串。例如，构造一个 n 个顶点的图，我们可以按固定顺序考虑所有可能的 $\binom{n}{2}$ 条边，对每条边决定“保留”（编码为 1）或“删除”（编码为 0）。

这样，任何一个图都唯一对应一个长度为 $\binom{n}{2}$ 的二进制字符串。智能体的任务就是一步步生成这个字符串。

关键思想在于，智能体对它所解决的“数学问题”本身一无所知。它只是学习一个策略——一个从“当前已生成的字符串片段”到“下一个字符概率分布”的映射。这个策略通过反复试错来调整：它生成许多完整的字符串（即构造许多图），每个字符串都会根据奖励函数得到一个分数。然后，智能体分析那些得分高（或低，取决于目标）的构造，并调整自己的策略，使得未来更倾向于做出与这些成功构造相似的决定。

为了更好的理解上述方法，我们介绍 Wagner 的工作。Wagner 选择了深度交叉熵方法作为基础工具。

交叉熵方法本质上是一种基于策略搜索的进化算法。其核心是维护一个参数化的策略网络（通常是一个神经网络），并通过迭代地“生成-评估-进化”来改进策略。我们结合组合构造问题，分解其工作流程。

假设我们要构造一个离散对象 x ，它由一系列决策 $a_1, a_2, \dots, a_T$ 生成，每个决策 a_t 属于一个有限的动作集合 $\mathcal{A}$ 。例如，在构造图时， $T = \binom{n}{2}$ ， $\mathcal{A} = \{0, 1\}$ 。存在一个奖励函数 $R(x)$ 评价最终对象 x 的优劣。我们的目标是找到最大化（或最小化） $R(x)$ 的对象 x^* 。

策略 π_θ 是一个参数为 θ 的函数，它给出在给定当前部分序列（状态 s_t ）下，选择各个动作的概率分布： $\pi_\theta(a|s_t)$ 。在顺序生成设定下，状态 s_t 通常就是已经做出的决策序列 $(a_1, \dots, a_{t-1})$ 。

我们希望学习参数 θ ，使得根据策略 π_θ 生成的对象 x 具有高的期望奖励 $\mathbb{E}_{x \sim \pi_\theta}[R(x)]$ 。

深度交叉熵方法通过以下迭代步骤逼近最优策略：

1. 采样阶段：根据当前策略 π_θ ，独立地生成 N 个完整的对象 $x_1, x_2, \dots, x_N$ ，并计算每个对象的奖励 $r_i = R(x_i)$ 。
2. 选择阶段：将所有样本按奖励排序，只保留排名前 $K = \lceil \rho N \rceil$ 的精英样本，其中 ρ 是精英保留比例（通常取 5% 到 20%）。
3. 更新阶段：分析这些精英样本的生成轨迹，即每一步的决策序列。对于每个精英样本的生成序列 $(a_1, \dots, a_T)$ ，构造训练数据：对于每一步 t ，状态 $s_t = (a_1, \dots, a_{t-1})$ ，动作 a_t 。然后，通过最小化交叉熵损失来更新策略：

$$L(\theta) = - \sum_{(s,a) \in \text{精英轨迹}} \log \pi_\theta(a|s)$$

损失函数 $L(\theta)$ 的交叉熵形式是核心。对于一个给定的（状态，动作）对 (s, a) ，我们希望策略网络在该状态下输出动作 a 的概率 $\pi_\theta(a|s)$ 尽可能大。 $-\log \pi_\theta(a|s)$ 衡量了预测概率与完美预测（概率为 1）之间的差异。最小化所有精英轨迹上这个差异的总和，就是在“拉伸”策略网络，使其决策分布向精英样本的经验分布靠拢。

这个过程反复进行，直到找到满意的构造或达到预设的迭代次数。算法 5.1 是该深度交叉熵方法的伪代码。

为了将上述算法付诸实践，我们需要将部分构造序列 s_t 编码成神经网络可以处理的固定维度的向量。Wagner 采用了简单而通用的编码方案，特别适用于生成二进制字符串的任务（如图的邻接矩阵上三角部分）。

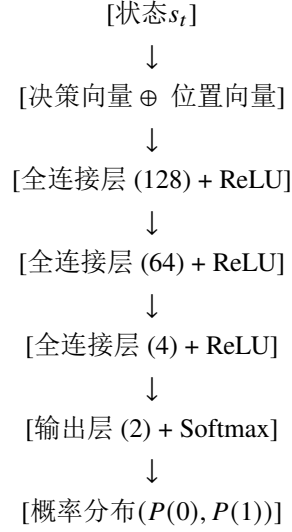
输入由两个向量拼接而成：

- **决策向量**：长度为 T ，第 i 个分量表示第 i 个位置是否已做出决策。如果已决策，则该分量为实际值（0 或 1）；如果尚未决策，则该分量为 0。这个向量记录了到目前为止的“历史”。
- **位置指示向量**：这是一个 one-hot 向量，只有第 t 个分量为 1，其余为 0。它明确告诉网络当前正在为哪个位置做决策。

为什么需要两个向量？决策向量提供了上下文，位置向量提供了焦点。网络需要结合二者来理解：“基于我已经做过的这些决定（决策向量），现在对于这个特定的位置（位置向量），我应该选择 0 还是 1 的概率更高？”

神经网络架构方面，Wagner 使用了简单的多层感知机（MLP）：

- 输入层：大小为 $2T$ （两个向量拼接）
- 隐藏层：通常为 2-3 个全连接层，使用 ReLU 激活函数。例如，文中提到使用 128、64 和 4 个神经元的三层结构
- 输出层：大小为 $|\mathcal{A}|$ ，对于二进制决策有两个神经元，通过 softmax 函数转换为概率分布 $\pi_\theta(0|s_t)$ 和 $\pi_\theta(1|s_t)$

**Algorithm 5.1:** 深度交叉熵方法 (Deep Cross-Entropy Method)

Input: 奖励函数 $R(\cdot)$,
 动作空间 $\mathcal{A}$, 序列长度 T ,
 策略网络 π_θ , 初始参数 θ_0 ,
 每轮采样数 N , 精英保留比例 $\rho \in (0, 1)$ (精英数量 $K = \lceil \rho N \rceil$),
 学习率 α , 停止条件

Output: 训练过程中找到的最佳构造 $x^* = \arg \max_x R(x)$

```

1 初始化策略网络参数  $\theta \leftarrow \theta_0$ 
2 while 未满足停止条件 (如找到反例或达到最大迭代次数) do
3   采样阶段:
4     1. 初始化样本集合  $\mathcal{S} \leftarrow \emptyset$ 
5       for  $i = 1$  to  $N$  do
6         初始化空序列  $s \leftarrow \epsilon$ 
7         for  $t = 1$  to  $T$  do
8           根据  $\pi_\theta(\cdot|s)$  采样动作  $a_t$ 
9            $s \leftarrow s \oplus a_t$ 
10        从  $s$  解码得到完整对象  $x_i$ , 计算奖励  $r_i = R(x_i)$ 
11        将  $(x_i, r_i, s)$  加入  $\mathcal{S}$ 
12   选择阶段:
13   将  $\mathcal{S}$  中样本按奖励  $r_i$  排序
14   令  $\mathcal{E}$  为排名前  $K$  的精英样本集
15   更新阶段:
16   初始化训练集  $\mathcal{D} \leftarrow \emptyset$ 
17   foreach 精英样本的生成序列  $(a_1, \dots, a_T) \in \mathcal{E}$  do
18     for  $t = 1$  to  $T$  do
19        $s_t \leftarrow (a_1, \dots, a_{t-1})$ 
20        $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t)\}$ 
21   计算交叉熵损失:
22
23
24   梯度下降更新:
25
26
27 return 训练过程中找到的最佳构造  $x^* = \arg \max_x R(x)$ 

```

$$L(\theta) = - \sum_{(s,a) \in \mathcal{D}} \log \pi_\theta(a|s)$$

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta L(\theta)$$

这种架构虽然简单, 但足以捕捉许多组合问题中决策间的依赖关系。对于图这类具有强烈结构化关系的对

象，图神经网络（GNN）可能是更自然的选择，因为它可以直接将图的结构（顶点、边）作为输入，并利用消息传递机制来聚合邻域信息。然而，简单 MLP 的通用性使得它可以快速应用于各种问题，而无需为每个新问题设计专门的网络结构。

Wagner 的论文展示了这一方法在多个图论和组合猜想上的成功应用。下面我们选取两个典型例子，深入阐述 AI 是如何工作的，以及它揭示了什么数学洞察。

示例一：特征值与匹配数之和的猜想

猜想：对于任意 $n \geq 3$ 的连通图 G ，其最大特征值 λ_1 与匹配数 μ 之和满足 $\lambda_1 + \mu \geq \sqrt{n-1} + 1$ 。

AI 设置详情：

- 对象表示： n 顶点图，编码为长度 $T = \binom{n}{2}$ 的二进制串。
- 奖励函数： $R(G) = -(\lambda_1(G) + \mu(G))$ ，目标是最小化该和值以寻找潜在反例。
- 任务规模：针对 $n = 19$ 进行搜索。
- 计算细节：每次奖励评估需要计算图的邻接矩阵的最大特征值和最大匹配，总训练迭代约 5000 轮。

AI 的发现过程：

1. 初始阶段：策略随机，生成各种稠密和稀疏的图。
2. 学习趋势：网络很快（约几百次迭代后）从奖励反馈中“领悟”到，稀疏图倾向于有更小的 λ_1 。同时，为了保持连通性（避免奖励函数可能未明确要求，但稀疏易导致不连通），它发现树是一个很好的候选类别：它们是极小的连通图。
3. 结构演化：从随机图，到稀疏非连通图，再到树，最后聚焦于一种特定的树形结构——“平衡双星”：一条中心边连接两个星图的中心，每个星有相近数量的叶子。这种结构能有效地在保持连通的同时，将最大特征值控制在较低水平，同时匹配数也较小。
4. 成功的反例：当 $n = 19$ 时，最终找到的图满足 $\lambda_1 = \sqrt{10} \approx 3.162$ ， $\mu = 2$ ，总和约 5.162，小于 $\sqrt{18} + 1 \approx 5.243$ ，成功证伪猜想。

这个案例展示了 AI 如何从零开始，通过纯粹的试错学习，发现了一种人类数学家可能未曾考虑过的图结构，并成功地用它推翻了猜想。

示例二：距离谱与 proximity 的猜想

猜想：关于图的距离矩阵特征值 ∂_i 与 proximity 参数 π ，满足 $\pi + \partial_{\lfloor 2D/3 \rfloor} > 0$ ，其中 D 是直径。

AI 设置详情：

- 奖励函数： $R(G) = -(\pi(G) + \partial_{\lfloor 2D/3 \rfloor}(G))$ ，目标是找到使该和尽可能小（甚至为负）的图。
- 任务规模：初始搜索 $n = 30$ 。
- 计算挑战：距离矩阵的特征值计算和 proximity 计算比邻接矩阵更耗时，尤其是对于非树图，需要全源最短路径算法。这限制了每轮能评估的样本数 N 。

AI 的发现过程：

1. 未能直接找到反例：经过几天训练，AI 在 $n = 30$ 时找到的最佳图其和值约为 0.4，仍为正数，不是反例。
2. 关键洞察——结构趋同：尽管不是反例，但一个强烈的模式浮现出来：在所有精英样本中，图的结构高度一致。它们都由一条长路径构成，在路径中点附近连接一个大星，而这个星的邻居们又各自形成小团。唯一的变化是这些小团的大小，如图 5.1 所示（引自原始论文）。
3. 从模式到反例：这个模式为人类研究者提供了清晰的“设计蓝图”。Wagner 等人据此手动构造了一个参数化的图族：一条长度为 d 的路径，在靠近中点的一个顶点上悬挂 m 个叶子（即星）。通过分析，他们发现当 $d = 12$ （直径 $D = 12$ ， $\lfloor 2D/3 \rfloor = 8$ ）且 n 足够大（ ≥ 190 ）时，这种“双尾彗星”图确实满足 $\pi + \partial_8 < 0$ ，从而成为反例。

这个案例凸显了 AI 在数学发现中的另一种角色——超级模式探测器。当直接搜索反例计算困难时，AI 可以通过输出一系列“接近最优”的、结构相似的候选解，来揭示潜在极值结构所必须满足的拓扑特征。这极大地缩减了人类的搜索空间，将问题从“在全体图中寻找”转化为“在具有某种特定模式的图族中优化参数”。

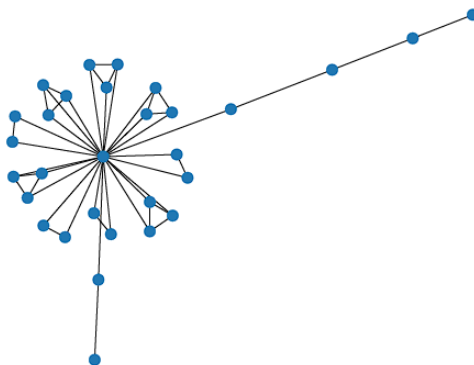


图 5.1: 强化学习构造反例

基于上述示例，可以看到强化学习用于数学构造的一般流程如下：

步骤 1：问题形式化

将具体的数学构造问题转化为强化学习框架，定义状态表示、动作空间、奖励函数和终止条件。这是最关键的步骤，需要深入理解数学问题的本质。一个好的形式化应该让奖励函数能够准确反映问题的目标，同时让状态表示包含足够的信息以供决策。

步骤 2：算法选择与实现

选择合适的强化学习算法。对于数学构造这类具有稀疏奖励、离散动作空间的问题，策略梯度方法和交叉熵方法等基于策略的方法通常表现较好。这些方法直接优化策略，而不需要学习复杂的价值函数，更适合探索空间巨大的组合问题。

步骤 3：训练过程

智能体通过与环境（即构造过程）的交互进行学习。在每个训练周期中，智能体使用当前策略生成多个完整的构造，计算每个构造的奖励，然后使用这些信息更新策略。这个过程反复进行，直到策略收敛或找到满足要求的构造。

步骤 4：结果提取与分析

训练结束后，使用学到的策略生成构造，并对其进行数学验证。如果找到了反例或极值构造，需要进一步分析其结构，以获取数学洞察。有时，即使没有找到反例，AI 发现的“接近最优”的构造也能为人类研究者提供有价值的线索。

方法的基本要点可以总结为：

1. 通用性框架：同一套强化学习代码可以用于攻击不同的数学猜想，只需更换奖励函数。这种通用性是传统方法难以比拟的。
2. 从经验中学习：强化学习智能体通过大量试错积累经验，逐渐学习到哪些决策模式更有可能产生高奖励的构造。这与人类数学家通过积累直觉和经验来指导研究有相似之处，但强化学习可以更系统、更广泛地探索可能性。
3. 处理高维组合空间：强化学习方法，特别是与神经网络结合时，能够处理极高维的状态和动作空间，这是传统穷举方法无法企及的。
4. 探索非常规结构：由于强化学习算法不依赖于人类先验直觉，它们有可能发现反直觉的、非常规的构造，这些构造可能正是人类研究者容易忽略的。
5. 稀疏奖励挑战：数学构造问题通常只有最终奖励，这使得学习非常困难。需要采用专门的技术，如课程学习（从简单问题开始逐步增加难度）或内在奖励（鼓励探索新状态），来辅助学习过程。
6. 可扩展性：一旦训练好一个策略，它可以快速生成大量高质量的构造，这对于研究构造的分布或寻找多个不同的反例非常有价值。

值得注意的是，虽然强化学习提供了强大的搜索能力，但它并不能保证找到最优解或反例。算法的成功取决于多种因素，包括问题形式化的质量、奖励函数的设计、算法的选择以及超参数的调整等。

从更广阔的视角看，强化学习在图论反例发现中的应用，代表了 AI for Math 的一种重要范式：当人类直觉和传统算法都难以企及的问题空间，AI 可以通过系统性的探索和学习，发现那些隐藏的、反直觉的结构。这些发现不仅能够验证或证伪猜想，更重要的是，它们能够丰富我们对数学结构的理解，启发新的理论方向。

5.3 PatternBoost 方法

在上一节中，我们探讨了如何利用强化学习和启发式搜索在离散的组合空间中进行高效构造。我们看到了纯强化学习方法（如交叉熵方法）的巨大潜力，同时也认识到其局限性：当构造对象的规模（序列长度）变得很大（例如几百个符号）时，一个朴素的神经网络很难有效地学习和预测整个结构。这种困难本质上是“探索”与“利用”之间的根本矛盾，在数学构造这一巨大、离散、结构复杂的空间中尤为突出。

具体来说，我们可以从两个视角来看待数学构造任务：

1. 局部视角：从一个给定的构造（比如一个图、一个矩阵）出发，通过一系列“小步”的修改（如添加或删除一条边、一个元素），尝试改进其满足目标约束的程度。这是一种“微调”式的优化，其优点是能够在已知的“好解”附近进行精细的勘探，但缺点是容易陷入局部最优，缺乏跳出当前解构模式的能力。例如，对于一个无三角形的图，局部搜索可以尝试在现有图的非三角形边上添加新边，但它很难构想出一个全新的、结构迥异的二分图布局。
2. 全局视角：从宏观上把握“好构造”的整体模式和结构特征。例如，通过分析成百上千个优秀的无三角形图，人类数学家可能会洞察到它们大多呈现出“二分图”的结构。全局视角能够提供跳出局部模式的新颖构想，但其缺点是难以精确地生成满足所有微观约束的具体实例。一个纯粹的生成模型可能学会了“二分性”这个模式，但它随机生成的图很可能仍然包含一些三角形。

在数学研究中，这种“局部调整”（如尝试修改一个已知反例）与“全局构想”（如基于对称性或极值原理猜想一个新的结构框架）通常是交替进行的。一个纯粹基于强化学习或局部搜索的方法，往往缺乏有效的全局构想能力。而一个纯粹的生成模型（如直接训练一个 Transformer 来输出构造），则可能因为无法精确满足复杂的、组合性的约束条件而生成大量无效的输出。

因此，一个自然的问题是：能否设计一种混合方法，让机器也能像数学家一样，在局部精细调整与全局模式学习之间进行有效迭代？这种方法需要既能利用局部搜索的精确性，又能汲取全局学习的创造性。这正是 PatternBoost 方法提出的核心动机。它试图解决的不是如何替换人类，而是如何构建一个能够模拟人类数学研究中“动手实验”与“反思抽象”这一循环的自动化系统。其目标数学问题是明确的：在那些结构复杂的组合空间中，寻找满足特定约束的、具有极值性质（如边数最多、数量最大）或作为猜想反例的构造。

PatternBoost 的精髓在于其简单而有力的迭代框架。它将整个构造搜索过程分解为两个交替的阶段：局部搜索与全局学习。这个循环可以用一个清晰的比喻来理解：想象一个社区的自行车设计师们（局部阶段）各自对自己的自行车设计进行微调优化，使其更舒适、更高效。这些优秀的自行车被投入使用，遍布大街小巷。新一代的设计师（全局阶段）观察街上这些已被优化过的各式各样的自行车，从中抽象出成功设计的共同特征和模式，并基于这些模式构想出全新的、但风格类似的自行车设计方案。这些新设计随后又会被设计师们拿回去进行微调优化，新一轮的循环就此开始。

在数学的语境下，这个循环被形式化为一个可重复的算法流程：

1. 局部阶段（产生优解）：使用一个（通常是简单的）局部搜索算法，从一个或多个“种子”构造出发，通过一系列允许的局部操作（如添加/删除元素、交换位置），探索其附近的构造空间，目标是得到一个或多个“高质量”的构造。这里的“高质量”由一个评分函数量化，它衡量构造在多大程度上满足了我们的目标（例如，对于最大化边数的无三角形图问题，评分就是图的边数；为了引导搜索，也可以设计一个惩罚存在三角形的评分函数）。
2. 全局阶段（学习模式）：收集在局部阶段产生的一批最优构造，将它们作为训练数据。使用一个能够捕捉序列或结构模式的机器学习模型（在 PatternBoost 中，主要使用基于 Transformer 的生成模型），来学习这些优解中隐含的“构造模式”。训练完成后，让这个模型去生成一批新的、与训练数据“类似”的构造，作为新的“种子”。

3. 迭代：将全局阶段生成的新种子，再次输入到局部搜索算法中，开始新一轮的循环。随着循环的进行，局部搜索的起点越来越好（因为它们来自一个已经学会了“好模式”的模型），而用于训练模型的数据集质量也越来越高（因为它们是经过不断优化的局部搜索结果）。理想情况下，这个过程会形成一个正反馈循环，不断逼近甚至发现最优的或突破性的构造。

这个框架的美妙之处在于其模块化和通用性。局部搜索算法和全局学习模型可以根据具体问题进行定制或替换。PatternBoost 论文的 authors 特意强调，他们希望提供一个对数学家友好、不需要深厚机器学习背景即可使用的工具。因此，他们在实验中大量使用非常简单的局部搜索，以及一个轻量级的 Transformer 实现，来证明即使在这种“朴素”设置下，方法依然有效。

接下来，我们将深入拆解 PatternBoost 工作流程中的每个关键组件，并以论文中的核心示例——“在 n 个顶点的图中，构造边数最多且不含三角形的图（即验证/发现 Mantel 定理的极值图）”——作为贯穿始终的案例。

核心组件一：局部搜索——从种子到优解

局部搜索是 PatternBoost 的基础引擎。它需要完成两项任务：从一个起点产生一个可行的、改进的构造；定义什么叫做“局部”移动。

让我们以论文中的核心示例来理解这一点：在 n 个顶点的图中，构造边数最多且不含三角形的图——这正是图论中著名的 Mantel 定理所描述的问题，其极值结构是完全二分图 $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$ 。

对于这个问题，论文采用了一个极其简单的两阶段贪婪策略作为局部搜索：

1. 消除违规：如果输入图中含有三角形，则反复随机删除一条所在三角形最多的边，直到图中没有任何三角形为止。这个操作是贪婪的，旨在快速降低“违规”程度。
2. 扩展优化：在得到一个无三角形图后，反复随机尝试添加一条新边。如果添加这条边不会产生新的三角形，则保留它；否则，拒绝这次添加。重复这个过程，直到无法再添加任何边而不产生三角形为止。

这个算法非常简单，它没有利用任何关于图结构的深层知识（比如二分性），纯粹依靠随机尝试。它的评分函数也很直接：在最终的无三角形图中，分数就是边的数量。作者使用这个简单的设置，是为了凸显后续 Transformer 全局学习带来的提升，而非局部搜索本身的智能。

为什么需要局部搜索？因为生成模型（如 Transformer）直接输出的构造，很可能在微观约束上并不完美。例如，一个学会了“二分图大致结构”的 Transformer 生成的图，可能仍包含零星几个三角形。局部搜索就像一个“校对员”或“打磨匠”，负责将这些粗糙的、可能有瑕疵的构想，修正为严格满足所有约束的、可被精确评估的数学对象。没有这一步，生成的大量结果将是无效的，无法进入后续的评估和迭代。

核心组件二：全局学习——用 Transformer 捕捉构造的“模式”

在收集到一批高质量的构造（例如，从大量局部搜索运行中选出的边数最多的那些图）后，PatternBoost 进入全局学习阶段。其核心思想是：这些优解中一定蕴含着某种可以泛化的“成功模式”，而机器学习模型的任务就是发现并学会这种模式。

为什么选择 Transformer？Transformer 架构在处理序列数据方面表现出色，尤其擅长捕捉长程依赖关系。一个数学构造（如图的邻接矩阵、矩阵的条目序列、点集的坐标列表）可以被“序列化”成一串符号。Transformer 通过其自注意力机制，能够学习序列中任意两个位置元素之间的关系，这对于理解组合结构的整体布局（比如图的两个部分之间的连接模式）至关重要。相比之前工作中使用的简单前馈神经网络，Transformer 能处理更长的序列，从而能够对更大规模的构造进行建模。

要将一个数学对象输入 Transformer，首先需要将其表示为一系列离散的符号。以 20 个顶点的无三角形图为例：

1. 矩阵表示：图可以用一个 20×20 的对称邻接矩阵表示，对角线为 0。由于对称性，我们只需要上三角部分的 190 个 0/1 元素。
2. 扁平化：将这 190 个元素按行连接成一个一维的二进制序列，例如 010010110...
3. 分词：直接使用长度为 190 的 0/1 序列效率很低，因为词汇表只有两个符号，序列过长会加大模型的学习难度。PatternBoost 采用了字节对编码技术——一种在自然语言处理中常用的压缩方法。BPE 会自动分析训练数据中的所有序列，找出频繁出现的子串，并将每个这样的子串分配一个新的、单独的 Token ID。经

过 BPE 处理后，原始的长二进制序列被压缩成由多个 Token 组成的短序列，每个 Token 可能代表 2 个、3 个或更多个原始比特。这不仅缩短了序列长度，提高了训练和生成效率，而且这些 Token 本身就on能对应着有意义的局部模式。

在实践中，BPE 词汇表的大小（即 Token 的数量）是一个需要调整的超参数。通常，它需要在模型容量（更大的词汇表意味着更短的序列，但 Token 嵌入层更大）和序列表示的信息量之间取得平衡。一个常见的经验法则是，词汇表大小可以与训练数据集的大小、模型的嵌入维度相关联。对于数学构造数据集，由于模式的规律性可能更强，相对较小的词汇表（如 100-500）有时就足够高效。另一个实际问题是动态性：当迭代过程中优解数据集更新时，是否要重新训练 BPE 词典？原则上，为了保持一致性，最好在整个 PatternBoost 循环中使用固定的分词方案。如果新数据引入了全新的高频模式，重新训练 BPE 可能会更优，但这会使得模型在不同迭代间的输出更难比较。一个折中方案是使用一个较大的初始训练集来训练 BPE，并在后续迭代中保持不变。

训练过程是标准的语言模型训练：给定一个 Token 序列（代表一个优解图），让 Transformer 模型去预测序列中下一个 Token 是什么。通过在所有优解序列上最小化预测误差（交叉熵损失），模型逐渐学会了“一个优秀构造的序列应该长什么样”的概率分布。

训练完成后，生成新构造的过程就像让模型“续写”故事：从一个起始 Token 开始，让模型根据已生成的 Token，预测下一个 Token 的概率分布，然后从这个分布中采样出下一个 Token，如此反复，直到生成一个完整的序列（以结束符结尾）。最后，将这个 Token 序列解码回原始的二进制表示（邻接矩阵），就得到了一个新的、由模型“构想”出来的图。

关键点在于：模型生成的新图，其序列模式与训练集中的优解图“相似”，但并非简单复制。它是在学习到的概率分布下的一次新采样，因此可能产生结构上类似但具体细节不同的新构造。这正体现了“全局学习”的创造力——它能够生成遵循成功模式但又是全新的起点。

将上述两个阶段连接起来，就构成了 PatternBoost 的主循环。论文中的实验清晰地展示了迭代的效果：

- 第零代（初始化）：从空图出发，运行 4 万次简单的局部搜索。得到的结果分布峰值在 66 条边左右，最优结果为 99 条边——仅出现 2 次。已知理论最优是 100 条边，即完全二分图 $K_{10,10}$ 。
- 第一代：取前 25% 的优解训练一个小型 Transformer。让 Transformer 生成 10 万个新序列，其中约 3.7 万个能正确解码为有效的 20 顶点邻接矩阵。将这 3.7 万个图作为种子，分别进行局部搜索。结果令人惊讶：产生了 46 个达到理论最优的图，以及 47 个 99 条边的图。而局部搜索的基线在 4 万次尝试中只找到 2 个 99 条边的图。并且，这 46 个最优图在结构上都是同构的——都是完全二分图 $K_{10,10}$ 或其同构体。
- 后续迭代：将新找到的优解加入训练集，继续微调 Transformer，并重复生成和搜索。经过几轮迭代后，模型几乎只生成完全二分图，并且很快学会了生成两部分大小相等的极优结构。

这个例子完美诠释了 PatternBoost 的威力：一个非常简单的局部搜索，配上一个轻量级的 Transformer，通过几次迭代，就从对问题一无所知（从空图开始随机搜索），到自动“发现”了该问题的极值结构（完全二分图）。Transformer 成功地从杂乱的成功案例中抽象出了“二分性”这一全局模式，并利用它极大地提升了局部搜索的起点质量。

PatternBoost 论文并非只展示成功的案例，而是精心挑选了一系列问题，构成了一个从“方法有效但不及人工”到“方法取得突破性发现”的连续谱。这有助于我们客观评估其能力和适用范围。

1. 困难问题：无 4-圈图的最大边数问题

这个问题要求构造一个 n 顶点、不含长度为 4 的圈且边数尽可能多的图。该问题在数学上比无三角形图困难得多。使用同样的简单局部搜索，在运行了 5000 万次局部搜索后得到的结果中最低分是 68，最高分是 89，结果分布峰值在 81，而对于 $n = 33$ ，可能达到的最大分数是 96。应用 PatternBoost 后，性能得到显著提升，最好的结果是包含有 91 条边的图，并且通过使用更大的 Transformer 模型和改进的分词策略（在矩阵每行后加分隔符），最终在 1.165 亿次局部搜索后找到了理论最优的 96 条边图。然而，达到最优所需的搜索量巨大，且对于更大的 n ，即使使用 PatternBoost，找到的解与已知下界仍有差距。这说明对于某些结构极其复杂、模式难以捕捉的问题，即使是增强后的 PatternBoost 也面临挑战，但其表现依然远优于纯局部搜索。

2. 表现优异的问题：避免“312”模式的矩阵最大积和式问题

在这个问题中，目标是寻找不含特定“312”模式的 0-1 矩阵，使其积和式（Permanent）最大。PatternBoost

使用与之前类似的简单设置，与一个由人类专家精心设计的、针对该问题特化的搜索算法进行对比。结果是，专门的人工算法仅以微弱优势胜出（找到的矩阵积和式约为 $5.2e6$ ，PatternBoost 约为 $5.1e6$ ）。这是一个极具启发性的结果：一个通用的、几乎不包含领域知识的 ML 方法，其表现能够逼近人类绞尽脑汁设计出的专业算法。更重要的是，两种方法找到的最优解集几乎不相交，这意味着它们探索了搜索空间的不同区域。当把人工算法找到的好解加入 PatternBoost 的训练集后，PatternBoost 又能发现一系列新的优质构造。这体现了人机协同的潜力：人类直觉可以提供高价值的种子，而机器可以在此基础上进行大规模的、不知疲倦的探索和泛化。

3. 取得突破的问题：超立方体生成子图的最小边数问题

这是一个存在 30 年之久的问题：关于在 d 维超立方体中，至少需要保留多少条边，才能使其生成子图的直径仍为 d 。有一个猜想估计这样的子图至少有 $2^d + \binom{d}{d/2} - 2$ 条边。对于 $d = 5$ ，猜想中的构造似乎是最优的。但当作者将 PatternBoost 应用于 $d = 6$ 时，方法成功地找到了一个仅含 81 条边的构造，优于猜想中给出的 $2^6 + \binom{6}{3} - 2 = 82$ 条边构造，从而推翻了该猜想。这是 PatternBoost 最引人注目的成果之一：它不仅仅是在优化已知的界，而是真正地解决了一个悬而未决的数学问题，做出了新的发现。这个反例的发现过程，完整地演绎了从自动化构造到数学发现的闭环：机器提出候选反例，人类进行验证和理论化。

此外，论文还在“网格中无等腰三角形点集”、“空间中无 5 点共球点集”、“饱和 Sperner 系”等多个问题上展示了 PatternBoost 的应用，其中一些改进了已知的最佳构造。这些案例共同表明，PatternBoost 是一个适用于多种组合极值问题的通用框架，其效力在很大程度上取决于问题本身是否存在可被机器学习模型捕捉的“隐式模式”。

尽管 PatternBoost 取得了令人印象深刻的成果，但它并非万能钥匙，其成功应用依赖于一系列条件，并面临诸多挑战：

1. 问题适应性：方法最擅长解决那些“最优构造存在清晰但复杂模式”的问题。如果最优构造是高度无序或随机的（例如某些 Ramsey 数下界的构造），或者评分函数极其崎岖、缺乏梯度，PatternBoost 可能难以学习到有效的模式。论文中的“无 4-圈”问题就比“无三角形”问题困难得多，部分原因可能在于极值图的结构更复杂、更难以编码和学习。
2. 局部搜索的质量：PatternBoost 的整体性能严重依赖于局部搜索组件的有效性。论文中使用的简单贪婪搜索虽然证明了概念，但在实际应用中，针对特定问题设计更智能的局部搜索（如利用问题的对称性、设计更精细的邻域操作）能极大提升循环效率。一个糟糕的局部搜索可能无法将 Transformer 生成的好起点充分优化，从而拖累整个进程。
3. 表示与分词的玄学：如何将一个数学对象有效地表示为序列，对 Transformer 的学习至关重要。不同的扁平化顺序（按行、按列、按对角线）、是否添加分隔符、BPE 词汇表的大小等，都会显著影响模型生成有效序列的比例和学习速度。目前，这方面的选择更多地依赖于经验和试验，缺乏系统的理论指导。这构成了一个重要的工程与理论研究方向。
4. 可解释性与数学洞察：PatternBoost 是一个强大的“发现引擎”，但它本质上是一个优化工具。它找到的反例或极值构造，其背后是否揭示了更深层的数学原理？模型学到的“模式”能否被翻译成人类可理解的数学概念或猜想？例如，在推翻超立方体猜想的案例中，找到的 81 条边的具体图结构，能否启发数学家提出关于极值图结构的新定理？目前，从机器输出的构造到人类数学知识的升华，仍需数学家的深度介入。如何让模型不仅输出构造，还能输出对构造的解释或假设，是一个前沿挑战。
5. 计算成本与停止标准：虽然 PatternBoost 使用的模型相对轻量，但大规模迭代仍然需要可观的 GPU 计算资源，特别是在处理大规模问题（ n 很大）时。局部搜索阶段可能涉及数百万次乃至上亿次对候选解的评估，这本身也可能是计算密集型的。在实践中，需要设定明确的停止标准以避免无限计算。常见的停止条件包括：模型在验证集上的性能（如生成有效样本的比例）不再提升；学习曲线（如训练损失）趋于平稳；在连续的若干次迭代中，找到的最优解质量没有显著改进；或者达到了预定的计算时间或资源预算。当满足这些条件时，迭代应当停止，并分析当前得到的最佳构造。

5.4 AI 构造反例：基于 LLM 智能体

在前几节中，我们探讨了两种反例构造方法：一种基于强化学习，通过策略网络在巨大的组合空间中系统性地搜索；另一种基于局部搜索与全局学习的迭代融合，让机器学会“好构造”的隐含模式。这两种方法各有千秋，但它们共享一个共同的前提：需要一个定义良好的形式化环境——无论是明确的动作空间、奖励函数，还是可执行的评分标准。

本小节系统阐述如何构建 LLM 智能体系统，使其能够自主构造数学猜想的反例，并结合验证机制确保构造的正确性。

一个完整的反例构造智能体系统，可以抽象为以下三层架构：

- **核心引擎层**：包括用于生成反例的生成模型。这一层提供了智能体的基本能力：理解数学语言、进行推理、生成构造。
- **代理层**：实现智能决策的核心逻辑，包括如何分解复杂问题、如何从错误中学习、如何检索相关知识等。这一层决定了智能体能否像人类研究者一样，在面对困难时有策略地调整思路，而不是盲目尝试。
- **应用层**：面向具体任务的目标导向层，负责将“推翻这个猜想”的高层目标转化为可执行的行动计划，并最终输出经过验证的反例。

在上述架构中，一个关键的设计选择是将反例的生成与验证分离：由大语言模型负责生成候选反例，然后交由独立的验证模块进行正确性判断。这种设计的核心优势在于：

- **无需完整形式化环境**：不需要预先将整个猜想及其背景理论全部形式化。智能体可以直接处理自然语言描述的问题，只在最终验证环节依赖形式化工具或人工检查。
- **验证目标相对简单**：只需判断构造是否满足命题的条件和结论，无需验证复杂的中间推理步骤。这种“最终结果检查”比验证一个完整的证明链条要简单。
- **迭代速度快**：生成后立即交由验证模块评判，形成“生成-验证-修正”的闭环，可以在短时间内进行大量尝试。

在这个架构中，验证模块的设计至关重要。它可以是形式化验证工具（如 Lean、Coq），可以是符号计算系统（如 Mathematica、SageMath），也可以是人类专家的介入。无论采用何种形式，验证模块需要输出明确的判断结果，并在判定错误时提供具体的反馈信息——例如，“这个构造不满足条件 A”或“它实际上没有否定结论 B”。这种可解释的输出不仅让生成模块可以进行有针对性的修正，也为后续的人工复核提供了依据。

核心机制一：迭代验证修复

迭代验证修复是智能体与 LLM 验证器交互的核心机制。其基本流程如下：

1. **初始生成**：智能体根据给定的数学猜想，生成一个候选反例构造及其简要说明。
2. **验证**：将候选反例提交给验证器，要求其判断是否正确。验证器输出判断结果，并附上详细的理由。
3. **错误反馈**：若验证器判定错误，它会输出具体的错误原因——例如，“这个构造不满足条件 A”或“它实际上没有否定结论 B”。
4. **反思修正**：生成模块接收错误反馈，结合问题上下文和之前的尝试历史，生成修正后的候选反例。这个过程不是简单的“再试一次”，而是基于反馈的有针对性改进。
5. **循环迭代**：重复步骤 2-4，直至验证通过或达到最大尝试次数。

这里的技术要点在于：错误反馈的质量至关重要。如果验证器只输出“错误”，生成模块就不知道如何修正。因此，需要设计提示让验证器输出详细、可操作的反馈。同时，修正过程需要保留历史上下文，避免重复同样的错误——智能体应该“记住”哪些思路已经尝试过且失败了。

核心机制二：反思分解

对于复杂的反例构造任务，直接生成完整反例往往超出 LLM 的能力范围。反思分解机制将复杂问题逐步拆解为可管理的子问题：

1. **规划**：智能体首先理解原始问题，生成一个大致构造思路。这一步类似于人类数学家面对难题时的“先想想大概方向”。

2. 子目标分解：将构造过程分解为多个子任务。例如，“首先构造一个满足条件 A 的函数，然后调整使其满足条件 B，最后验证其破坏结论 C”。每个子目标相对简单，更容易实现。
3. 递归求解：对每个子目标，可以再次应用迭代验证修复机制——甚至可以将子目标进一步分解。这种递归分解使得智能体能够处理复杂度远超其单次生成能力的问题。
4. 结果合成：当所有子目标都得到验证后，系统将它们组合成完整的反例描述，并进行最终的整体验证。

子目标分解的质量直接影响最终成功率。分解得太粗，子目标仍然难以解决；分解得太细，又可能引入过多的组合复杂性。子目标之间的依赖关系也需要妥善管理——有些子目标可能必须顺序完成，有些则可以并行探索。

案例——已成功的反例构造实践

为了理解上述机制如何在实际中运作，让我们考察一个已经取得成功的案例：北京大学 AI4Math 团队对交换环论中 Anderson 猜想的处理。

Anderson 猜想由美国数学家 David F. Anderson 于 2014 年提出，关注的是交换代数中“准完备局部环”的一类深刻性质，是交换环论中一个相当艰深的问题。该猜想提出后十余年间，始终无人取得实质性突破。北京大学北京国际数学研究中心董彬教授课题组与合作者组建的 AI4Math 团队，决定将这一难题作为检验其智能体能力的试金石。他们自主构建了一套双智能体协作框架——由自然语言推理智能体 Rethlas 和形式化验证智能体 Archon 组成，两个智能体分工明确：Rethlas 负责文献检索与数学推理，Archon 则负责将推理结果转化为严格的形式化证明。

负责数学推理的智能体 Rethlas 并未试图直接证明该猜想，而是通过跨领域检索，将“整环完备化”理论与 Anderson 猜想联系起来，构造出了一个反例，从而否定了该猜想。这意味着 Anderson 猜想被否定而非证实——在数学研究中，否定一个猜想与证明一个猜想具有同等重要的价值，因为它划定了理论的边界。

随后，负责形式化验证的智能体 Archon 将该反例构造转化为约 19000 行的 Lean 形式化代码，并在形式化过程中自主发现了初始方案的逻辑漏洞并予以修正。当所需的一个数学概念在 Lean 的现行形式化数学库中尚未被收录时，Archon 还通过检索和比对，自主找到了一条等价的替代路径。最终，Archon 完成了全部代码的生成。

这一案例清晰地展示了“生成-验证”范式在反例构造任务中的核心优势。在生成阶段，智能体敢于提出否定性的构造，而非一味寻求证明——Rethlas 通过跨领域联想，将看似不相关的“整环完备化”理论与 Anderson 猜想联系起来，这种类比与迁移属于传统上被视为人类智慧核心的“概念性跳跃”。在验证阶段，形式化验证环节为这种看似大胆的构造提供了严格的最终保障——Archon 不仅完成了上万行代码的生成，还在过程中自主发现并修复了逻辑漏洞。两个智能体分工协作，共同完成了一个人类数学家十余年未能解决的问题。

基于大语言模型的反例构造智能体，是 AI for Math 的一种新范式。与需要完整形式化环境的传统方法相比，它具有显著的优势：

- 低门槛：不需要将整个猜想及其背景理论预先形式化，可以直接处理自然语言描述的猜想。这使得大量尚未形式化的数学问题可以被 AI 探索。
- 高灵活性：可以处理各种类型的数学对象——从数论中的整数集合，到分析中的函数，再到几何中的构造。只要能用语言描述，就能尝试用语言构造。
- 可交互性：智能体的输出是人类可读的自然语言，数学家可以理解、评判、修正。这种人机可读的特性，使得协作成为可能。
- 快速迭代：生成-验证-修正的闭环可以在短时间内进行大量尝试，探索人类研究者可能忽略的思路。

然而，这种方法也有其固有的局限。生成模块（大语言模型）在处理需要多步复杂推理的问题时，仍然可能犯错或陷入循环，反思分解机制可以缓解这一问题，但不能完全消除。模型的能力受限于其训练数据，对于极其前沿或小众的数学领域，模型可能缺乏必要的背景知识。

构造数学猜想反例，是检验 LLM 数学创造力的试金石。通过将 LLM 的生成能力与外部验证机制相结合，我们可以构建一种高效的反例发现范式。这种范式无需预先将整个问题形式化，门槛更低，迭代更快，有望让更多数学工作者受益。北京大学 AI4Math 团队对 Anderson 猜想的成功处理，正是这一范式有效性的有力证明：智能体不仅构造出了一个人类数学家十余年未能找到的反例，还通过形式化验证确保了其正确性，两个环节均由 AI 智能体自主完成。虽然大语言模型本身的可靠性仍需提升，但通过将其定位于“生成候选”而非“验证结论”

的角色，我们可以充分发挥其探索优势，同时将严谨性的责任交由专门的验证模块承担。这种分工协作的模式，使得反例构造任务成为 LLM 数学创造力的理想试验场。

参考文献

- [1] Boris Alexeev and Dustin G Mixon. “Forbidden Sidon subsets of perfect difference sets, featuring a human-assisted proof”. In: *arXiv preprint arXiv:2510.19804* (2025).
- [2] Pieter-Tjerk de Boer et al. “A tutorial on the cross-entropy method”. In: *Annals of Operations Research* 134.1 (2005), pp. 19–67.
- [3] François Charton et al. “PatternBoost: Constructions in Mathematics with a Little Help from AI”. In: *arXiv preprint arXiv:2411.00566* (2024).
- [4] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. 2nd. MIT Press, 2018.
- [5] Adam Zsolt Wagner. “Constructions in combinatorics via neural networks”. In: *arXiv preprint arXiv:2104.14516* (2021).

第六章 AI for PDE (数值计算)

6.1 PDE 问题概览与传统数值方法

偏微分方程 (Partial Differential Equations, PDEs) 是描述物理、化学、生物、工程乃至社会经济等领域中连续介质系统演化的核心数学工具。从热传导、流体运动到量子力学和金融期权定价, PDEs 模型了现实世界中变量随空间和时间变化的规律。传统上, 求解这些方程依赖于发展成熟的数值方法, 如有限差分法、有限元法和谱方法, 它们构成了科学与工程计算的基石。然而, 随着问题复杂度的不断提升——无论是高维空间、复杂几何边界、强非线性特性, 还是数据匮乏的逆问题——传统方法的局限性也日益凸显。本章节将深入探讨 PDE 问题的基本分类、传统数值方法的数学思想、实现细节、理论极限与计算挑战, 并以此为出发点, 揭示为何近年来基于人工智能的求解方法, 特别是物理信息神经网络 (Physics-Informed Neural Networks, PINNs), 被视为应对这些挑战的一种富有前景的新范式。我们将通过详尽的数学示例、算法伪代码、复杂度分析和可视化对比, 构建一个全面而深入的理解框架。

为了理解 AI 如何介入, 我们必须首先理解问题的本质。一个偏微分方程建立了未知函数 (通常是时空的函数) 与其关于各独立变量的偏导数之间的关系。它通常形式化为:

$$F\left(x_1, \dots, x_n, u, \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_n}, \frac{\partial^2 u}{\partial x_1^2}, \dots\right) = 0, \quad \mathbf{x} \in \Omega$$

其中, $u = u(\mathbf{x})$ 是未知函数, $\mathbf{x} = (x_1, \dots, x_n)$ 是定义在区域 $\Omega \subset \mathbb{R}^n$ 内的自变量 (通常是空间和时间坐标), F 是一个给定的算子。这个等式就像一个“语法规则”, 规定了函数 u 在其定义域内每一点的行为必须满足的局部约束。然而, 这个规则本身通常不足以唯一确定一个解, 我们需要额外的“语境”信息, 即边界条件和 (对于含时问题) 初始条件。一个完整的 PDE 问题, 由控制方程、初值条件和边界条件共同构成, 我们称之为一个定解问题。数学上需要研究其解的存在性、唯一性和稳定性 (适定性)。而在计算实践中, 我们的目标是在无法获得解析解的情况下, 构造一个尽可能逼近真解的数值解。

从数学结构上, PDEs 可以根据其最高阶导数的特性进行分类, 这种分类深刻影响了方程的解的行为以及求解它们的数值方法的性质。对于二阶线性 PDE, 通常可归纳为以下三种经典类型:

1. 椭圆型方程 (Elliptic PDEs) 椭圆型方程通常描述稳态 (与时间无关) 的平衡或分布问题, 其解具有“平滑化”和“全局依赖”的特性。一个经典的例子是泊松方程 (Poisson's equation), 它描述了势场 (如静电势、重力势) 在给定源分布下的稳定状态:

$$-\nabla^2 u = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

其中 ∇^2 是拉普拉斯算子 (Laplacian), f 是已知的源项。特别地, 当 $f = 0$ 时, 称为拉普拉斯方程 (Laplace's equation)。椭圆型方程没有时间维度, 其解在整个区域内同时被决定, 边界上的任何扰动都会瞬间影响到内部所有点 (尽管影响强度随距离衰减)。典型的边界条件是狄利克雷 (Dirichlet) 条件 (指定边界上的函数值 u)、诺伊曼 (Neumann) 条件 (指定边界上的法向导数 $\partial u / \partial n$) 或两者的混合 (Robin 条件)。椭圆型问题在数学上对应着寻找能量泛函的极小值点。

2. 抛物型方程 (Parabolic PDEs) 抛物型方程引入了时间变量, 描述了扩散、热传导等耗散过程。最著名的代表是热传导方程 (Heat equation):

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u + f(\mathbf{x}, t), \quad \mathbf{x} \in \Omega, t > 0$$

其中 $\alpha > 0$ 是扩散系数。这类方程在时间上是单向演化的 (不可逆), 具有“平滑”初始扰动的特性。求解时需要初始条件 $u(\mathbf{x}, 0) = u_0(\mathbf{x})$ 以及边界条件。信息从初始时刻和边界向内传播, 时间步进是自然的求解

策略。

3. 双曲型方程 (Hyperbolic PDEs) 双曲型方程同样描述时间演化过程，但特性是波动和无耗散，如声波、光波和弹性波的传播。标准形式是波动方程 (Wave equation)：

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u + f(\mathbf{x}, t), \quad \mathbf{x} \in \Omega, t > 0$$

其中 c 是波速。这类方程具有特征线，扰动沿这些特征线以有限速度传播，并可能保持不连续性（如激波）。

它们需要初始条件（包括初始位移和初始速度）及边界条件。

除了上述三种经典类型，现实世界中还存在大量非线性 PDE（如描述流体的纳维-斯托克斯方程、描述相变的 Allen-Cahn 方程）、方程组（耦合多个未知函数）。这些复杂性给解析求解和数值求解都带来了巨大挑战。

传统数值方法的核心思想是将连续的 PDE 问题离散化，转化为一个离散的、有限维的代数问题（通常是线性或非线性方程组），然后利用计算机求解。这个过程可以看作是在一个网格或基函数集合所张成的有限维空间中，寻找原 PDE 解的近似。不同的离散化策略催生了不同的数值方法家族，它们各有其数学原理、优势和适用场景。表 6.1 概括了这四大主流方法的核心理念与典型应用场景：

表 6.1: 四大主流数值方法对比

方法名称	核心离散化思想	典型应用领域	关键优势	主要挑战
有限差分法 (FDM)	用网格点函数值的差商近似导数	规则区域上的流体、热传导	概念简单，易于实现，理论成熟	复杂几何适应性差，高维灾难
有限元法 (FEM)	区域剖分为单元，用局部多项式基函数逼近解，基于变分原理	结构力学，复杂几何流体，电磁学	几何适应性强，理论严谨，应用最广	网格生成复杂，高维计算量大
有限体积法 (FVM)	划分控制体积，对守恒律积分，保持物理通量平衡	计算流体动力学，燃烧，多相流	严格保持物理守恒性，鲁棒性强	高阶格式复杂，理论分析较难
谱方法	用全局光滑基函数（如傅里叶级数、切比雪夫多项式）展开解	湍流模拟，量子化学，气象预报	对光滑解有指数级收敛精度	要求解光滑，几何适应性差，矩阵稠密

接下来，我们将深入每种方法的数学细节、实现示例，并分析其计算复杂度与误差特性。

1. 有限差分法 (Finite Difference Method, FDM)

想象我们在求解区域上铺设一张规则的、带有节点的网格，就像一张坐标纸。PDE 告诉我们的是函数在每一点的变化率（导数）与该点函数值的关系。FDM 的核心就是用相邻节点函数值的差值（差商）来近似表示这一点上的导数。这是对微积分基本定义“导数即差商的极限”最直接的数值实现。

数学表述与示例为：考虑一个简单但经典的二维泊松方程在单位正方形区域上的狄利克雷问题：

$$-\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f(x, y), \quad (x, y) \in (0, 1) \times (0, 1)$$

$$u(x, y) = g(x, y), \quad (x, y) \in \partial\Omega$$

我们在 x 和 y 方向分别以步长 $\Delta x = \Delta y = h = 1/(N+1)$ 进行均匀离散，得到网格点 $(x_i, y_j) = (ih, jh)$ ，其中 $i, j = 0, 1, \dots, N+1$ 。边界点 ($i = 0, N+1$ 或 $j = 0, N+1$) 的函数值由边界条件 g 给出。对于内部点 (i, j) ，我们用中心差分近似二阶导数：

$$\frac{\partial^2 u}{\partial x^2} \Big|_{(x_i, y_j)} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}, \quad \frac{\partial^2 u}{\partial y^2} \Big|_{(x_i, y_j)} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}$$

其中 $u_{i,j} \approx u(x_i, y_j)$ 。将这两个近似代入泊松方程，对每个内部点我们得到一个方程：

$$-\frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} = f_{i,j}$$

整理后：

$$4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1} = h^2 f_{i,j}$$

这形成了一个关于所有内部点未知数 $u_{i,j}$ 的大型线性方程组。该方程组具有稀疏的、带状的系数矩阵（每个方程仅涉及 5 个未知数），可以用高效的迭代法（如共轭梯度法、多重网格法）求解。

Algorithm 6.1: 二维泊松方程的有限差分法求解

Input: 网格剖分数 N （每个方向上的内部点数），
 源项函数 $f(x, y)$ ，
 边界条件函数 $g(x, y)$ ，
 求解器参数（迭代次数、收敛容差等，可选）
Output: 数值解矩阵 $U \in \mathbb{R}^{(N+2) \times (N+2)}$ （包含边界点）

- 1 初始化步长 $h = 1/(N+1)$ ，创建解矩阵 U 维度为 $(N+2) \times (N+2)$
 - 2 施加边界条件：
 - 3 **for** 所有边界点 (i, j) ($i = 0$ 或 $i = N+1$ 或 $j = 0$ 或 $j = N+1$) **do**
 - 4 $U[i][j] \leftarrow g(x_i, y_j)$
 - 5 组装线性系统 $Au = b$ ：
 - 6 **for** 每个内部点 (i, j) ($i = 1, \dots, N, j = 1, \dots, N$) **do**
 - 7 构建方程： $4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1} = -h^2 f_{i,j}$
 - 8 **if** 邻点位于边界 **then**
 - 9 将其值（已知量）移至右端项 b
 - 10 注：系数矩阵 A 是一个稀疏、对称正定的块三对角矩阵（按自然编号时），每行最多约有 5 个非零元，维度为 $N^2 \times N^2$
 - 11 求解稀疏线性系统：
 - 12 选择求解器（如共轭梯度法、多重网格法）
 - 13 求解 $Au_{\text{vec}} = b$ ，得到内部点解向量 u_{vec}
 - 14 重组解矩阵：
 - 15 将解向量 u_{vec} 映射回 U 的内部点位置
 - 16 **return** 数值解矩阵 U
-

对于 d 维空间中的问题，若在每个维度上离散为 N 个内部点，则总自由度（未知数个数）为：

$$N_{\text{dof}} = N^d$$

这一关系揭示了有限差分法的根本性挑战：当维数 d 增加时，未知数数量呈指数增长。例如，当 $N = 100$ 时，二维问题的未知数为 10^4 ，而三维问题则增至 10^6 ，四维问题更是达到 10^8 。这种“维数灾难”（Curse of Dimensionality）使得传统数值方法在处理高维 PDE 问题时面临巨大的计算和存储压力。

有限差分法的计算开销主要来源于两个阶段：系统组装与系统求解。

系统组装阶段需要为每个内部节点生成对应的离散方程。由于内部节点总数为 N^d ，而每个节点的离散格式（如五点差分）仅涉及常数个邻近节点，因此组装阶段的时间复杂度为

$$T_{\text{assemble}} = O(N^d)$$

系统求解阶段是整个计算过程的主要瓶颈。对于所得到的稀疏线性系统 $Au = b$ ，不同的求解策略呈现出迥异的

复杂度特征。

直接法如高斯消元，虽在理论上是通用的求解手段，但对于稀疏矩阵而言，由于消元过程中会产生填充元，使得系数矩阵的稠密程度显著增加。对于一般的二维问题，其计算复杂度可高达 $O((N^d)^3) = O(N^{3d})$ ，这在实际中通常不可接受。

经典迭代法如 Jacobi，每步迭代仅需 $O(N^d)$ 操作。这类方法的收敛速度通常为线性，即误差以固定比率逐次衰减。对于椭圆型问题，收敛速度由系数矩阵的谱半径决定，所需的迭代次数与网格尺寸 h 的平方成反比，即 $O(h^{-2}) = O(N^2)$ 。因此，经典迭代法的总复杂度为

$$T_{\text{classical}} = O(N^2 \cdot N^d) = O(N^{d+2})$$

现代迭代法如共轭梯度法，能显著提升收敛效率。对于由 Poisson 方程离散所得的对称正定系统，其条件数满足 $\kappa(A) = O(h^{-2}) = O(N^2)$ 。共轭梯度法的迭代次数约为 $O(\sqrt{\kappa(A)}) = O(N)$ ，从而总复杂度为

$$T_{\text{CG}} = O(N \cdot N^d) = O(N^{d+1})$$

最优方法如多重网格法，通过在不同尺度的网格上交替进行误差修正，能够实现与未知数数量成正比的求解效率。对于充分光滑的问题，多重网格法的计算复杂度可达

$$T_{\text{solve}} = O(N^d)$$

这是有限差分法在理论上能达到的最优复杂度，使得整个计算过程的总复杂度仍保持在 $O(N^d)$ 量级。

尽管线性系统 A 具有稀疏性——每个方程仅涉及常数个未知数——但存储需求仍需要谨慎处理。若采用压缩行存储或压缩列存储等稀疏格式，仅需记录非零元素及其对应的行、列索引。对于五点差分格式，每个内部点对应 5 个非零元素，因此总存储量为 $O(N^d)$ 。若采用稠密存储策略，即保留矩阵中所有元素（包括大量零元），则存储量将高达 $O(N^{2d})$ 。这种方案在绝大多数实际问题中完全不可行，因此实际应用必须采用稀疏存储策略，将空间复杂度控制在 $O(N^d)$ 。

有限差分法的误差根源在于用差商对导数的近似。以中心差分为例，由 Taylor 展开可得

$$\frac{u(x+h) - u(x-h)}{2h} = u'(x) + \frac{h^2}{6} u'''(x) + O(h^4)$$

对于二阶导数，其中心差分近似为

$$\frac{u(x+h) - 2u(x) + u(x-h)}{h^2} = u''(x) + \frac{h^2}{12} u^{(4)}(x) + O(h^4)$$

由此可见，中心差分格式的局部近似误差（即截断误差）为 $O(h^2)$ 量级。这意味着当网格加密时，每个离散点上的局部误差以 h^2 的速度趋于零。

离散化误差是指数值解与真解之间的整体差异。在真解足够光滑时，离散化误差与截断误差具有相同的量级。具体地，存在与 h 无关的正常数 C ，使得

$$\|u - u_h\| \leq Ch^2$$

其中 $\|\cdot\|$ 可取 L^2 范数或 L^∞ 范数。这一关系表明，当网格步长减半时，误差约缩小至原来的四分之一。因此，上述中心差分格式被称为具有二阶收敛精度。

更一般地，若采用 p 阶精度的差分格式，其离散化误差满足

$$\|u - u_h\| = O(h^p)$$

其中 p 由差分格式的构造方式决定。需要指出的是，实际收敛阶受限于真解的光滑性：若真解缺乏足够的高阶

导数，则实际收敛速度可能低于理论值。

有限差分法的核心优势在于其概念直观、实现简单。该方法直接基于导数的定义构造离散格式，物理意义清晰，便于初学者理解和掌握。对于规则区域（如矩形、立方体）上的问题，有限差分法能够高效地生成稀疏线性系统，且其理论体系——包括稳定性分析、收敛性证明和误差估计——已发展得相当成熟，为数值解的可靠性提供了坚实的理论保障。

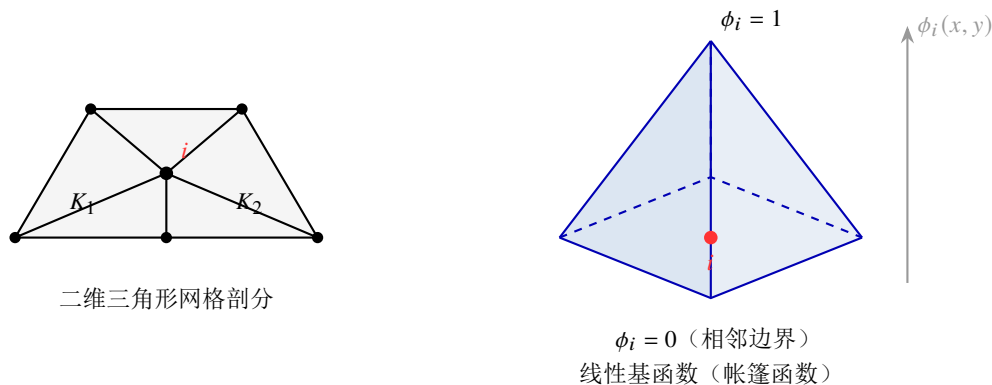
然而，有限差分法也面临着明显的局限性。首先，该方法对计算区域的几何形状适应性较差，在处理复杂或不规则边界时，构造高精度且满足边界条件的差分格式往往十分困难，通常需要借助坐标变换或浸入边界法等复杂技术。其次，随着空间维数的增加，自由度呈指数级增长，导致计算量和存储需求急剧膨胀，这一“维数灾难”问题使得传统有限差分法在高维问题中难以直接应用。

2. 有限元法 (Finite Element Method, FEM)

如果说有限差分法 (FDM) 是“逐点”地施加规则，那么有限元法 (FEM) 则是“分片”地构建解。其核心思想是：将复杂的求解区域划分成许多互不重叠的小块，比如三角形或四边形，这些小块称为“单元”。在每个单元上，我们用一个简单的函数（例如低次多项式）来近似描述真实的解，就像用许多小块多边形瓷砖拼贴出一块复杂形状的地板，并在每一块瓷砖上绘制简单的图案。通过将这些“局部近似”巧妙地组合在一起，我们就能得到一个在整个区域上的整体近似解。

与有限差分法直接离散偏微分方程不同，有限元法处理的是原方程的“弱形式”（积分形式）。弱形式通过对积分区域的“平均化”，降低了对解的光滑性要求——允许解在某些地方导数不连续，这为处理复杂问题提供了便利。同时，某些边界条件（如自然边界条件）会在弱形式中自动体现，无需额外处理。有限元法的核心是伽辽金 (Galerkin) 法：我们在一个由分片多项式基函数张成的有限维函数空间中寻找近似解，并要求偏微分方程的残差与该空间中的所有“测试函数”正交，从而保证近似解在整体意义下尽可能接近真实解。

图 6.1: 二维有限元法示意图：左图为三角形单元剖分，节点 i 与若干单元相连；右图为对应线性基函数 $\phi_i(x, y)$ 的“帐篷函数”示意，其在节点 i 处取值为 1，在支撑边界处取值为 0，并在每个相邻三角形单元上呈线性变化。



接下来，我们以二维泊松方程为例，介绍有限元法的四个主要步骤：

$$-\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f(x, y), \quad (x, y) \in \Omega \subset \mathbb{R}^2$$

$$u(x, y) = g(x, y), \quad (x, y) \in \partial\Omega$$

有限元法的求解过程可分为以下四个主要步骤。

步骤一：区域剖分

将求解区域 Ω 剖分为许多小三角形（或四边形）单元，形成三角网格。与有限差分法不同，有限元法的单元可以灵活地贴合复杂几何边界。记网格为 $\mathcal{T}_h = \{K\}$ ，其中 h 表示单元的特征尺寸（通常取最大单元的外接圆直径）。所有单元顶点的集合构成节点集。

步骤二：构造有限元空间

在每个单元 K 上，我们需要一个“局部蓝图”来描述解在该单元上的形态。这个蓝图就是形函数。以三角形单元上的线性元为例，在单元 K 上定义三个形函数 $N_1^K(x, y)$, $N_2^K(x, y)$, $N_3^K(x, y)$ ，它们满足：

- 在每个单元上，形函数是 x 和 y 的线性函数，形式简单，易于积分和求导
- 在对应的顶点（节点）上取值为 1，在其他两个顶点上取值为 0

这意味着，如果我们知道单元 K 三个顶点上的函数值 $c_{i_1}, c_{i_2}, c_{i_3}$ ，就可以通过形函数唯一地构造出单元上的近似解：

$$u_h(x, y)|_K = c_{i_1} N_1^K(x, y) + c_{i_2} N_2^K(x, y) + c_{i_3} N_3^K(x, y)$$

这里，形函数充当了“插值器”的角色，将离散的节点值光滑地延拓到整个单元内部。

接下来，我们需要将这些“局部蓝图”拼合成一个全局的近似函数。为此，我们为每个全局节点 i 定义一个全局基函数 $\phi_i(x, y)$ 。它的构造方式非常直观：将包含节点 i 的所有单元上，与该节点对应的那个形函数“拼接”起来，而在其他单元上取 0。全局基函数 ϕ_i 具有以下关键性质：

- 局部支撑： ϕ_i 只在节点 i 周围的一圈单元上非零，其余区域均为 0。这保证了后续计算的稀疏性。
- 节点插值性： ϕ_i 在节点 i 处值为 1，在其他节点处值为 0。

整个区域上的近似解 $u_h(x, y)$ 可以表示为所有节点基函数的线性组合：

$$u_h(x, y) = \sum_{i=1}^M c_i \phi_i(x, y)$$

其中 M 为节点总数， c_i 是待求的未知系数，其物理意义为节点处的近似解值。通过这种方式，形函数提供了每个单元上的“局部描述”，而全局基函数则负责将这些局部描述“无缝拼接”成整个区域上的连续函数（对于线性元，解在单元边界上自动连续）。

步骤三：形成弱形式与离散系统

为了得到有限元方程，我们先推导泊松方程的弱形式。将原方程乘以一个“测试函数” v ，并在区域 Ω 上积分，利用散度定理（高维分部积分）将二阶导数转化为一阶导数：

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega + \int_{\partial\Omega} (\nabla u \cdot \mathbf{n}) v \, dS$$

对于狄利克雷边界条件（给定边界上的函数值），我们在边界上强制近似解等于已知值 g ，因此测试函数 v 在边界上取 0，边界积分项自动消失。这样，问题转化为：寻找 u 满足边界条件，使得上述积分等式对所有允许的测试函数成立。

现在，我们将近似解 $u_h = \sum_{i=1}^M c_i \phi_i$ 代入，并取测试函数 v 为每一个基函数 ϕ_j （即伽辽金法）。这样得到 M 个方程：

$$\sum_{i=1}^M c_i \int_{\Omega} \nabla \phi_i \cdot \nabla \phi_j \, d\Omega = \int_{\Omega} f \phi_j \, d\Omega, \quad j = 1, 2, \dots, M$$

记

$$A_{ji} = \int_{\Omega} \nabla \phi_i \cdot \nabla \phi_j \, d\Omega, \quad b_j = \int_{\Omega} f \phi_j \, d\Omega$$

则上述方程组可写成简洁的矩阵形式：

$$\mathbf{A} \mathbf{c} = \mathbf{b}$$

其中 $\mathbf{A}$ 称为刚度矩阵， $\mathbf{b}$ 称为载荷向量， $\mathbf{c}$ 为未知系数向量。

步骤四：组装与求解

刚度矩阵和载荷向量的计算并不需要在全局区域上直接进行，而是利用基函数的局部支撑性质：每个 ϕ_i 只在少数几个单元上非零。因此，我们可以在每个单元上分别计算局部贡献，再将所有单元的贡献“组装”到全局

矩阵和向量中。具体来说：

- 在每个单元 K 上，计算单元刚度矩阵和单元载荷向量；
- 按照节点编号，将单元贡献累加到全局矩阵 A 和全局向量 b 的对应位置。

由于每个基函数仅与其相邻节点的基函数有重叠，刚度矩阵 A 是稀疏的——非零元比例很低，且通常对称正定（对于泊松方程）。这一稀疏性使得我们可以用高效的迭代求解器（如共轭梯度法）或直接求解器（如稀疏 LU 分解）来求解线性方程组，即使节点数达到百万级别也能在合理时间内完成。

求解得到系数向量 c 后，代入 $u_h(x, y) = \sum c_i \phi_i(x, y)$ ，便得到了原问题在整个区域上的有限元近似解。

Algorithm 6.2: 有限元法求解二维泊松方程（三角形线性元）

Input: 节点坐标数组 **nodes**，维度为 $M \times 2$ ，
单元连接数组 **elements**，每个单元包含 3 个节点索引，
右端项函数 $f(x, y)$ ，
狄利克雷边界节点列表 **dirichlet_nodes** 及其对应的边界值
Output: 节点解向量 u ，其中 $u_i \approx u(\mathbf{x}_i)$

- 1 初始化全局刚度矩阵 A （采用稀疏存储格式）和载荷向量 b 为零
 - 2 **for** 每个三角形单元 $K \in \mathbf{elements}$ **do**
 - 3 获取单元的三个节点坐标 $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$
 - 4 计算单元刚度矩阵 $A_K \in \mathbb{R}^{3 \times 3}$:

$$(A_K)_{ij} = \int_K \nabla \phi_i \cdot \nabla \phi_j d\Omega$$
 - 5 计算单元载荷向量 $b_K \in \mathbb{R}^3$:

$$(b_K)_i = \int_K f \phi_i d\Omega$$
 - 6 将 A_K 组装到全局矩阵 A 的对应位置（根据单元的局部-全局节点映射）
 - 7 将 b_K 组装到全局向量 b 的对应位置
 - 8 处理狄利克雷边界条件:
 - 9 **for** 每个边界节点 $i \in \mathbf{dirichlet_nodes}$ **do**
 - 10 强制设置 $u_i = g(\mathbf{x}_i)$
 - 11 修改矩阵 A 的第 i 行和第 i 列（通常将第 i 行第 i 列置为 1，右端项相应调整）
 - 12 求解稀疏线性系统 $Au = b$ （采用稀疏直接法如 UMFPACK，或迭代法如共轭梯度法）
 - 13 **return** 节点解向量 u
-

对于二维三角形网格，节点数 M 与单元数大致成正比。更一般地，在 d 维空间中，为了达到特征尺寸 h 的离散精度，所需的自由度（节点数）约为 $O(h^{-d})$ 。这一关系与有限差分法完全一致，同样揭示了维数灾难的本质——当空间维数增加时，自由度呈指数级增长，直接导致计算量和存储需求的急剧膨胀。

由有限元离散所得的刚度矩阵 A 具有三个重要性质：稀疏性、对称性和正定性（对于椭圆型问题）。稀疏性源于每个基函数仅在其支撑的有限个单元上非零，因此每个节点仅与相邻节点产生耦合，矩阵中绝大多数元素为零。采用稀疏存储格式时，存储需求仅为 $O(M)$ ，即与自由度数成线性关系。对称性和正定性则使得共轭梯度法等高效迭代算法得以应用。

有限元法的计算复杂度由组装和求解两个阶段共同构成。组装阶段需要对每个单元进行积分计算，由于每个单元的计算量为常数，因此组装的总复杂度为 $O(M)$ 。求解阶段的复杂度则取决于所采用的线性求解器。若使用稀疏直接法（如基于多重前代消去的求解器），对于二维问题，复杂度通常介于 $O(M^{1.5})$ 与 $O(M^2)$ 之间；若采用多重网格法这类最优迭代求解器，则求解复杂度可降至 $O(M)$ ，实现与自由度数成线性比例的最优效率。

关于有限元法的收敛精度，对于线性元（即采用一次多项式近似），当真解充分光滑时，误差满足以下估计：

$$\|u - u_h\|_{L^2} = O(h^2), \quad \|u - u_h\|_{H^1} = O(h)$$

其中 L^2 范数度量函数值本身的误差， H^1 能量范数度量函数值及其一阶导数的整体误差。这意味着当网格加密一倍时，函数值误差约缩小至原来的四分之一，而导数误差约缩小至原来的二分之一。

更一般地，若采用 p 次多项式单元，收敛阶可提升为

$$\|u - u_h\|_{L^2} = O(h^{p+1})$$

这一性质揭示了有限元法中精度与单元阶次的内在关系：提高多项式的次数能够显著加快收敛速度。然而，这种精度的提升并非没有代价——高次单元在每个单元上需要更多的积分点和自由度，且程序实现的复杂度也相应增加。因此，在实际应用中，需要在精度、效率和实现难度之间做出权衡：线性元（ $p = 1$ ）因其简单性而广受欢迎，二次元（ $p = 2$ ）在精度与效率之间取得良好平衡，而更高次单元则通常用于需要极高精度的特殊问题。

有限元法的核心优势在于其对复杂几何形状的极强适应性。通过非结构网格（如三角形、四面体网格），有限元法可以精确贴合任意复杂的边界，这使得它在处理工程实际问题时具有无可比拟的灵活性。此外，有限元法拥有严格的数学理论基础——基于索伯列夫空间和变分原理，其稳定性分析和误差估计体系已发展得相当成熟。正因如此，有限元法已成为结构力学、流体力学、热传导等工程领域最主流的数值方法。

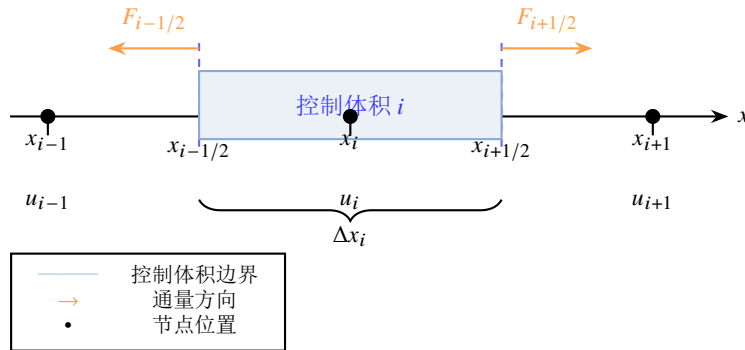
然而，有限元法也面临着不容忽视的挑战。首先，高维问题中的维数灾难依然存在——随着空间维数的增加，单元和节点数量呈指数增长，计算成本急剧攀升。其次，生成高质量的计算网格（尤其是三维非结构网格）本身就是一项极具挑战性的任务，往往需要专门的前处理软件和大量的人工干预，且网格质量直接影响数值解的精度和稳定性。对于涉及移动边界、自由表面或大变形的动态问题，网格可能需要频繁重构，进一步增加了计算复杂度和实现难度。

3. 有限体积法 (Finite Volume Method, FVM)

有限体积法的思想根植于物理世界中最基本的守恒定律。它将求解区域划分为许多互不重叠的控制体积（通常就是网格单元），然后对每个控制体积直接应用守恒定律：物理量（如质量、动量、能量）的变化率，等于通过其边界进入的净通量加上内部源项。这种“从局部守恒到全局守恒”的朴素思想，使得 FVM 在离散层面天然保持守恒性，这是其区别于有限差分法和有限元法最显著的特征。

与有限差分法直接近似导数不同，FVM 在积分形式上离散守恒律方程。通过散度定理，将控制体积内的体积分转化为边界上的通量积分，从而将连续方程转化为一组离散的平衡方程。这种处理方式使得 FVM 特别适合处理存在激波、间断等强非线性现象的流动问题——在这些问题中，解的导数可能不存在，但积分形式的守恒律依然成立。图 6.2 以一维情形为例，展示了控制体积的划分与通量交换的物理图像：

图 6.2: 一维有限体积法示意图：每个控制体积以节点 x_i 为中心， u_i 为节点 x_i 处的解值， $F_{i\pm 1/2}$ 为界面 $x_{i\pm 1/2}$ 上的通量，通量在界面 $x_{i-1/2}$ 和 $x_{i+1/2}$ 处交换



在一维中，每个控制体积以节点 x_i 为中心，左右边界分别位于 $x_{i-1/2}$ 和 $x_{i+1/2}$ 。守恒律要求：

$$\text{控制体积内量的变化率} = \underbrace{F_{i-1/2}}_{\text{左边进入}} - \underbrace{F_{i+1/2}}_{\text{右边流出}} + \underbrace{\int_{\text{体积}} S dx}_{\text{内部源项}}$$

其中 F 表示通过边界的通量（单位时间单位面积传递的物理量）。这一方程构成了有限体积法离散化的物理基础。

考虑一维稳态对流扩散方程：

$$\frac{d}{dx} \left(au - v \frac{du}{dx} \right) = f, \quad x \in (0, L)$$

其中 a 为对流速度（常数）， $v > 0$ 为扩散系数， f 为源项。该方程已写成守恒形式 $\frac{dF}{dx} = f$ ，其中总通量 F 由对流项和扩散项组成：

$$F = au - v \frac{du}{dx}$$

对控制体积 $V_i = [x_{i-1/2}, x_{i+1/2}]$ 积分：

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{dF}{dx} dx = F|_{x_{i+1/2}} - F|_{x_{i-1/2}} = \int_{x_{i-1/2}}^{x_{i+1/2}} f dx$$

这一积分形式不涉及导数，允许解存在间断，体现了 FVM 对弱解的适应性。

接下来进行离散化。记 $F_{i+1/2}$ 为界面 $x_{i+1/2}$ 上的通量近似，即 $F_{i+1/2} \approx F(u_i, u_{i+1})$ ，右端源项近似为 $f_i \Delta x_i$ （ Δx_i 为控制体积宽度），则离散方程为：

$$F_{i+1/2} - F_{i-1/2} = f_i \Delta x_i$$

问题的核心在于如何构造界面通量 $F_{i+1/2}$ ，即根据相邻节点值 u_i, u_{i+1} 近似该界面上的物理通量。对于不同的物理机制，采用不同的重构方式：

- 扩散项 $-v \frac{du}{dx}$ ：通常采用中心差分，因为扩散是各向同性的物理过程：

$$-v \frac{du}{dx} \Big|_{i+1/2} \approx -v \frac{u_{i+1} - u_i}{\Delta x}$$

其中 $\Delta x = x_{i+1} - x_i$ （均匀网格假设）。

- 对流项 au ：对流具有方向性，信息沿流动方向传播，因此需要根据流速方向选择“上游”值，以确保数值稳定性：

$$au|_{i+1/2} \approx \begin{cases} au_i, & a \geq 0 \quad (\text{从左向右流}) \\ au_{i+1}, & a < 0 \quad (\text{从右向左流}) \end{cases}$$

这种迎风格式（Upwind Scheme）物理上合理：界面处的对流通量由上游的物理量决定。迎风格式无条件稳定，但只有一阶精度。

将上述两部分组合，得到总通量的离散形式：

$$F_{i+1/2} = au_{\text{upwind}} - v \frac{u_{i+1} - u_i}{\Delta x}, \quad \text{其中 } u_{\text{upwind}} = \begin{cases} u_i, & \text{if } a \geq 0 \\ u_{i+1}, & \text{if } a < 0 \end{cases}$$

代入离散方程并整理，得到关于节点值 u_i 的线性方程组：

$$-\left(\frac{v}{\Delta x} + \max(0, -a)\right) u_{i-1} + \left(\frac{2v}{\Delta x} + |a|\right) u_i - \left(\frac{v}{\Delta x} + \max(0, a)\right) u_{i+1} = f_i \Delta x_i$$

这是一个三对角系统，可用高效的 Thomas 算法直接求解。

有限体积法的最大优势在于其天然的守恒性。由于该方法直接从积分形式的守恒律出发，每个控制体积上的离散方程都精确满足物理量的收支平衡，内部界面上的通量在全局组装时相互抵消，从而在离散层面严格保持整体守恒。这一特性使得有限体积法特别适合处理流体力学、传热学等涉及质量、动量、能量守恒的多物理场问题，即使在粗网格上也能保证解的物理合理性。此外，有限体积法对复杂几何具有良好的适应性，可以结合

非结构网格处理工程中的复杂计算域，加之其较强的数值稳定性（尤其是迎风格式），使其成为当前计算流体力学（CFD）领域商业软件的主流方法。

然而，有限体积法也存在一定的局限性。高阶精度格式的构造比有限元法更为复杂，需要在单元界面处进行通量重构，并常常引入限制器（limiter）来抑制非物理振荡，这对格式设计和程序实现提出了较高要求。与此同时，严格的数学误差分析（如先验误差估计和后验误差估计）在有限体积法框架下不如有限元法那样系统和完善，理论基础的完备性稍显不足。在非结构网格上处理高维问题时，如何平衡精度、稳定性和计算效率，仍然是有限体积法研究和应用中需要持续关注的课题。

Algorithm 6.3: 有限体积法求解一维稳态对流扩散方程（迎风格式）

Input: 网格节点坐标数组 $x[0..N]$ ，其中 N 为单元数（节点数为 $N+1$ ），
对流速度 a （常数），扩散系数 $\nu > 0$ ，
源项函数 $f(x)$ ，
边界值 $u_0 = u(x_0)$ ， $u_N = u(x_N)$ （狄利克雷边界条件）

Output: 节点解向量 $\mathbf{u} = [u_0, u_1, \dots, u_N]^T$ ，其中 $u_i \approx u(x_i)$

- 1 计算各控制体积的长度 $\Delta x_i = x_{i+1} - x_i$ ， $i = 0, 1, \dots, N-1$
 - 2 初始化三对角系数矩阵 $\mathbf{A} \in \mathbb{R}^{(N+1) \times (N+1)}$ 和右端项向量 $\mathbf{b} \in \mathbb{R}^{N+1}$ 为零
 - 3 **for** 每个内部控制体积 $i = 1, 2, \dots, N-1$ **do**
 - 4 计算左界面通量系数（ $i-1/2$ 界面）：

$$a_{\text{conv}}^{\text{left}} = \max(a, 0) \quad (\text{迎风格式})$$

$$d^{\text{left}} = \frac{\nu}{\Delta x_{i-1}}$$
 - 5 计算右界面通量系数（ $i+1/2$ 界面）：

$$a_{\text{conv}}^{\text{right}} = \min(a, 0) \quad (\text{迎风格式})$$

$$d^{\text{right}} = \frac{\nu}{\Delta x_i}$$
 - 6 组装到三对角矩阵：

$$A_{i,i-1} = -\left(d^{\text{left}} + \max(0, -a)\right)$$

$$A_{i,i} = d^{\text{left}} + d^{\text{right}} + |a|$$

$$A_{i,i+1} = -\left(d^{\text{right}} + \max(0, a)\right)$$
 - 7 计算源项贡献（中点积分近似）：

$$b_i = f(x_i) \cdot \frac{\Delta x_{i-1} + \Delta x_i}{2}$$
 - 8 处理狄利克雷边界条件：

$$A_{0,0} = 1, \quad b_0 = u_0 \quad (\text{左边界})$$

$$A_{N,N} = 1, \quad b_N = u_N \quad (\text{右边界})$$
 其余边界行元素置零（保持矩阵结构）
 - 9 求解三对角线性系统 $\mathbf{A}\mathbf{u} = \mathbf{b}$ （采用 Thomas 算法，时间复杂度 $O(N)$ ）
 - 10 **return** 节点解向量 $\mathbf{u}$
-

4. 谱方法 (Spectral Methods)

谱方法采用一种与有限元法截然不同的“全局”策略。有限元法使用局部支撑的“小帐篷”基函数，每个基函数只影响其周围的一小片区域；而谱方法则使用在整个区域上定义的光滑、振荡的全局函数作为基函数，如正弦函数、余弦函数、切比雪夫多项式、勒让德多项式等。其思想是将未知解展开为这些全局基函数的级数，然后通过某种准则（如要求偏微分方程的残差在配置点上为零，或与所有基函数正交）来确定展开系数。

这种全局逼近策略的背后是一种深刻的权衡：如果所求问题的解本身是光滑的（即无穷可微），那么用同样光滑的全局基函数去逼近它会极其高效。与有限差分法或有限元法的代数收敛（误差随自由度增加按幂律下降）不同，谱方法可以达到指数收敛——这意味着增加少量基函数就能使误差急剧下降，以极少的自由度获得极高的精度。

以定义在区间 $[0, 1]$ 上的 Fourier 基函数 $\sin(k\pi x)$ 、 $\cos(k\pi x)$ 以及定义在 $[-1, 1]$ 上的 Chebyshev 基函数 $T_k(x) = \cos(k \arccos x)$ 为例，低阶基函数（ k 较小）波形平滑、波长较长，而高阶基函数（ k 较大）则呈现快速振荡、波长变短的特征。在数值实现中，常采用 Chebyshev-Gauss 配置点，此类节点在边界附近分布密集，向区间中部逐渐稀疏。这种节点分布有助于抑制 Runge 现象（即采用等距节点进行高次多项式插值时，在区间两端附近产生的剧烈振荡失真），从而提高高阶插值的稳定性。

谱方法使用在整个区间上振荡的基函数，低阶基函数描述解的大尺度特征，高阶基函数捕捉精细结构。

考虑定义在区间 $[-1, 1]$ 上的一维泊松方程：

$$u''(x) = f(x), \quad x \in (-1, 1)$$

并配以狄利克雷边界条件 $u(-1) = a$, $u(1) = b$ 。我们选择切比雪夫多项式作为基函数：

$$T_k(x) = \cos(k \arccos x), \quad k = 0, 1, 2, \dots$$

切比雪夫多项式在区间 $[-1, 1]$ 上关于权重函数 $1/\sqrt{1-x^2}$ 正交，且在边界附近自然形成更密集的节点分布，使其特别适合处理边界层问题。近似解表示为截断展开：

$$u_N(x) = \sum_{k=0}^N \hat{u}_k T_k(x)$$

其中 $\hat{u}_k$ 为待求的展开系数。

谱方法主要有两种实现方式：

- 伽辽金谱方法：将近似解代入微分方程，要求残差与所有测试函数 $T_j(x)$ （ $j = 0, 1, \dots, N$ ）在加权内积意义下正交：

$$\int_{-1}^1 (u_N''(x) - f(x)) T_j(x) \frac{dx}{\sqrt{1-x^2}} = 0$$

这导出一个关于系数 $\hat{u}_k$ 的代数方程组。由于切比雪夫多项式的正交性和导数关系的特殊结构，对于某些常系数线性方程，刚度矩阵可以是对角或近似对角的，计算较为高效。

- 配置点法（伪谱法）：这是工程应用中更为常见的形式。我们选择一组特殊的配置点——切比雪夫-高斯-洛巴托（Chebyshev-Gauss-Lobatto）节点：

$$x_j = \cos\left(\frac{\pi j}{N}\right), \quad j = 0, 1, \dots, N$$

这些节点在区间端点处恰好为 -1 和 1 ，且越靠近边界分布越密集。配置点法要求近似解 $u_N(x)$ 在每个内部配置点上精确满足微分方程：

$$u_N''(x_j) = f(x_j), \quad j = 1, 2, \dots, N-1$$

而边界条件直接强加在端点节点上：

$$u_N(x_0) = a, \quad u_N(x_N) = b$$

为了计算 $u_N''(x_j)$ ，需要引入微分矩阵。记 $u_j = u_N(x_j)$ 为节点上的函数值，则导数可以通过矩阵乘法计算：

$$u_N'(x_j) = \sum_{k=0}^N D_{jk} u_k, \quad u_N''(x_j) = \sum_{k=0}^N D_{jk}^{(2)} u_k$$

其中 D 为一阶微分矩阵， $D^{(2)} = D \cdot D$ 为二阶微分矩阵。这两个矩阵都是稠密的，其元素可通过切比雪夫多项式的解析公式显式给出。于是，离散化方程在内部配置点上写为：

$$\sum_{k=0}^N D_{jk}^{(2)} u_k = f(x_j), \quad j = 1, 2, \dots, N-1$$

结合边界条件，即得到一个关于节点值 $\{u_k\}$ 的线性代数方程组。

算法6.4展示了配置点谱方法求解一维泊松方程的完整流程：

Algorithm 6.4: 配置点谱方法求解一维泊松方程（切比雪夫基函数）

Input: 截断阶数 N （多项式最高次数），

右端项函数 $f(x)$ ，

边界值 $a = u(-1)$ ， $b = u(1)$

Output: 配置点上的解向量 $\mathbf{u}$ ，其中 $u_j \approx u(x_j)$

1 生成切比雪夫-高斯-洛巴托节点：

$$x_j = \cos\left(\frac{\pi j}{N}\right), \quad j = 0, 1, \dots, N$$

2 构造一阶微分矩阵 $\mathbf{D} \in \mathbb{R}^{(N+1) \times (N+1)}$ （标准公式，稠密矩阵）

3 计算二阶微分矩阵 $\mathbf{D}^{(2)} = \mathbf{D} \cdot \mathbf{D}$

4 提取内部节点对应的子矩阵和右端项：

内部节点索引： $j = 1, 2, \dots, N-1$

$\mathbf{A}_{\text{int}} = \mathbf{D}^{(2)}[1:N, 1:N]$ （大小为 $(N-1) \times (N-1)$ ）

$\mathbf{b}_{\text{int}} = f(x_{1:N}) - \mathbf{D}^{(2)}[1:N, 0] \cdot a - \mathbf{D}^{(2)}[1:N, N] \cdot b$

5 求解稠密线性系统 $\mathbf{A}_{\text{int}} \mathbf{u}_{\text{int}} = \mathbf{b}_{\text{int}}$ （采用 LU 分解，时间复杂度 $O(N^3)$ ）

6 组合完整解向量： $\mathbf{u} = [a, \mathbf{u}_{\text{int}}, b]^T$

7 **return** 配置点上的解向量 $\mathbf{u}$

谱方法在自由度的使用上极为高效。在一维问题中，解的自由度由截断阶数 N 决定，仅为 $O(N)$ ，与网格步长无关。扩展到 d 维空间时，若采用张量积形式的基函数，自由度则为 $O(N^d)$ 。这意味着，要达到同样的分辨率，谱方法所需的自由度数远少于有限差分法或有限元法。

然而，这种高效性的代价体现在代数系统的结构上。配置点法产生的微分矩阵是稠密的——每个配置点上的导数计算都依赖于所有节点上的函数值，因此离散得到的代数系统矩阵也是稠密的。在一维问题中，存储成本为 $O(N^2)$ ，直接求解（如 LU 分解）的计算成本为 $O(N^3)$ 。对于二维问题，若采用 N^2 个配置点，则矩阵规模为 $N^2 \times N^2$ ，存储和求解成本分别升至 $O(N^4)$ 和 $O(N^6)$ ，这一增长趋势在高维情形下变得难以承受。因此，实用中的高维谱方法往往需要借助快速变换（如快速傅里叶变换）或迭代求解技术来规避直接处理稠密矩阵。

谱方法最引人瞩目的特点是其收敛速度。如果真解是无限光滑的（即所有阶导数存在且连续），那么随着截断阶数 N 的增加，近似误差将呈指数衰减：

$$\|u - u_N\| \sim O(e^{-cN})$$

其中 $c > 0$ 为常数。这意味着，每增加几个基函数，误差就会下降一个数量级。要达到 10^{-6} 的精度，有限元法可能需要成千上万个自由度，而谱方法往往只需要几十个。这种指数收敛速度远快于有限差分法或有限元法的代数收敛 $O(N^{-p})$ ，使谱方法成为高精度计算的理想工具。

不过，这种惊人的收敛速度有一个关键前提：解必须是光滑的。如果真解存在间断或尖角，情况则截然不同。用光滑的全局基函数去逼近不光滑的函数，会产生著名的吉布斯现象——在间断点附近出现剧烈的非物理振荡，且不会随着 N 增大而消失，只会使振荡频率越来越高。从收敛性角度看，当解不光滑时，谱方法的收敛速度会急剧退化为代数收敛 $O(N^{-1})$ 甚至更差。这一敏感性构成了谱方法最主要的应用限制。

谱方法的最大优势在于其对光滑问题展现出的无与伦比的精度和效率。指数收敛意味着只需极少的自由度即可获得极高精度的数值解，这使得谱方法成为需要高精度计算的领域的首选方法，如湍流的直接数值模拟（DNS）、量子化学计算、气象预报中的谱模式等。同时，对于周期性问题和规则区域上的问题，谱方法能够充分利用快速傅里叶变换等技术，实现高效计算。

然而，谱方法的局限性也同样明显。它对于解的光滑性要求极为苛刻，一旦解存在间断或局部剧烈变化，收敛性就会急剧恶化，甚至出现非物理振荡。此外，谱方法对复杂几何形状的适应性较差——传统的谱方法要求计算域为矩形、圆环等规则形状，难以处理工程中常见的不规则区域。虽然谱元法（Spectral Element Method）等

变体通过将计算域划分为多个子区域并在每个子区域上使用谱逼近，在一定程度上结合了有限元法的几何灵活性和谱方法的高精度，但实现复杂度和计算成本也随之显著增加。最后，谱方法产生的稠密矩阵特性使得高维问题的计算成本急剧上升，限制了其在三维以上问题中的广泛应用。

6.2 PINNs 的基本框架与数学原理

尽管传统数值方法取得了巨大成功，构成了现代科学计算的支柱，但在面对以下新兴挑战时，它们显露出了固有的局限性。这些局限性恰恰构成了以 PINNs 为代表的人工智能方法介入 PDE 求解领域的核心动机。

挑战一：高维问题的“维数灾难”

这是最根本的挑战。传统基于网格的方法，其自由度（网格点、单元、基函数数量）随空间维度 d 指数增长。例如，在 d 维单位超立方体中，若每维离散为 N 个点，则 FDM 和 FEM（在准均匀网格下）的自由度约为 N^d 。对于一个中等维度 $d = 10$ ，即便每维只取 $N = 10$ 个点，总自由度也高达 10^{10} ，这已经超出了常规计算能力。而谱方法虽然自由度也是 $O(N^d)$ ，但其稠密矩阵的存储和运算成本 $O(N^{2d})$ 更是灾难性的。神经网络作为一种函数逼近器，其参数量（权重和偏置）主要取决于网络的宽度和深度，而不直接依赖于输入空间的维数。PINNs 以空间和时间坐标作为输入，直接输出解在该点的值，这种“点对点”的求解方式，绕开了对整个高维空间进行整体离散化的需要。虽然训练过程也需要在许多点上计算损失，但这些点是随机或自适应采样的，其数量可以独立于维度进行控制，为缓解维数灾难提供了一种可能途径。

挑战二：复杂几何、逆问题与自动化

对于复杂和不规则的求解区域，FEM 和 FVM 虽然能通过网格生成来适应，但高质量网格的生成本身计算成本高，且对于涉及拓扑变化（如裂缝扩展）、移动边界（如自由液面）或反演几何形状（逆问题）的场景，网格可能需要动态调整，十分繁琐且需要专业知识。PINNs 是本质上无网格的方法。网络的输入仅仅是坐标点 (x, y, z, t) ，只要能在区域内（包括边界）采样点，就可以构建损失函数进行训练，对几何形状的复杂性极不敏感。这在求解逆问题时优势尤为明显：例如，在只知道部分内部观测数据（如几个传感器的读数），需要同时反推物理参数、未知源项甚至边界形状的问题中，PINNs 可以通过在损失函数中统一纳入 PDE 残差、边界条件（如果已知）和数据拟合项，自然地将其表述为一个统一的优化问题。神经网络参数同时编码了解和待反演参数，通过一次训练即可同时得到它们，而无需为每一种可能的参数或几何假设重新生成网格和求解正问题。

挑战三：融合多源异构数据与不确定性量化

传统的数值求解器通常是“纯物理驱动”的：给定完整的控制方程和定解条件，输出确定的解。然而，在实际应用中，我们常常拥有一些来自实验或观测的稀疏、带噪声的数据，同时也可能对某些物理参数或边界条件存在认知不确定性。传统方法很难将这些数据和不确定性无缝地、自然地融入到求解过程中，通常需要复杂的数据同化（如四维变分）或随机 PDE 框架。PINNs 的损失函数具有天然的融合能力与概率解释扩展潜力。它可以将 PDE 残差损失、边界/初值条件损失与数据拟合损失加权求和。通过最小化这个总损失，神经网络学习到的解会同时满足物理规律和实际观测数据，成为一种“物理驱动 + 数据驱动”的混合模型。进一步地，可以通过贝叶斯框架或引入随机变量，使 PINNs 能够量化预测的不确定性，这对于基于仿真的决策至关重要。

挑战四：求解流程的自动化与通用性

传统数值方法的实施通常需要大量领域专家的介入：根据问题类型选择合适的方法（FEM vs. FVM）、生成高质量网格、设计稳定的离散格式、处理复杂的边界条件、选择和调试求解器参数（如迭代容差、预处理子）等。这个过程专业化程度高，不易自动化，可移植性差。PINNs 提供了一个相对统一的框架：对于一大类 PDE 问题，其工作流程都可以抽象为“定义网络架构、构建物理信息损失（利用自动微分）、在域内和边界采样坐标点、训练网络优化损失”。现代深度学习框架（如 PyTorch, TensorFlow）提供的自动微分（Autograd）功能，使得计算 PDE 残差（往往涉及高阶偏导数）变得异常简单，只需编写原方程代码，无需手动推导复杂的离散格式。这大大降低了 PDE 求解的入门门槛，并有利于构建通用求解工具和代码库。

接下来，我们详细介绍 PINNs 的基本框架与数学原理。

从数学角度看，求解一个 PDE 问题本质上是寻找一个满足特定方程和边界/初始条件的函数。这是一个典型的函数逼近问题：我们希望在无穷维的函数空间中，找到一个满足复杂约束的特定函数。传统数值方法通过将

连续问题离散化，将无穷维问题转化为有限维的线性或非线性系统来求解。这种离散化过程虽然有效，但也在一定程度上“固化”了我们对解的表示方式（例如，基函数的线性组合），并且将计算复杂度与网格分辨率紧密耦合。

正是在这样的背景下，物理信息神经网络应运而生。其核心思想是：能否利用神经网络强大的函数逼近能力，直接去逼近那个满足 PDE 约束的解函数？这个想法并非凭空产生。早在 20 世纪 90 年代，就有研究者探索用神经网络求解微分方程。但其真正的复兴和广泛应用，得益于 2010 年代深度学习的革命性进展：高效自动微分框架的出现、大规模并行计算硬件的普及、以及对神经网络理论理解的深化。Raissi 等人在 2019 年提出的“物理信息神经网络”框架，清晰地勾勒出了这一范式的数学轮廓，使其成为一个统一且灵活的求解器。

PINNs 的根本动机在于，它将 PDE 求解从一个纯粹的数值离散问题，转变为一个受物理定律约束的机器学习问题。神经网络不再仅仅是一个黑箱拟合工具，而是成为一个嵌入物理先验知识的“白箱”或“灰箱”模型。这种视角的转换，为解决传统方法难以处理的复杂问题（如高维、逆问题、数据同化）提供了新的可能性。

1. 核心思想：损失函数作为物理规律的“看守者”

PINNs 的基本思想直观而有力。我们可以将其概括为以下几个关键步骤：

第一步：将未知解参数化。用一个神经网络来逼近 PDE 的解。神经网络的输入是空间坐标 $\mathbf{x}$ 和时间 t ，输出是解的近似值 $\tilde{u}(\mathbf{x}, t; \theta)$ ，其中 θ 代表网络的所有可调参数（权重和偏置）。与传统的数值方法不同，这里没有网格的概念——神经网络本身就是一个全局的函数逼近器。

第二步：将物理定律编码为损失函数。PDE 本身、初始条件和边界条件不再作为必须严格满足的“硬约束”，而是被转化为一个可优化的“软约束”，即损失函数的一部分。网络训练的目标是使这个损失函数最小化。这种做法体现了一种深刻的视角转变：与其在离散点上精确满足方程，不如在整个定义域上寻找一个“尽可能”满足所有约束的函数。

第三步：通过优化进行求解。利用基于梯度的优化算法（如 Adam、L-BFGS）来调整网络参数 θ ，使得损失函数不断减小。当损失足够小时，我们就认为神经网络输出 $\tilde{u}$ 是原 PDE 的一个可接受的近似解。

这个框架的巧妙之处在于，它充分利用了现代深度学习框架中的自动微分技术。为了计算 PDE 残差（即方程左右两边的差值），我们需要计算近似解 $\tilde{u}$ 对输入变量（如 x, t ）的偏导数。在传统编程中，这需要手动推导或使用符号计算，既繁琐又容易出错。而在 TensorFlow、PyTorch 等现代框架中，自动微分可以精确地计算出任意阶的导数，其计算效率与网络前向传播相当。这使得将复杂的微分算子嵌入损失函数变得异常简便。

因此，PINNs 的核心数学原理可以总结为：将一个 PDE 的求解问题，转化为一个以神经网络参数为优化变量，以物理规律（PDE、初边值条件）和数据拟合误差的加权和为目标的非线性优化问题。

2. 数学表述：统一的损失函数框架

现在，让我们形式化地描述一个典型的 PDE 问题，以及 PINN 如何对其进行建模。

考虑一个定义在空间域 $\Omega \subset \mathbb{R}^d$ 和时间区间 $[0, T]$ 上的 PDE 问题。我们寻找函数 $u(\mathbf{x}, t)$ ，使其满足：

- 控制方程（PDE）：

$$\mathcal{N}[u](\mathbf{x}, t) = f(\mathbf{x}, t), \quad (\mathbf{x}, t) \in \Omega \times [0, T]$$

其中 $\mathcal{N}$ 是一个微分算子（例如，对于热方程， $\mathcal{N}[u] = \frac{\partial u}{\partial t} - \kappa \nabla^2 u$ ）。

- 初始条件（IC）：

$$u(\mathbf{x}, 0) = h(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

- 边界条件（BC）：

$$\mathcal{B}[u](\mathbf{x}, t) = g(\mathbf{x}, t), \quad (\mathbf{x}, t) \in \partial\Omega \times [0, T]$$

其中 $\mathcal{B}$ 是边界算子（如狄利克雷条件 $\mathcal{B}[u] = u$ ，或诺伊曼条件 $\mathcal{B}[u] = \nabla u \cdot \mathbf{n}$ ）。

此外，有时我们还有一些观测数据点 $\{(\mathbf{x}_d^i, t_d^i), u_d^i\}_{i=1}^{N_d}$ ，这些数据可能来自实验测量或不完整的数值解。

PINN 方法用一个参数化的神经网络 $\tilde{u}(\mathbf{x}, t; \theta)$ 来近似真实解 $u(\mathbf{x}, t)$ 。接下来，我们构造一个复合损失函数来同时编码所有这些约束：

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_i \mathcal{L}_i(\theta) + \lambda_d \mathcal{L}_d(\theta)$$

其中各项的含义如下：

1. PDE 残差损失 $\mathcal{L}_f$ ：衡量近似解在域内部满足控制方程的程度。通过在域内采集一组“残差点”或“配置点” $\mathcal{T}_f = \{(\mathbf{x}_f^i, t_f^i)\}_{i=1}^{N_f}$ ，计算：

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \mathcal{N}[\tilde{u}](\mathbf{x}_f^i, t_f^i; \theta) - f(\mathbf{x}_f^i, t_f^i) \right|^2$$

这是整个损失函数的核心，它强制神经网络学习隐藏在 PDE 背后的物理规律。

2. 边界条件损失 $\mathcal{L}_b$ ：强制近似解在边界上满足给定条件。在边界上采样一组点 $\mathcal{T}_b = \{(\mathbf{x}_b^i, t_b^i)\}_{i=1}^{N_b}$ ，计算：

$$\mathcal{L}_b(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \left| \mathcal{B}[\tilde{u}](\mathbf{x}_b^i, t_b^i; \theta) - g(\mathbf{x}_b^i, t_b^i) \right|^2$$

3. 初始条件损失 $\mathcal{L}_i$ ：强制近似解在初始时刻满足给定条件。在初始时间面上采样点 $\mathcal{T}_i = \{(\mathbf{x}_i^i, 0)\}_{i=1}^{N_i}$ ，计算：

$$\mathcal{L}_i(\theta) = \frac{1}{N_i} \sum_{i=1}^{N_i} \left| \tilde{u}(\mathbf{x}_i^i, 0; \theta) - h(\mathbf{x}_i^i) \right|^2$$

4. 数据损失 $\mathcal{L}_d$ （可选）：如果有观测数据，用于强制近似解与数据吻合：

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} \left| \tilde{u}(\mathbf{x}_d^i, t_d^i; \theta) - u_d^i \right|^2$$

公式中的 $\lambda_f, \lambda_b, \lambda_i, \lambda_d$ 是正权重系数，用于平衡不同损失项的量级和重要性。这些权重的设置对训练成功至关重要——它们决定了网络在拟合 PDE 残差、边界条件、初始条件和观测数据之间的优先级。

最终，PDE 的求解转化为一个优化问题：

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

通过梯度下降法或其变种（如 Adam）迭代更新 θ ，最终得到的网络 $\tilde{u}(\mathbf{x}, t; \theta^*)$ 即为所求的近似解。

在实践中，损失函数中的权重系数 λ 的设置往往是一个经验性的难题。PDE 残差损失 $\mathcal{L}_f$ 涉及高阶导数，其数值通常远小于边界/初始条件损失 $\mathcal{L}_b, \mathcal{L}_i$ 。如果简单地将所有权重设为 1，优化过程可能会被 $\mathcal{L}_b$ 和 $\mathcal{L}_i$ 主导，导致网络快速满足边值/初值条件，但 PDE 残差依然很大，得到一个不满足物理定律的平凡解。

在实践中，上述权重系数 λ 的设置往往是一个经验性的难题。PDE 残差损失 $\mathcal{L}_f$ 涉及高阶导数，其数值通常远小于边界/初始条件损失 $\mathcal{L}_b, \mathcal{L}_i$ 。如果简单地将所有权重设为 1，优化过程可能会被 $\mathcal{L}_b$ 和 $\mathcal{L}_i$ 主导，导致网络快速满足边值/初值条件，但 PDE 残差依然很大，得到一个不满足物理定律的平凡解。

常见的调优策略包括：

- 经验试错：根据损失项初始量级的比例手动设置权重，例如使 $\lambda_f \mathcal{L}_f(0) \approx \lambda_b \mathcal{L}_b(0) \approx \lambda_i \mathcal{L}_i(0)$ 。这种方法简单直观，但需要多次尝试。
- 学习率退火：在训练过程中动态调整权重，例如根据各损失项下降的速度或当前值进行自适应调整。一种常见做法是让权重与相应损失项的平均值成反比，使得各项损失对总损失的贡献保持均衡。
- 基于梯度的平衡：通过监测不同损失项对参数更新的相对贡献（即梯度范数）来调整权重，使各项对优化的影响大致均衡。其基本思路是：如果一个损失项产生的梯度范数过大，就降低其权重；反之则增加其权重。
- 自适应的概率模型：将每个损失项视为一个高斯分布的负对数似然，其方差（即权重的倒数）作为可训练参数，通过最大似然估计同步优化。这种方法将权重选择问题转化为自动学习过程，但实现相对复杂。

损失函数的计算依赖于在域内、边界和初始时刻采集的离散点集。这些点的分布策略直接影响训练的效率 and 最终解的精度。常用的采样策略包括：

- 均匀随机采样：最简单的方法，在定义域内均匀随机采样。适用于解变化平缓的问题。
- 拉丁超立方采样：一种分层随机采样方法，将每个维度划分为等概率区间，然后在每个区间内各取一个样

本。这种方法能保证投影到每个坐标轴上的点分布均匀，在中等维度下比纯随机采样具有更好的覆盖性。

- 拟蒙特卡洛序列：如 Sobol 序列、Halton 序列等低差异序列，能够生成比随机采样分布更均匀的点集，通常能加速收敛并提供更稳定的训练过程。
- 自适应重要性采样：在训练过程中，根据当前 PDE 残差的大小动态调整采样点的分布，在残差大的区域采集更多点。这种方法可以更有效地分配计算资源，集中处理困难区域。

初始采样点的数量和分布是需要调整的超参数。通常，边界和初始条件点需要足够密集以确保约束被准确施加，而内部配置点的数量则决定了网络对 PDE 的学习充分程度。经验上，配置点数量通常远多于边界点和初始点。

PINN 训练流程如图 6.3 所示。PINN 训练包括训练点采样、前向传播与自动微分、损失构造以及参数更新，并通过迭代优化直至收敛。

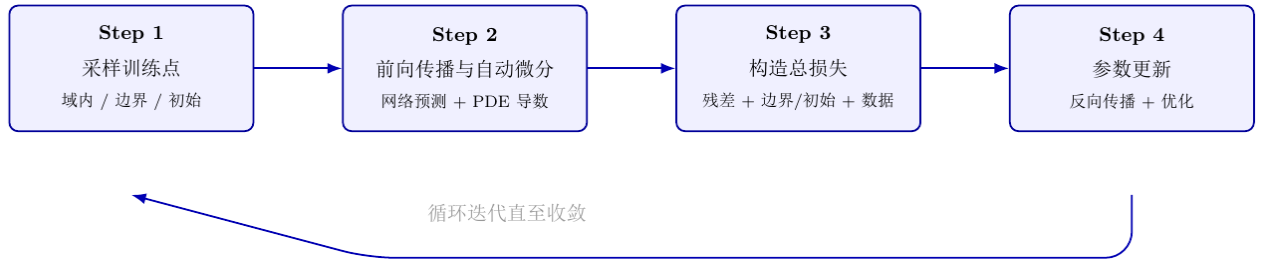


图 6.3: PINN 训练流程

3. 网络架构：为何选择全连接网络？

在 PINNs 的早期和大多数工作中，选择的基础架构是多层感知机（Multilayer Perceptron, MLP），也称为全连接前馈神经网络。这一选择并非偶然，而是由 MLP 的数学性质和 PDE 问题的特点共同决定的。

从数学上看，MLP 受到万有逼近定理（Universal Approximation Theorem）的支持。该定理指出，只要隐藏层神经元足够多，一个单隐藏层的 MLP 就能以任意精度逼近任何定义在紧致集上的连续函数。对于更深的网络，其表达能力则更强。PDE 的解通常是我们期望的光滑函数（至少在域内部），因此 MLP 在理论上有能力逼近它。

定理 6.1 (万有逼近定理)

令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数， I_d 是一个 d 维的单位超立方体 $[0, 1]^d$ ， $C(I_d)$ 是定义在 I_d 上的连续函数集合。对于任何一个函数 $f \in C(I_d)$ ，存在一个整数 m ，以及一组实数 $v_i, b_i \in \mathbb{R}$ 和实数向量 $\mathbf{w}_i \in \mathbb{R}^d$ ($i = 1, \dots, m$)，使得由

$$F(\mathbf{x}) = \sum_{i=1}^m v_i \varphi(\mathbf{w}_i^T \mathbf{x} + b_i)$$

定义的函数 F 满足

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \quad \forall \mathbf{x} \in I_d$$

其中 $\epsilon > 0$ 是任意小的正数。

万有逼近定理保证了 MLP 作为通用函数逼近器的能力：只要网络足够宽，它能够以任意精度逼近任何连续函数。这一理论基石为 PINNs 的可行性提供了严格的数学支撑。

从问题结构来看，PDE 的输入通常是空间坐标和时间，维度较低（1D、2D、3D 加上时间），而输出是解的数值，通常是标量或低维向量。MLP 天然适合处理这种“坐标到值”的映射关系。与卷积神经网络（CNN）假设输入具有局部空间结构、循环神经网络（RNN）假设输入具有时序结构不同，MLP 没有对数据的结构做出任何先验假设，这使得它成为一个真正通用的函数逼近器，可以平等地处理各种几何形状和方程类型。

一个典型的用于 PINN 的 MLP 可以表示为：

$$\begin{aligned}\mathbf{z}^{(0)} &= \mathbf{x} \quad (\text{输入层, 包含空间坐标和时间}) \\ \mathbf{z}^{(l)} &= \sigma(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \dots, L-1 \quad (\text{隐藏层}) \\ \tilde{\mathbf{u}} &= \mathbf{W}^{(L)}\mathbf{z}^{(L-1)} + \mathbf{b}^{(L)} \quad (\text{输出层})\end{aligned}$$

其中 σ 是非线性激活函数，如 $\tanh$ 、 $\sin$ 或 ReLU 。在 PINN 中， $\tanh$ 因其光滑性（无穷可微）和有界性（输出值域为 $[-1, 1]$ ）而尤为常用，因为光滑性保证了自动微分计算高阶导数的稳定性，而有界性则有助于训练过程的稳定收敛。

然而，MLP 并非唯一选择。根据具体问题的特性，研究者也探索了其他架构。表 6.2 对比了几种常见架构在 PINN 中的适用场景和优缺点。

表 6.2: PINN 中常用神经网络架构对比

架构类型	核心思想	适用场景	优点	缺点与挑战
MLP	全连接, 通用函数逼近器	通用, 低维问题, 不规则几何	理论保障充分（万有逼近定理），实现简单，与自动微分天然兼容	存在“谱偏差”，高维问题时参数效率低
CNN	局部连接, 权重共享, 平移不变性	规则网格数据（如图像化物理场），具有局部相关性的问题	参数效率高，能有效提取局部特征，适合处理高维空间	对不规则几何处理复杂，需要将输入投影到规则网格
RNN/LSTM	循环连接, 记忆历史状态	强时间依赖性问题, 序列预测	天然适合时序建模，能捕捉长时间依赖	训练易遇梯度消失/爆炸，并行化程度低
Transformer	自注意力机制, 全局依赖建模	复杂几何, 长程相互作用, 点云数据	能建模任意距离的依赖关系，灵活性强	计算和内存开销大，训练需要较大数据量
KAN	可学习的激活函数, 基于 Kolmogorov-Arnold 定理	函数逼近, 可解释性需求高的场景	参数量更少，可能具有更好的可解释性	尚处于探索阶段，理论和方法仍在发展

选择何种架构应视具体问题而定。对于大多数标准的、中低维度的正向 PDE 问题，一个足够深和宽的 MLP 通常是可靠且简便的起点。当问题具有明显的结构特性（如时空序列、规则域上的图像数据）时，选择对应的专用架构（RNN、CNN）可能带来效率和解质量的提升。Transformer 和 KAN 等新兴架构则代表了前沿探索方向，适用于传统架构效果不佳的复杂场景。

4. 误差来源的三重奏：逼近、优化与泛化

理解 PINN 的误差来源对于评估其性能和可靠性至关重要。与传统的数值方法不同，PINN 的误差由三个相互关联的部分构成，我们可以称之为“误差三重奏”：

- 逼近误差：**即使存在一组最优的网络参数 θ^* ，神经网络函数空间 $\{\tilde{u}(\cdot; \theta)\}$ 也可能无法完美表示真实解 u 。这是由网络架构的有限表达能力（如深度、宽度、激活函数的选择）决定的。万有逼近定理保证了在无限宽或无限深的情况下误差可以任意小，但在实践中网络是有限规模的，其表达能力必然受到限制。对于具有奇异性、边界层或高频振荡的解，逼近误差可能尤为显著——因为这些复杂特征需要网络具有足够的容量才能准确捕捉。此外，激活函数的光滑性也会影响逼近能力：例如，ReLU 网络只能表示分段线性函数，而 $\tanh$ 网络则可以逼近光滑函数。
- 优化误差：**训练过程可能无法找到全局最优的 θ^* ，而是陷入一个局部最优解 $\tilde{\theta}$ ，使得 $\mathcal{L}(\tilde{\theta}) > \mathcal{L}(\theta^*)$ 。神经网络的损失函数通常是非凸的，在高维参数空间中存在大量的鞍点和局部极小值。优化算法的选择（Adam 还是 L-BFGS?）、学习率的调度策略、网络参数的初始化方式都会显著影响最终收敛到的参数点。此外，损失函数中不同项之间的数值量级差异（如前所述）也会导致优化困难，使得某些约束被忽视。因此，即使网络理论上能够表示精确解，不恰当的优化过程也可能无法找到它。

3. 泛化/离散化误差：即使我们找到了一个在训练样本点上损失很低的网络，也不能保证它在整个连续域上都满足 PDE。这是因为损失函数是在一组有限的点集 $\{\mathcal{T}_f, \mathcal{T}_b, \mathcal{T}_i\}$ 上计算的，而不是在整个连续域上。如果这些采样点分布不合理——例如，在解变化剧烈的区域采样不足，或者在边界附近采样稀疏——网络可能会在未采样的区域产生很大的误差。这类似于传统数值方法中的离散化误差，但在 PINN 中，采样点的选择和分布更加灵活，既可以均匀随机采样，也可以采用自适应策略在困难区域加密。然而，这种灵活性也带来了新的挑战：如何设计采样策略以保证整个域上的精度，目前仍然是一个开放的研究问题。

这三类误差相互耦合，使得 PINN 的理论分析远比传统数值方法复杂。例如，一个容量足够大的网络（逼近误差小）可能更难以优化（优化误差大），而一个在训练点上拟合完美的网络可能在未采样点处表现糟糕（泛化误差大）。在实践中，我们更多地依赖后验误差评估来评判解的质量：在训练完成后，在一个独立的、密集的测试点集上评估损失，检查解的物理合理性（如对称性、守恒律），并与已知的解析解或高精度数值解进行对比。此外，许多物理系统满足守恒定律（如质量、能量、动量守恒），我们可以计算 PINN 解对应的这些守恒量，检查其是否在物理上合理（例如，在封闭系统中是否随时间保持恒定），这是一种有效的物理一致性检验手段。

5. 理论可信度：何时可以信任 PINN？

鉴于上述挑战，一个自然的问题是：在什么情况下，我们可以对 PINN 的解抱有较高的信心？虽然完整的理论仍在发展中，但基于现有研究和经验，我们可以勾勒出一些指导原则：

- 解的光滑性：当 PDE 的解足够光滑（没有奇点、间断或强边界层）时，MLP 更容易逼近，谱偏差的影响较小，PINN 通常表现良好。椭圆型方程（如泊松方程）在光滑域和光滑边界条件下是这类问题的典型代表。相反，对于含有激波的流体问题或具有边界层的奇异摄动问题，标准 PINN 往往难以捕捉这些局部剧烈变化，需要引入特殊处理（如自适应采样、域分解或冲击捕捉技术）。
- 问题的维度：对于空间维度不超过 3 维（加上时间）的问题，PINN 的采样和训练相对可控，能够以合理的计算成本获得满意的精度。对于更高维的问题（如高维 PDE、不确定性量化中的随机 PDE），标准 PINN 会面临“维数灾难”——随着维度增加，需要指数级增长的采样点才能覆盖整个空间。此时需要特别的架构（如张量分解网络）或采样策略（如稀疏网格、低差异序列）来应对这一挑战。
- 问题的适定性：PINN 最适合求解适定的正向问题——即已知完整的方程形式、所有参数、以及完备的边界条件和初始条件。对于逆问题（需要同时推断方程参数或边界条件）或数据稀缺的情况，虽然 PINN 也能处理，但由于解空间的不确定性增大，失败的风险会显著增加。此时，引入额外的正则化或利用物理先验知识显得尤为重要。
- 充分的训练与验证：训练损失下降并不等同于解的高精度。必须使用独立的、密集的测试点集来验证解的精度，而不仅仅是依赖训练损失。检查解的物理合理性（如对称性、单调性、守恒律）也是重要的验证手段。对于时间相关问题，可以检查解是否满足能量耗散或质量守恒等物理性质。
- 与传统方法的一致性：在可能的情况下，将 PINN 的解与高精度的传统数值解（如谱方法或精细网格的有限元解）进行比较。一致性是建立信心的有力证据。即使无法获得全局的精确解，在局部区域或简化情况下进行对比验证也是有益的实践。

当面对不满足上述条件的问题时（例如，含有激波的流体问题、高维随机 PDE、几何形状极其复杂），则需要诉诸于更高级的 PINN 变体或混合方法，如域分解、自适应采样、多尺度网络结构等。

然而，我们必须清醒地认识到，PINNs 并非旨在全面取代传统数值方法。在传统方法擅长的领域（如规则区域上的线性 PDE、大规模工业级稳态仿真），它们在精度、效率、可靠性和成熟度上仍然具有压倒性优势。PINNs 的出现，更像是为我们的 PDE 求解工具箱添加了一套全新的、互补的工具。它特别适合于那些传统方法处理起来困难或繁琐的场景：高维问题、复杂几何与逆问题、以及需要灵活融合物理模型与多源数据的场景。

6.3 应用实例

在前面的小节中，我们系统地介绍了物理信息神经网络的基本框架、训练技巧与稳健化方法。理解这些理论与技术是基础，而了解它们如何被应用于解决真实的科学计算问题，则是掌握 PINN 方法的最终环节。本节我们

将通过一系列具有代表性的应用案例，展示 PINN 如何在多个科学与工程领域中，面对复杂的偏微分方程问题，发挥其独特优势。

本节将深入剖析每个案例背后的数学问题本质，解释 PINN 方法的设计动机与核心思想，并通过对结果与局限性的讨论，勾勒出 PINN 方法的应用边界与潜力。我们将按照以下逻辑展开每个案例：问题背景与数学表述、传统方法的局限与 PINN 的动机、PINN 模型的具体设计、结果分析与对照、总结与启示。这种结构旨在清晰地揭示，对于一个具体的数学问题（PDE），PINN 是如何介入，并从方法论层面提供了何种新的可能性。

例一：求解带有复杂源项与几何的泊松方程（正向问题）

● 问题背景与数学表述

泊松方程是数学物理中最基本的一类椭圆型偏微分方程，描述了势场（如静电场、重力场、稳态温度场）的分布。其一般形式为：

$$-\nabla^2 u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

并配备适当的边界条件，例如狄利克雷边界条件 $u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega$ 。

● 传统方法的局限与 PINN 的动机

泊松方程的求解是计算科学中的经典问题，传统数值方法如有限元法已非常成熟。然而，当源项 $f(\mathbf{x})$ 具有剧烈变化、奇异性或高振荡特性时，或者计算域 Ω 的几何形状非常复杂（如多连通域、带有尖锐凹角的域）时，传统方法可能面临严峻挑战。例如，为捕捉高振荡源项的影响，有限元法需要极其精细的局部网格，导致计算量剧增；对于复杂几何，高质量的网格生成本身就是一个难题，往往需要大量的人工干预。PINN 作为一种无网格方法，其优势在于能够自然地处理复杂几何——只需在定义域内采样坐标点作为输入，无需显式网格生成。对于复杂源项，PINN 理论上可以凭借神经网络的通用逼近能力进行拟合，而无需在源项剧烈变化的区域预先加密网格。

● PINN 模型的具体设计

针对上述问题，PINN 模型的设计如下：

1. 网络架构：采用多层感知机（MLP），输入为空间坐标 $\mathbf{x}$ ，输出为标量场 $\hat{u}(\mathbf{x}; \theta)$ 。隐藏层通常使用 $\tanh$ 激活函数以保证光滑性。
2. 损失函数：损失函数由两部分构成：

- 物理损失（PDE 残差）：在计算域 Ω 内采样 N_f 个“残差点” $\{\mathbf{x}_f^i\}_{i=1}^{N_f}$ ，计算：

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} |-\nabla^2 \hat{u}(\mathbf{x}_f^i; \theta) - f(\mathbf{x}_f^i)|^2.$$

其中， $\nabla^2 \hat{u}$ 通过自动微分（AutoDiff）计算。

- 边界损失：在边界 $\partial\Omega$ 上采样 N_b 个点 $\{\mathbf{x}_b^i\}_{i=1}^{N_b}$ ，计算：

$$\mathcal{L}_b(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} |\hat{u}(\mathbf{x}_b^i; \theta) - g(\mathbf{x}_b^i)|^2.$$

总损失为 $\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f + \lambda_b \mathcal{L}_b$ 。权值 λ_f, λ_b 可以手动设定或采用自适应加权策略。

3. 训练：通过梯度下降法最小化总损失 $\mathcal{L}(\theta)$ 。

● 结果分析与对照

在具有高振荡源项 $f(x) = 50 \sin(20\pi x) \sin(20\pi y)$ 的方形域上，PINN 能够以极少的采样点（通常几千个）获得与有限元法在百万级网格上相当的精度。更重要的是，对于带有复杂几何（如星形域、带有孔的域）的问题，PINN 无需进行网格生成，只需在域内随机采样即可直接求解。实验表明，当几何边界复杂时，PINN 的训练收敛速度和最终精度主要受限于边界条件损失的处理方式——硬约束嵌入（将边界条件直接编码进网络输出）通常比软约束（通过损失项惩罚）表现更好。

● 总结与启示

本案例表明，PINN 在处理复杂几何和复杂源项的正向问题时展现出独特的灵活性。无网格特性使其免去

了耗时的网格生成过程，而自动微分则简化了高阶导数的计算。然而，需要注意的是，对于具有奇异性（如点源）的问题，标准 PINN 可能难以捕捉局部剧烈变化，此时需要引入自适应采样或域分解等技术来提升局部精度。

例二：伯格方程与流体动力学中的正向问题

• 问题背景与数学表述

伯格方程是一个非线性的抛物型偏微分方程，是流体力学中 Navier-Stokes 方程的简化模型，用于描述一维粘性流体的运动，兼具对流和非线性效应。其形式为：

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-L, L], \quad t \in [0, T]$$

其中 $u(x, t)$ 是流体速度， ν 是粘性系数。需要给定初始条件 $u(x, 0) = u_0(x)$ 和边界条件（通常为狄利克雷或周期性边界条件）。

• 传统方法的局限与 PINN 的动机

伯格方程虽然形式相对简单，但因其非线性特性，能够产生丰富的物理现象，如激波的形成与传播。当粘性系数 ν 很小时，方程趋于双曲型，解可能产生陡峭的梯度（近似的间断）。用传统的谱方法或有限差分法求解时，需要精心设计风格（如迎风格式、TVD 格式）来数值稳定地捕捉激波，避免非物理振荡。此外，传统方法通常需要在时空网格上进行离散，时间步长受到 CFL 条件的约束。

PINN 求解伯格方程，本质上是将时空坐标 (x, t) 作为输入，直接学习整个时空域上的解场 $\hat{u}(x, t; \theta)$ 。其吸引力在于：

1. 无需求解离散的代数方程组，避免了设计稳定格式的麻烦；
2. 一次训练得到整个时空的解，便于后续分析和可视化；
3. 易于处理参数化问题，例如将 ν 也作为输入，学习解随粘性系数的变化。

• PINN 模型的具体设计

损失函数设计如下：

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f + \lambda_i \mathcal{L}_i + \lambda_b \mathcal{L}_b$$

其中：

- $\mathcal{L}_f$: PDE 残差损失，在时空域内采样点计算 $\left| \frac{\partial \hat{u}}{\partial t} + \hat{u} \frac{\partial \hat{u}}{\partial x} - \nu \frac{\partial^2 \hat{u}}{\partial x^2} \right|^2$ 。
- $\mathcal{L}_i$: 初始条件损失，在 $t = 0$ 的线上采样计算 $|\hat{u}(x, 0; \theta) - u_0(x)|^2$ 。
- $\mathcal{L}_b$: 边界条件损失，在 $x = -L$ 和 $x = L$ 的边界上采样计算 $|\hat{u}(\pm L, t; \theta) - g(t)|^2$ 。

对于小粘性（如 $\nu = 0.01/\pi$ ）的问题，解会在短时间内形成陡峭梯度。为捕捉这一特征，通常需要在激波区域附近进行更密集的采样，或采用自适应采样策略。

• 结果分析与对照

在小粘性伯格方程的标准测试问题中（初始条件为 $u_0(x) = -\sin(\pi x)$ ，周期性边界条件），PINN 能够准确捕捉激波的位置和形状。与传统的有限差分法（如迎风格式）相比，PINN 在激波附近不会产生过冲或非物理振荡，但代价是激波区域可能被一定程度地“抹平”，这是由于神经网络的全局光滑性导致的。研究表明，增加采样点密度或引入自适应采样可以显著改善激波的分辨率。

• 总结与启示

伯格方程案例展示了 PINN 处理非线性 PDE 的能力。值得注意的是，PINN 无需特殊设计即可稳定地捕捉激波，避免了传统方法中复杂的数值格式选择。然而，对于极低粘性的近双曲情形，标准 PINN 仍面临挑战——激波可能被过度抹平，或者训练难以收敛。这一问题的解决通常需要结合自适应采样、激波检测技术或域分解方法。

例三：参数反演（逆向问题）

• 问题背景与数学表述

逆向问题在实际工程和科学实验中极为常见：我们通过观测获得系统的一部分数据（如某些点的测量值），需要据此推断未知的方程参数。传统方法解决此类问题通常需要复杂的优化迭代和多次正向求解，计算成本高昂。PINN 天然适合处理此类问题，因为它能够将观测数据作为软约束无缝地融入损失函数。

考虑一个带未知参数的扩散方程：

$$\frac{\partial u}{\partial t} - \kappa \frac{\partial^2 u}{\partial x^2} = 0, \quad x \in [0, L], \quad t \in [0, T]$$

配备初始条件 $u(x, 0) = h(x)$ ，边界条件 $u(0, t) = g_1(t)$ ， $u(L, t) = g_2(t)$ ，其中扩散系数 κ 是未知的。我们在时空域中的某些点 (x_d^i, t_d^i) 上观测到了场的值 u_d^i ，这些观测可能带有噪声。目标是从这些稀疏数据中，同时复原出整个场 $u(x, t)$ 和参数 κ 。

• 传统方法的局限与 PINN 的动机

传统方法处理此类参数反演问题通常采用“先正向求解，再优化参数”的两步迭代策略：在每一步迭代中，使用当前的参数估计值求解正向问题，然后根据观测数据与计算值的差异更新参数。这种方法需要多次完整的正向求解，计算成本高昂，且对正向求解器的精度和稳定性高度敏感。

PINN 采用端到端的联合学习方式，将参数反演与正向求解融合在同一个优化框架中。这使得我们可以利用自动微分同时计算关于网络参数和物理参数的梯度，避免了多次正向求解的循环。

• PINN 模型的具体设计

1. 网络架构：输入为 (x, t) ，输出为 $\hat{u}(x, t; \theta)$ 。
2. 待学习参数：网络权重 θ 和物理参数 κ 。将 κ 视为一个可训练的张量（标量）。
3. 损失函数：

$$\mathcal{L}(\theta, \kappa) = \lambda_f \mathcal{L}_f(\theta, \kappa) + \lambda_i \mathcal{L}_i(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta)$$

其中前三项与正向问题相同， $\mathcal{L}_f$ 的计算中包含了未知参数 κ 。新增的 ** 数据损失项 ** 为：

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |\hat{u}(x_d^i, t_d^i; \theta) - u_d^i|^2$$

4. 联合优化：通过梯度下降法同时更新 θ 和 κ ，最小化总损失。

• 结果分析与对照

在标准测试中，假设我们在时空域中仅有少量观测点（例如 $N_d = 20$ ），且观测数据包含一定噪声。PINN 能够以较高精度同时复原出完整的解场和未知参数 κ 。研究表明，参数 κ 的反演精度对观测点的数量和分布较为敏感——在解变化剧烈的区域布置更多观测点通常能获得更好的反演效果。与传统的基于伴随的方法相比，PINN 的实现更加简洁，且不需要推导伴随方程。

• 总结与启示

参数反演案例展示了 PINN 处理逆向问题的独特优势：它将正向求解与参数优化统一在一个框架中，使得我们可以利用自动微分高效地计算关于所有未知量的梯度。这种端到端的学习方式简化了逆向问题的求解流程，尤其适合处理观测数据稀疏、噪声较大的情况。然而，需要警惕的是，当观测数据过于稀疏或参数空间存在多解性时，反演问题可能是不适定的，此时需要引入额外的正则化项或先验知识来保证解的合理性。

例四：源项反演与数据同化（逆向问题）

• 问题背景与数学表述

另一个重要的逆向问题类型是源项反演，即 PDE 的右端项 $f(\mathbf{x})$ 或初始条件未知，需要通过观测数据来推断。这在诸如生物医学成像、污染源定位、热源识别等领域有广泛应用。

考虑稳态泊松方程，其源项 $f(\mathbf{x})$ 未知：

$$-\nabla^2 u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad u(\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega$$

我们在域内某些点 $\{\mathbf{x}_d^i\}$ 上观测到解的值 u_d^i ，目标是根据这些数据反演出源项 $f(\mathbf{x})$ 和全场 $u(\mathbf{x})$ 。

- 传统方法的局限与 PINN 的动机

与参数反演不同，源项反演需要反演的是一个函数而非单个参数，这显著增加了问题的复杂度。传统方法通常将源项离散为有限维参数（如用一组基函数展开），然后进行参数优化，但离散化带来的误差可能影响反演精度，且基函数的选择本身就是一个难题。

PINN 提供了一种优雅的解决方案：将未知源项也用神经网络参数化，从而将函数反演问题转化为两个网络的联合优化问题。这种处理方式避免了显式的离散化假设，使得源项可以在连续函数空间中表示。

- PINN 模型的具体设计

联合求解的 PINN 架构包含两个网络：

- 解网络： $\hat{u}(\mathbf{x}; \theta)$ ，输出解的近似值
- 源项网络： $\hat{f}(\mathbf{x}; \phi)$ ，输出源项的近似值

损失函数需同时约束两个网络：

$$\mathcal{L}(\theta, \phi) = \lambda_f \mathcal{L}_f(\theta, \phi) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta)$$

其中 PDE 残差损失变为：

$$\mathcal{L}_f(\theta, \phi) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| -\nabla^2 \hat{u}(\mathbf{x}_f^i; \theta) - \hat{f}(\mathbf{x}_f^i; \phi) \right|^2$$

边界损失 $\mathcal{L}_b$ 与数据损失 $\mathcal{L}_d$ 形式不变。通过联合优化 θ 和 ϕ ，可以同时复原出解场和源项。

- 结果分析与对照

在二维问题中，假设真实源项为局部高斯型分布（如 $f(x, y) = \exp(-(x - 0.5)^2 - (y - 0.5)^2)$ ），观测数据为稀疏的 20 个点。PINN 能够准确恢复源项的位置和形状，即使观测点没有覆盖源项的核心区域。与传统的 Tikhonov 正则化方法相比，PINN 不需要预先假设源项的光滑性或形式，具有更强的表达能力。

- 总结与启示

源项反演案例展示了 PINN 处理函数型逆向问题的能力。通过引入额外的源项网络，PINN 将函数反演问题转化为可微分的联合优化问题，使得我们可以利用自动微分高效求解。然而，需要注意的是，源项反演问题通常具有严重的非唯一性——不同的源项可能产生几乎相同的解场。因此，在应用中需要结合先验知识（如源项的非负性、稀疏性）来约束解空间，或者通过增加观测数据来消除歧义。

例五：求解高维偏微分方程

- 问题背景与数学表述

很多重要的物理和金融模型涉及高维空间中的偏微分方程。例如，量子多体问题的薛定谔方程、金融衍生品定价的布莱克-舒尔斯方程（考虑多个标的资产时）、以及描述随机过程的福克-普朗克方程。这些方程的维度 d 可能从几十到数百，传统方法无法处理。

考虑一个基于 d 个标的资产的欧式一篮子期权定价问题，其定价函数 $u(\mathbf{s}, t)$ 满足高维 Black-Scholes 方程：

$$\frac{\partial u}{\partial t} + \frac{1}{2} \sum_{i,j=1}^d \rho_{ij} \sigma_i \sigma_j s_i s_j \frac{\partial^2 u}{\partial s_i \partial s_j} + r \sum_{i=1}^d s_i \frac{\partial u}{\partial s_i} - ru = 0$$

其中 $\mathbf{s} = (s_1, \dots, s_d) \in \mathbb{R}_+^d$ ，终值条件为 $u(\mathbf{s}, T) = \max\left(\frac{1}{d} \sum_{i=1}^d s_i - K, 0\right)$ 。

- 传统方法的局限与 PINN 的动机

传统网格类数值方法（如有限差分、有限元）在高维情况下完全失效，因为所需的网格点数量随维度 d 呈指数增长（ $O(N^d)$ ），这就是所谓的“维数灾难”。即便像蒙特卡洛方法这类概率算法，在高维积分或求解高维 PDE 时也可能面临收敛缓慢的问题。

神经网络，特别是 MLP，其参数数量是随输入维度线性或多项式增长的，而非指数增长。因此，PINN 被寄予厚望，有望成为打破维数诅咒、求解高维 PDE 的有力工具。其基本形式不变：将高维坐标 $\mathbf{x} \in \mathbb{R}^d$ 作

为输入，输出标量场 $u(\mathbf{x}; \theta)$ ，损失函数由高维 PDE 的残差和边界条件构成。

• PINN 模型的具体设计与改进方案

直接用标准 PINN 求解高维问题面临巨大挑战：高维空间采样效率低、损失函数地形复杂、计算高维混合偏导数开销大。为此，研究者提出了多种改进方案：

1. 可分 PINNs (Separable PINNs, SPINNs): 核心思想是将高维解函数近似为多个低维函数乘积的求和（可分离结构），从而极大降低计算复杂度。具体而言，假设解可以表示为：

$$u(\mathbf{x}) \approx \sum_{k=1}^r \prod_{i=1}^d \psi_{i,k}(x_i)$$

其中每个 $\psi_{i,k}$ 是一维函数，由子网络表示。这种分解使得前向传播和梯度计算不再直接依赖于全维度的点积，将计算复杂度从 $O(N^d)$ 降至 $O(dNr)$ 。

2. 域分解 (cPINNs/XPINNs): 将高维计算域划分为若干子域，在每个子域上训练一个独立的 PINN，并通过在子域交界面上施加连续性条件（作为额外的损失项）来保证全局解的连续性。总损失函数形式为：

$$\mathcal{L}_{\text{total}} = \sum_{k=1}^K \mathcal{L}_{\text{PINN}}^{(k)} + \sum_{\text{interfaces}} \lambda_{\Gamma} \mathcal{L}_{\Gamma}$$

其中 $\mathcal{L}_{\Gamma}$ 惩罚界面上的解或通量的不连续性。域分解能够将高维问题分解为多个较低维的子问题，降低训练难度。

3. 自适应采样与特征映射：在高维空间中使用基于残差的自适应采样，将计算资源集中在残差较大的区域。同时，利用随机傅里叶特征等映射将输入投影到更高维的特征空间，可以缓解神经网络的“谱偏差”问题，提升对高频成分的学习能力。

• 结果分析与对照

对于 $d = 50$ 的期权定价问题，标准 PINN 在合理训练时间（数小时）内能够获得与蒙特卡洛参考解相对误差在 1% 以内的结果。而传统网格法在如此高的维度下根本无法实施。SPINNs 进一步将训练时间缩短了一个数量级，同时保持相当的精度。

• 总结与启示

高维案例展示了 PINN 在突破维数诅咒方面的巨大潜力。然而，我们也需要清醒地认识到，当维度超过几百时，即使 PINN 也会面临采样效率、训练稳定性等方面的严峻挑战。目前，高维 PINN 的成功案例主要集中在具有特殊结构（如可分离、低秩）的问题上，对于一般的高维 PDE，仍然需要更深入的理论和方法创新。很多重要的物理和金融模型涉及高维空间中的偏微分方程。例如，量子多体问题的薛定谔方程、金融衍生品定价的布莱克-舒尔斯方程（考虑多个标的资产时）、以及描述随机过程的福克-普朗克方程。这些方程的维度 d 可能从几十到数百。

例六：复杂多物理场耦合问题

• 问题背景与数学表述

许多前沿科学与工程问题涉及多个物理场的相互作用，例如流固耦合（Fluid-Structure Interaction, FSI）、热-流-固耦合等。这些问题的控制方程通常是一组相互耦合的非线性 PDE，求解极具挑战性。

以一个简化的二维流固耦合问题为例：

- 流体域 Ω_f ：不可压缩流体的运动由 Navier-Stokes 方程描述：

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \quad \nabla \cdot \mathbf{u} = 0$$

- 固体域 Ω_s ：弹性体的变形由线弹性动力学方程描述：

$$\rho_s \frac{\partial^2 \mathbf{d}}{\partial t^2} = \nabla \cdot \boldsymbol{\sigma}(\mathbf{d}) + \mathbf{f}_s$$

- 交界面条件：在流固交界面 Γ 上，需满足：

- 运动学条件（速度连续）： $\mathbf{u} = \frac{\partial \mathbf{d}}{\partial t}$

- 动力学条件（应力连续）： $\sigma_f \cdot \mathbf{n} = \sigma_s \cdot \mathbf{n}$

• 传统方法的局限与 PINN 的动机

传统分区迭代求解需要在每个时间步在流体求解器和固体求解器之间反复传递数据，直至收敛。这种解耦策略不仅计算成本高，而且存在数值稳定性问题（如“附加质量”不稳定性）。此外，对于复杂几何和变形，网格更新和重新生成也是一大挑战。

PINN 提供了一种整体求解的可能性。可以构建一个统一的神经网络，其输入是时空坐标 $(\mathbf{x}, t)$ 以及一个域标识符，其输出是所有相关物理场（如流体速度 $\mathbf{u}$ 、压力 p 、固体位移 $\mathbf{d}$ ）。损失函数则囊括了所有域内的 PDE 残差、各域的边界条件、以及交界面的耦合条件。这种“整体求解”的思路避免了分区迭代带来的不确定性和稳定性问题。

• PINN 模型的具体设计

1. 网络架构：一个共享主干的多头输出网络，输入为 $(\mathbf{x}, t)$ ，输出为流体速度 $\mathbf{u}$ 、压力 p 、固体位移 $\mathbf{d}$ 。可以通过域标识符或直接在损失函数中区分不同区域。
2. 损失函数：

$$\mathcal{L} = \lambda_f \mathcal{L}_{NS} + \lambda_s \mathcal{L}_{Elastic} + \lambda_{b,f} \mathcal{L}_{b,f} + \lambda_{b,s} \mathcal{L}_{b,s} + \lambda_\Gamma \mathcal{L}_\Gamma$$

其中耦合条件损失为：

$$\mathcal{L}_\Gamma = \lambda_{kin} \left\| \mathbf{u} - \frac{\partial \hat{\mathbf{d}}}{\partial t} \right\|_\Gamma^2 + \lambda_{dyn} \left\| \hat{\sigma}_f \cdot \mathbf{n} - \hat{\sigma}_s \cdot \mathbf{n} \right\|_\Gamma^2$$

3. 训练策略：所有场量及其导数均可通过网络输出的自动微分得到。

多物理场耦合问题给 PINN 训练带来极大挑战，核心问题是损失项尺度不平衡。流体和固体方程在量纲、数值尺度和物理特性上差异巨大，导致损失函数中各项的梯度数量级相差悬殊，训练可能被某一物理场主导，而忽略其他物理场。应对策略包括：

- 自适应损失加权：如基于神经正切核（NTK）分析的方法，通过分析 PINN 训练初期不同损失项收敛速率的差异，自动调整损失权重，平衡各物理场的训练进度。
- 分阶段训练：先独立训练满足部分强约束的子问题（如稳态流场或静态固体变形），再将预训练模型作为初始化，加入耦合条件进行联合精细调优。
- 无量纲化：将所有物理量进行无量纲化处理，使各物理场的数值尺度相近，从根本上缓解尺度不平衡问题。

• 结果分析与对照

在经典的一维活塞流固耦合问题中，PINN 能够准确捕捉流体压力和固体位移的耦合演化，与解析解或高精度数值解吻合良好。对于二维问题，PINN 在耦合界面处的应力连续性得到较好满足，但训练时间显著增加，且对超参数（如网络深度、损失权重）较为敏感。

• 总结与启示

多物理场耦合案例展示了 PINN 处理复杂系统整体建模的能力。通过统一的网络和损失函数，PINN 能够同时学习多个相互作用的物理场，避免了传统分区迭代的复杂性和稳定性问题。然而，多物理场问题的训练难度显著增加，需要精心设计网络结构、损失权重和训练策略。当前，PINN 在复杂多物理场问题上的应用仍处于探索阶段，对于具有强耦合、大变形或湍流特征的复杂问题，仍需结合物理约束和数值技巧才能获得可靠结果。

本节通过一系列从简到繁的应用案例，展示了物理信息神经网络如何从一个新颖的数学思想，演化为解决实际科学计算问题的有力工具。每个案例都围绕一个核心的数学问题展开，阐释了 PINN 的设计如何响应这些问题的特定需求与挑战。

我们看到，PINN 的魅力在于其简洁性和灵活性：它将复杂的 PDE 求解问题转化为一个神经网络优化问题，通过设计损失函数来编码所有的物理知识、边界条件和观测数据。这种范式使得研究人员可以用相对统一的代码框架，探索正向、逆向、高维、多物理场等多样化的问题。

然而，我们也必须清醒地认识到，PINN 并非“万能钥匙”。其在处理间断、超高维问题时的局限性，以及训练过程的黑箱性和对超参数的敏感性，都是当前活跃的研究领域。PINN 的成功应用，往往依赖于对问题物理本质的深刻理解，以及对神经网络训练技巧的熟练掌握。它更像是一种需要与传统数值方法互补、在某些特定场景下能发挥奇效的“特长生”，而非全方位的“替代者”。在将 PINN 用于任何严肃的科学或工程研究时，交叉验证至关重要。这包括：

1. 与传统高精度数值解对比（在可行的情况下）。
2. 网格收敛性分析：增加残差点密度，观察解是否收敛。
3. 物理合理性检查：验证解是否满足基本的物理直觉或次要的守恒律。

参考文献

- [1] Simon Arridge et al. “Solving inverse problems using data-driven models”. In: *Acta Numerica* 28 (2019), pp. 1–174.
- [2] Peter Benner et al. *Model reduction and approximation*. SIAM, 2017.
- [3] Steven L Brunton and J Nathan Kutz. “Promising directions of machine learning for partial differential equations”. In: *Nature Computational Science* 4.7 (2024), pp. 483–494.
- [4] Claudio Canuto et al. *Spectral methods: Fundamentals in single domains*. Springer, 2006.
- [5] Junwoo Cho et al. “Separable physics-informed neural networks”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.
- [6] Jean Donea and Antonio Huerta. *Finite element methods for flow problems*. John Wiley & Sons, 2003.
- [7] Lawrence C Evans. *Partial differential equations*. American Mathematical Society, 2010.
- [8] Joel H Ferziger and Milovan Perić. *Computational methods for fluid dynamics*. Springer, 2012.
- [9] Jacob Fish and Ted Belytschko. *A first course in finite elements*. John Wiley & Sons, 2007.
- [10] Jochen Garcke and Michael Griebel. *Sparse grids and applications*. Springer, 2012.
- [11] Jian Guan et al. “DaPINN: Dimension-augmented physics-informed neural networks”. In: *arXiv preprint* (2023).
- [12] Ehsan Haghighat et al. “A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics”. In: *Computer Methods in Applied Mechanics and Engineering* 379 (2021), p. 113741.
- [13] Ameya D Jagtap and George Em Karniadakis. “Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations”. In: *Communications in Computational Physics* 28.5 (2020), pp. 2002–2041.
- [14] Ameya D Jagtap, Ehsan Kharazmi, and George Em Karniadakis. “Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems”. In: *Computer Methods in Applied Mechanics and Engineering* 365 (2020), p. 113028.
- [15] George Em Karniadakis et al. “Physics-informed machine learning”. In: *Nature Reviews Physics* 3.6 (2021), pp. 422–440.
- [16] Aditi Krishnapriyan et al. “Characterizing possible failure modes in physics-informed neural networks”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 26548–26560.
- [17] Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations*. SIAM, 2007.
- [18] Randall J LeVeque. *Finite volume methods for hyperbolic problems*. Cambridge University Press, 2002.
- [19] S Liao et al. “Physics-informed neural networks for PDE problems: A comprehensive review”. In: *arXiv preprint* (2025).

- [20] J David Logan. *Applied partial differential equations*. Springer, 2015.
- [21] Lu Lu et al. “DeepXDE: A deep learning library for solving differential equations”. In: *SIAM Review* 63.1 (2021), pp. 208–228.
- [22] Lu Lu et al. “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators”. In: *Nature Machine Intelligence* 3.3 (2021), pp. 218–229.
- [23] K W Morton and D F Mayers. *Numerical solution of partial differential equations*. Cambridge University Press, 2005.
- [24] Nasim Rahaman et al. “On the spectral bias of neural networks”. In: *International Conference on Machine Learning*. PMLR, 2019, pp. 5301–5310.
- [25] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (2019), pp. 686–707.
- [26] Justin Sirignano and Konstantinos Spiliopoulos. “DGM: A deep learning algorithm for solving partial differential equations”. In: *Journal of Computational Physics* 375 (2018), pp. 1339–1364.
- [27] Matthew Tancik et al. “Fourier features let networks learn high frequency functions in low dimensional domains”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 7537–7547.
- [28] Lloyd N Trefethen. *Spectral methods in MATLAB*. SIAM, 2000.
- [29] Rui Wang et al. “Physics-informed neural networks for fluid-structure interaction”. In: *arXiv preprint* (2023).
- [30] Sifan Wang, Yujun Teng, and Paris Perdikaris. “On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks”. In: *Computer Methods in Applied Mechanics and Engineering* 393 (2022), p. 114823.
- [31] Sifan Wang, Yujun Teng, and Paris Perdikaris. “Understanding and mitigating gradient flow pathologies in physics-informed neural networks”. In: *SIAM Journal on Scientific Computing* 43.5 (2021), A3055–A3081.
- [32] Olek C Zienkiewicz, Robert L Taylor, and Jian Z Zhu. *The finite element method: Its basis and fundamentals*. Elsevier, 2013.